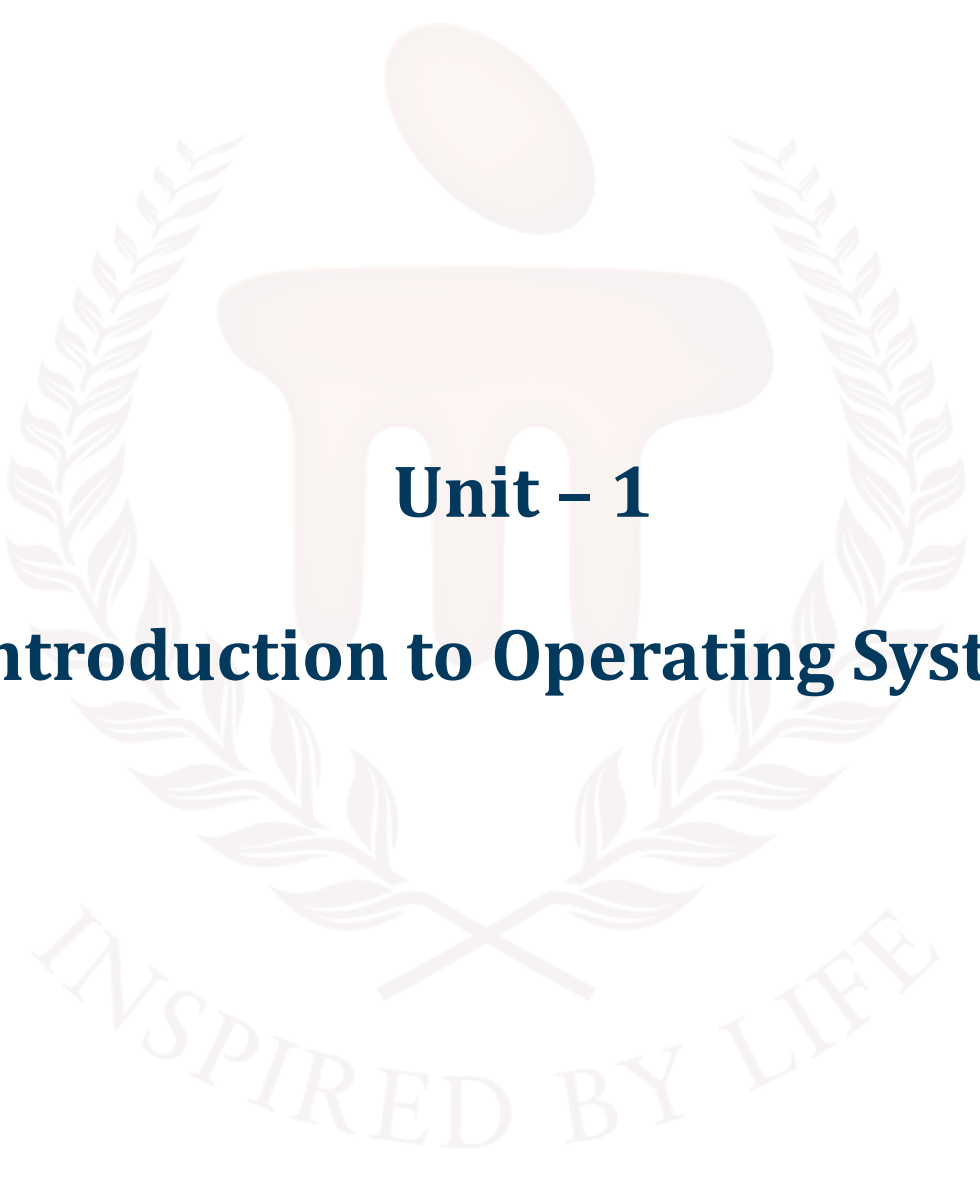


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 1

Introduction to Operating System

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 – 5
1.1	Objectives	-	-	
2	Operating System – Basic Concepts	-	1	6 – 16
2.1	OS Services	-	-	
2.2	Operating system Goals & Functions	-	-	
2.3	Views of Operating System	-	-	
2.4	Evolution of Operating systems	-	-	
3	Types of Operating System	1, 2, 3, 4, 5, 6	2	17 - 27
3.1	Simple Batch System	-	-	
3.1.1	Spooling	-	-	
3.2	Multiprogramming System	-	-	
3.3	Time sharing System	-	-	
4	Summary	-	-	28
5	Glossary	-	-	29 – 30
6	Terminal Questions	-	-	31
7	Terminal Answers	-	-	32
8	Answers	-	-	33
9	References	-	-	34

1. INTRODUCTION

In this unit, learners will explore the foundational concepts and functionalities of an operating system (OS), which serves as the crucial intermediary between a computer's hardware and the user or application software. Just like a government facilitates the smooth functioning of society, an operating system governs all hardware and software resources within a computer system. Without an operating system, modern computing would be chaotic, inefficient, and practically impossible for most users.

At the outset, learners will develop a clear understanding of what an operating system is, along with its critical role in managing the core resources of a computer—namely, the CPU, memory, storage, and input/output devices. They will learn how the OS schedules tasks for execution, allocates memory to processes, manages files and directories, and oversees input/output operations. These responsibilities form the backbone of system efficiency and stability.

The unit will then explore different types of operating system architectures, including single-processor, multiprocessor, and clustered systems. Through comparative analysis, learners will gain insights into how these systems differ in performance, scalability, reliability, and cost. They will also examine computer architectures such as asymmetric and symmetric multiprocessing systems understanding how multiple CPUs can collaborate to improve performance.

As part of this learning journey, students will investigate the evolution of operating systems across four generations. From the early days of no OS in the first generation, to batch processing in the second, the emergence of multiprogramming in the third, and the widespread use of personal computers and GUI-based OS in the fourth, the unit presents a historical perspective on how OS technology has adapted to meet growing user and business demands.

Another important focus area is the classification of operating systems based on their processing models batch systems, multiprogrammed systems, and time-sharing systems. Through real-world examples and illustrative diagrams, students will understand how different OS models handle task execution, system resources, and user interactions. They will also explore modern concepts such as multitasking, virtual memory, and spooling, which enhance the performance and responsiveness of systems.

Key OS services such as program execution, file manipulation, I/O operations, inter-process communication, and security will be detailed, demonstrating how the operating system not only acts as a resource manager but also as a service provider. Additionally, learners will discover how the OS

handles error detection and system protection, ensuring reliable operation in multi-user environments.

By the end of this unit, learners will be equipped with a solid conceptual framework to understand how operating systems function at both user and system levels. They will appreciate the OS as both a user-friendly interface and a sophisticated control mechanism that enables efficient and secure computing.

1.1. Objectives

By the end of this session, you will be able to:

- Define an operating system and describe its primary purpose.
- List different types of operating systems (e.g., batch, multiprogramming, time-sharing).
- Identify core functions of an OS such as memory management, process scheduling, and file handling.
- Explain how an operating system manages hardware and software resources.
- Illustrate the role of the OS in a multiprogramming or time-sharing environment.
- Describe the differences between single-processor, multiprocessor, and clustered systems.



2. OPERATING SYSTEM

There are numerous hardware and software resources available on a computer system that can be needed to finish a task. Commonly needed resources include CPU, memory, input/output devices, and file storage space. The operating system manages the aforementioned resources and distributes them to certain users and applications as needed to complete a given task. The operating system can thereby manage a computer system's resources internally, making it the resource manager. The processor, memory, files, and I/O devices are the resources

An operating system is essentially an interface that connects the user and the computer. You should be aware that all computers need an operating system in order to run other applications. The operating system primarily manages how various system and application programs access and utilise the hardware resources.

Scheduling is the method used to control the CPU and guarantee that every running task receives adequate processing time. Memory management keeps program conflicts at bay while guaranteeing that every application has enough memory to function. Communication between software and external devices, including keyboards, mouse, printers, and displays, is managed by I/O management. Through the management of files, directories, permissions, and access rights, the operating system's file management system keeps data on storage devices organized.

Apart from resource management, the operating system offers a graphical or command-line user interface that enables users to communicate with the machine. Additionally, it acts as the platform for application software. Basic functions including file handling, input/output processing, and network connectivity are provided by the OS to programs.

The operating system is frequently likened to a government since it facilitates the efficient operation of other apps but does not carry out actions that are directly beneficial to the user. The OS offers a stable and consistent environment for software to work in, much like a government does for a society.

2.1 OS Services

An operating system is an interface that offers services to programs as well as users. It offers a setting in which the software can run. Additionally, it gives users instructions on how to run programs in a

practical way. Both programs and their users can benefit from some features that the operating system offers. It goes without saying that the OS offers distinct services.

The common services that an operating system offers are as follows:

1. Execution of programs
2. Input/output activities
3. Manipulation of file systems
4. communication
5. Identification of errors
6. Allocation of resources
7. Protection

Execution of programs : An operating system needs to be able to operate and load a variety of tasks into memory. It must be possible for the program to terminate, either normally or abnormally. The full execution of the written code or program is referred to as a process. The operating system is responsible for a number of tasks. The operating system loads programs into memory, runs them, manages their execution, helps with process synchronization, and facilitates process communication.

Input/Output activities :

The operating system controls the communication between the user and device drivers. Input/Output devices are necessary for any process that is operating. An Input/Output device or a file may be engaged in Input/Output. Read and write activities, or I/O operations, are carried out with the aid of input-output devices. Access to I/O devices is granted by the operating system as necessary.

Manipulation of file systems : A file is an arrangement of connected data that represents some content. The secondary storage devices can hold files that are stored on the computer. to be stored for a long time. Magnetic disks and tapes, as well as optical disk drives like CDs and DVDs, are examples of storage media. A group of directories for convenient use and understanding is called a file system. Some files can be found in these directories. In relation to file management, an operating system performs a number of important tasks.

- The operating system grants the software access to conduct a file operation. Programs must have the ability to read and write files.
- A file can be created or deleted by the user through an interface that the operating system provides.

- A user interface for creating and deleting folders is provided by the operating system.
- The operating system provides an interface for creating a backup of the file system.

Communication: The operating system controls communication between all of the processes in the computer system, which consists of a group of processors that do not share memory, peripheral devices, or a clock. Through the network's communication channels, several processes can speak with each other. An operating system carries out a number of important communication-related tasks. Data transfer between two processes may be necessary. The two processes may be on the same computer or separate computers, but they are linked via a computer network.

Identification of errors: One component that could cause the entire system to malfunction is an error. The operating system continuously scans the system for flaws in order to prevent certain scenarios. This alleviates the user's concern about a malfunctioning system due to an error in any of its components. Anywhere and at any time, the problem could happen. The computer system's CPU, I/O devices, or memory hardware could all be the source of the issue.

The following tasks are carried out by an operating system:

- In order to fix faults and provide consistent computation, the operating system regularly checks for potential problems.

Allocation of resources: Resources must be distributed to each user or job when there are several operating concurrently. CPU scheduling algorithms are used to optimize CPU utilization, and the operating system uses schedulers to manage various resources.

Protection: A multi-user computer system's owners wish to regulate how their data is used. No process should be able to interfere with another when multiple disjoint processes are running simultaneously. The computer system's processes all need to be managed and secured.

2.2 Operating system Goals & Functions

OS Goals

An operating system has two different kinds of goals: primary goals and secondary goals.

- An operating system's main objective is to create a convenient and easy-to-use environment. Although using the operating system is not required, things get more challenging when the user is responsible for scheduling every task and it is also highly challenging to translate user code into machine code. Therefore, we utilize an operating system to serve as a mediator between the hardware and ourselves. You simply need to provide the operating system orders, and it will take care of the rest. Therefore, the operating system requires to be easy to use.
- Effective Computer System Use Efficiency is an operating system's secondary objective. The operating system should handle all resource management so that all available resources are used to their full potential and that no resource is left idle when a request for it is present.

Functions of an Operating System

An operating system carries out a variety of tasks in order to accomplish its objectives. They are:

- **Process Management** : One of the primary roles of an operating system is process management. In any computing environment, multiple processes (or programs in execution) may compete for access to the CPU. However, a processor can handle only one process at a time. The operating system ensures fair and efficient allocation of the CPU to these processes. It maintains a queue of processes in the ready state and schedules them based on predefined algorithms.

To achieve this, the OS implements **CPU scheduling algorithms** such as:

- **First Come First Serve (FCFS)**: Processes are executed in the order they arrive.
- **Shortest Job First (SJF)**: The process with the shortest burst time is given priority.
- **Round Robin (RR)**: Each process gets a fixed time slot in rotation.
- **Priority Scheduling**: Each process is assigned a priority; higher-priority processes are executed first.

The OS must also handle process creation, termination, and coordination. It keeps track of the **process control block (PCB)** for each process, which stores vital information like the program counter, register values, and process state.

- **Memory Management**: Memory is a critical resource in any computer system. The operating system is responsible for allocating and deallocating memory to different processes. When a process is initiated, its data, code, and execution context are loaded into the main memory.

Once the process completes execution, that memory space must be released for use by other processes.

Efficient memory management ensures that multiple processes can be accommodated without conflict or data corruption. Techniques like **segmentation** and **paging** are used to divide memory into manageable units. The OS must also keep track of free and used memory segments and ensure that processes do not interfere with each other's memory space (protection).

- **I/O Device Management:** Modern computer systems use a wide range of Input/Output devices such as keyboards, printers, disk drives, and network cards. Since multiple processes may require these devices, the operating system takes responsibility for managing and coordinating access. Instead of allowing direct device access, the OS provides a standardized interface through device drivers. This ensures controlled, safe, and efficient use of hardware.

The OS handles:

- I/O requests from processes
- Device allocation and deallocation
- Buffering, caching, and spooling

For instance, when a user prints a document, the OS queues the print job, manages the spooler, and sends data to the printer in an organized manner.

- **File Management:** Every computer system maintains a **file system** to store data and programs permanently. The operating system manages all aspects of file creation, deletion, access, and protection. This includes managing files and directories, along with the data structures used to store metadata.

One commonly used structure is the **File Allocation Table (FAT)**, which stores file-related information such as:

- File name
- File size
- File type

- File location on disk
- Access permissions

The OS ensures that files are stored efficiently and that unauthorized users cannot access protected files. It provides file handling commands and manages access rights to maintain the integrity and confidentiality of stored data.

- **Virtual Memory Management:** When running large applications, the physical memory (RAM) may not be sufficient to hold all the required data and code. The operating system addresses this limitation using **virtual memory**, a technique that gives the illusion of a much larger memory space. In virtual memory systems, only the **necessary parts of a program are loaded into RAM** at any one time. The remaining data resides on the disk and is brought into memory as needed through a process known as **paging**. This improves system efficiency by allowing multiple large programs to run simultaneously, even on limited physical memory. The OS maintains a **page table** that maps virtual addresses to physical addresses. It also handles **page faults** (when a required page is not in memory) by swapping pages between RAM and disk.

The operating system (OS) is primarily made to fulfill two fundamental functions:

1. It regulates how the resources of the computer system are distributed and utilized among different users and tasks.
2. It primarily offers an interface between the programmer and the computer hardware that enables coding, application program building, and debugging easier and possible.

2.3 Views of Operating System

An operating system (OS) is a critical component of any computer system, acting as an intermediary between the user and the hardware. To understand its significance, it can be examined from two major perspectives—**the user's view** and **the system's view**. These views reflect the dual nature of the OS, which simultaneously serves human users and the system's hardware and software components. Each perspective reveals different priorities, functionalities, and design goals of the OS.

Two Views of Operating System are:

1. User's View
2. System View

1. User's View

From a user's perspective, the operating system is essentially a tool that enables interaction with the computer. This view focuses on the **interface** provided by the OS—such as the desktop, graphical icons, command-line utilities, or touch-based navigation. In this context, the operating system is primarily evaluated based on how easy and convenient it is for users to perform tasks like opening applications, accessing files, browsing the internet, or editing documents.

This view is particularly relevant in **single-user systems** like personal laptops, smartphones, or desktop computers, where only one individual typically interacts with the device at a time. In such environments, the **goal is not optimal resource utilization** but rather **ease of use and user satisfaction**. The design of these systems prioritizes **responsiveness, intuitive controls, and aesthetics**. For example, operating systems like Windows, macOS, Android, and iOS are designed with extensive user interface (UI) considerations to make them accessible even to users with minimal technical knowledge.

In these systems, users are generally unaware of the underlying complexities of managing memory, scheduling tasks, or handling device communications. The OS hides these details and provides a seamless experience. Users expect the system to respond quickly, open files on demand, and allow smooth multitasking. This expectation drives the OS to handle background operations in a way that does not disturb the user experience. For instance, when a user plays music while writing a document and receiving notifications, the OS manages all these activities without requiring the user to understand how CPU cycles or memory blocks are distributed among the tasks.

Thus, the **user's view** treats the operating system as a **friendly facilitator** that bridges the gap between complex hardware functions and human commands. Its success is measured by the quality of the user experience, the simplicity of operation, and the reliability in carrying out user instructions.

2. System View

On the other hand, from the **system's perspective**, the operating system functions as a **resource manager**. This view highlights the OS's responsibility to efficiently and fairly manage the hardware and software resources of the computer system. These resources include the **CPU, memory, input/output devices, file systems**, and network interfaces. Unlike the user's view, which emphasises usability, the system view focuses on **optimization, control, and allocation**.

In this context, the OS is responsible for ensuring that resources are **allocated properly to various tasks and reclaimed when no longer needed**. When multiple programs or users request the same resource—such as access to the CPU or a storage device—the OS must **resolve conflicts and schedule usage** in a way that maintains system stability and performance. For example, it uses scheduling algorithms to determine which process should run next, or memory management techniques to allocate RAM among active processes without conflicts.

The operating system must also **enforce protection and security policies** to prevent one process from interfering with another or accessing restricted resources. This becomes especially crucial in multi-user or multi-tasking environments, such as servers or enterprise systems, where the system view dominates design priorities. In these systems, performance, scalability, and fault tolerance are often more important than user interface features.

The OS acts as a **control center** that coordinates all operations within the system. It tracks the status of each resource, monitors system health, logs activities, and ensures that the execution of applications conforms to system policies. For example, when an application attempts to open a file, the OS checks whether the application has the necessary permissions and whether the file is already in use by another process. These checks are essential for maintaining system integrity.

In the system view, the OS is also responsible for **providing abstraction**. It abstracts complex hardware operations into simpler services and APIs (Application Programming Interfaces) that developers and system software can use. This abstraction layer makes it easier to build, maintain, and run applications without needing to tailor them to specific hardware models.

2.4 Evolution of Operating Systems

The First Generation(1940-1950):

In 1940, the first electrical computer was invented, but it lacked an operating system. Users used to have complete access to computers and create programs for every task in complete computer language. During computer creation, the programmer can just execute and resolve basic mathematical computations; an operating system is not necessary for these calculations.

The Second Generation(1955-1965)

GMOS, the first operating system (OS), was developed in the early 1950s. The operating system for the IBM computer was created by General Motors. Because it gathers all similar operations in groups or batches and then sends them to the operating system via a punch card to finish all jobs in a machine, the second-generation operating system was based on a single stream batch processing system. Every time a job is finished (naturally or abnormally), control is transferred to the operating system, which cleans up after finishing a job before reading on and starting the next one in a punch card. Subsequently, professional operators began using large, modern machines known as mainframes.

The Third Generation (1965-1980)

Operating system designers were able to create a new operating system in the late 1960s that could multiprogram, or execute several operations in a single computer program, at the same time. Multiprogramming's introduction was crucial to the development of operating systems that enable a CPU to stay active at all times by handling multiple jobs on a computer at once. In 1961, the DEC PDP-1 marked the beginning of a new phase in the amazing rise of minicomputers during the third generation. Personal computers of the fourth generation are the result of these PDPs.

The Fourth Generation(1980-present day)

The evolution of the personal computer is linked to the fourth generation of operating systems. Nonetheless, the third-generation minicomputers and the personal computer have many similarities. At the time, personal computers were extremely expensive; minicomputers were only slightly less expensive. The development of Microsoft and the Windows operating system played a significant role

in the creation of personal computers. In 1975, Microsoft developed the first Windows operating system. Bill Gates and Paul Allen had the idea to advance personal computers after releasing the Microsoft Windows operating system. Thus, in 1981, they released the MS-DOS.

Windows has emerged as the most widely used and well-liked operating system. Windows then released a number of operating systems, including Windows 95, Windows 98, Windows XP, and the most recent, Windows 7. The Windows 10 operating system is currently used by the majority of Windows users. In addition to Windows, Apple is another well-known operating system that was created in the 1980s and was created by Steve Jobs, one of the company's co-founders. The operating system was dubbed Mac OS or Macintosh OS.

SELF-ASSESSMENT QUESTIONS - 1

Fill in the Blanks:	
1	An operating system acts as an _____ between the user and the hardware.
True or False	
2	The operating system is responsible for managing both software and hardware resources.(True/False)
Multiple choice Questions	
3	Which of the following is NOT a function of the operating system? a) Process management b) Web browsing c) Memory management d) File system management
4	Which memory management technique loads only the necessary parts of a program into memory? a) Paging b) Segmentation c) Virtual memory d) Partitioning
Fill in the Blanks:	
5	The table used by the operating system to store file information is called _____.

6	In a _____ multiprocessing system, each processor is assigned a specific task.
True or False	
7	Virtual memory allows a program to use more memory than is physically available.(True/False)
8	In symmetric multiprocessing, only one processor is active at a time.(True/false)
Multiple choice Questions	
9	Which of the following is a single-user, single-tasking operating system? a) MS-DOS b) UNIX c) Linux d) Windows 10
10	Which generation introduced multiprogramming operating systems? a) First b) Second c) Third d) Fourth



3. TYPES OF OPERATING SYSTEM

1. Simple Batch System
2. Multiprogrammed
3. Time-Shared

3.1 Simple Batch System

A Simple Batch System is an early type of operating system designed to improve the efficiency of computer usage during a time when user interaction with the computer was minimal. These systems were developed when computers were extremely costly, and the primary focus was on reducing the idle time of the CPU.

In this approach, jobs (programs) are grouped into batches and then executed one after another without manual intervention between each job. A batch is simply a set of jobs that are submitted to the computer together, typically by an operator. These jobs are often similar in nature, requiring the same setup or compiler, making it efficient to run them together.

To illustrate this, consider the example mentioned in the content. Suppose there are ten programs that need to be executed—some written in Java, others in C++. Normally, without a batch system, we would have to load the Java compiler each time we run a Java program, and the same for the C++ compiler. This repetitive loading is inefficient.

In a batch system, we can organise all the Java programs into a batch, load the Java compiler just once, and then execute all Java jobs in sequence. The same applies to C++ jobs. This technique significantly reduces overhead, as it avoids the need to load the compiler repeatedly for each job.

The batch system requires an operator who collects similar jobs from users, batches them together, and submits the entire batch to the Operating System (OS) for processing. The OS takes care of executing the jobs one after another and producing the required outputs

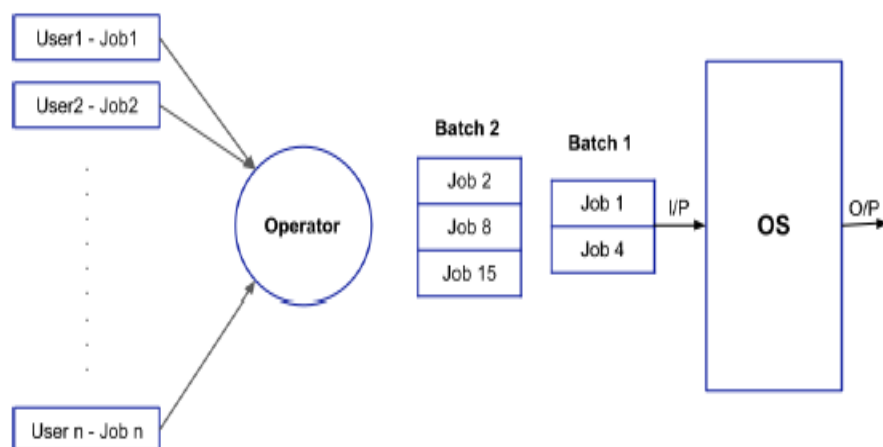


Figure 3 : Simple Batch System

The figure 3 illustrates the process of how jobs are collected, organised, and executed in a batch processing environment. On the left side of the figure, multiple users—such as User 1, User 2, and so on up to User n—submit their individual jobs (e.g., Job 1, Job 2, Job n). These jobs are not sent directly to the operating system. Instead, they are first handled by an **operator**, who acts as a coordinator.

The **operator** is responsible for grouping similar types of jobs together into **batches**. For example, in the figure, two batches are formed: **Batch 1** includes Job 1 and Job 4, while **Batch 2** includes Job 2, Job 8, and Job 15. The criteria for forming these batches could be based on programming language, resource requirement, or job type. This step ensures that similar jobs are processed together, improving execution efficiency.

After forming the batches, the operator submits them sequentially to the **operating system (OS)**, which is depicted as a large rectangular box in the diagram. The **input arrow (I/P)** pointing to the OS indicates that it receives the batches from the operator. The OS then begins processing the jobs one after another in the order they were batched, without any user intervention during execution.

Finally, the **output arrow (O/P)** represents the results produced by the operating system after processing the batch. This output is delivered after all jobs in the batch have been executed. The figure clearly shows the flow from job submission by users, through batch formation by the operator, to job execution and result generation by the operating system.

Benefits :

A reduction in the system's overall execution time for all programs.

1. Multiple users can share the Batch Operating System.
3. Batch processing makes handling huge, repetitive tasks simple.
4. In batch systems, no specialized hardware or system support is needed to input data.
5. A number of users can share it.
6. The batch system's idle time is extremely low. It makes it possible for us to effectively handle the heavy workload.

Drawbacks:

1. Between two batches, manual interventions are necessary.
2. Because batch loading and unloading times are significantly longer than execution times, the CPU utilization is low.
3. A longer turnaround time.
4. Execution order is not strictly predictable
5. Behavior that cannot be changed.
6. Any mistake could cause a job to go into an endless loop.
7. Occasionally turns out to be expensive.

3.1.1 Spooling:

SPOOL stands for simultaneous online peripheral operations. It is a type of buffering mechanism or procedure where data is momentarily stored for usage and execution by a program, device, or system. Until the computer or software asks for it to be executed, data is delivered to and kept in memory or another volatile storage.

Printers, punch card readers, and other peripheral devices (batch processing) in computer systems operate very slowly in comparison to the system as a whole. It was soon determined that obtaining input and output from the system constituted a bottleneck. This is where the spool is needed.

Data, instructions, and processes from various sources are gathered for subsequent execution using spooling, which functions similarly to a standard request queue. It is typically kept up to date on the physical memory and buffers of computers.

I/O device-specific interrupts or buffers. The first instruction in the queue will be popped and executed since the spool is processed in a FIFO fashion.

The acronym for "Simultaneous Peripheral Operations Online" is "spooling." Therefore, in a spooling, multiple I/O operations can be carried out concurrently, that is, when the CPU is running a process, multiple I/O operations can also be carried out concurrently. The following illustration will improve our comprehension of the idea:

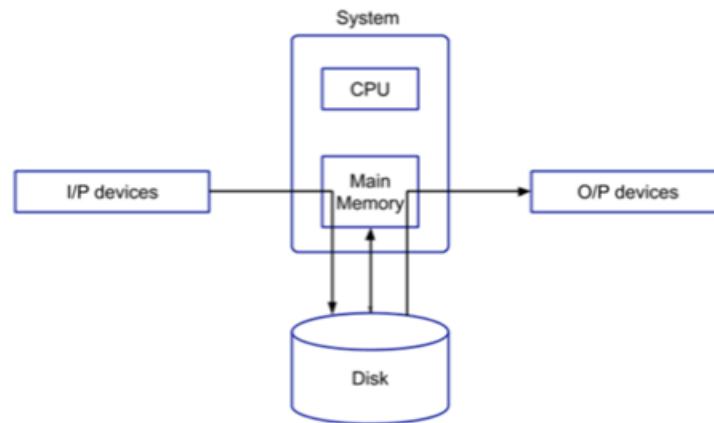


Figure 4 : Spooling

As can be seen from the figure above, the main memory retrieves the input data after it has been saved in a secondary device of some sort. This method has the advantage because the CPU typically uses the information kept in main memory. Given that many input devices can be used simultaneously, all of these devices have the ability to save data in secondary memory or the disk. The CPU will then run certain instructions on the data after the main memory has retrieved each piece of information from the secondary memory one at a time. Since the primary memory and secondary memory are both digital, data transfer between them happens quickly.

Additionally, the input devices don't have to wait for their turn when the CPU is working on a job. They don't have to wait for their turn to enter their data into the secondary memory. The CPU will spend the majority of its time in the execution phase as a result. Therefore, in this scenario, the CPU won't be idle.

For instance, multiple papers may need to be printed in a printer spool. The printer can then retrieve the documents off the spool and print each one individually.

Benefits:

- The CPU does not have to wait for the I/O operation to occur since there is no interaction between the CPU and I/O devices. It takes a long time to do the I/O operations.
- Since the CPU is kept working most of the time, it is not in an idle state, which is a desirable circumstance.
- Multiple I/O devices can operate at the same time.

The distinction between buffering and spooling.

- Spooling allows one job's I/O to be managed alongside some of another job's processes.
- Only one job is managed at a time during buffering.
- Compared to buffering, spooling is more efficient.
- A buffer is a little, distinct region of memory used for buffering. However, spooling can utilize all of the RAM.

3.2 Multiprogramming System

Multiprogramming is the term used to describe shared processing when two or more programs run simultaneously in memory. Multiprogramming is predicated on a single shared CPU. By arranging tasks so that the CPU always has one to complete, multiprogramming raises CPU utilization. A multiprogramming system's memory arrangement is depicted in the following figure.

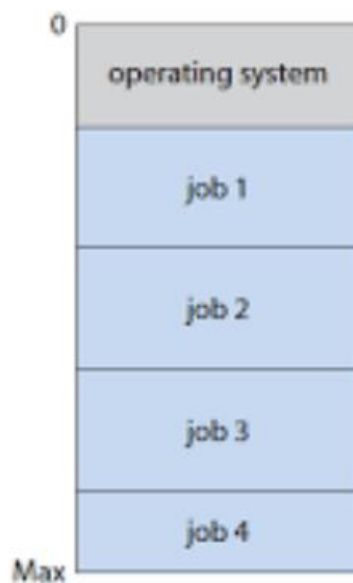


Figure 5 : memory layout of Multiprogrammed System

- An operating system performs the following multiprogramming tasks. The operating system maintains multiple tasks in memory simultaneously.

- The operating system selects and starts to run one of the jobs in memory from this collection, which is a subset of the jobs stored in the job pool.
- Unless there are no jobs to process, multiprogramming operating systems use memory management programs to keep an eye on the status of all running programs and system resources.

Benefits:

- High and effective CPU use.
- The user perceives that numerous apps are receiving CPU allocations nearly at the same time.
- The CPU is never idle.
- Effective use of resources and a quicker response time.
- Short-term tasks are finished more quickly than long-term ones, and throughput is higher

Drawbacks:

- CPU scheduling is necessary, and memory management is necessary to support numerous tasks in memory.
- Jobs that take a long time must wait.
- It might be challenging at times to keep track of every process.
- It is necessary to schedule CPU usage.
- Effective memory management is necessary.
- It is not feasible to engage with users while the program is running.

Working of a multi-programmed system:

- When a job in a multi-programmed system initiates an I/O task, the operating system stops it, selects a new job from the job pool (waiting line), assigns CPU to the new job, and begins working on it. This job performs CPU-bound tasks, while the prior job continues to perform I/O operations. Let's imagine the CPU selects a third job and begins running it after the second job completes an I/O task. When an I/O operation is finished and a job returns for CPU duties, the CPU is assigned to it.
- By doing this, the system avoids wasting CPU time waiting for the I/O process to finish. Thus, the ultimate objective of multiprogramming is to maintain CPU activity as long as there are active processes. In this manner, it is possible to run numerous programs on a single processor by running a portion of one program at a time, followed by another portion of another program, and so on. As a result, the CPU is never idle. It has the CPU assigned to it.

- Program A executes for a while before entering a waiting state in the picture below. Program B starts running in the interim. Thus, the CPU allows program B to run and avoids wasting its resources.

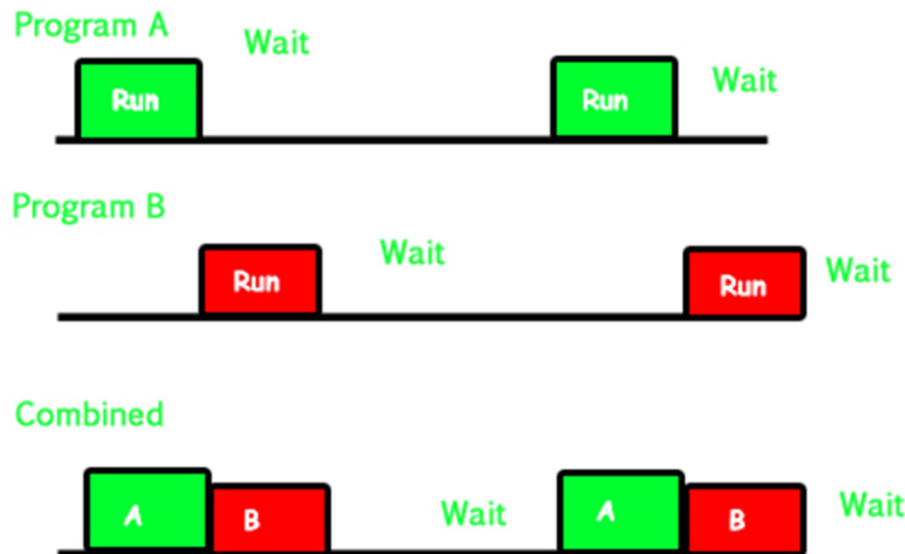


Figure 6 : Working of Multiprogrammed System

3.3 Time Sharing System

Multitasking, as the name implies, is the practice of carrying out several tasks (such as programs, threads, processes, etc.) simultaneously. Through the use of multitasking, contemporary operating systems enable us to simultaneously listen MP3 music, edit papers in Microsoft Word, and browse Google Chrome.

Multitasking is an extension of multiprogramming. While multiprogramming allows multiple programs to reside in memory, multitasking adds the ability for users to interact with those programs using time-sharing. One of the main ways that multitasking and multiprogramming are different is that multitasking is built on time sharing in addition to the concept of context switching, while multiprogramming does not.

An operating system that supports multitasking uses the time-sharing principle to run multiple processes simultaneously. Therefore, in a time-sharing environment, we set a time, known as the time quantum, and when a process begins to execute, it does so for that duration only. After that, other processes will have a chance to execute for that duration only. In the following cycle, the first process will be conducted once more for that time quantum alone, and then the second process will be executed. This process will go on. The operation of a time-sharing operating system is depicted in the following figure.

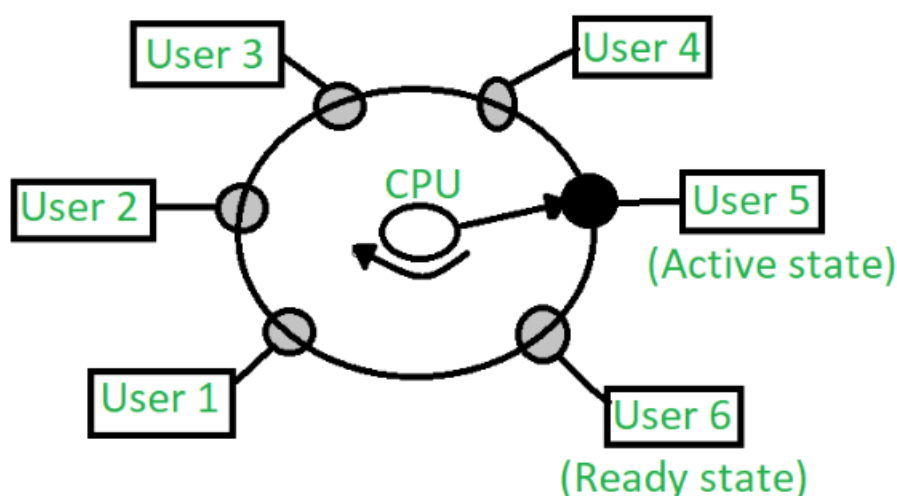


Figure 7 : Time Sharing Operating System

The figure 7 illustrates how the CPU is shared among multiple users in a time-sharing environment. In this system, several users (User 1 to User 6) are connected to a central CPU, and each is allocated a short, fixed slice of processing time in a cyclic order. The black-filled circle pointing to User 5 signifies that this user is currently in the active state, meaning the CPU is executing User 5's task during its allocated time slice. Meanwhile, User 6 is shown in the ready state, indicating that their task is lined up next for execution as soon as the CPU finishes processing the current one. The remaining users (User 1 to User 4) are also part of the circular queue and will be scheduled for execution in subsequent time slots. This round-robin scheduling mechanism ensures that all users receive fair access to the CPU without any one user monopolizing system resources. It creates the illusion that all users are working simultaneously, even though the CPU is actually switching rapidly between tasks. This figure effectively demonstrates the fundamental concept of time sharing, where system responsiveness and user interactivity are maintained through quick context switching.

The operation of multitasking systems:

- In a time-sharing system, each process is given a defined amount of time to complete its task. Assume that processes P1, P2, P3, and P4 are all prepared to run. Each of them is therefore given a time quantum, such as ten nanoseconds (10 ns), for which they will execute. When a process (let's say P1) starts running, it runs for that amount of time (10 ns). The CPU begins executing the other process (let's say P2) for the designated amount of time after 10 ns.
- As a result, the CPU causes the processes to run in accordance with sharing time slices. Another process starts running as soon as the time quantum of the first one ends.
- Essentially, a context switch is taking place here as well, but it happens so quickly that the user can interact with each program independently while it is running. The user is thereby given the impression that several jobs or processes are running concurrently. However, at any given moment, only one process or job is truly running. Time sharing works best while multitasking since each process uses a reasonable amount of CPU time.
- Multitasking is the more broad term for having several tasks, threads, processes, and programs operating simultaneously. This phrase refers to situations in which several jobs share a single processing resource (such as CPU and memory) in contemporary operating systems.

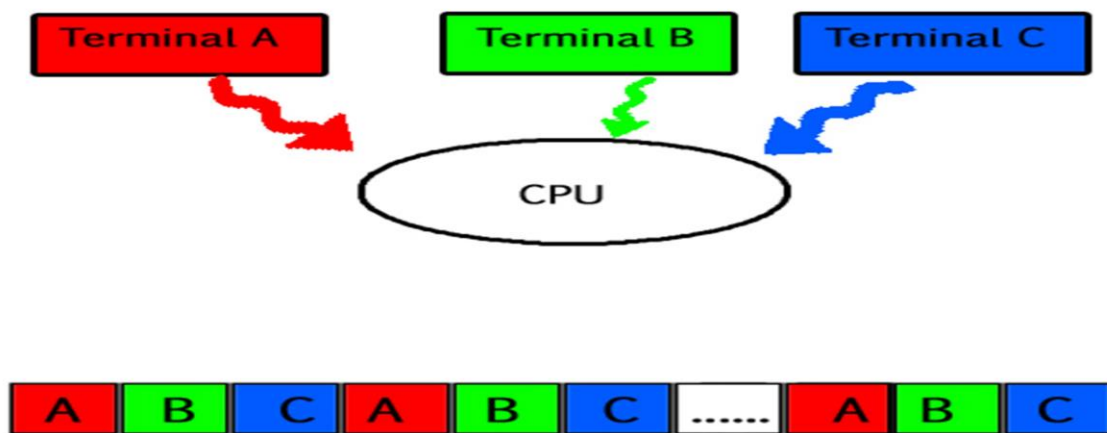


Figure 8 : operation of Multitasking System

The figure 8 illustrates how a multitasking operating system handles multiple processes or terminals—specifically **Terminal A**, **Terminal B**, and **Terminal C**—by allowing them to share the CPU in an organized manner. Each terminal represents a separate task or program, and they are color-coded: Terminal A (red), Terminal B (green), and Terminal C (blue). These terminals are connected

to the CPU, which sequentially executes small parts of each task by allocating a time slice to every terminal in a rotating order.

The bottom part of the figure shows a timeline that visualizes how the CPU cycles through tasks A, B, and C repeatedly. It begins with executing a portion of A, then moves to B, then to C, and the cycle continues in this same pattern. This approach creates the **illusion of parallelism**, making it appear as if all three tasks are running simultaneously, while in reality, the CPU is rapidly switching between them.

This form of task management is essential in modern operating systems to maintain responsiveness and efficiency. It ensures fair CPU time distribution among tasks, maximizes resource utilization, and improves the overall user experience by quickly cycling through processes without noticeable delay.

Benefits:

1. Every process has an equal chance to execute since each process is assigned an equal amount of time.
2. In most situations, the CPU will be busy, which is a useful feature.

Drawbacks:

1. Because every process is given an equal chance, the process with a greater priority will not be given the opportunity to be completed first.

SELF-ASSESSMENT QUESTIONS - 2

Fill in the Blanks:	
11	In _____ operating systems, each task gets a fixed time slot to execute.
12	_____ is used to temporarily store data for input/output operations.
True or False	
13	Batch operating systems provide immediate user interaction.(True/False)
Multiple Choice Questions	
14	Which type of operating system allows multiple users to access the system simultaneously? a) Batch OS b) Multiprogramming OS c) Time-sharing OS

	d) Real-time OS
True or False	
15	The command-line interface is more user-friendly than the graphical user interface.(True/false)



4. SUMMARY

In this unit, you have learnt An **Operating System (OS)** is essential system software that manages computer hardware and software resources, providing a stable environment for users and application programs. Acting as an interface between users and the hardware, the OS handles core functions such as **process scheduling, memory management, file system control, device coordination, and system security.**

Operating systems come in various types, including **batch systems, multiprogramming systems, and time-sharing systems**, each designed to improve system efficiency, multitasking, and user interaction. Advanced OS architectures support **single-processor, multiprocessor, and clustered systems**, enabling scalability and reliability in modern computing environments.

The OS facilitates **user interfaces** either command-line or graphical allowing interaction with system components. It also ensures secure, synchronized communication between processes and devices, and handles errors and resource allocation effectively.

Over time, operating systems have evolved through four generations—from basic machine-level control to sophisticated, user-friendly systems like **Windows, Linux, and MacOS**. Today's operating systems support multitasking, virtual memory, and network communication.

The OS is the backbone of a computer system, responsible for resource management, system coordination, and providing an accessible computing environment, making it indispensable for both users and developers.

5. GLOSSARY

Operating System (OS)	- System software that manages computer hardware, software resources, and provides services to applications.
Kernel	- The core part of the operating system that controls all hardware operations and manages system resources.
Process	- A program in execution; it includes the current activity, program counter, and resource information.
Thread	- The smallest unit of execution within a process; allows parallel execution inside a single process.
Multitasking	- The ability of the OS to execute multiple tasks or processes simultaneously.
Multiprogramming	- Technique where multiple programs are loaded into memory and the CPU switches between them for execution.
Time Sharing	- A method that allows multiple users or tasks to share system resources efficiently through time slicing.
Virtual Memory	- A memory management capability that allows a system to compensate for shortages of physical memory by using disk space.
Spooling	- Simultaneous Peripheral Operations Online; a process where data is temporarily stored to be processed later (e.g., print queue).
Buffering	- Temporary storage of data while it is being transferred between two locations, usually between I/O devices and the CPU.
File System	- The method used by the OS to store, organize, and retrieve files on storage devices.
System Call	- A request by a program to the operating system to perform a specific operation like file handling or process control.
Scheduler	- A component of the OS that decides which process should be executed next by the CPU.

Deadlock	- A condition where two or more processes are unable to proceed because each is waiting for the other to release resources.
Device Driver	- Specialized software that allows the operating system to communicate with hardware devices.
Swapping	- The process of moving processes in and out of main memory to ensure optimal use of memory resources.
Shell	- The user interface for accessing OS services, typically through a command-line or graphical interface.
Clustered System	- A group of independent computers that work together as a single system for high availability and load balancing.
Symmetric Multiprocessing (SMP)	- A system where each processor has equal access to resources and can perform any task.
Asymmetric Multiprocessing (AMP)	- A system where each processor is assigned a specific task; usually one master controls the rest.

6. TERMINAL QUESTIONS

1. Explain the key functions of an operating system.
2. Differentiate between single-processor, multiprocessor, and clustered systems.
3. Define multiprogramming. How does it improve CPU utilization?
4. Explain the working principle of Time Sharing system
5. List and explain any five services provided by an operating system.
6. Discuss the evolution of operating systems through four generations. How did each contribute to modern OS development?
7. Compare batch processing, multiprogramming, and time-sharing operating systems with examples.
8. Explain the architecture and types of multiprocessor systems. How do symmetric and asymmetric multiprocessing differ?
9. Describe the core components of an operating system and how they interact to manage a computer system efficiently.
10. What are the major goals of an operating system? How does it achieve efficient resource management and user convenience?

7. TERMINAL ANSWERS

1. Refer: Section 1.1.3 Functions of an Operating System
2. Refer: Section 1.1.1 computer Architecture
3. Refer: Section 3.2 Multi programming
4. Refer: Section 3.3 Time sharing System
5. Refer: Section 2.1 OS Services
6. Refer: Section 1.1.6 Operating System Generations
7. Refer: Section 1.2.1 Batch Operating System, 1.2.2 Multiprogrammed System and 1.2.3 Time sharing System
8. Refer: Section 1.1.1 Computer Architecture , Multiprocessor system, symmetric and asymmetric multiprocessing
9. Refer: Section 2.1 OS Services
10. Refer: Section 2.2 Operating system Goals & Functions

8. ANSWERS

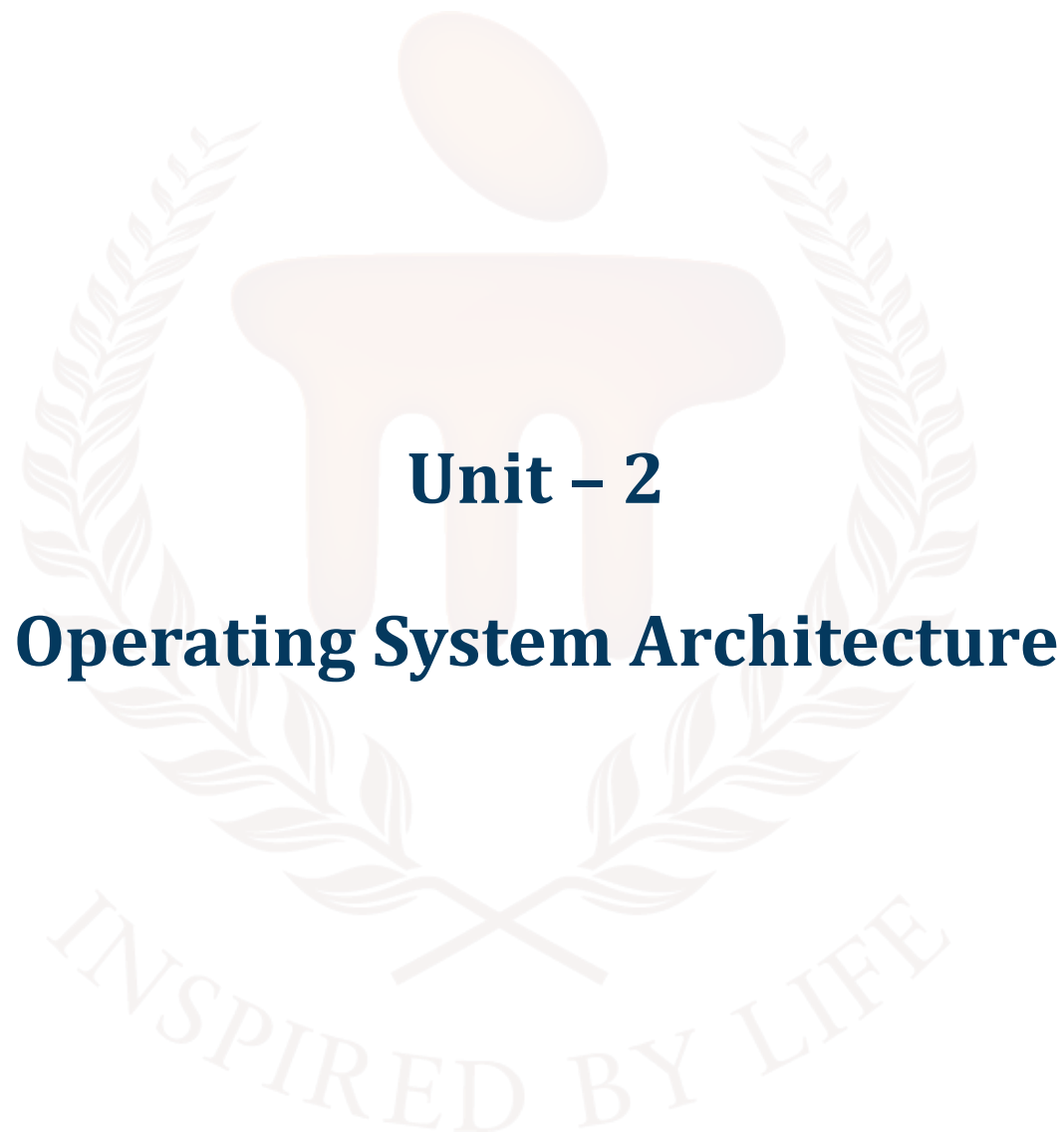
Self-Assessment Questions

1. Interface
2. True
3. Web browsing
4. Virtual memory
5. File Allocation Table (FAT)
6. Asymmetric
7. True
8. False
9. MS-DOS
10. Third
11. time-sharing
12. Spooling
13. False
14. Time-sharing OS
15. False

9. REFERENCES

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 10th edition, Wiley Publications.





Unit – 2

Operating System Architecture

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 – 5
1.1	Objectives	-	-	
2	Operating system Structure	1 , 2 , 3 , 4 , 5	1	6 - 19
2.1	Simple Structure	-	-	
2.2	Monolith Structure	-	-	
2.3	Micro-Kernel Structure	-	-	
2.4	Exo-Kernel Structure	-	-	
2.5	layered structure	-	-	
2.6	Modular Structure	-	-	
2.7	Virtual Machines	-	-	
3	System components	-	2	20 – 26
4	Protection	-	3	27 – 31
5	Summary	-	-	32
6	Glossary	-	-	33 – 34
7	Terminal Questions	-	-	35
8	Terminal Answers	-	-	36
9	Answers	-	-	37
10	References	-	-	38

1. INTRODUCTION

In this unit, learners will explore the fundamental architecture and structure of operating systems, which serve as the backbone for all computer operations. The unit introduces various structural models of operating systems including simple, monolithic, microkernel, exokernel, layered, modular, and virtual machine-based designs. Each structure offers a different approach to organizing and managing system resources, and learners will examine the benefits and limitations of each model in terms of performance, maintainability, security, and flexibility. By comparing these architectures, learners will gain insights into how design choices influence the efficiency and reliability of operating systems.

Beyond structure, the unit focuses on key system components that work together to perform core tasks. These include process management, memory management, file and storage systems, I/O device control, network management, security measures, and the command interpreter system. Learners will understand how these components interact to coordinate hardware and software functions effectively. Special attention is also given to system protection mechanisms that safeguard data and system integrity, ensuring that resources are used appropriately and securely in multi-user or multi-process environments. Overall, this unit provides a comprehensive understanding of how operating systems are designed and operated to support modern computing requirements.

By the end of this unit, learners will have a strong foundational understanding of the internal design and organisation of operating systems. They will be able to compare and contrast different structural models, understand how core system components interact, and recognise the importance of protection in maintaining a secure and stable computing environment. This knowledge is essential for anyone pursuing further studies or careers in computer science, software engineering, or system administration.

1.1. Objectives

By the end of this unit, learners will be able to:

- Define key terms related to operating system structures and components.
- List different types of operating system architectures such as monolithic, microkernel, and modular structures.
- Describe the purpose and function of various OS architectures.
- Explain the roles of system components like process management, memory management, and file systems.
- Compare and contrast different operating system structures in terms of modularity, performance, and security.
- Assess the strengths and limitations of specific OS architectures in real-world scenarios.



2. OPERATING SYSTEM

An operating system is a framework that facilitates communication between user applications and system hardware. Given its intricate structure, the operating system should be carefully designed to facilitate easy use and modification. Partially building the operating system is a simple method to accomplish this. Each of these components needs to have distinct inputs, outputs, and functionalities.

A variety of structures can be used to implement an operating system. The way the several common components of the operating system are integrated into the kernel and related to one another determines the OS's structure. Based on this, the operating system has the following structures:

- **Simple Structure**
- **Monolith Structure**
- **Micro-Kernel Structure**
- **Exo-Kernel Structure**
- **Layered Structure**
- **Modular Structure**
- **Virtual Machines**

2.1 Simple Structure

Such operating systems are tiny, basic, and constrained systems without a clear structure. Interfaces and functionality tiers are not clearly distinguished from one another. Among these operating systems is MS-DOS. Programs can access the fundamental I/O operations in MS-DOS applications. If one of the user programs on these operating systems fails, the system as a whole crashes.

Unlike MS-DOS, it is preferable for operating systems to be modular. More control over the computer system and its different applications would result from that. Additionally, the programmers could build internal procedures and conceal information as needed without altering the external requirements because to the modular nature.

The MS-DOS structural diagram is displayed below.

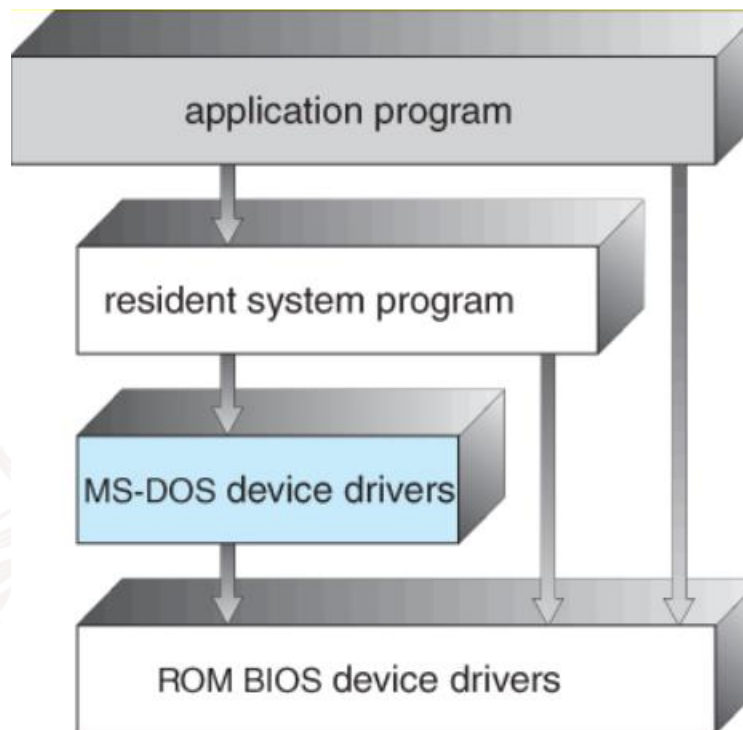


Figure 1 : The MS-DOS structural diagram

Benefits:

- **Simple Development:** Development is made simple in systems with minimal interfaces, particularly when just a small number of features need to be provided.
- **Improved Performance:** Compared to other operating system types, this system can offer superior performance because it has less layers and integrates directly with hardware.

Drawbacks:

- **Regular System Failures:** Such a system is not robust due to its bad design. The entire operating system crashes if one program fails. As a result, basic operating systems frequently experience system failures.
- **Poor Maintainability:** Because operating systems' levels are closely interconnected, changes made to one might have a significant effect on other layers, eventually rendering code unmanageable.

2.2 Monolithic Structure:

The monolithic operating system is an extremely basic operating system where the kernel directly controls memory management, device management, file management, and process management.

Every resource in the system is available to the kernel. Each operating system component is housed within the kernel of monolithic systems. The 1970s saw the introduction of operating systems with monolithic architecture.

The kernel serves as an interface between application programs and the underlying hardware and has access to all resources. Older banking systems had a monolithic kernel structure, which encourages timesharing and multiprogramming.

The monolithic kernel is another name for the monolithic operating system. This is an outdated operating system that banks use for time-sharing and other minor activities like batch processing. All hardware components are controlled by the monolithic kernel, which functions as a virtual machine.

It is not the same as a microkernel, which has fewer functions. User space and kernel space are the two sections that make up a microkernel. Inter-process communication, or IPC, is how the two components communicate with one another. The benefit of microkernel is that it allows the other server to take over in the event of a server failure.

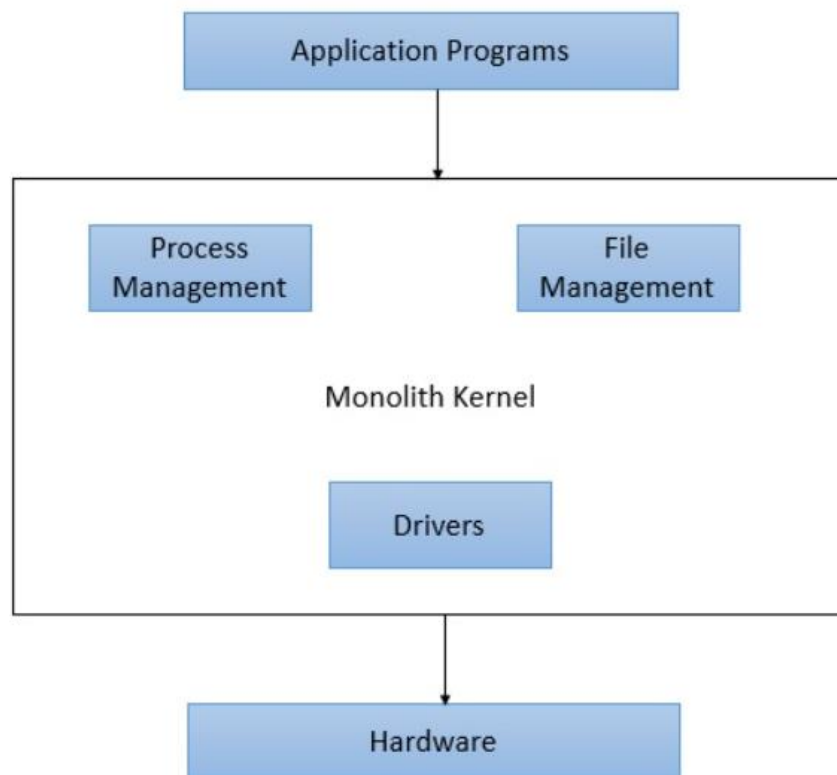


Figure 2 : The monolith kernel

Benefits:

- **Simple Development:** Since the kernel is the only layer that can be developed with all of the main features, it is simpler to design and create.
- **Performance:** The kernel runs better since it has direct access to the hardware and is in charge of memory management and other functions.

Drawbacks:

- **Crash Prone:** Because the kernel controls every operation, the operating system as a whole crashes if one of its operations fails.
- **Enhancement is challenging:** Adding a new service to a monolith operating system without affecting its other services is extremely challenging.

2.3 Micro Kernel:

The micro-kernel has numerous kernels, each of which is specialised in a different service, whereas the monolith structure had a single kernel. The system becomes more stable when each microkernel is built independently of the others. In the event that one kernel dies, the operating system will continue to function using the features of the other kernel.

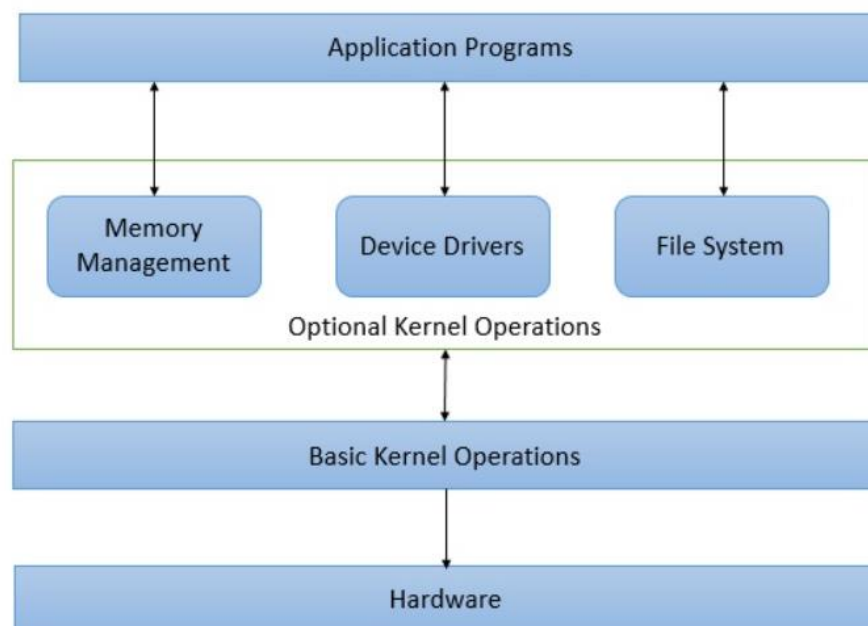


Figure 3 : The Micro Kernel

Benefits:

- **Stable and Reliable:** Because numerous kernels are executing at the same time, there is virtually little likelihood of the operating system failing. Using stable kernels, the operating system can continue to deliver other functionalities even if one is unavailable.
- **Maintainability:** The code size is manageable due to the minimal kernel size. A microkernel code base can be improved without affecting other microkernel code bases.

Drawbacks :

- **Complex to develop:** It is challenging to develop an architecture based on a microkernel.
- **Performance Degradation:** When compared to monolith design, multi-kernel and multi-modular communication may cause performance issues.

2.4 Exo-Kernal

MIT was the design and development site for the Exo-Kernal Structured operating system. This architecture sought to minimize kernel size while enabling direct hardware resource management by application programs. It was intended to allow application programmers to write high-performance code while the exo-kernel took care of other tasks by eliminating the operating system's abstraction for hardware resources.

Benefits:

- **High Performance:** Better code can make the most use of memory and perform better because application programs can allocate memory.
- **Application Control:** Because the operating system does not protect resource management, an application software has greater authority over system resources and is able to perform bespoke operations on them.

Drawbacks:

- **Unreliable and unsafe:** Badly written code has the potential to destroy a system because security resides at the application program level.
- **Complex Design:** Exo-Kernel design is a challenging process.

2.5 Layered structure:

An operating system can be divided up and still have a great deal of control over the system. The OS is divided into a number of layers (levels) in this manner. The user interface (layer N) is at the top,

while the hardware (layer 0) is at the bottom. Because of the way these levels are constructed, only the lower level layers' functions are utilized by each layer. Debugging is made easier because if an error arises during the debugging of lower level layers, it must only affect that layer because the lower level levels have already been debugged.

This structure's primary drawback is the requirement to modify and transmit data at each tier, which increases system overhead. Furthermore, since a layer can only employ lower level layers, careful layer planning is required. Examples of this framework include UNIX.

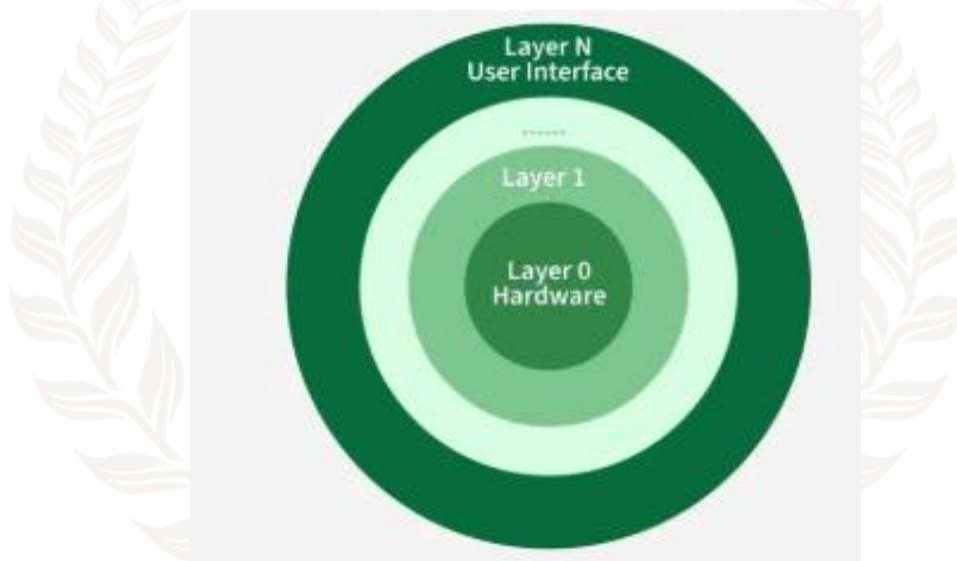


Figure 4 : Layered Structure

The diagram illustrates how the entire operating system is divided into multiple layers, numbered 0 through n. Every layer needs to fulfill a distinct purpose of its own. The following are some guidelines on how the layers should be implemented.

1. The User Interface layer needs to be the uppermost layer.
2. The hardware layer needs to be the innermost layer.
3. While a given layer can access all of the levels below it, it is unable to access the layers above it.

In other words, layer n-1 has access to all layers between n-2 and 0, but not to the nth layer.

Therefore, in order for the user layer to communicate with the hardware layer, the response must pass via each layer from n-1 to 1. The design and implementation of each layer must ensure that it only requires the services offered by the layers beneath it.

Benefits:

- **Modularity:** This architecture encourages modularity since each layer solely completes the tasks for which it is intended.
- **Simple debugging:** It is very simple to troubleshoot because the layers are distinct. The developer can only search that specific layer to debug if an error arises in the CPU scheduling layer, as opposed to a monolithic system where all the services are present at once.
- **Easy update:** Any changes made to one layer won't have an impact on the others.
- **No direct hardware access:** The innermost layer in the design is the hardware layer. Unlike the Simple system, which allowed the user to have direct access to the hardware, a user can utilize the hardware's services but cannot alter or access it directly.
- **Abstraction:** Each layer concentrates on its own tasks. The other layers' implementations and functionalities are therefore abstract to it.

Drawbacks:

- A reduced application performance compared to a basic structure; careful layer design is necessary because upper layers utilize only the lower layers' features.
- **Implementation is intricate and meticulous:** Layers must be arranged properly since they can access the services of the layers behind them. The backing storage layer, for instance, makes use of the memory management layer's services. Hence, it needs to be kept beneath the layer of memory management. This means that sophisticated implementation follows high modularity.
- **Slower in execution:** When a layer wishes to communicate with another layer, it must submit a request that requires passage through every layer between the two layers. As a result, it becomes slower than the Monolithic system, which is faster. Therefore, a design that has more layers could be extremely inefficient.

2.6 Modular Structure:

The modular structure of an operating system is a modern approach that aims to combine the performance efficiency of monolithic systems with the flexibility and maintainability of micro-kernel designs. It reflects an architectural philosophy where the system is built as a collection of independent, interchangeable components called *modules*, each responsible for a specific function. While the modular structure still relies on a central kernel, this kernel is streamlined and designed to

support dynamic loading and unloading of modules during system operation. This enables a more adaptable and organized operating system.

At the heart of a modular system lies a **core kernel** that provides only the most essential services—such as low-level memory management, process scheduling, and inter-process communication. All other functionalities, such as device drivers, file systems, and networking services, are implemented as modules. These modules are not part of the core but can be loaded into memory and connected to the kernel when required. They communicate with the kernel through standardized interfaces, allowing them to function as part of the system while remaining logically separate.

A defining characteristic of this structure is **dynamic extensibility**. During boot time, only the core components necessary for initialization are loaded. As the system runs and new services are needed—such as when new hardware is connected or when advanced system functions are invoked—appropriate modules can be inserted into the system without requiring a restart. This greatly improves system flexibility and responsiveness.

One prominent example of an operating system with a modular architecture is **Sun Solaris OS**, which was designed to support enterprise computing environments requiring high stability, scalability, and extensibility. Other modern operating systems such as **Linux** also incorporate modular design principles, allowing for the use of **loadable kernel modules (LKMs)** to extend system functionality on demand.

Benefits:

1. High Customizability

Modularity allows developers and system administrators to customize the operating system to suit specific needs. If a particular functionality is not required—such as support for a legacy device or a specific file system—it can simply be excluded. New features, such as support for emerging hardware or security tools, can be added by integrating new modules without altering the core kernel or affecting existing components. This results in a more streamlined and purpose-built system, especially useful in environments like embedded systems or specialised servers.

2. Simplified Maintenance and Development

Since each module is designed to perform a specific task independently, developers can focus on one module at a time. This isolation simplifies development, testing, and debugging. If a bug or issue arises, it is easier to pinpoint and resolve it within a specific module, reducing the risk of unintentional changes affecting the entire system. This also facilitates parallel development, where different teams can work on different modules simultaneously.

3. Improved Verifiability and Reliability

The separation of concerns inherent in modular design contributes to better system reliability. Since modules are logically isolated, a failure in one module does not necessarily compromise the entire operating system. Furthermore, modules can be independently verified to meet specific security or performance criteria before they are integrated. This enhances the overall robustness of the system, especially in security-critical or mission-critical environments.

4. Dynamic Feature Management

With modular systems, features can be added or removed at runtime. For example, a device driver can be loaded only when the corresponding hardware is connected, and unloaded when it is no longer needed. This contributes to better resource utilization and system performance. Additionally, this ability supports systems that require high availability, as new capabilities can be added without needing to reboot.

Drawbacks:

1. Performance Overhead

Although the modular structure offers many advantages, it may introduce slight performance penalties compared to more tightly integrated architectures like monolithic systems. This is mainly due to the additional layers of communication between modules and the kernel, as well as the runtime cost of managing modules. Function calls between separate modules and the kernel may not be as direct or efficient as within a single, unified codebase.

2. Complex Integration

Ensuring that all modules interact correctly with the core kernel and with each other requires careful planning and adherence to strict interface guidelines. A mismatch in interfaces,

improperly designed modules, or failure to follow the system's module-loading protocols can lead to instability or crashes. This integration complexity can increase the overall development effort, especially when maintaining compatibility across different kernel versions or hardware platforms.

3. Security Risks from External Modules

The ability to dynamically load and unload modules also introduces potential security risks. If an unauthorized or malicious module is loaded into the system, it could gain kernel-level access and compromise the entire system. Therefore, modular systems must include mechanisms to authenticate and verify the integrity of modules before loading them, and administrators must enforce strict policies regarding what modules are permitted.

2.7 Virtual Machine:

The concept of a **Virtual Machine (VM)** represents a powerful and flexible approach in operating system design, where the physical hardware of a computer system—such as the CPU, memory, and storage—is abstracted and emulated by software. This abstraction allows multiple operating systems and applications to run independently on the same physical machine, each within its own isolated virtual environment. A virtual machine acts as a self-contained execution environment that simulates a complete computer system, offering users the same interface and capabilities as a physical computer.

The core component that enables virtual machines is known as a **Virtual Machine Monitor (VMM)** or **hypervisor**. This software layer sits between the physical hardware and the virtual machines. The hypervisor manages and allocates hardware resources—such as CPU time, RAM, disk space, and I/O devices—to each VM. There are two main types of hypervisors: *Type 1 (bare-metal)* hypervisors, which run directly on the host's hardware (e.g., VMware ESXi, Microsoft Hyper-V); and *Type 2* hypervisors, which run on top of a conventional operating system (e.g., VirtualBox, VMware Workstation).

Virtual machines are created through **provisioning**, a process that sets up the virtual hardware, including assigning virtual CPUs, RAM, storage, and network interfaces. Each VM also requires its own operating system, which can be different from the host system, enabling multiple environments to

coexist. For instance, a single physical computer can run Windows, Linux, and macOS virtual machines simultaneously, each isolated from the others.

One of the defining features of virtual machines is their **execution context**—each VM operates independently, with its own user space and system space. This means that the failure or behavior of one virtual machine has no impact on the others. Such isolation is essential for testing, software development, and secure application deployment.

Benefits:

1. High Customizability

Virtual machines offer a high level of configurability. Users can define the hardware specifications of a VM according to their needs, such as adjusting CPU cores, allocating memory, or assigning disk space. Because these resources are virtual, they can be modified or reconfigured without affecting the physical system. This makes VMs ideal for testing different operating systems, running legacy applications, or simulating various hardware environments. Snapshots and cloning features also make it easy to replicate or revert configurations, further enhancing flexibility.

2. Strong Security and Isolation

Security is a key strength of virtual machine environments. Since VMs operate independently in isolated spaces, malicious software or security vulnerabilities in one VM cannot directly affect others or the host system. This isolation is especially valuable in cloud computing, enterprise networks, and software testing, where untrusted applications or users must be contained. Furthermore, virtual machines can be easily monitored and restricted through the hypervisor, which enforces strict access controls between VMs and the hardware.

3. Resource Optimization and Efficiency

Virtualization enables better utilization of hardware by allowing multiple workloads to share the same physical infrastructure. Instead of running one operating system on one physical machine, multiple VMs can be consolidated on a single host. This improves overall system

efficiency, reduces hardware costs, and simplifies management. For organizations, this translates into lower operational costs, energy savings, and easier scalability.

Drawbacks:

1. Reduced Performance Compared to Native Systems

Although virtual machines provide flexibility and isolation, they typically introduce **performance overhead**. Because the hypervisor must mediate between each VM and the physical hardware, access to resources is not as direct as in native or modular operating systems. Tasks that require intensive computation, high-speed I/O, or low-latency response—such as gaming, real-time data processing, or graphics rendering—may perform less efficiently in virtual environments. The abstraction layer adds latency and consumes system resources, which can limit the performance of applications running inside VMs.

2. High Storage and Memory Requirements

Running multiple virtual machines on a single host requires substantial amounts of **disk space and RAM**. Each VM needs its own operating system installation and system files, which can be several gigabytes in size. Memory must be allocated to each VM, which can lead to resource contention if the host system is not adequately provisioned. In environments with many VMs, storage management and memory allocation become critical concerns.

3. Complex Management

Managing a virtualized environment can become complex, especially as the number of virtual machines increases. Tasks such as monitoring performance, handling backups, applying updates, and ensuring network connectivity require additional tools and expertise. Although hypervisors provide interfaces and automation capabilities, administrators must carefully manage system resources and plan capacity to avoid performance bottlenecks or system failures.

Applications:

Virtual machines are widely used across various fields:

- **Software Development and Testing:** Developers can test software in different operating systems without needing separate physical machines.
- **Server Consolidation:** Organizations can reduce hardware usage by running multiple server instances on fewer physical machines.
- **Disaster Recovery:** VMs can be backed up and restored easily, offering a reliable way to recover from failures.
- **Cloud Computing:** Virtualization is the backbone of cloud services, allowing providers to offer scalable infrastructure-as-a-service (IaaS) models.

Table 1: Comparison of Different Structures

Structure Type	Key Feature	Examples	Performance	Modularity	Security
Simple	No clear separation of modules	MS-DOS	High	Low	Low
Monolithic	All services in a single large kernel	Unix	High	Low	Medium
Microkernel	Minimal kernel, services in user space	Minix, QNX, macOS	Medium	High	High
Exokernel	Exposes hardware directly to applications	MIT Exokernel	Very High	Medium	Depends
Layered	Hierarchical OS layers	THE OS	Medium	Medium	Medium
Modular	Dynamically loadable kernel modules	Linux, Solaris	High	High	Medium
Virtual Machine	Multiple virtual OS instances	VMware, KVM	Medium	High	High

SELF-ASSESSMENT QUESTIONS - 1

True or False	
1	A monolithic structure separates OS services into independent user-level modules. (True/False)
Multiple choice Questions	
2	Which of the following is a feature of a microkernel structure? A. All services run in kernel space B. Only essential services are in kernel space C. No separation of services

	D. All services run in user space
Fill in the Blanks:	
3	A _____ OS structure allows application programs to directly manage hardware resources.
4	The _____ structure divides an operating system into distinct levels, where each layer uses services from the layer below.
Multiple choice Questions	
5	Which OS structure allows dynamic loading of modules? A. Monolithic B. Layered C. Modular D. Exokernel
True or False	
6	In modular operating systems, all services are hardcoded into the kernel. (True/False)
7	Virtual machines offer low security because they have direct access to hardware. (True/False)



3. SYSTEM COMPONENTS

An operating system is made up of several parts that work together to accomplish specific tasks. Even though the majority of operating systems have different structures, their constituent parts are logically similar. Every component needs to be a clearly defined area of a system that accurately explains its inputs, outputs, and functions.

An operating system consists of the following eight components:

1. Process Management
2. I/O Device Management
3. File Management
4. Network Management
5. Main Memory Management
6. Secondary Storage Management
7. Security Management
8. Command Interpreter System

Process Management:

A program or a portion of a program that is loaded into main memory is called a process. To complete a task, a process requires specific resources, such as CPU time, memory, files, and I/O devices. The operating system's numerous concurrently executing processes are controlled by the process management component.

The following tasks are under the operating system's purview in relation to process management:

- **Creation and Termination of Processes:**

When a user launches an application or a system task calls for a new process, the OS is in charge of starting it. It loads the application into memory, assigns a unique identity (PID), and distributes resources like memory and CPU cycles. The operating system makes sure that the process is appropriately deleted from memory and its resources are released when it has finished its task or needs to be terminated (either normally or as a result of failures).

- **Context switching and scheduling**

Multiple processes can live in memory and operate simultaneously thanks to multiprogramming, which is supported by modern operating systems. The scheduler uses a selected algorithm (such as round-robin or priority scheduling) to determine which process should be run next. Context switching is the technique by which the operating system loads the state of the next scheduled process while saving the state of the current process (context).

- **Communication Across Processes (IPC)**

Processes frequently have to talk to one another. To help with this, the OS offers IPC techniques including shared memory, pipes, and message forwarding. These are essential for organizing tasks, particularly in multiprocessing environments or client-server models.

- **Control of Synchronization and Concurrency**

There is a chance of data corruption or inconsistency when several processes access shared resources or data at the same time. The OS offers synchronization tools like monitors, mutexes, and semaphores to deal with this. In order to preserve data integrity, these technologies make sure that only one process at a time can access a crucial portion of code or resource.

- **Handling Deadlock**

When several processes are waiting for a resource that is being held by another, it is called a deadlock and stops them all from moving further. Deadlocks must be identified and resolved by the operating system using techniques such as deadlock recovery (e.g., killing a process or preempting resources), deadlock avoidance (e.g., Banker's algorithm), or deadlock prevention (e.g., cautious resource allocation).

- **Allocation and Deallocation of Resources**

The operating system makes sure that no conflicts arise and that every process has access to the resources it requires. When resources are no longer required, it must also recover them. Maintaining system performance and stability requires this.

- **Support for Multithreading**

Many contemporary processes have several threads of execution, or are multithreaded. By effectively managing these threads, the OS provides advantages including enhanced resource sharing and quicker processing.

I/O Device Management:

An operating system's function includes concealing from the user the quirks of particular hardware devices. I/O Device Management gives users a pleasant and effective programming environment while providing an abstract level of hardware devices and preserving application information to guarantee correct device utilization and avoid problems.

The responsibilities of the I/O Device Management component are as follows:

- **Device Drivers:** To communicate with hardware, the operating system utilizes specialised software known as device drivers. Every device has a driver that controls information unique to that device.
- **Buffering and Caching:** The operating system makes use of buffers, or temporary storage, and caches to manage speed variations between devices and the CPU.
- **Interrupt Handling:** When a device requires attention, it notifies the CPU via interrupts. The OS controls these interruptions and makes sure that ongoing tasks are not significantly disrupted.
- **I/O Scheduling:** To increase efficiency and reduce waiting times, the OS prioritizes and arranges I/O requests in a similar manner to CPU scheduling. For peripheral devices to be accessed and controlled effectively, securely, and without interference, I/O device management is crucial.

File Management:

One of an operating system's most noticeable features is file management. Information can be physically stored on computers in a variety of formats; the most popular ones are magnetic tape, disk, and drum. According to the definition provided by the file's inventor, a file is a collection of related data. Programs, data, and source and object forms are mostly represented as files. Alphanumeric, numeric, and alphabetic data files are all possible.

The operating system manages mass storage devices, including drives and types, to put the abstract idea of a file into practice. To make them easier to utilize, files are typically arranged into folders. Files, other folders, and other things might be found in these directories.

In relation to file management, the operating system is in charge of the following tasks:

- **File Storage and Organization:** The operating system logically arranges files and keeps data in hierarchical directories and subdirectories.
- **File Access techniques:** Depending on the requirements of the application, it provides a number of access techniques, including sequential, direct, and indexed.
- **File Naming and Metadata:** A unique name and related metadata, such as size, type, rights, and creation/modification date, are assigned to every file.
- **Security and Permissions:** To limit access to files and safeguard private information, the operating system imposes read, write, and execute permissions.
- **File System Management:** It facilitates standardized user interaction with storage media by managing various file systems, such as FAT32, NTFS, and ext4.

Network Management:

Since network administration encompasses a variety of components, the term is frequently broad. The practice of overseeing and managing a computer network is known as network management. A group of different kinds of computers connected to one another is called a computer network.

Fault analysis, quality of service maintenance, network provisioning, and performance management are all included in network management.

Using stable (nonvolatile) storage media to backup files

The characteristics of network management are as follows:

- **Protocol Support:** To enable system-to-system communication, the OS supports a number of network protocols, including TCP/IP.
- **Data Transmission:** It manages data packet transmission, routing, and packing.
- **Network Security:** To safeguard data while it is being transmitted, it implements firewalls, access controls, and encryption.
- **Resource Sharing:** The operating system permits network sharing of files, printers, and other resources.
- **Connection Management:** It keeps track of open connections and efficiently controls network traffic and bandwidth.

Main Memory Management:

A vast collection of words or bytes, each with its own address, makes up memory. The CPU and I/O devices share this store of easily accessible data. Being a volatile storage device, main memory loses its contents as soon as the system fails or the power goes out.

In relation to memory management, the operating system is in charge of the following tasks:

- **Memory allocation** is the process of allocating memory to processes as required and recovering it when it is no longer required.
- **Address Translation:** This technique isolates the memory space of each process by mapping logical addresses to physical addresses using virtual memory.
- **Swapping and Paging:** This allows more processes to operate simultaneously by implementing strategies like paging or segmentation to load portions of processes into memory only when necessary.
- **Memory Protection:** Preserves stability and security by preventing a process from accessing memory that hasn't been allotted to it.

Memory management is an essential component since it has a direct impact on system speed and multitasking capacity.

Secondary Storage Management:

A computer system's primary function is to carry out programs. During execution, these programs and the data they access need to be in main memory. The computer system must have secondary storage to support main memory backups since the main memory is insufficient to permanently hold all of the data and programs.

Disks are the primary online storage medium used by the majority of contemporary computer systems, both for data and programs. The majority of programs, including editors, formatters, compilers, assemblers, sorting routines, and others, are stored on the disk until they are loaded into memory. After that, they use the disk as both the source and the destination of their processing.

In relation to disk management, the operating system is in charge of the following tasks:

- **Space Allocation:** Ensures effective use of disk space by tracking and allocating disk blocks to files.
- **Disk scheduling:** This technique minimizes latency and seek time by optimizing read/write operations utilizing algorithms (such as SCAN or SSTF).
- **Free Space Management:** Maintains track of available disk space for new data storage.

- **Data Organization and Retrieval:** Uses file systems to organize data so that it may be easily accessed, searched, and navigated.

This part makes sure that data is neither lost or corrupted during write/read cycles and that storage is used efficiently.

Security Management:

All tasks and activities that take place within the computer system are essentially the responsibility of the operating system. An operating system's different processes need to be shielded from one another's actions. For this reason, there are a number of ways to make sure that only programs that have received the appropriate permission from the operating system can access files, memory segments, CPUs, and other resources.

A process can only run within its own address space, for instance, thanks to memory addressing technology. No process can take over the CPU without giving it up thanks to the timer. No process is permitted to do its own input/output in order to safeguard the integrity of the different peripheral devices.

In relation to security management, the operating system is in charge of the following tasks:

- **Authentication:** Uses digital certificates, biometrics, or passwords to confirm a user's identity.
- **Authorization:** Based on user responsibilities and permissions, it either grants or refuses access.
- **Encryption:** Protects private information while it's being transmitted or stored.
- **Auditing and Logging:** Monitors system activities for forensic investigation and security monitoring.
- **Malware Protection:** Detects and blocks threats by integrating with or offering antivirus and antimalware software.

Command Interpreter System:

The command interpreter is one of the most crucial parts of an operating system. The main conduit between the user and the rest of the system is the command interpreter.

The Command Interpreter System calls one or more underlying system programs or system calls in order to carry out a user command. Control statements provide the operating system a lot of commands. Control statements are read and interpreted by a program that runs automatically.

Examples of this software are the Windows DOS command window, Bash of Unix/Linux, and C-Shell of Unix/Linux. It is referred to as the shell.

An operating system is a sophisticated piece of software. The Operating System performs a wide range of other tasks in addition to the duties and components listed above. Below is a list of some of them:

- **Security:** It stops unwanted access to programs and data by using passwords and similar methods.
- **Control over system performance:** Documenting the time lag between a service request and the system's response.
- **Accounting:** Monitoring the time and resources utilized by different jobs and users is known as job accounting.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
8	What does the memory management unit handle? A. File naming B. I/O scheduling C. Memory allocation and protection D. Device driver loading
9	Which of the following is not a system component of an operating system? A. Process Management B. Web Browsing C. File Management D. Network Management
Fill in the Blanks:	
10	_____ management handles communication between software and hardware devices.
11	The _____ provides a user interface for entering commands in an operating system.
True or False	
12	The operating system's memory management prevents one process from accessing another's memory space. (True/False)

4. PROTECTION

When a computer system has several users and multiple processes running at the same time, the different processes need to be shielded from one another.

The term "protection" describes a method or technique for limiting program, process, or user access to the resources specified by a computer system. The procedures that an operating system has put in place to guarantee the security and integrity of the system are referred to as system protection. Several strategies are used in system protection to stop unwanted access, abuse, or alteration of the operating system and its components.

A framework for implementing rules that specify how and by whom system resources (such as memory, files, CPU time, I/O devices, and network access) can be utilized is provided by protection. Protection focuses on internal regulation—making sure that one user or activity does not negatively impact another—in contrast to security, which is concerned with safeguarding against external dangers.

For instance, protection makes sure that no software can read, alter, or interfere with the other's files or memory space when two users are using the same system.

Operating System Protection Mechanisms

An operating system implements protection using a mix of policies and methods. The system's technological tools are called mechanisms, and the rules governing access are called policies. Separating these two is necessary to provide scalability and flexibility.

An operating system can offer system protection in a number of ways:

- **User authentication:** Before users may access the system, the operating system requires them to authenticate themselves. Passwords and usernames are frequently used for this. Users must authenticate themselves before they can access any system resources. Usually, authentication entails Passwords and usernames, Two-factor authentication (2FA) and

biometric verification (such as fingerprint or face recognition). This guarantees that the system is only accessible by authorized users.

- **Access control:** To identify which users or processes are authorized to access particular resources or carry out particular operations, the operating system makes use of access control lists, or ACLs. For instance, a user may have complete access to their own papers but read-only access to a system configuration file.
- **Encryption:** To safeguard private information and stop unwanted access, the operating system may employ encryption. Data that has been encrypted becomes unintelligible until a key is used to decrypt it. Operating systems can encrypt network transfers (like SSL/TLS), files and folders, even whole disks (like BitLocker and FileVault). Even if physical access to storage is obtained, this stops data theft.
- **Firewall:** A firewall is a piece of software that uses pre-established security rules to monitor and regulate all network traffic, both inbound and outbound. A firewall serves as a safeguard between the system and outside networks. Based on pre-established security standards, it permits or prohibits data packets and keeps an eye on all incoming and outgoing network traffic. Unauthorized remote access, hostile user or bot intrusions, and the spread of malware through network connections are all prevented by firewalls.
- **Antivirus software:** Antivirus software guards against malware, viruses, and other harmful software. These programs search for, identify, and eliminate trojans, worms, viruses, spyware, and other harmful software. While some operating systems rely on third-party solutions, others include built-in protection (such as Windows Defender).
- **System patches and updates:** To stop known vulnerabilities from being exploited, the operating system needs to be kept up to date with the most recent security patches and updates. Frequent updates address bugs and security flaws that an attacker could exploit. Operating systems usually come with scheduled patch management, automatic update methods, and rollback capabilities in case updates cause new issues. Maintaining strong protection requires being up to date.

The operating system makes sure that all system resource access is managed. External I/O devices are shielded from unauthorized access attempts by the operating system. Passwords are used by the OS to provide authentication functionality for every user.

Complex design: Since each virtual component of the machine abstracts the underlying hardware, it must be meticulously planned.

Protection's function is to offer a framework for putting policies into practice that specify how computer system resources are to be used. Certain policies are established during the system's design, while others are created by the system's administration or by system users to safeguard their personal data and applications. Because each program has unique resource usage guidelines that can change over time, system security is not just the operating system's designer's problem. To prevent their system from being abused, application programmers should also create a safeguard.

A mechanism is not the same as a policy. Policy determines what will be done, while mechanisms dictate how it will be done. Policies vary from place to place and throughout time. For the system to be flexible, the mechanism and the policy must be kept apart.

Benefits:

- **System Integrity and Security:** By guaranteeing that only authorized users and applications have the capacity to access or alter system resources, operating system protection measures contribute to the stability and dependability of the system. Permissions, user authentication, and access control lists, for instance, all aid in preventing unwanted modifications to system files or configurations.
- **Safeguards Private Information:** Security features including user authentication, encryption, and safe storage guarantee that private information about individuals and organizations is kept private.

For systems that handle financial data, medical records, or secret information—where hacks could have major repercussions—this is extremely important.

- **Creates a safe environment for apps and users:** Applications can operate in a reliable environment free from harmful software compromise thanks to a secured operating system. Apps cannot interfere with the core system or one another when isolation techniques are used (such as user-mode vs. kernel-mode separation).

- **Permits the safe exchange of data and resources:** Without running the risk of data corruption or unwanted access, several users and apps can coexist and share files, printers, memory, or processing time.
- **Avoids Malware and Additional Security Risks:** Real-time threat detection, firewalls, sandboxing, and antivirus software to stop or limit malware are all part of modern OS protection. Patches and system updates also lessen vulnerabilities that hackers could take advantage of.
- **Encourages Adherence to Security Standards:** Strict data protection laws (such as GDPR, HIPAA, and ISO/IEC 27001) apply to many businesses.

Drawbacks:

- **Complex Management:** It can be difficult and time-consuming to maintain a safe operating system, particularly in settings with many of users. System administrators must set up appropriate permissions, keep abreast of dangers, and routinely audit activity.
- **Performance Overhead:** Memory and processing power are used by security features including intrusion detection systems, encryption, and real-time monitoring. System performance may suffer as a result, especially on machines that are older or have less resources.
- **Problems with Compatibility:** Certain security measures might not work with specialised hardware or outdated software. For example, if an application needs higher rights that are prohibited by default security settings, it may not be able to execute.
- **Inauthentic Feeling of Safety:** Users might disregard safe computing practices (such as avoiding dubious emails or websites) because they believe the system's built-in security is sufficient.
- **Higher Expenses:** Investing in extra software, hiring expert personnel (such security analysts), and providing frequent training are frequently necessary for high-level defense. Ongoing operational costs are further influenced by penetration testing, compliance audits, and regular system updates.

SELF-ASSESSMENT QUESTIONS - 3

Fill in the Blanks:	
13	_____ is the process of verifying the identity of a user before allowing access to the system.
True or False	
14	Encryption is used to protect data during storage and transmission.
Multiple Choice Questions	
15	What is the main purpose of access control in protection? A. To delete files B. To block all users C. To define who can access what D. To schedule I/O operations

5. SUMMARY

- In this topic, Students have explored the various structural models of operating systems, including simple, monolithic, microkernel, exokernel, layered, modular, and virtual machine designs. They understood how each architecture impacts system performance, maintainability, and modularity, as well as how these models handle essential services and components.
- Understood how each structure influences system performance, maintainability, modularity, and how it handles different services and components.
- Learnt about the key functional areas of an operating system such as process management, memory management, file system, I/O device management, network management, security services, and the command interpreter.
- They have gained insight into how these components work together to manage hardware resources, run applications, and maintain system stability.
- Learnt the significance of protecting system resources from unauthorized access and ensuring safe execution of multiple processes and users.
- They have explored common protection mechanisms including user authentication, access control, encryption, firewalls, antivirus software, and regular updates that help secure the operating system.

6. GLOSSARY

Operating System (OS)	- Software that manages hardware resources and provides services to computer programs.
Kernel	- The core part of an operating system that controls all major functions of hardware and manages system resources.
Monolithic Structure	- An OS structure where all services run in a single large kernel space, offering high performance but low modularity.
Microkernel	- An OS design where only essential functions run in kernel space, and other services run in user space for better reliability and security.
Exokernel	- An OS structure that exposes hardware resources directly to applications, allowing high customization but requiring complex design.
Layered Structure	- An operating system design where functions are separated into hierarchical layers, improving modularity and ease of debugging.
Modular Structure	- A design where the OS uses a basic kernel and loads additional services as modules, enabling flexibility and customizability.
Virtual Machine (VM)	- A software-based simulation of a computer system that runs multiple operating systems on a single physical machine.
Process Management	- The OS function that handles creation, scheduling, execution, and termination of processes.
Memory Management	- Controls and coordinates computer memory, allocating space to processes and ensuring safe access.
File System	- The component that manages data storage, file naming, access permissions, and file organization on storage devices.
I/O Management	- Handles communication between the system and input/output devices using drivers and scheduling.
Network Management	- Oversees data communication between systems, including protocol handling, data transfer, and security.

Command Interpreter (Shell)	- The user interface of the OS that reads and executes commands (e.g., Bash in Linux, Command Prompt in Windows).
Protection	- A set of mechanisms to control access to system resources and prevent unauthorized or harmful actions by users or programs.
Authentication	- The process of verifying the identity of a user, typically through passwords, PINs, or biometrics.
Access Control	- Rules and mechanisms that determine what resources a user or process can access and what operations they can perform.
Encryption	- The process of converting data into a secure format to prevent unauthorized access.
Firewall	- A security system that monitors and controls incoming and outgoing network traffic based on predefined rules
Antivirus Software	- Programs designed to detect, prevent, and remove malicious software from a system.

7. TERMINAL QUESTIONS

1. Define a microkernel and mention one of its key benefits.
2. What is the purpose of process management in an operating system?
3. List any two system protection mechanisms used in operating systems.
4. What is the difference between a monolithic and modular OS structure?
5. Mention any two roles of memory management in an operating system.
6. Compare layered and modular structures in terms of design, performance, and maintainability.
7. Explain how an operating system manages I/O devices.
8. Describe the structure and purpose of virtual machines in an OS.
9. Discuss how an operating system ensures protection in a multi-user environment.
10. Explain the functions of any four core components of an operating system.

8. TERMINAL ANSWERS

1. Refer to Section 2.3- Micro-Kernel Structure.
2. Refer to Section 3 – Process Management.
3. Refer to Section 4- system protection mechanisms.
4. Refer to Sections 2.2- monolithic OS structure and 2.6 -modular OS structure.
5. Refer to Section 3 – Main Memory Management.
6. Refer to Sections 2.5 –layered Structure and 2.6- modular structure.
7. Refer to Section 3 – I/O Device Management
8. Refer to Section 2.7- virtual machines
9. Refer to Section 4 – protection
10. Refer to Section 3- components of an operating system

9. ANSWERS

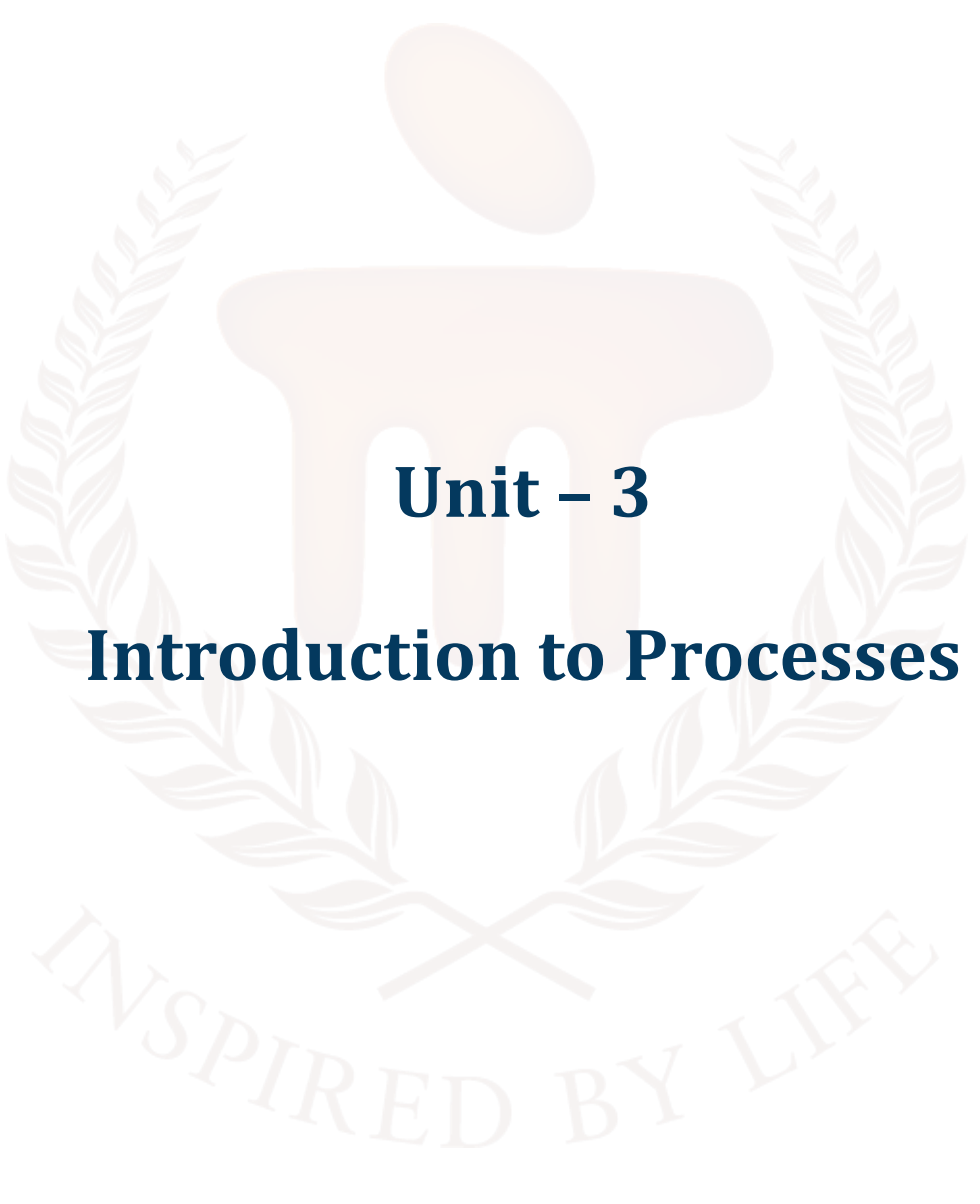
Self-Assessment Questions

1. False
2. B- Only essential services are in kernel space
3. Exokernel
4. Layered
5. C- Modular
6. False
7. False
8. C -Memory allocation and protection
9. B -Web Browsing
10. I/O device
11. command interpreter (or shell)
12. True
13. Authentication
14. True
15. C. To define who can access what

10. REFERENCES

- **Silberschatz, A., Galvin, P. B., & Gagne, G.** (2018). *Operating System Concepts* (10th Edition), Wiley.
- **Tanenbaum, A. S., & Bos, H.** (2015). *Modern Operating Systems* (4th Edition), Pearson.
- **Stallings, W.** (2018). *Operating Systems: Internals and Design Principles* (9th Edition), Pearson.





Unit – 3

Introduction to Processes

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 – 5
1.1	Objectives	-	-	
2	Process Concept	1 , 2 , 3 , 4	1	6 - 17
2.1	The Process	-	-	
2.2	Process State	-	-	
2.3	Process Control Block	-	-	
3	Operation on Process	-	2	18 - 23
3.1	The Process Creation	-	-	
3.2	The Process Termination	-	-	
4	Cooperating processes	5	3	24 – 31
4.1	Mechanisms for Process Cooperation	-	-	
5	Summary	-	-	32
6	Glossary	-	-	33- 34
7	Terminal Questions	-	-	35
8	Terminal Answers	-	-	36
9	Answers	-	-	37 – 38
10	References	-	-	39

1. INTRODUCTION

In this unit, students will learn all about processes, which are a key part of how modern operating systems work. A process means a program that is running, and it includes not just the code, but also the data it uses, the memory, and other resources it needs to work.

The unit starts by explaining the basic ideas of a process—like what it looks like inside, the different states it can be in (such as ready, running, or waiting), and the important parts that make up a process. A special focus is given to something called the Process Control Block (PCB). This is a structure that helps the operating system keep track of what each process is doing, its priority, and which resources it is using. Knowing how the PCB works is important to understand how the OS switches between different processes quickly.

Next, students will learn how processes are created and ended. This includes how parent and child processes are formed and how they are connected. System calls like `fork()`, `exec()`, and `wait()` will be introduced, as they are important for managing the life of a process.

The unit also explains the difference between a process and a thread. A process has its own memory, but threads are like smaller tasks that run inside a process and share the same memory. Learning this helps students understand how multithreading makes programs faster and more efficient by running different parts at the same time.

Then, the topic of cooperating processes is introduced. These are processes that interact with each other. Students will learn how processes can share data and work together using inter-process communication (IPC) methods like shared memory and message passing.

The unit also looks at the problems that can happen when multiple processes run at the same time—like race conditions and deadlocks. To solve these problems, the unit explains how synchronization tools like locks, semaphores, and monitors are used to make sure everything runs safely and in the right order.

By the end of this unit, learners will be able to:

-

2. PROCESS CONCEPT

In modern operating systems, the concept of a process is central to understanding how programs are managed and executed. A process is not just a static file stored on disk—it is a dynamic execution of a program. To put it simply, while a program is a passive collection of instructions saved in a file, a process is that program when it is actually running and performing tasks.

When a user or system starts a program, the operating system creates a process for it. This process contains not only the program's code but also other essential information required for proper execution. This includes the process's current state, such as whether it is ready, running, or waiting; the program counter, which keeps track of which instruction is currently being executed; register values, memory allocations, and input/output resources. All this information is managed by the operating system to keep the process functioning correctly.

One important structure that holds this information is called the Process Control Block (PCB). The PCB is used by the OS to track each process and helps in performing actions such as scheduling and context switching—especially when multiple processes are running simultaneously.

Each process operates in its own address space, meaning it has its own separate memory, which helps protect it from other processes. This separation is key to providing process isolation, which increases system stability and security.

Understanding processes is essential for grasping how multitasking works in operating systems. It allows systems to run several applications at once while managing resources efficiently. As such, the study of processes provides a foundation for exploring more advanced operating system topics like scheduling, synchronization, and communication between processes.

2.1 The Process

A process is a dynamic entity, which is a running program.

A series of information executions is called a process. The process has a finite lifespan. The same software could be running in several processes, each utilizing its own resources and data.

A program is composed of program statements and is a static entity.

The instructions are in the program. A program only exists in one location in space and never goes away. The action is not carried out by a software alone.

A process is a program that is being run. In a contemporary time-sharing system, a process is the unit of labor. Thus, a system is made up of a number of processes, including user processes running user code and operating system processes running system code. It is possible for the CPU (or CPUs) to be multiplexed among all of these processes, allowing them to run simultaneously. The operating system can increase computer productivity by alternating the CPU across processes.

A job or task is another name for a procedure. The text component, also referred to as the computer code, is only one aspect of a process. Additionally, the value of the program counter and the contents of the processor's registers indicate the current activity. The process stack, which holds temporary data (such function parameters, return addresses, and local variables), and the data section, which houses global variables, are typically also included in a process. A heap is a type of memory that is dynamically allocated during the course of a task. A process's memory structure is displayed in Figure1

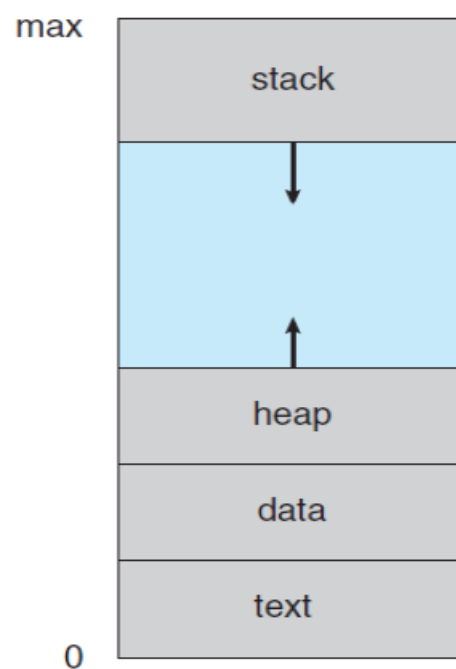


Figure 1 : process in memory

We make it clear that a program is not a process in and of itself. A file with a list of instructions saved on disk (commonly referred to as an executable file) is an example of a program, which is a passive entity. On the other hand, a process is an active entity that has a program counter that indicates which

instruction to execute next along with a list of related resources. The loading of an executable file into memory turns a program into a process.

Components of a Process

Every process consists of several key components that allow it to function correctly. These components include:

1. Object Program (Executable Code)

This is the actual **machine-level code** of the program that is to be executed. It consists of compiled instructions ready to run on the processor. When the process is created, the object program is loaded into memory and is managed by the operating system.

2. Information (Data)

Processes require **data** to perform their operations. This data can include user input, intermediate results, variables, constants, and any other form of information that the process needs to function. The data segment of a process contains both global and static variables. Additionally, each process has its own **stack and heap** for handling function calls and dynamic memory allocations, respectively.

3. Resources

Processes often require certain **system resources** to execute properly. These can include:

- CPU time
- Memory
- Input/output devices
- Files
- Network connections

Before execution can proceed, the process must acquire all the required resources. The operating system is responsible for allocating and managing these resources fairly among all running processes.

4. Execution Status (Process State)

Each process has an **execution status** that helps the operating system manage its execution. This status may include:

- **Process ID (PID):** A unique identifier assigned to each process.
- **Program Counter (PC):** Indicates the address of the next instruction to execute.
- **Registers:** Store the process's current working data.
- **Process State:** Defines the current state of the process (e.g., Ready, Running, Waiting, Terminated).
- **Scheduling Information:** Includes priority and other data needed by the CPU scheduler.

These elements are stored in a data structure known as the **Process Control Block (PCB)**. The PCB is crucial for context switching and process management.

Multiple Instances of a Program

It is important to understand that **the same program can run in multiple processes simultaneously**. For example, if two users open the same text editor, the operating system creates two separate processes. Each process runs the same code but operates with **its own data, memory, and resources**. They are independent in terms of execution, although they are derived from the same program file.

This concept is essential in **multitasking environments**, where multiple processes can run concurrently, providing a responsive user experience and efficient resource usage.

2.2 The Process State

A **process** is a dynamic entity that changes states throughout its execution. These **state transitions** are governed by the operating system as it manages system resources and schedules tasks. At any point, a process can be in one of the following **five main states**:

- **New:** The process is being created. It has not yet been admitted into the pool of executable processes by the operating system.

- **Running:** The process is currently executing instructions on the CPU. Only **one process per core** can be in this state at a time.
- **Waiting:** The process is **temporarily inactive**, waiting for an external event (such as an I/O operation or signal) to complete before it can resume execution.
- **Ready:** The process is loaded into memory and **ready to run**, but is **waiting for CPU time**. Multiple processes can be in this state simultaneously.
- **Terminated:** The process has completed execution or has been forcefully stopped. It is no longer active and awaits cleanup.

Although only one process can run on a processor at a time, **many processes may be in the ready state**, waiting for their turn. These states are typically visualized in a **process state diagram** (like Figure 2), which illustrates the transitions driven by scheduling, I/O events, and termination.

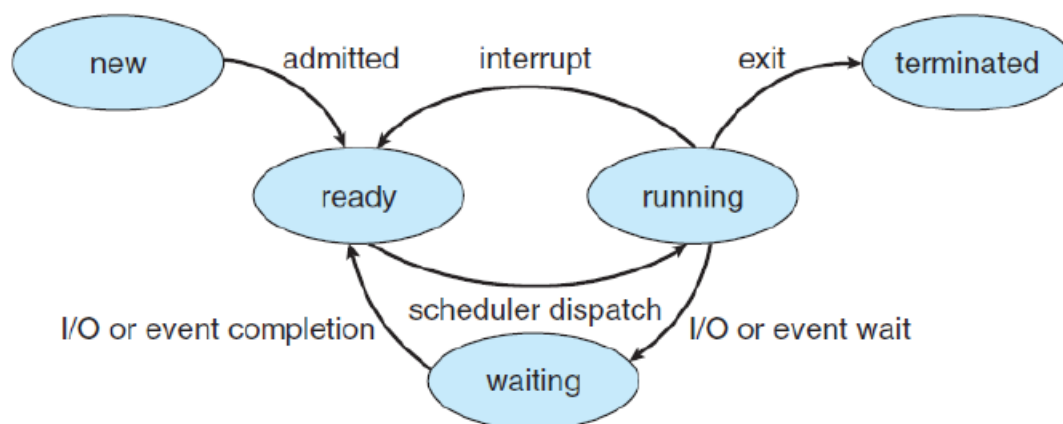


Figure 2 : Process States

Process State Transitions

In an operating system, processes transition between different states based on system events and scheduling decisions. Here's how each transition works:

1. Scheduler Dispatch (Ready → Running)

When the CPU becomes available, the process scheduler selects a process from the *ready* queue and dispatches it for execution. The selected process then transitions from the **ready** state to the **running** state.

2. I/O or Event Wait (Running → Waiting)

If a process requires an I/O operation (e.g., reading from disk) or must wait for an external event, it transitions from **running** to **waiting**. This avoids blocking the CPU while waiting.

3. Interrupt (Running → Ready)

A process in the **running** state can be interrupted (e.g., due to time-slice expiration in a multitasking OS). The OS saves its state and places it back in the **ready** queue, allowing another process to run.

4. Event Completion (Waiting → Ready)

Once the I/O or event the process was waiting for completes, it moves from the **waiting** state back to the **ready** queue, ready to be scheduled again.

5. Exit (Running → Terminated)

When a process finishes its execution or is terminated by the system/user, it moves from the **running** state to the **terminated** state, completing its lifecycle.

The operating system responds to process state changes by adding the process PCB to the list that matches the new state. There can be multiple processes in a ready-and-waiting state, but only one process can be active on any one processor at once.

Suspended Process

A **suspended process** is one that is **temporarily halted** and **not eligible for execution** until further notice. This mechanism allows the operating system to manage limited system resources more effectively.

Features of a Suspended Process:

1. **Not Immediately Executable:** A suspended process cannot resume execution until it is explicitly allowed.
2. **May or May Not Be Waiting for an Event:** Suspension is independent of event-waiting; a process may be suspended even if it isn't waiting for I/O or other signals.

3. **Suspended by Various Agents:** The suspension can be triggered by the **OS**, the **parent process**, the **process itself**, or an external **agent**.
4. **Requires Explicit Reactivation:** The process stays suspended until the suspending agent issues a command to resume it.

Additionally, suspended processes are often **swapped out** of main memory to **disk storage**, freeing up memory for other active processes.

Causes of Process Suspension:

1. **Swapping:** To make room for a higher-priority process in memory.
2. **Timing:** A process may be paused until its designated time slot arrives.
3. **User Request:** An interactive user might suspend a process to debug or examine it.
4. **Parent Process Request:** To synchronize or control the behavior of child processes.

2.3 The Process Control Block

The operating system uses process control blocks (PCBs), also known as task control blocks, to represent each process. The PCB is seen in Figure 3. It includes a variety of details related to a certain process, such as:

Pointer	Process State
Process Number	
Program Counter	
CPU registers	
Memory Allocation	
Event Information	
List of open files	

Figure 3 : Process Control Block

- A **pointer** is a reference to a different process control block. The scheduling list is kept up to date using pointers.

- **Process State:** This can be new, ready, operating, waiting, or any combination of these.
- **Program Counter:** It provides the address of the subsequent command that needs to be carried out for this procedure.
- **Event information:** This field provides details about the event that the process is waiting for when it is in the blocked state.
- **CPU register:** It shows accumulators, index registers, stack pointers, and general purpose registers, among other things. The computer architecture completely determines the number and kind of registers.
- **Information about Memory Management:** This data may contain the base and limit register values. When the process ends, this information helps with memory deallocation.
- **Accounting Information:** This data comprises tasks or processes, account numbers, time limitations, CPU and real-time usage, and more. CPU scheduling, I/O resource management, file management information, priority, and other details are also included in the process control block. The PCB only acts as a storage for any data that might differ depending on the process.
- Hardware registers and flags are set to the values supplied by the loader or linker upon the creation of a process. The pointer to the relevant stack frame is kept in the PCB, and the contents of the processor register are typically preserved on the stack once that process is suspended. When the process is scheduled to run again, the hardware state can be restored in this manner.

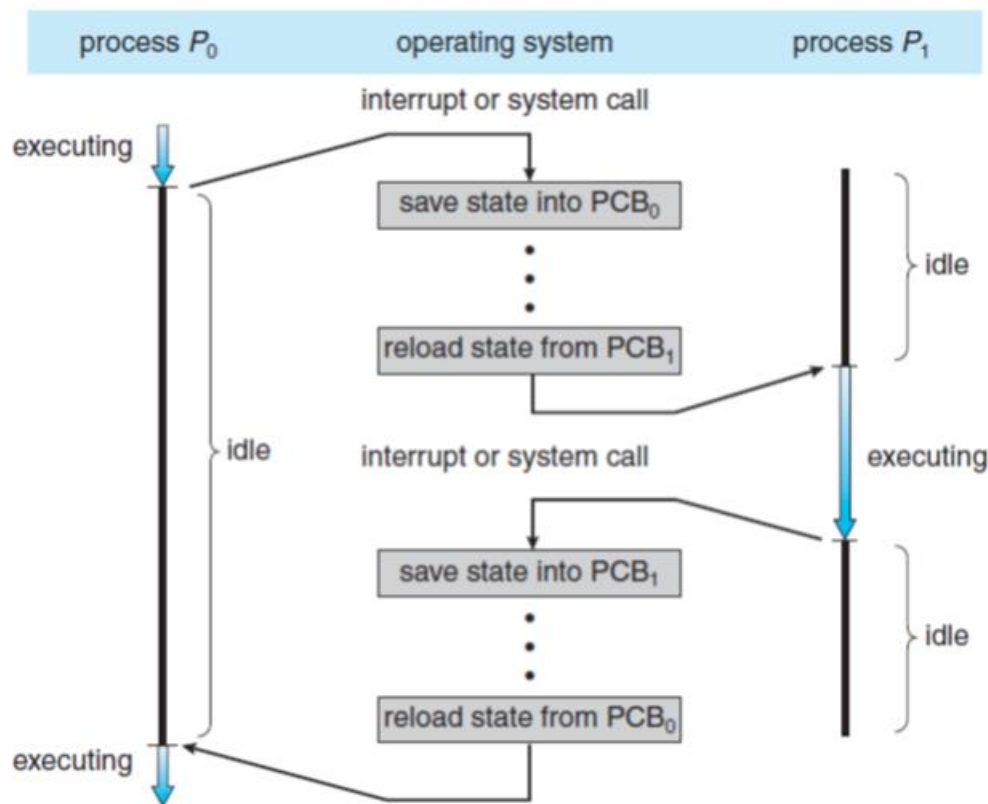


Figure 4 : CPU switch from process to process

Context Switching

In systems where **multiple processes share a single CPU**, the operating system employs a technique called **context switching** to create the illusion of concurrent execution. Since the CPU can only execute one process at a time, it must **rapidly switch** between processes to ensure all get a fair share of CPU time.

During a context switch, the OS performs two key actions:

1. **Saves the state** of the currently running process in its **Process Control Block (PCB)**. This includes the program counter, registers, and other vital information.
2. **Loads the state** of the next scheduled process from its PCB so it can **resume execution** exactly where it left off.

While context switching is essential for multitasking, it comes with a cost known as **context switch overhead**. This is the time during which the CPU is not performing useful work (i.e., not executing process instructions), as it is busy saving and restoring states. Excessive context switching can

degrade system performance, particularly in environments with many short-lived processes or threads.

Therefore, a key goal of efficient operating system design is to **minimize context switch overhead** by optimizing scheduling algorithms and reducing unnecessary switches, thereby improving overall system responsiveness and throughput.

Benefits of the Process Concept

The process abstraction is fundamental to modern operating systems and brings several key advantages that enhance both performance and reliability.

1. **Isolation:** Each process operates in its own dedicated memory space. This isolation ensures that a faulty or malicious process cannot directly interfere with the memory or execution of another process. As a result, system **stability** and **security** are significantly improved, since a crash in one process doesn't bring down the entire system.
2. **Resource Control:** The operating system can efficiently **track and allocate resources**—such as CPU time, memory, and I/O devices—on a per-process basis. This allows for better **accountability** and **fairness**, ensuring that no single process can monopolize system resources.
3. **Parallel Execution:** By supporting multiple processes, operating systems can perform **parallel processing**, especially on multi-core processors. Different processes can run simultaneously on separate cores, improving system **throughput** and enabling responsive multitasking.
4. **Modularity and Flexibility:** Applications can be broken down into **modular components**, each implemented as a separate process. For example, a web browser might run the UI, network handler, and rendering engine as separate processes. This makes the system more **maintainable**, scalable, and fault-tolerant.

Threads:

In traditional computing models, a **process** was designed to execute a **single thread of execution**. This meant that within a process, only one task could run at a time. For instance, in a word processing application, if the process was single-threaded, it could either handle user typing or run a spell

checker—but not both simultaneously. This limited the responsiveness and multitasking capability of applications.

To overcome this limitation, modern operating systems introduced the concept of **multithreading**, allowing a single process to contain and manage **multiple threads**. Each thread represents an independent path of execution within the same process and shares the process's memory and resources. In a multithreaded word processor, one thread can handle keyboard input, another can run the spell checker, and a third might perform auto-saving all at the same time.

This model is particularly powerful on **multicore processors**, where multiple threads can be executed in parallel across different CPU cores. This leads to better performance, increased efficiency, and a smoother user experience. Multithreading is widely used in modern software, from web browsers to operating systems, as it enables tasks to be handled concurrently, making applications more responsive and capable of performing complex operations simultaneously.

Difference between Process and threads

Aspect	Process	Thread
Definition	An independent program in execution.	A smaller unit of execution within a process.
Execution Unit	Has its own memory space and resources.	Shares memory and resources with other threads of the same process.
Memory	Has its own memory (heap, stack, etc.).	Shares memory space with other threads.
Communication	Inter-process communication (IPC) is required and complex.	Communication is easier and faster via shared memory.
Creation	More overhead to create and manage.	Less overhead; faster to create.
Crash Impact	One process crashing usually doesn't affect others.	One thread crashing can crash the whole process.
Context Switching	Slower due to switching between memory spaces.	Faster since memory is shared.
Examples	Web browser, text editor, database server.	Spell-checker, background sync in a browser.

SELF-ASSESSMENT QUESTIONS - 1

Multiple choice Questions	
1	Which of the following is NOT a valid process state? A. Running B. Waiting C. Terminated D. Executable
2	Which of the following allows multiple threads to share the same memory? A. Different processes B. Zombie processes C. Threads of the same process D. Parent and child processes
3	The Process Control Block (PCB) does NOT include: A. Program Counter B. Stack Memory C. CPU Registers D. Process State
Fill in the Blanks:	
4	A _____ is a program in execution.
5	_____ switching refers to the act of saving the state of one process and loading another.
6	A process enters the _____ state when it is waiting for an I/O operation to complete.
7	In a multithreaded process, threads share _____ and resources.
True or False	
8	A program and a process are the same thing. (True/False)
9	Each process has its own PCB maintained by the operating system. (True/False)
10	Context switching increases system performance significantly. (True/False)

3. OPERATION ON PROCESS

Modern operating systems are built to manage many simultaneous tasks. These tasks, known as **processes**, form the foundation of every program or operation a computer performs. Understanding how these processes are created, managed, and terminated is essential to understanding how operating systems function and optimize performance.

Processes are dynamic entities they are **created**, they **execute**, and eventually they **terminate**. The operating system provides structured mechanisms and environments that allow these operations to happen in an organized, controlled, and efficient manner.

The **operations on processes**, focusing specifically on:

1. **Process Creation**
2. **Process Termination**

These operations are foundational to multitasking, parallel execution, resource allocation, and system stability.

3.1 Process Creation

Process creation is the operation through which a **new process is initialized and added** to the system's process table. In operating systems that support multitasking, a currently running process can create another process. The creating process is referred to as the **parent process**, while the newly formed process is its **child**.

This hierarchical structure allows the OS to implement inheritance mechanisms, where the child inherits some resources or characteristics of the parent. Most operating systems allow a process to create multiple children, forming a **process tree** or **process hierarchy**.

When Are Processes Created?

Processes are created in many typical situations:

- When a user launches an application (e.g., opening a browser).

- When a system service or daemon starts (e.g., background services in Linux).
- When a process needs to delegate a task (e.g., a parent process creates a child to handle I/O).
- When a script or command runs in the terminal or shell.

In client-server applications, the server often spawns a new process for each incoming connection to handle client requests concurrently.

Process Identifier (PID)

Each process is assigned a unique **Process Identifier (PID)**. This identifier is used:

- Internally by the OS to track and manage the process.
- By user-level applications (e.g., ps, top, kill) to refer to specific processes.
- By parent processes to interact with their children (e.g., via wait() or kill()).

In systems like UNIX/Linux, the root of the process tree is typically a system process like init or systemd, which spawns all other system and user processes.

Process Creation in UNIX/Linux and Windows

In UNIX-based systems, a process is created using the fork() system call. When a process calls fork(), the OS creates a **duplicate** of the current process. The child process gets a copy of the parent's memory space, file descriptors, and execution state.

The newly created child process can then replace its memory space with a new program using exec(). Together, fork() and exec() allow a process to create a new, unrelated program instance.

Example:

```
c
CopyEdit
pid_t pid = fork();
if (pid == 0) {
    // Child process
    execl("/bin/ls", "ls", NULL);
} else {
```



```
// Parent process
wait(NULL);
}
```

In Windows, process creation is handled by the `CreateProcess()` function. This function creates a new process and a primary thread within it. Unlike UNIX, Windows does not separate the creation and execution steps as `fork()` and `exec()` do; everything is handled in one function call.

Concurrent vs. Sequential Execution

When a parent creates a child process, one of two behaviors may occur:

1. **Concurrent Execution** – The parent continues execution independently of the child. Both run simultaneously.
2. **Sequential Execution** – The parent waits for the child to complete before resuming its own execution. This is useful when the parent depends on the result of the child process.

The decision to execute concurrently or sequentially is a matter of design and often determined by the requirements of the parent process.

Process Tree Structure

When processes continuously spawn other processes, a **tree-like structure** is formed. For example:

```
swift
CopyEdit
init (PID 1)
├── systemd
│   ├── NetworkManager
│   └── login
├── bash
│   ├── vim
│   └── gcc
```

In this example, `bash` is a shell process that spawns a text editor and a compiler as child processes.

This hierarchical organization allows for better resource management, security isolation, and process control.

3.2 Process Termination

Just as processes are created, they must also be terminated. A process is terminated once it has:

- Finished executing its instructions,
- Encountered a fatal error,
- Been forcibly stopped by another process or the OS.

When a process terminates, the OS performs **cleanup operations**, such as releasing memory, closing open files, and removing the process from the scheduler.

Voluntary Termination:

A process can end voluntarily by calling the `exit()` system call (or an equivalent in higher-level languages). This indicates that the process has completed its intended task.

A process may pass an **exit status** to its parent, which can be used to determine whether the process terminated successfully or failed.

Example in C:

```
c
CopyEdit
int main() {
    // Code logic
    return 0; // Equivalent to exit(0)
}
```

Involuntary Termination:

The OS or a parent process can forcibly terminate a child process. This happens in scenarios such as:

- **Resource overuse:** The child uses more CPU or memory than allowed.

- **Security violation:** The child attempts unauthorized access.
- **Logic error:** The child enters an infinite loop or becomes unresponsive.
- **Parent logic:** The parent no longer requires the child to continue.

In UNIX systems, a process can be terminated using the `kill()` system call, where the parent sends a signal like `SIGTERM` or `SIGKILL`.

Cascading Termination:

In some operating systems, when a parent process terminates, all its children are automatically terminated. This behavior is known as **cascading termination**.

This ensures that no child process is left running without a supervising parent. Cascading termination is useful in structured environments where processes depend on each other.

Zombie Processes

A **zombie process** is a process that has completed execution but still has an entry in the process table. It occurs when the parent hasn't yet collected the termination status using the `wait()` system call.

Zombie processes don't consume memory or CPU, but if too many accumulate, they can clog the process table and degrade system performance.

Example:

```
c
CopyEdit
pid_t pid = fork();
if (pid == 0) {
    // Child finishes quickly
    exit(0);
} else {
    // Parent does not call wait()
    sleep(100);
}
```

In this case, the child becomes a zombie until the parent collects its status.

Orphan Processes

An **orphan process** is a process whose parent has terminated. In UNIX/Linux, such processes are automatically adopted by init (PID 1), which then takes responsibility for cleaning them up.

This behavior ensures system hygiene and prevents orphans from becoming zombies

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
11	Which system call is used in UNIX to create a new process? A. create() B. fork() C. exec() D. spawn()
12	A process that has completed execution but still has an entry in the process table is called: A. Orphan B. Daemon C. Zombie D. Suspended
Fill in the Blanks:	
13	In UNIX, a child process is created using the _____ system call.
True or False	
14	A process can create multiple child processes in a multitasking system.(True/False)

4. COOPERATING PROCESS

In the field of operating systems, understanding how processes interact is fundamental to the efficient design and implementation of multitasking systems. Processes that run in an operating system may either be **independent** or **cooperating**. This distinction forms the basis for resource sharing, concurrent execution, and inter-process communication mechanisms in both single-processor and multi-core environments.

Independent vs. Cooperating Processes

A **process** is an instance of a program in execution, complete with its own memory space, registers, and execution context. If a process executes in isolation, without any interaction with other processes, it is considered **independent**. Independent processes neither affect other processes nor are affected by them. They do not share data, resources, or any information with other processes in the system.

Conversely, **cooperating processes** are those that **can affect or be affected** by other processes. This effect typically arises through **shared data** or **inter-process communication (IPC)** mechanisms. For instance, two processes accessing a shared file, shared memory, or sending messages back and forth are classified as cooperating processes. Such cooperation is often intentional and designed to accomplish tasks that would be inefficient or impossible to perform individually.

Why Use Cooperating Processes?

The operating system allows and facilitates cooperation between processes for several key reasons:

1. *Information Sharing*

One of the primary motivations behind process cooperation is the need to **share information**. In multi-user systems or networked environments, it is common for multiple users or processes to need access to the same data—for example, a shared file or database. Cooperating processes can safely and concurrently access such resources with the help of synchronization techniques provided by the operating system.

2. *Computation Speedup*

To improve performance and reduce computation time, a single complex task can be broken down into smaller subtasks that execute **concurrently**. Each of these subtasks may run in a separate process or thread, especially on **multi-core or multi-processor systems**. This parallelism leads to **faster computation**, provided that the processes are properly coordinated and do not contend for the same resources.

3. *Modularity*

Another benefit of cooperating processes is **modular system design**. Large systems can be developed in smaller, independent modules (processes or threads), each responsible for a specific function. This modularity simplifies **development, debugging, and maintenance**, and also enhances system scalability and flexibility.

4. *Convenience for Users*

Even a single user may wish to run multiple applications simultaneously, such as listening to music, editing documents, and downloading files. Each application runs as a separate process, and some of them might even cooperate (e.g., a document editor accessing files through a background sync service). Cooperating processes make such multitasking more **fluid and efficient**.

4.1 Mechanisms for Process Cooperation

For processes to cooperate effectively, the operating system must provide mechanisms for **inter-process communication (IPC)**. These include:

- **Shared Memory:** Multiple processes access the same region of memory. This is fast but requires proper synchronization (e.g., semaphores or mutexes) to avoid conflicts.
- **Message Passing:** Processes communicate by sending and receiving messages. This method is more structured and avoids the need for shared memory but may be slower due to system calls.
- **Pipes, FIFOs, and Sockets:** Useful for communication between processes on the same or different machines.

Resources such as files, I/O devices, and network ports can also be shared, necessitating proper control to avoid data corruption and race conditions.

Nondeterministic Behavior of Cooperating Processes

One of the most challenging aspects of cooperating processes is their **nondeterministic behavior**. Since these processes execute concurrently, the exact order in which their instructions are carried out may vary every time they run. This behavior is influenced by factors such as CPU scheduling, system load, and synchronization delays.

For example, suppose two cooperating processes are both writing to a shared terminal. One process prints “ABC” while the other prints “CBA.” The output on the terminal could be a combination like “ABCCBA,” “CBABAC,” or “ACBCBA,” depending on the timing and sequence of executions. However, not all outputs are valid. For example, an output like “AABBCC” may not be possible depending on how state changes and interleavings occur in real-time.

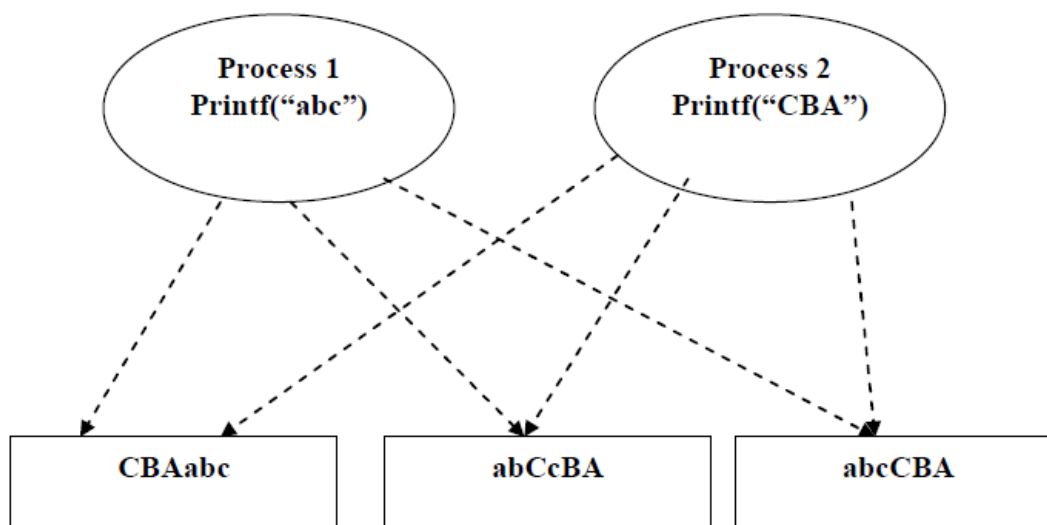


Figure 5 : Nondeterministic Behavior of Cooperating Processes

The **unpredictability** inherent in concurrent programming is what makes it both a powerful and complex paradigm. When multiple processes or threads execute simultaneously and interact with shared resources, the outcome is no longer deterministic. This introduces a range of challenges that, if left unmanaged, can compromise the correctness and reliability of a system.

Among the most common issues in concurrent systems are:

- **Race Conditions:** These occur when two or more processes access shared data concurrently and at least one of them modifies the data. Because the timing of execution can vary, the final result may be inconsistent or unexpected, depending on which process completes its operation first. Race conditions are notoriously difficult to detect and reproduce, making them a serious concern in concurrent systems.
- **Deadlocks:** A deadlock arises when two or more processes wait indefinitely for resources that are held by each other. This circular waiting pattern means none of the involved processes can proceed, effectively halting part or all of the system. Deadlocks are especially problematic in systems where resource management is critical, such as databases and operating systems.
- **Starvation:** Starvation occurs when a process waits indefinitely to acquire a needed resource because other processes continuously gain preference. Unlike deadlocks, where all involved processes are stuck, in starvation, one or more processes continue while another is perpetually delayed. This can result from flawed scheduling algorithms or unfair resource allocation policies.

To address these challenges, developers use **synchronization mechanisms**—techniques designed to coordinate the execution of concurrent processes and manage access to shared resources. Key synchronization tools include:

- **Locks (or Mutexes):** These are the simplest form of synchronization. A lock ensures that only one process can access a critical section of code at a time. While effective, improper use of locks can lead to deadlocks or performance bottlenecks.
- **Semaphores:** Semaphores are more general than locks and can control access to a resource pool with multiple instances. They can be used for both mutual exclusion and signaling between processes.
- **Monitors:** A monitor is a high-level abstraction that combines mutual exclusion and condition synchronization. It encapsulates shared variables, operations, and the synchronization required, making concurrent programming safer and more structured.
- **Condition Variables:** These allow threads to wait for certain conditions to become true before proceeding. They are often used within monitors or in conjunction with locks to facilitate coordination between threads.

By carefully applying these tools, developers can enforce **critical sections**, avoid unsafe interleavings, and ensure that processes cooperate in a predictable and controlled manner. However, effective synchronization requires thoughtful design, as misuse can introduce new problems such as deadlocks or degraded performance.

In essence, while concurrent programming unlocks the potential of modern multi-core processors and distributed systems, its complexity demands disciplined practices and robust synchronization strategies to build safe and efficient applications.

Reproducibility and Replication

In modern computing systems, cooperating processes—processes that communicate and work together to achieve a common goal—are central to many applications, from web servers to distributed databases. However, these systems introduce a level of complexity that often leads to **nondeterministic behavior**, meaning the same set of inputs can result in different outputs or execution paths each time a program is run. This lack of predictability poses challenges, particularly in the areas of **testing, debugging, and system reliability**.

Reproducibility refers to the ability to recreate the exact behavior of a system during a particular execution. This is crucial for identifying and fixing bugs, as developers need to consistently reproduce an issue to understand its root cause. In cooperating systems, achieving reproducibility is difficult due to factors such as race conditions, timing variations, and asynchronous communication between processes. To mitigate this, developers rely on specialized tools and techniques. **Loggers** capture detailed execution traces, **debuggers** allow for step-by-step inspection of program state, and **simulators** can emulate system behavior under controlled conditions. These tools enable engineers to replay execution sequences and identify the exact circumstances under which failures occur.

On the other hand, **replication** is a deliberate technique used in system design to improve performance, availability, and fault tolerance. It involves running multiple copies of the same process or service in parallel. In environments such as web hosting or cloud computing, this allows systems to handle many client requests simultaneously by distributing the load among multiple replicated processes. This approach, known as **load balancing**, ensures efficient resource utilization and faster response times.

Replication also enhances **fault tolerance**. If one process fails or becomes unresponsive, others can continue to operate, minimizing downtime and service disruption. This is especially important in mission-critical systems such as online banking, healthcare platforms, or telecommunications networks, where system failures can have significant consequences. Techniques such as **process checkpointing**, **state synchronization**, and **redundant execution** are employed to keep replicas in sync and recover quickly from failures.

Interestingly, while replication can improve system robustness, it can also complicate reproducibility. For instance, when multiple replicated processes interact or race to complete tasks, their execution paths may diverge in unpredictable ways. This interplay between reproducibility and replication presents an ongoing challenge in systems engineering. Developers must strike a balance between designing for resilience and maintaining the ability to debug and validate the system effectively.

SELF-ASSESSMENT QUESTIONS - 3

True or False	
15	_____ is the process of verifying the identity of a user before allowing access to the system.
Multiple Choice Questions	
16	Which of the following is a characteristic of cooperating processes? A. They do not share any data or resources B. They are executed in isolation C. They may share data and affect each other D. They always run sequentially
17	Which of the following is NOT a reason for using cooperating processes? A. Information sharing B. Modularity C. Computation speedup D. Complete process isolation
18	Which IPC mechanism allows processes to share a common memory space? A. Message passing B. Shared memory

	C. Sockets D. Signals
19	Which of the following problems can arise in cooperating processes? A. Race conditions B. Deadlocks C. Starvation D. All of the above
20	In message passing, processes communicate by: A. Sharing code segments B. Reading and writing to shared disk C. Sending and receiving messages D. Updating the same stack memory
Fill in the blanks	
21	A process that affects or is affected by another process is called a ____ process.
22	In ____ memory IPC, multiple processes access a common memory space.
23	Message passing is ____ than shared memory but provides better structure
24	A condition where multiple processes are stuck waiting for each other indefinitely is known as a ____.
25	____ processes do not communicate or interfere with other processes.
True or False	
26	Cooperating processes cannot access shared data. (True/False)
27	Independent processes do not share memory, data, or resources with other processes. (True/false)
28	Shared memory is always slower than message passing. (True/false)
29	Race conditions occur when two processes try to access shared data without synchronization. (True/False)

30	Starvation is a situation where a process always gets priority and completes before others. (True/False)
----	---



5. SUMMARY

- In this unit, students have learnt that a process is a program in execution, distinct from a static program file. They understood that a process includes code, data, resources, and the execution context managed by the operating system.
- a process is an active entity with a defined memory structure including stack, heap, and code segments. They also recognized the role of processes as the basic units of work in a multitasking operating system.
- The five primary states of a process: New, Ready, Running, Waiting, and Terminated. They also explored how processes transition between these states based on scheduling and events.
- PCB stores all critical information about a process, such as its state, program counter, and memory management details. They understood that the PCB enables the OS to manage and switch between processes efficiently.
- How processes are created and terminated, including parent-child relationships and process identifiers. They also studied the conditions under which processes voluntarily or involuntarily terminate.
- The new processes can be initiated by user actions, system functions, or other processes. They explored mechanisms such as `fork()` in UNIX and `CreateProcess()` in Windows.
- The processes can terminate normally or be forcefully stopped due to errors or control decisions. They also understood concepts like zombie and orphan processes and how the OS handles them.
- The difference between independent and cooperating processes. They explored how cooperation enables shared resource access, faster computation, and modular design.
- inter-process communication techniques such as shared memory, message passing, and sockets. They also recognized the importance of synchronization to prevent issues like race conditions and deadlocks.

6. GLOSSARY

Process	- A program in execution that includes the program code, current activity, and resources used during execution
Program	- A passive set of instructions stored on disk that becomes a process when loaded into memory for execution.
Process State	- The current status of a process, such as New, Ready, Running, Waiting, or Terminated.
Process Control Block (PCB)	- A data structure used by the operating system to store all the information about a process, including process ID, state, program counter, registers, and memory management details
Context Switching	- The procedure of saving the state of a currently running process and loading the state of another process, allowing multitasking on a CPU.
Multithreading	- A technique where a single process contains multiple threads, each performing different tasks concurrently within the same memory space
Thread	- A lightweight execution unit within a process that shares the same memory and resources as other threads in the same process.
Process Creation	- The operation by which a new process is initialized, usually by a parent process, and added to the system's process table.
Process Termination	- The process of ending a process, either normally after execution or abnormally due to an error or external command.
Parent Process	- A process that creates one or more child processes during execution.
Child Process	- A process created by another (parent) process, often inheriting some of its resources.
Zombie Process	- A terminated process that still has an entry in the process table because its parent has not yet collected its exit status.

Orphan Process	- A process whose parent has terminated. In UNIX-like systems, such processes are adopted by the init process.
Cooperating Processes	- Processes that can affect or be affected by other processes, typically through shared data or inter-process communication.
Independent Process	- A process that runs without interacting or sharing data with other processes.
Inter-Process Communication (IPC)	- A mechanism that allows processes to communicate and synchronize their actions while executing.
Shared Memory	- A method of IPC where multiple processes access the same memory segment to exchange information.
Message Passing	- An IPC technique where processes communicate by sending and receiving messages through the operating system
Race Condition	- A situation where the behavior of a software system depends on the sequence or timing of uncontrollable events, often due to improper synchronization.
Deadlock	- A state where two or more processes are unable to proceed because each is waiting for the other to release a resource.
Semaphore	- A synchronization primitive used to control access to shared resources by multiple processes or threads.

7. TERMINAL QUESTIONS

1. Define a process. How is it different from a program?
2. List and explain any three states of a process.
3. What is a Process Control Block (PCB)? Mention its two contents.
4. What is context switching and why is it important?
5. What are cooperating processes? Give an example.
6. Explain process creation in UNIX and Windows with examples.
7. Differentiate between a process and a thread with at least four points.
8. Describe the lifecycle of a process with a state diagram.
9. What are zombie and orphan processes? How does the OS handle them?
10. Describe two mechanisms used for inter-process communication. Give advantages of each.

8. TERMINAL ANSWERS

1. Refer to Section 2: Process Concept
2. Refer to Section 2.2: The Process State
3. Refer to Section 2.3: Process Control Block
4. Refer to Section 2.3: Context Switching
5. Refer to Section 4: Cooperating Processes
6. Refer to Section 3.1: Process Creation
7. Refer to Threads section under Section 2: Process Concept
8. Refer to Section 2.2: The Process State
9. Refer to Section 3.2: Process Termination
10. Refer to Section 4.1: Mechanisms for Process Cooperation.

9. ANSWERS

Self-Assessment Questions

1. D. Executable
2. C. Threads of the same process
3. B. Stack Memory
4. Process
5. Context
6. Waiting
7. Memory
8. False – A program is passive; a process is active.
9. True
10. False – It adds overhead; efficiency depends on minimizing it.
11. B. fork()
12. C. Zombie
13. fork()
14. True
15. False – They do, often intentionally.
16. C. They may share data and affect each other
17. D. Complete process isolation
18. B. Shared memory
19. D. All of the above
20. C. Sending and receiving messages
21. Cooperating
22. Shared
23. Slower
24. Deadlock
25. Independent
26. False – They often do to achieve communication or synchronization.
27. True
28. False – Shared memory is generally faster but requires careful synchronization.
29. True

30. False – Starvation means a process waits too long without getting required resources.



10. REFERENCES

- **Silberschatz, A., Galvin, P. B., & Gagne, G.** (2018). *Operating System Concepts* (10th Edition),Wiley.
- **Tanenbaum, A. S., & Bos, H.** (2015). *Modern Operating Systems* (4th Edition),Pearson.
- **Stallings, W.** (2018). *Operating Systems: Internals and Design Principles* (9th Edition),Pearson.

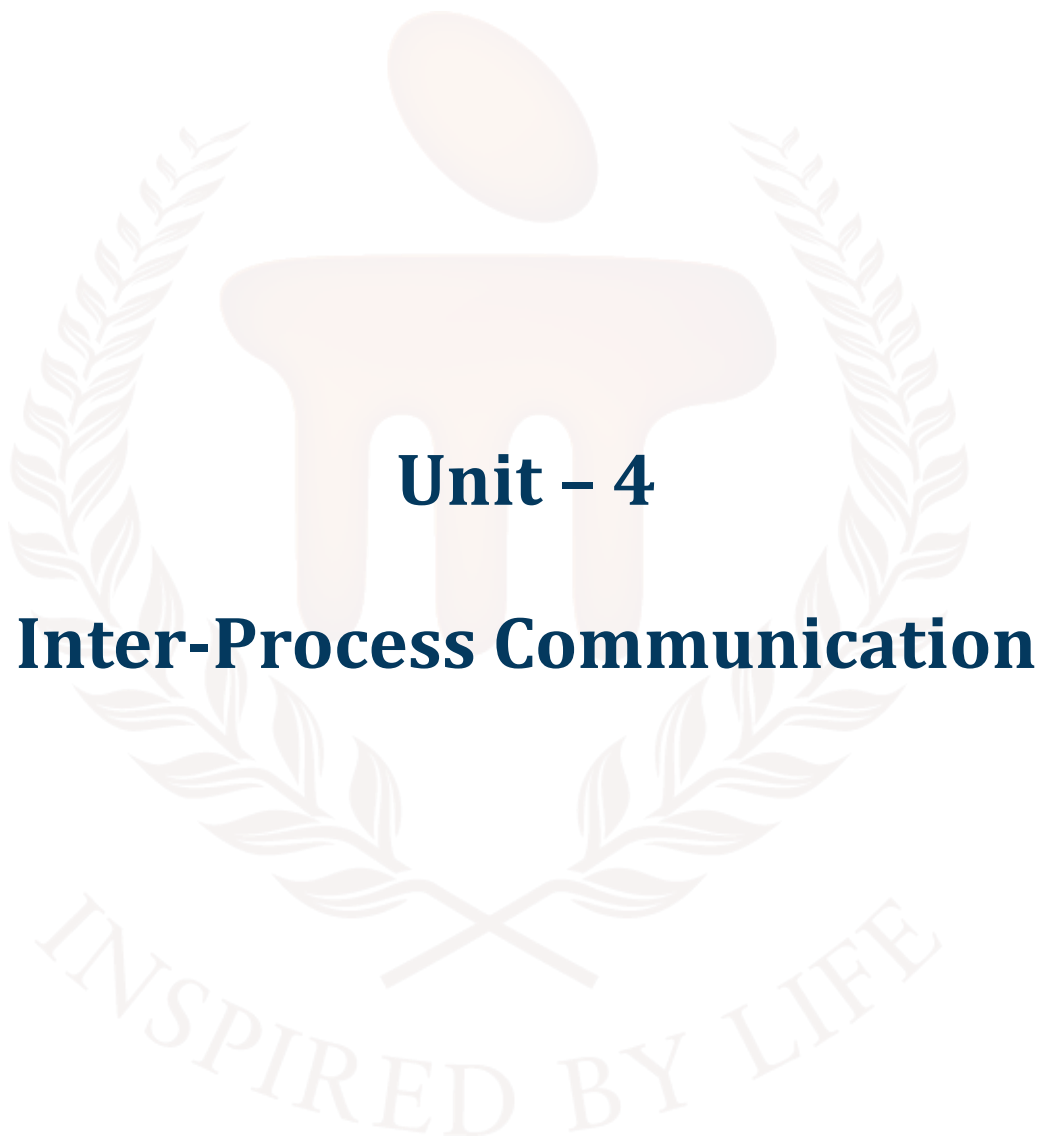


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 4

Inter-Process Communication

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Inter-Process Communication	1, 2	1	6 - 9
2.1	Shared Memory Model	-	-	
3	Message Passing Model	3	2	10 - 16
4	Process Scheduling	4	3	17 - 33
4.1	The schedulers	-	-	
5	Summary	-	-	34
6	Glossary	-	-	35 - 36
7	Terminal Questions	-	-	37
8	Terminal Answers	-	-	38
9	Answers	-	-	39
10	References	-	-	40

1. INTRODUCTION

In this unit, learners will explore essential mechanisms that support multitasking in modern operating systems—namely, Inter-Process Communication (IPC) and Process Scheduling. As computer systems often run multiple processes simultaneously, it becomes necessary for these processes to communicate, coordinate, and share information effectively. IPC provides the framework for this interaction, ensuring that processes can synchronise their activities, share data, and notify each other about events or state changes. The two primary models of IPC introduced in this unit are the shared memory model and the message passing model. Each model offers distinct methods for enabling communication, with shared memory focusing on common memory spaces and message passing involving the exchange of structured data packets.

The shared memory model is typically faster and suited for large volumes of data, but it demands careful synchronisation to avoid issues like race conditions. On the other hand, message passing provides better process isolation and is particularly useful in distributed systems where processes do not share the same memory space.

The unit also focuses on the concept of process scheduling, which ensures fair and efficient allocation of CPU time among competing processes. Learners will gain insight into the different types of schedulers—long-term, short-term, and medium-term—that manage various stages of a process's life cycle. These schedulers play a crucial role in determining which processes enter the system, which ones get CPU time, and which should be swapped out to manage memory effectively.

By studying IPC and process scheduling, learners will develop a foundational understanding of how operating systems manage multiple processes, ensure system responsiveness, and optimise performance. These concepts are critical for designing reliable, scalable, and high-performing computing environments in both centralised and distributed systems.

1.1. Objectives

By the end of this unit, learners will be able to:

- Define Inter-Process Communication (IPC), shared memory, and message passing.
- List different types of schedulers in an operating system.
- Explain how shared memory and message passing models enable communication between processes.
- Describe the roles of long-term, short-term, and medium-term schedulers.
- Illustrate the process of communication between processes using either shared memory or message passing.
- Demonstrate how different scheduling algorithms affect process execution using simple examples.
- Compare and contrast shared memory and message passing models in terms of efficiency and complexity.



2. INTER-PROCESS COMMUNICATION

In modern operating systems, multiple processes often need to cooperate to complete complex tasks. These processes may need to share data, synchronise actions, or notify each other about important events. This communication between processes, known as **Inter-Process Communication (IPC)**, is a fundamental concept that allows processes to work together efficiently.

Since processes usually run in separate memory spaces to ensure protection and stability, they cannot directly access each other's memory. Therefore, the operating system provides mechanisms to enable these processes to communicate safely and reliably.

Two primary models facilitate IPC:

- **Shared Memory**
- **Message Passing**

Both methods have their advantages and use cases, and understanding these helps in designing efficient multitasking systems.

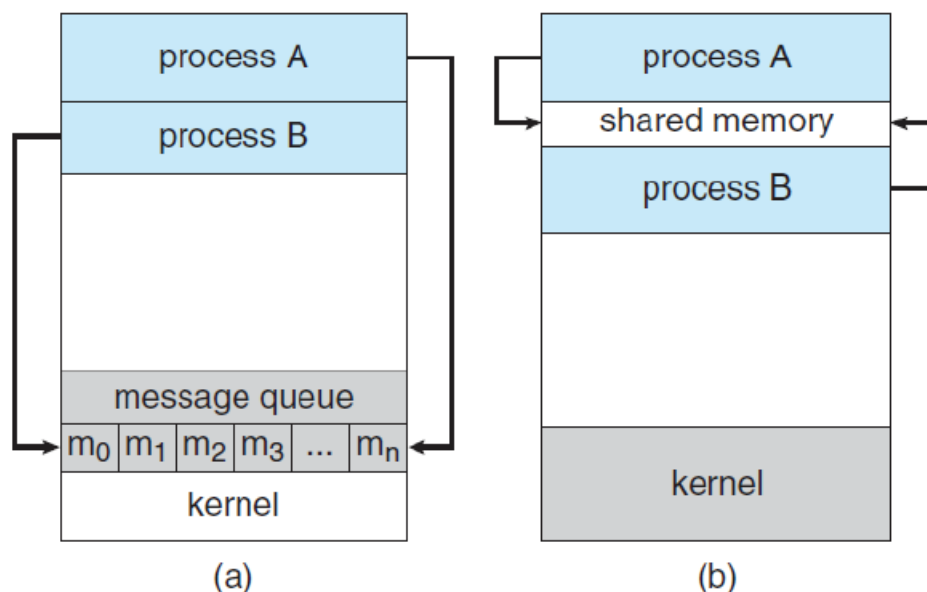


Figure 1: Communication Model (a) Message Passing model (b) Shared Memory model

2.1 Shared Memory Model

The shared memory model is based on the idea that two or more processes can **share a portion of the system's memory**. This shared space acts as a **common buffer** or **workspace**, enabling processes to exchange information by simply reading from and writing to this area.

The basic steps in a shared memory model are:

1. **Creation:** One process, or sometimes the operating system, creates a shared memory segment.
2. **Attachment:** Other cooperating processes attach to this shared memory segment, making it accessible in their address space.
3. **Communication:** Processes read and write data in this shared region to exchange information.
4. **Detachment and Cleanup:** Once communication is done, processes detach from the shared memory, and eventually, the segment is destroyed to free up resources.

This approach eliminates the overhead of data copying and system calls needed in message passing, leading to **faster communication**, especially for **large data transfers**.

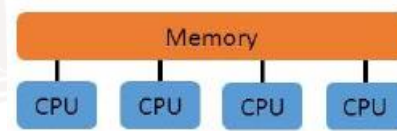


Figure 2: Shared Memory

Benefits :

- **High Efficiency:** Since all processes involved access the same memory region, there's no need to duplicate data. This direct access minimises overhead, especially when transferring large amounts of data. For example, in multimedia applications or large-scale data processing, copying entire files or video frames would be costly in terms of performance.
- **Fast Communication:** Shared memory is one of the fastest IPC mechanisms because it bypasses the kernel for data exchange after initial setup. Once the shared segment is established and attached, processes can operate on memory just like local variables. This low

latency makes it ideal for time-sensitive applications such as real-time monitoring systems or high-frequency trading platforms.

- **Good for Large Data Transfers:** When dealing with large data structures like arrays, images, or logs, copying data through pipes or sockets becomes inefficient. Shared memory allows a single, centralised copy of the data that can be used by multiple processes, significantly improving throughput.

Drawbacks:

Despite its speed and efficiency, the shared memory model comes with significant challenges that developers must address.

1. **Complex Synchronisation:** The biggest challenge is ensuring **data consistency** when multiple processes access shared memory concurrently. For instance, if one process is writing data while another reads it, there's a risk that the reader will get partially updated or corrupted data. These kinds of conflicts are known as **race conditions**.

To handle this, synchronisation tools like:

- **Semaphores** – Counting mechanisms used to manage access to shared resources.
- **Mutexes (Mutual Exclusion Locks)** – Ensure that only one process can access a critical section at a time.
- **Condition variables** – Used to synchronise complex interactions between threads or processes.

Proper use of these mechanisms is crucial. However, it also increases the complexity of program design and the potential for bugs, such as deadlocks or priority inversion.

2. **Limited to the Same Machine:** Shared memory segments exist within the RAM of a single computer. Therefore, they are not suitable for communication between processes on different machines. For distributed systems or cloud-based architectures, other IPC methods like sockets, Remote Procedure Calls (RPC), or message queues are more appropriate.
3. **Security and Safety Risks:** When multiple processes can access and modify the same memory region, there's a risk of one process inadvertently or maliciously corrupting the memory. Without strict access controls and synchronisation, this can lead to data corruption, crashes, or

vulnerabilities. Careful programming and memory protection techniques are necessary to mitigate these risks.

Use Cases of Shared Memory

The shared memory model is particularly well-suited to specific kinds of applications, including:

- **Multimedia Processing:** Video editing software, game engines, or real-time audio processing where large buffers must be shared between processes.
- **Database Systems:** High-performance databases may use shared memory to allow client processes to access a common data buffer.
- **Simulation and Modeling:** Scientific simulations often run multiple processes to simulate complex systems. Shared memory allows them to quickly exchange intermediate data.
- **Operating System Kernels:** Some OS kernels use shared memory for communication between kernel and user processes, or among subsystems.

SELF-ASSESSMENT QUESTIONS - 1

Multiple choice Questions	
1	In the shared memory model, synchronisation between processes is required to avoid: A. Memory leaks B. Deadlocks C. Race conditions D. Page faults
Fill in the Blanks:	
2	The _____ model allows processes to communicate by reading and writing to a common memory region.
True or False	
3	Shared memory requires no synchronisation mechanisms between processes. (True/False).

3. MESSAGE PASSING MODEL

Imagine two people who cannot speak directly but need to exchange information. They decide to write letters back and forth. Each letter contains a message that conveys their thoughts, questions, or commands. This simple analogy reflects the concept of **Message Passing** in operating systems.

In a multitasking operating system, different programs or **processes** run independently, often isolated from each other for security and stability reasons. Because they don't share the same memory space, they cannot directly access each other's data. Instead, they communicate by sending and receiving messages through the operating system.

Message Passing is a fundamental method used by processes to exchange data, synchronise actions, and coordinate tasks, especially when processes are running concurrently and may reside on different machines in a distributed environment.

In the message passing model, **processes maintain completely separate address spaces**. They communicate by passing **structured messages**—bundles of data and metadata—through channels managed by the operating system or middleware. The communication may occur between processes on the same system or across different systems in a network.

1. **Message Creation:** The sender constructs a message containing the data to be transmitted. This may include control information such as message type, length, or priority.
2. **Transmission:** The operating system or a communication middleware transmits the message from the sender to the recipient through a communication channel, such as a socket, pipe, or message queue.
3. **Reception:** The receiver retrieves the message using a `receive()` operation and processes its contents.

Since each process has its own memory space, the system must **copy the message** between the processes' memory, ensuring **isolation and protection**.

Because processes maintain separate memory spaces, messages are copied between the sender and receiver, ensuring memory protection.

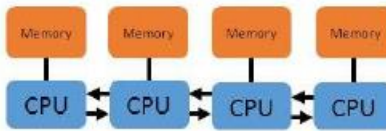


Figure 3: **Message Passing**

Why is Message Passing Important?

When multiple processes run simultaneously, they often need to:

- Share data
- Synchronise their operations
- Notify each other about events
- Coordinate access to resources

If these processes tried to directly read or write each other's memory, it would lead to data corruption, security risks, and instability. Message passing provides a controlled and structured way to achieve this communication.

Message Passing Operations

Most message passing systems provide two fundamental operations:

- **send(destination, message):** Sends a message to a destination process.
- **receive(source, buffer):** Receives a message from a source process into a local buffer.

The message may be:

- **Fixed-length:** Suitable for simple commands or small data items.
- **Variable-length:** Allows flexible and dynamic message structures, used in more complex communication scenarios.

The underlying system handles message buffering, copying, and queuing, depending on whether the communication is **synchronous** or **asynchronous**.

In computer systems, especially in concurrent and distributed environments, **interprocess communication (IPC)** is vital for processes to exchange data and coordinate their actions. One of the

most commonly used IPC mechanisms is **message passing**, where processes communicate by sending and receiving structured messages. This model is particularly useful in systems where processes operate in separate memory spaces or even on different physical machines.

Message passing can be classified into **two primary types** based on how the communication occurs: **synchronous** and **asynchronous**. Each of these approaches offers unique trade-offs in terms of timing, coordination, efficiency, and system complexity.

1. Synchronous Message Passing

In **synchronous message passing**, the communication between the sender and the receiver is tightly coordinated. Both parties must be actively engaged in the communication process at the same time.

- The **sender** sends a message and **waits** (blocks) until the receiver acknowledges that it has received the message.
- The **receiver** is also **blocked** until a message arrives. Once it does, it processes the message and may optionally send a reply or acknowledgment.

This behavior can be compared to a **telephone conversation**: the sender cannot proceed with the rest of their activity until the receiver responds, and the communication is instantaneous and mutual.

Benefits

- **Guarantees Delivery Timing:** The sender knows that the message has been received before proceeding. This makes it ideal for situations where **strict synchronisation** is required.
- **Simple Buffer Management:** Since both sender and receiver are synchronised, there's usually **no need for intermediate message queues or buffers**.
- **Deterministic Communication:** Useful in **real-time systems** and control applications where timing is crucial.

Drawbacks

- **Blocking Delays:** If the receiver is not ready or is slow to respond, the sender remains blocked, which can lead to **reduced performance**.

- **Deadlocks:** Improper implementation can result in situations where two or more processes are waiting for each other indefinitely.
- **Limited Scalability:** Synchronous messaging is **not ideal for systems with many asynchronous or distributed components.**

Use Cases

- **Real-Time Systems:** Industrial automation or robotics where precise timing is critical.
- **Client-Server Applications:** Where a client must wait for a response before proceeding (e.g., a banking transaction).
- **Protocol Implementations:** Certain network protocols require acknowledgment-based communication.

2. Asynchronous Message Passing

In **asynchronous message passing**, the sender and receiver operate **independently**. Communication does not require both parties to be synchronised in time.

- The **sender** sends the message and immediately **continues its execution** without waiting for the receiver's acknowledgment.
- The **receiver** retrieves the message **whenever it is ready**, either through polling or event-driven mechanisms.

This model is analogous to **sending an email**: the sender dispatches the message and moves on, while the recipient can read and respond at their convenience.

Benefits

- **Non-blocking Communication:** Processes can operate independently, improving **overall system responsiveness and concurrency.**
- **Scalability:** Well-suited for **large-scale systems** such as cloud services and distributed architectures.
- **Improved Efficiency:** No idle time for the sender, which can continue working on other tasks.

Drawbacks

- **Buffering Required:** Messages must be **temporarily stored** (buffered) until the receiver is ready to process them.
- **Risk of Queue Overflow:** If messages are produced faster than they are consumed, **message queues may overflow**, causing errors or dropped messages.
- **Complexity in Handling Order and Consistency:** Developers may need to implement logic for **message ordering, delivery guarantees, and fault tolerance**.

Use Cases

- **Distributed Systems:** Services in a microservices architecture or cloud infrastructure often communicate asynchronously.
- **User Interfaces:** Event-driven programming (like GUI apps) where actions are handled as they occur.
- **Logging and Monitoring:** Applications log events asynchronously to avoid interrupting their primary workflow.

Comparison Table:

Aspect	Synchronous Message Passing	Asynchronous Message Passing
Timing Dependency	Tight coupling: both sender and receiver synchronise.	Loose coupling: communication is decoupled in time
Blocking Behavior	Sender blocks until message is received	Sender continues immediately after sending
Efficiency	Less efficient in concurrent systems	Highly efficient in high-concurrency environments
Buffering Needs	Minimal or no buffering	Requires message buffering or queueing
Risk of Deadlocks	Higher, especially in tightly coupled systems	Lower, but risks of queue overflow
Use Case Examples	Real-time control, remote procedure calls	Event-driven systems, message queues, cloud services

Underlying Technologies and Implementations

Synchronous Communication Tools

- **Remote Procedure Calls (RPC):** Function calls that block until the remote system responds.
- **Blocking Sockets:** A client sends data and waits until a response is received.
- **System Calls in OS Kernels:** Many low-level system functions are implemented synchronously for determinism.

Asynchronous Communication Tools

- **Message Queues:** Such as RabbitMQ, Kafka, or POSIX message queues.
- **Non-blocking Sockets:** Often used in high-performance web servers and event-driven frameworks.
- **Event Buses:** Used in microservice architectures to loosely couple components via events.
- **Actors (e.g., Akka in Scala):** Each actor processes messages asynchronously and independently.

3. Hybrid Approaches

Modern systems often **combine both synchronous and asynchronous message passing** to leverage the benefits of both:

- A **web application** might use asynchronous messaging for internal service communication but use synchronous HTTP for front-end interaction.
- A **database system** might use asynchronous logging (write-ahead logs) but synchronous commits for transaction safety.

These hybrid systems rely on **middleware or communication protocols** that abstract the details and offer developers a flexible API for both communication models

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
4	Message passing is particularly useful in: A. Single-CPU systems B. Shared-memory systems only C. Distributed systems D. BIOS-level programming
5	Which of the following supports asynchronous communication? A. Mutex B. Sockets C. Round Robin D. Paging
Fill in the Blanks:	
6	In message passing, the two fundamental operations are _____ and _____.
True or False	
7	Message passing is not suitable for communication across different systems. (True/False)
Fill in the Blanks:	
8	In synchronous communication, both sender and receiver must be _____ at the same time.
True or False	
9	In asynchronous message passing, the sender does not wait for the receiver. (True/False)

4. PROCESS SCHEDULING

In modern computer systems, multiple processes often run concurrently. These systems must manage the execution of numerous processes efficiently to maximise performance and resource utilisation. Process scheduling is a core concept in operating systems, responsible for deciding which process should run at any given time.

Goals of Process Scheduling

The primary goals of process scheduling include:

1. **Maximising CPU Utilisation:** Ensuring that the CPU is active and processing tasks at all times.
2. **Implementing Time Sharing:** Enabling interactive use by switching the CPU frequently among processes so that users can interact with multiple programs simultaneously.
3. **Balancing Load:** Fairly distributing resources among all processes.
4. **Minimising Response Time:** Especially important in interactive systems.
5. **Throughput and Turnaround Time:** Improving the number of processes completed in a given time and reducing the time it takes for a process to complete.

In single-processor systems, only one process can execute at any given time. The remaining processes must wait for the CPU to become free. The scheduler decides the order in which these waiting processes get CPU time, significantly influencing system performance.

Queue Scheduling

Scheduling in operating systems often employs queues to manage the flow and execution of processes. Each process that enters the system is placed in a queue based on its current state and requirements.

Types of Queues

1. **Job Queue (Task Queue):** This queue contains all processes that have been submitted to the system. These processes may be in any state—waiting, ready, or running.
2. **Ready Queue:** It consists of all the processes that reside in main memory and are ready to run. These processes are waiting for the CPU. Typically, this queue is implemented as a linked list, with a header pointing to the first and last elements. Each Process Control Block (PCB) in the list contains a pointer to the next PCB, forming a chain of ready processes.

3. **Device Queues:** These queues hold processes that are waiting for specific I/O devices (e.g., disk, printer, network). When a process requests an I/O operation, it is removed from the CPU and placed in the appropriate device queue.

Process scheduling is often represented by a queueing diagram, like the one shown in Figure 4. A queue is represented by each rectangular box. There are two different kinds of queues: a group of device queues and the ready queue. The arrows show how processes move through the system, while the circles show the resources that supply the queues.

First, a new process is added to the ready queue. There, it waits to be dispatched, or chosen for execution.

One of the following things could happen once the process has been given the CPU and is running:

In the event of an interrupt, the process may:

- forcefully remove itself off the CPU and return to the ready queue;
- start a new child process and wait for the child's termination; or
- send an I/O request and subsequently be placed in an I/O queue.

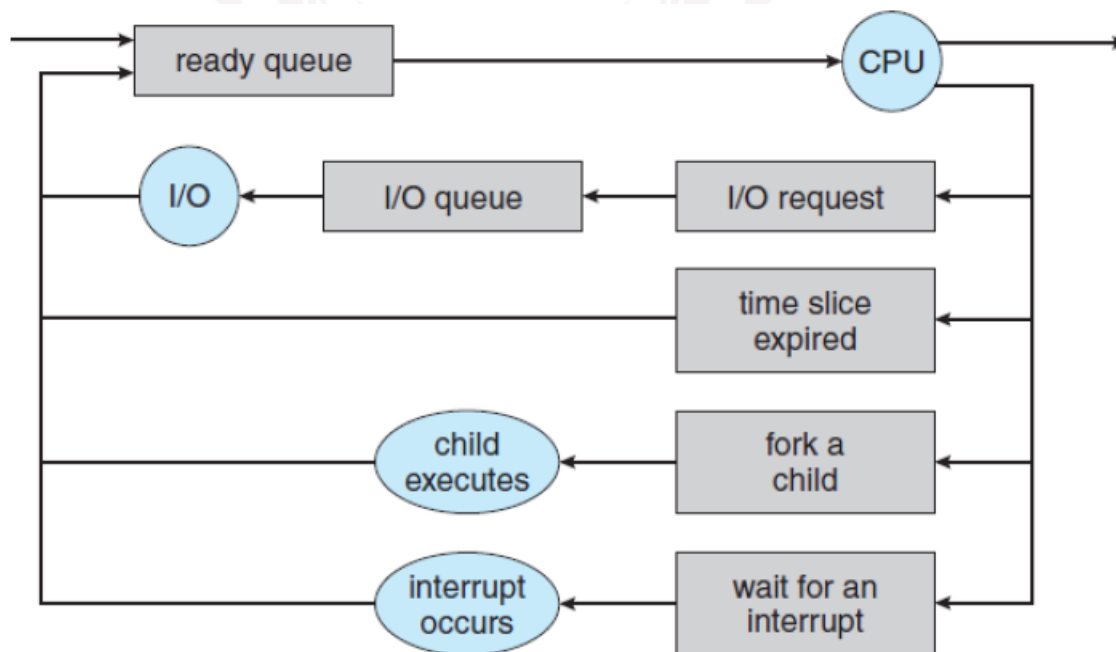


Figure 4: Process Scheduling

4.1 The schedulers

In a multiprogramming environment, multiple processes reside in the system simultaneously. These processes compete for CPU time, memory, and I/O resources. The operating system (OS) is responsible for deciding which process gets access to these resources and when. This decision-making is handled by components known as **schedulers**. Scheduling ensures efficient use of the CPU and other system resources while maintaining fairness and meeting performance goals.

There are **three primary types of schedulers** in an operating system:

1. **Long-Term Scheduler (Job Scheduler)**
2. **Short-Term Scheduler (CPU Scheduler)**
3. **Medium-Term Scheduler (Swapper)**

Each of these plays a unique and critical role in the management of process execution.

1. Long-Term Scheduler (Job Scheduler)

The **Long-Term Scheduler**, also known as the **Job Scheduler**, is a critical component of the operating system responsible for controlling the **admission of new processes** into the system. When users submit jobs, they are initially stored in a secondary storage area, commonly referred to as the **job pool** or **job queue**. From this pool, the long-term scheduler selects a **subset of jobs** and loads them into **main memory**, thereby making them eligible for execution by the CPU.

The primary goal of the long-term scheduler is to **manage the degree of multiprogramming**, which denotes the number of processes in memory at any given time. A well-regulated level of multiprogramming ensures that system resources—especially the CPU and I/O devices—are utilised efficiently without being overburdened.

Key Responsibilities:

The long-term scheduler has several important responsibilities that influence the performance and responsiveness of the system:

- **Admission Control:** It decides **which jobs from the job queue** should be admitted into the system for processing.

- **Workload Balancing:** It aims to maintain an optimal balance between **CPU-bound** and **I/O-bound** processes to prevent bottlenecks and underutilisation of system components.
- **Resource Management:** By controlling the number of processes in memory, the long-term scheduler helps **prevent resource saturation**, which can lead to performance degradation or system instability.

CPU-Bound vs. I/O-Bound Processes

Understanding the nature of different workloads is essential for effective long-term scheduling:

- **CPU-bound processes:**
 - Spend the majority of their time performing **computational operations**.
 - Make infrequent I/O requests.
 - Tend to monopolise the CPU if not balanced properly.
- **I/O-bound processes:**
 - Spend more time performing **input/output operations** than computations.
 - Frequently enter **waiting states**, thus relinquishing the CPU often.
 - Help keep I/O devices busy, but may leave the CPU underutilised if not balanced with CPU-bound jobs.

A **balanced mix** of these two types of processes ensures that both the CPU and I/O devices remain **consistently active**, leading to higher **system throughput** and better overall performance.

Unlike the short-term scheduler, which operates continuously, the long-term scheduler is invoked **infrequently**. It typically runs:

- When a **new batch of jobs** is submitted to the system.
- When the **number of active processes** in memory falls below a certain threshold.
- When the system anticipates additional workload capacity due to completed jobs or freed-up resources.

In most cases, this scheduler executes **once every few seconds or minutes**, depending on the load and type of operating system (especially in **batch processing systems**).

Consider a **batch processing system** where users submit multiple jobs for future execution. These jobs are stored temporarily on a **mass storage device**, such as a disk, until the system is ready to process them.

The **long-term scheduler** periodically evaluates the system's current workload and **selects appropriate jobs** from the job queue based on several factors:

- **Available memory space**
- **Process priority or job length**
- **CPU/I/O characteristics of each job**
- **Overall system load**

Selected jobs are then moved from the **job pool** into **main memory**, where they transition into the **ready state**, waiting for CPU allocation from the **short-term scheduler**.

The efficiency of the long-term scheduler significantly impacts:

- **System responsiveness:** By controlling how many jobs enter memory.
- **Throughput:** By keeping resources utilised through effective workload balancing.
- **System stability:** By preventing overload and maintaining a manageable number of active processes.

A **poorly designed long-term scheduler** may result in:

- **CPU underutilisation** (if mostly I/O-bound jobs are loaded)
- **I/O device idling** (if too many CPU-bound jobs are active)
- **Excessive memory usage**, leading to paging or thrashing.

2. Short-Term Scheduler (CPU Scheduler)

The **Short-Term Scheduler**, also known as the **CPU Scheduler**, is responsible for selecting one of the **ready processes** from main memory and allocating the **CPU** to it. It plays a central role in **process execution** and **CPU utilisation**, ensuring that processes are executed efficiently and fairly.

Unlike the long-term scheduler, which controls how many processes enter the system, the short-term scheduler decides **which process runs next** from the ready queue—making it **the most frequently invoked scheduler** in the system.

➤ **CPU Allocation:**

- The most fundamental task of the short-term scheduler is to allocate the **CPU to one of the ready processes**. These processes reside in the **ready queue**, having all the resources needed to execute except the CPU itself.
- The scheduler continuously **monitors the ready queue** and selects a process based on the current **scheduling algorithm** (e.g., Round Robin, FCFS, Priority Scheduling).
- The selected process is moved to the **running state**, and control of the CPU is transferred to it.
- This decision must be made quickly and efficiently, as it happens **hundreds or thousands of times per second**.

Example: If the scheduler uses Round Robin, it might allocate the CPU to each ready process in equal time slices (quantum), cycling through them.

➤ **Process Switching (Context Switching)**

Another vital responsibility is managing **process switching**, also known as **context switching**. This occurs when the CPU moves from executing one process to another.

- The scheduler **saves the state** (registers, program counter, etc.) of the currently running process in its **Process Control Block (PCB)**.
- It then **loads the state** of the next scheduled process from its PCB.
- This ensures that each process resumes **exactly where it left off** when it regains the CPU.

Though necessary, context switching introduces overhead—since no productive work is done during the switch—so it must be optimised.

Example: In a time-sharing system, when the time slice for a process expires, the CPU is switched to the next ready process, preserving the state of the previous one.

➤ **Fairness and Efficiency**

The short-term scheduler must maintain a balance between **fairness** and **efficiency**:

- **Fairness** ensures that all processes in the ready queue get their turn to execute and are **not starved** of CPU time.
- **Efficiency** requires minimising CPU idle time and avoiding unnecessary context switches.

The scheduler may implement **priority levels**, **aging** (to prevent starvation), or **dynamic priorities** to keep the system both fair and responsive.

Example: In Priority Scheduling, a low-priority process might wait indefinitely. To ensure fairness, the system may gradually **increase the process's priority over time** (aging).

➤ **Maintaining Responsiveness**

This responsibility is particularly important in **interactive and real-time systems**, where users expect immediate feedback.

- The scheduler must ensure that **interactive processes** (such as those waiting for user input) are given quick access to the CPU.
- **Latency-sensitive tasks** must be prioritised appropriately so that user-facing applications remain responsive.

Example: In a GUI-based operating system, the short-term scheduler may give preference to mouse or keyboard input handling processes to keep the interface smooth and reactive.

➤ **Ready Queue Management**

- All processes that are **prepared to execute** (i.e., they have all required resources except the CPU) are placed in the **ready queue**.
- The ready queue is typically implemented as a **linked list**, **priority queue**, or **array**, depending on the scheduling algorithm used.
- Each process in the ready queue has an associated **Process Control Block (PCB)** that contains important context information such as:
 - Process ID

- Program counter
- CPU registers
- Process state
- Priority level (if used)

Example: In a priority scheduling system, the ready queue might be implemented as a heap to always pick the highest-priority process quickly.

➤ **Monitoring and Selection**

- The **short-term scheduler continuously monitors** the ready queue to decide which process should be executed next.
- The choice depends on the **scheduling policy** currently in effect. Common scheduling algorithms include:
 - **First-Come, First-Served (FCFS)**
 - **Round Robin (RR)**
 - **Shortest Job Next (SJN)**
 - **Priority Scheduling**
 - **Multilevel Queue Scheduling**
- The selected process is then assigned the CPU, changing its state from **Ready** to **Running**.

Example: In Round Robin scheduling with a time quantum of 100 milliseconds, the scheduler gives each process up to 100 ms on the CPU before switching to the next one.

➤ **Handling Interrupts and Process Events**

The short-term scheduler is invoked when specific **events or conditions occur** that affect the current process's ability to continue execution:

a) **I/O Request**

- When a process currently using the CPU **requests I/O** (e.g., reading from disk), it must wait for the I/O operation to complete.
- The process is moved to the **I/O queue**, and the scheduler is called to pick another ready process to run.

b) Timer Expiry (Preemption)

- In time-sharing systems, a **timer interrupt** is used to preempt the current process after a defined time slice (quantum).
- This ensures **fair access** to the CPU among all processes.
- The scheduler then selects the next process to run.

c) Process Completion

- When a process completes its execution, it releases the CPU.
- The scheduler is then triggered to choose the next available process from the ready queue.

d) Higher Priority Process Becomes Ready

- In **priority-based systems**, if a new process with **higher priority** arrives or becomes ready, it may preempt the currently running process.
- The current process is returned to the ready queue, and the scheduler immediately runs the higher-priority process.

➤ Context Switching

- When the scheduler selects a new process to run, the operating system must perform a **context switch**:
 - Save the **state of the current process** (program counter, CPU registers, etc.) in its PCB.
 - Load the **state of the new process** from its PCB.
- Although necessary, context switching introduces a performance overhead. Therefore, the scheduler must be optimised to minimise context switch duration.

Example: On modern systems, a context switch may take a few microseconds, but if done too frequently, it can impact CPU performance due to lost cycles.

3. Medium-Term Scheduler

The **Medium-Term Scheduler**, also known as the **Swapper**, is a specialised component of the operating system responsible for managing **memory utilisation** in a multiprogrammed environment. It operates **between** the long-term and short-term schedulers and serves a

memory-balancing role by temporarily removing processes from the main memory (RAM) and transferring them to **secondary storage** (typically known as the **swap space**).

This action known as **swapping** allows the system to:

- Free up valuable main memory for active or high-priority processes.
- Improve overall CPU utilisation by ensuring that memory does not become a bottleneck.
- Prevent **thrashing**, a state where excessive paging activity hinders system performance.

The medium-term scheduler plays a **crucial role** in systems with **limited RAM** or **high levels of concurrency**, where many processes compete for limited memory resources.

Key Responsibilities

1. Memory Management

- The primary responsibility of the swapper is to **control the number of processes in main memory**.
- It ensures that the **available physical memory** is not overwhelmed by too many loaded processes.
- When memory usage exceeds a certain threshold, the scheduler **swaps out** one or more processes to reduce memory pressure.

Example: If 10 processes are loaded in memory and RAM usage crosses 90%, the medium-term scheduler might suspend 2-3 idle or low-priority processes.

2. Suspension of Processes

- The scheduler selects **candidate processes** for suspension based on their **state**:
 - **Blocked processes** waiting for I/O or other events.
 - **Low-priority processes** that are not time-sensitive.
 - **Idle processes** that are not using CPU time actively.
- Once identified, the process's **state and memory image** are saved to disk (swap area), and it is **removed from main memory**.
- The process is then marked as **suspended** (or swapped out).

Criteria for suspension may include how long a process has been blocked, its priority, memory footprint, and I/O requirements.

3. Process Resumption (Swapping In)

- When system conditions improve (e.g., memory becomes available), the medium-term scheduler brings **suspended processes back into memory**.
- The process is **reloaded** from swap space, its context is restored, and it is placed back in the **ready queue** for the CPU scheduler to pick up.
- This dynamic control of process presence in RAM ensures **effective memory reallocation** and **process fairness**.

Swapped-in processes continue execution as though they were never interrupted, assuming memory and I/O states are restored properly.

4. Prevention of Thrashing

- **Thrashing** occurs when the system spends more time **paging** than executing actual processes due to excessive process activity and insufficient memory.
- The medium-term scheduler prevents thrashing by **reducing the number of active processes** at a time.
- By keeping only a **manageable number of processes** in memory, it ensures fewer page faults and better performance.

It works as a buffer between memory overload and stable operation, helping to smooth out spikes in resource demands.

Swapping Process Flow

The operation of the medium-term scheduler are:

1. Swapping Out (Suspending a Process)

A process may be **swapped out** or suspended under several conditions:

➤ **When the Process is Waiting for I/O**

- If a process initiates an I/O request (e.g., waiting for disk input or user interaction), it enters a **blocked state**.
- Since it cannot proceed with execution until the I/O operation completes, **keeping it in main memory is inefficient**.
- The medium-term scheduler identifies such blocked processes as **candidates for suspension** and moves them to secondary storage, thereby **freeing up valuable RAM**.

➤ **When System Memory is Overutilised**

- In highly multiprogrammed systems, the total memory demand may exceed the available physical RAM.
- The OS detects **memory overcommitment** or frequent **page faults**, which may lead to **thrashing** (excessive swapping of pages in and out).
- To reduce memory load and prevent performance degradation, the swapper selectively suspends processes, prioritising those that are least likely to run soon.

➤ **When Higher-Priority Processes Need Space**

- If a **high-priority** process is admitted but there is insufficient memory to accommodate it, the medium-term scheduler may **evict** lower-priority processes temporarily.
- This ensures that critical or time-sensitive tasks are given precedence and system responsiveness is preserved.

2. Moving the Process to Secondary Storage

- Once a process is selected for suspension:
 - Its **current state** (register values, program counter, memory contents) is saved into its **Process Control Block (PCB)**.
 - The process's **memory pages** are written to **swap space** (usually on the system's hard drive or SSD).
 - The process's **state in the PCB** is updated to reflect that it is now in a **"suspended"** or **"swapped out"** state (commonly called *Suspended Ready* or *Suspended Blocked*, depending on its original state).

Swapping is a relatively expensive operation compared to other scheduling activities, due to disk I/O latency. Therefore, it is only used when necessary.

3. Managing Suspended Processes

- Suspended processes remain in secondary storage **without consuming main memory**.
- However, the OS **tracks them via PCB entries**, keeping information about their state, priority, and resource needs.
- These processes are **ineligible for CPU execution** until they are brought back into RAM.
- The system monitors **overall load and memory usage** to determine when suspended processes can be resumed.

4. Swapping In (Resuming a Process)

- When system conditions improve—such as:
 - Memory becomes available.
 - A process completes, freeing space.
 - A previously suspended process becomes eligible for CPU execution.

The **medium-term scheduler** selects one or more suspended processes to be swapped back in.

Process of Swapping In:

- The process's data is **retrieved from swap space**.
- Its memory context is **restored into RAM**.
- Its **state is updated** from *Suspended Ready* to *Ready* or from *Suspended Blocked* to *Blocked*, depending on whether it was waiting on an event.
- The process is **reinserted into the ready or waiting queue**, where it can be chosen by the short-term scheduler for execution.

Example 1: A Time-Sharing System

Imagine a scenario where:

- There are 10 processes in memory, and a new real-time process (e.g., a video call application) enters the system.
- Due to limited memory, the OS cannot admit this process without offloading others.
- The medium-term scheduler identifies two low-priority, blocked processes waiting on network I/O and **swaps them out**.
- The new process is loaded and begins executing.
- Once the real-time process terminates and memory is freed, the previously suspended processes are **swapped back in** and resume their lifecycle.

Example 2:

Suppose a system is running five processes, but only three can fit in memory efficiently. The medium-term scheduler might **swap out two background or low-priority processes**, allowing more critical processes to continue executing without memory contention. Once these processes finish or memory frees up, the suspended processes can be brought back for execution.

The medium-term scheduler is directly involved in the **transition between**:

- **Blocked → Suspended Blocked**: When a blocked process is swapped out.
- **Suspended Blocked → Blocked**: When a swapped-out blocked process is reloaded into memory.
- **Ready → Suspended Ready**: A ready process may also be suspended during high memory load.
- **Suspended Ready → Ready**: The process becomes eligible again for execution once memory permits.

These transitions enable **fine-grained control** over which processes occupy memory at any given time, 1. Maximising system stability and responsiveness.

The medium-term scheduler is especially significant in the following scenarios:

- **Systems with High Multiprogramming Levels**

With many processes in the system, the swapper helps control memory load and enables smooth multitasking.

- **Limited RAM Availability**

Systems with small amounts of RAM can still support multiple concurrent processes using swapping techniques.

- **Batch Systems and Time-Sharing Systems**

In systems where the number of jobs may temporarily exceed what memory can support, the swapper maintains system balance.

- **Preventing System Degradation**

By preventing thrashing, the medium-term scheduler indirectly helps **maintain CPU efficiency** and **reduce I/O bottlenecks** caused by frequent paging.

The **Medium-Term Scheduler** operates primarily by **dynamically managing memory** through a process known as **swapping**. It decides when to **temporarily remove (swap out)** certain processes from the main memory to **optimise resource allocation** and **maintain system stability**, especially under high memory pressure.

Benefits and Impact

- **Increases System Stability:** Keeps the active memory footprint within manageable limits.
- **Improves Performance:** By ensuring enough memory is available for processes that are actively running.
- **Enables Higher Multiprogramming:** Without significantly impacting system responsiveness.
- **Reduces Thrashing:** Especially in systems using virtual memory.

Relation to Other Schedulers

- The **Long-Term Scheduler** controls how many processes enter the system.
- The **Short-Term Scheduler** decides which of the ready processes runs next.
- The **Medium-Term Scheduler** acts as a balancing mechanism, adjusting **the number of processes in memory** at any given time.

Scheduler Table:

Scheduler Type	Frequency	Focus Area	Responsibility	System Impact
Long-Term Scheduler	Infrequent	Job Admission	Decides which jobs enter the system	Controls degree of multiprogramming
Short-Term Scheduler	Very Frequent	CPU Allocation	Chooses next process for CPU execution	Affects throughput and responsiveness
Medium-Term Scheduler	Occasional	Memory Management	Swaps processes in and out of memory	Balances memory load; avoids thrashing

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
10	Which of the following is NOT a type of scheduler in an operating system? A. Long-Term Scheduler B. Medium-Term Scheduler C. Parallel Scheduler D. Short-Term Scheduler
11	The short-term scheduler is also known as the: A. Memory manager B. Job scheduler C. Swapper D. CPU scheduler
Fill in the blanks	
12	A _____ is responsible for selecting which process should run next from the ready queue.
True or False	
13	The long-term scheduler runs frequently to maintain CPU usage.(True/False)
Fill in the blanks	
14	The medium-term scheduler reduces memory load by _____ processes.
True or False	

15	Context switching involves saving and restoring the process state. (True/False)
----	---



5. SUMMARY

- In this topic, learners have learnt how processes communicate and synchronise their actions to perform coordinated tasks. They have understood the importance of IPC in enabling process cooperation and system efficiency.
- They have learnt that processes can exchange data by accessing a common memory region. They have seen how this model offers high-speed communication but requires careful synchronisation to avoid conflicts and how processes exchange information through structured messages without sharing memory space. They have explored synchronous and asynchronous communication methods along with their use cases and trade-offs.
- Learners will learn how the operating system selects processes for execution to optimise CPU usage and maintain system performance. They have explored various goals of scheduling, such as fairness, throughput, and response time.
- They have learnt about the roles of long-term, short-term, and medium-term schedulers in managing process flow and memory. They have understood how these schedulers coordinate to maintain system stability and resource efficiency.

6. GLOSSARY

Inter-Process Communication (IPC)	- Mechanisms that allow processes to communicate and synchronise their actions when running concurrently.
Shared Memory	- An IPC method where multiple processes access a common memory region for data exchange.
Message Passing	- An IPC technique where processes send and receive structured messages through the operating system or middleware.
Synchronous Communication	- A type of message passing where the sender and receiver must both be active during the communication
Asynchronous Communication	- A type of message passing where the sender and receiver do not need to be active at the same time
Race Condition	- A situation where the outcome of a process depends on the sequence or timing of other uncontrollable events.
Semaphore	- A synchronisation tool used to control access to shared resources and prevent race conditions
Mutex (Mutual Exclusion)	- A locking mechanism that ensures only one process can access a critical section at a time.
Scheduler	- A synchronisation tool used to control access to shared resources and prevent race conditions.
Long-Term Scheduler (Job Scheduler)	- Manages the admission of new processes into the system from the job queue.
Short-Term Scheduler (CPU Scheduler)	- Selects from the ready queue which process should run next on the CPU.

Medium-Term Scheduler (Swapper)	- Temporarily removes processes from memory to control load and prevent thrashing.
Process Scheduling	- The method of assigning CPU time to various processes to achieve efficiency and responsiveness.
Context Switching	- The process of saving and restoring the state of a CPU so that multiple processes can share a single CPU resource.
Ready Queue	- A list of processes that are loaded into memory and waiting to use the CPU.
Blocked Process	- A process that is waiting for an event such as I/O completion and cannot proceed until it occurs..
Swapping	- The act of moving a process between main memory and secondary storage to manage memory usage.
Thrashing	- A condition where excessive paging or swapping activity prevents useful work from being done.system

7. TERMINAL QUESTIONS

1. What is Inter-Process Communication (IPC) and why is it important?
2. State any two differences between shared memory and message passing models.
3. List any two benefits of using shared memory for IPC.
4. What is a context switch in process scheduling?
5. What role does the medium-term scheduler play in process management?
6. Explain the shared memory model in IPC with its advantages and drawbacks..
7. Describe synchronous and asynchronous message passing with examples.
8. Discuss the goals of process scheduling in operating systems.
9. Compare the functions of long-term, short-term, and medium-term schedulers.
10. Explain how message passing works in a distributed system and mention its challenges.

8. TERMINAL ANSWERS

1. Refer to Section 2 – Inter-Process Communication
2. Refer to Section 2 – IPC Models
3. Refer to Section 2.1 – Shared Memory Model
4. Refer to Section 4 – Short-Term Scheduler
5. Refer to Section 4 – Medium-Term Scheduler
6. Refer to Section 2.1 – Shared Memory Model
7. Refer to Section 3 – Message Passing Model
8. Refer to Section 4 – Process Scheduling
9. Refer to Section 4 – The Schedulers
10. Refer to Section 3 – Message Passing Model

9. ANSWERS

Self-Assessment Questions

1. C. Race conditions
2. Shared memory
3. False
4. C. Distributed systems
5. B. Sockets
6. send, receive
7. False
8. active
9. True
10. C. Parallel Scheduler
11. D. CPU scheduler
12. Short-term scheduler (or CPU scheduler)
13. False
14. swapping out (or suspending)
15. True

10. REFERENCES

- **Silberschatz, A., Galvin, P. B., & Gagne, G.** (2018). *Operating System Concepts* (10th Edition),Wiley.
- **Tanenbaum, A. S., & Bos, H.** (2015). *Modern Operating Systems* (4th Edition),Pearson.
- **Stallings, W.** (2018). *Operating Systems: Internals and Design Principles* (9th Edition),Pearson.

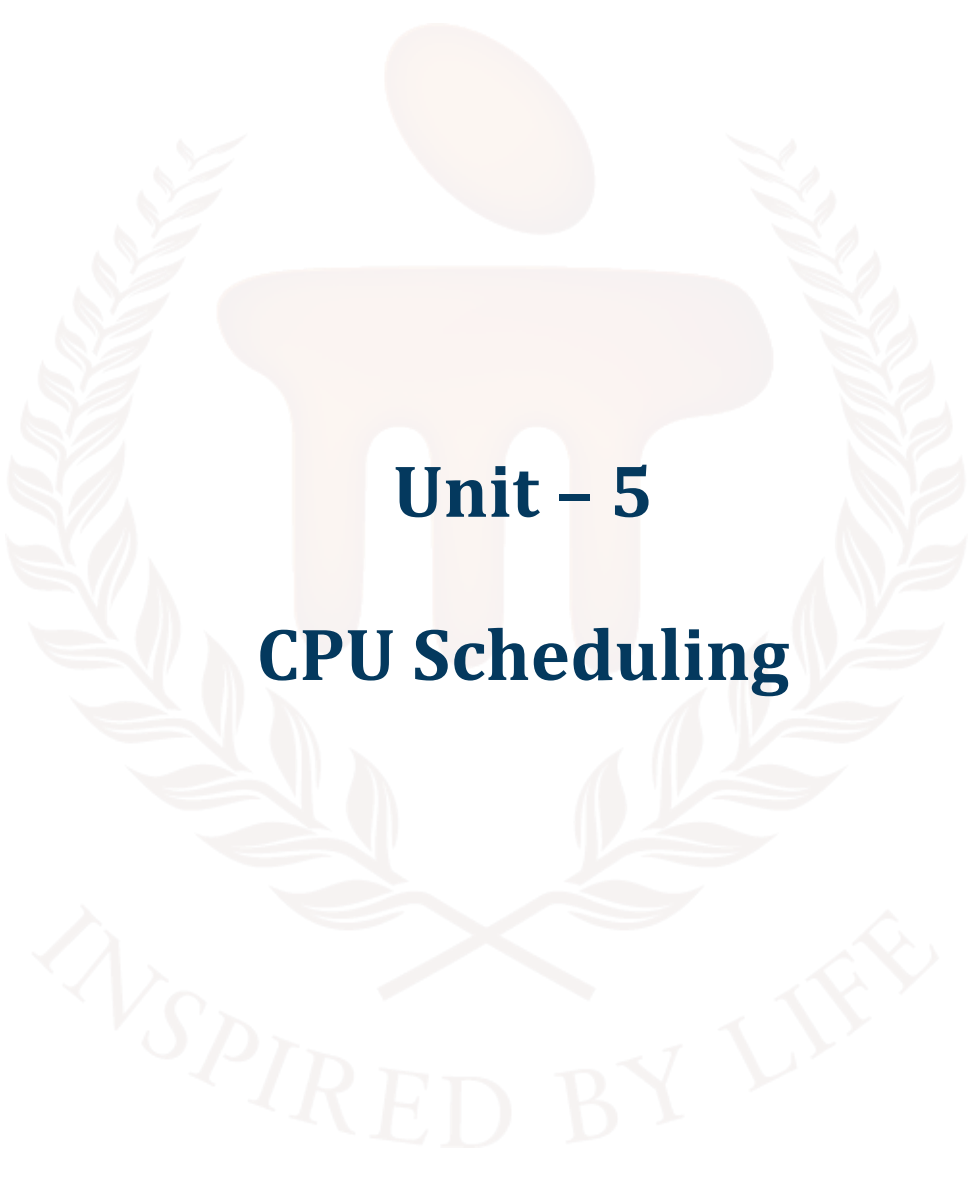


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3

DCA2108

OPERATING SYSTEMS



Unit – 5

CPU Scheduling

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Objectives	-	-	
2	CPU Scheduler	-	-	5 - 10
2.1	Scheduling criteria	-	1	
3	Scheduling Algorithms	-	-	11 - 30
3.1	First-Come, First-Served (FCFS)	-	-	
3.2	Shortest Job First	-	-	
3.3	Priority Scheduling	-	-	
3.4	Round Robin Scheduling	-	-	
3.5	Scheduling for Multilevel Queue	1	-	
3.6	Multilevel Feedback-Queue Scheduling	2	2	
4	Summary	-	-	31
5	Glossary	-	-	32 - 33
6	Terminal Questions	-	-	34
7	Answers	-	-	35 - 36
8	References	-	-	37

1. INTRODUCTION

In this unit, learners will gain a solid understanding of CPU scheduling, a critical component of operating systems responsible for managing how processes share the processor. The unit begins by examining the role of the CPU scheduler and the key criteria used to evaluate scheduling effectiveness, such as CPU utilisation, throughput, turnaround time, waiting time, and response time.

Learners will then explore a variety of scheduling algorithms, each suited for different scenarios and workloads. These include First-Come, First-Served (FCFS), which processes tasks in the order they arrive, and Shortest Job First (SJF), which gives priority to tasks that require the least processing time. Priority Scheduling introduces the concept of ranking processes, while Round Robin Scheduling is designed for time-sharing systems and ensures that each process receives a fair share of CPU time.

The unit also covers more advanced techniques, such as Multilevel Queue Scheduling, where processes are grouped based on characteristics and managed separately, and Multilevel Feedback-Queue Scheduling, which dynamically adjusts queues to improve responsiveness and fairness. By the end, learners will be able to compare and analyse these algorithms, understanding their strengths, weaknesses, and practical applications in real-world systems.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the purpose of scheduling processors in an OS.
- Explain the different types of CPU-based scheduling algorithms
- Explain how scheduling decisions are impacted by scheduling factors
- Use Gantt charts to demonstrate how various scheduling algorithms operate.
- Determine the typical wait and turnaround times for straightforward scheduling issues.



2. CPU SCHEDULER

When the central processing unit (CPU) is not actively working on a task, the operating system has the responsibility of choosing one of the waiting programs to begin execution. This decision-making task is carried out by a specific component of the operating system known as the short-term scheduler, which is often called the CPU scheduler. Its main job is to pick one process from a list of processes that are prepared and waiting in memory, and assign the CPU to that process so it can start running.

The ready queue, containing waiting processes, may not always follow the order of arrival. Depending on the scheduling strategy, its structure can vary (e.g., FIFO queue, priority queue, tree, or linked list). Depending on the scheduling strategy being used, the ready queue may take several different forms. It might be structured like a tree, organised according to priority levels, arranged as a simple FIFO queue, or even set up as an unordered linked list with no strict arrangement.

Regardless of the method used to organise it, the key idea remains the same: each process in the ready queue is waiting for its opportunity to use the CPU. The operating system keeps track of these waiting processes using data structures called process control blocks (PCBs). These PCBs store important information about each process, and they are what actually make up the entries in the ready queue.

Preemptive Scheduling

The operating system needs to make decisions about which process should be allowed to use the CPU, and these decisions are triggered by specific events. There are four primary circumstances where such decisions are made:

1. When a process voluntarily gives up the CPU because it has requested input/output operations or is waiting for one of its child processes to finish execution. In this case, the process changes from the running state to the waiting state.
2. When a process is interrupted—possibly by a hardware signal or an internal timer—it stops running and moves back to the ready state. This means the operating system must decide which process to run next.
3. When a process has been waiting for a resource, such as the completion of an input/output operation, and that wait ends. At that point, the process shifts from the waiting state back into the ready state, making it eligible for execution once more.

4. When a process finishes all of its tasks and completes execution, it exits the running state. The system must then choose a new process to assign to the CPU.

Based on how and when these decisions are made, scheduling techniques can be broadly grouped into two types: nonpreemptive (also called cooperative) and preemptive.

In nonpreemptive scheduling, once a process has been given control of the CPU, it retains control until it finishes its task or switches to the waiting state. No other process can interrupt or take over the CPU during that time. Only when the process voluntarily gives up the CPU does the scheduler get involved. This method is relatively simple and was used in older systems like Microsoft Windows 3.x.

On the other hand, preemptive scheduling allows the operating system to interrupt a process and replace it with another process—even if the first one has not finished or moved to the waiting state. This approach became a standard feature starting with Windows 95 and continues to be part of all newer Windows versions. Preemptive scheduling makes it easier to ensure that no single process keeps the CPU for too long, allowing for better responsiveness and improved system performance, especially in multi-tasking environments.

Some computer systems only support cooperative scheduling due to hardware limitations, as preemptive scheduling requires timer hardware that may not be present in all systems. These timers are crucial for the operating system to regain control from a process without waiting for it to finish voluntarily.

Despite the advantages of preemptive scheduling, it introduces certain challenges, especially when it comes to shared resources. To illustrate, consider two processes that are working with the same data. If the first process is in the middle of updating the shared data and gets interrupted, the second process might try to read the data before the update is complete. This could result in reading incorrect or inconsistent information. Such situations can lead to unexpected behaviour or errors unless the operating system or the processes themselves use careful methods—such as synchronisation mechanisms—to manage access to shared data safely.

Dispatcher

In the broader task of managing how the CPU is allocated among different processes, one important component is the dispatcher. This is a specialized module within the operating system that plays a key role once the short-term scheduler has selected which process should run next.

Once a process is chosen to receive control of the CPU, the dispatcher steps in to actually carry out that transition. The dispatcher is responsible for transferring CPU control to the selected process and preparing it for execution or resumption. This transition from one process to another involves several steps, all of which are critical to maintaining system stability and performance.

The tasks performed by the dispatcher include:

- **Switching the context:** This means saving the current state of the outgoing process (such as the contents of CPU registers and program counter) and loading the saved state of the incoming process. This ensures that the new process continues exactly from where it last left off.
- **Changing to user mode:** The operating system operates in different modes, such as kernel mode and user mode. Before the selected process can begin running, the dispatcher switches the CPU to user mode so that the process can safely execute without directly interfering with system-level operations.
- **Jumping to the correct location in the user program:** The dispatcher ensures that the CPU starts executing the new process from the right point. This may be the beginning of the process or wherever it was interrupted or paused last time.

Since the dispatcher is activated every time the CPU switches from one process to another, it is important that it works efficiently. The amount of time it takes for the dispatcher to stop the currently running process and start the next one is called dispatch latency. This time delay should be kept as short as possible because it affects the responsiveness of the system. If the dispatch latency is too long, it can reduce overall performance and make the system feel slower, especially when many processes are competing for CPU time.

2.1. Scheduling Criteria

To manage how processes are selected for execution, the operating system's short-term scheduler must consider a number of key factors. These factors, called scheduling criteria, guide the scheduler in choosing which process should be given access to the CPU next. The importance of each criterion may vary depending on what the system aims to achieve. Some systems are built for speed and responsiveness, while others prioritise fairness or maximising the number of tasks completed within a given time.

The main criteria used in scheduling decisions:

1. Process Priority

Every process can be assigned a priority level, which indicates how urgently it needs the CPU compared to other processes. The idea is straightforward: processes with a higher priority value are selected before those with lower priorities.

- Priorities can be fixed (static) at the time the process is created, or they can change over time (dynamic) depending on the process's behavior or how long it has been waiting.
- This method is especially helpful in systems where certain tasks, such as real-time operations or essential operating system functions, must be executed without delay.
- A key challenge in priority-based scheduling is starvation. This happens when low-priority processes are kept waiting for long periods because higher-priority processes continuously take precedence.
- To prevent starvation, techniques such as aging are used. In aging, the priority of a waiting process is gradually increased over time, making it more likely to be chosen eventually.

2. Burst Time (Required CPU Time)

Another important criterion is the CPU burst time—the amount of time a process will need the CPU before it either finishes its task or pauses to perform an input/output operation.

- Scheduling strategies often favour processes with shorter burst times. This approach can significantly improve the number of processes completed in a given time frame, known as system throughput.
- Giving preference to shorter tasks also helps reduce the overall waiting time for other processes in the queue.
- For example, the Shortest Job Next (SJN) algorithm uses this idea by always choosing the process that needs the least amount of CPU time. This minimises the average time each process takes to complete its execution.
- While efficient, this method can be difficult to implement accurately, since exact burst times are not always known in advance.

3. Waiting Time

This refers to the total time a process spends waiting in the ready queue before it is assigned the CPU.

- An effective scheduler works to keep the average waiting time as low as possible across all processes.

- Lower waiting times lead to better performance, especially in batch processing systems, where large numbers of tasks are submitted for execution at once.
- Reducing waiting time also contributes to fairness, as it ensures that no process is left waiting unnecessarily while others continue to receive attention.

4. Response Time

Response time measures how quickly a process begins to execute once it becomes ready.

- Specifically, it is the time between a process entering the ready queue and the moment it starts executing on the CPU for the first time.
- Response time is particularly important in interactive and time-sharing systems, where immediate feedback is expected.
- Schedulers that are designed to minimise response time create a more responsive and user-friendly system.
- Unlike waiting time, which covers the entire time spent waiting throughout the process's life, response time focuses only on the initial delay before the process starts.

These scheduling criteria help the operating system decide which process to run at any given time. By carefully balancing these factors—such as prioritising urgent tasks, reducing delays, and ensuring fair treatment—the scheduler can improve both system efficiency and user satisfaction.

SELF-ASSESSMENT QUESTIONS - 1

Fill in the Blank	
1	The dispatcher is responsible for _____ from one process to another.
True or False	
2	In non-preemptive scheduling, once a process starts executing, it can be interrupted by another process. (True/False)
Multiple Choice Questions	
3	Which of the following is not a CPU scheduling criterion? a) Throughput b) Burst Time c) Waiting Time d) File Size

4	Which scheduler selects the next process to run from the ready queue? a) Long-term scheduler b) Short-term scheduler c) Medium-term scheduler d) Disk scheduler
Fill in the Blank	
5	_____ scheduling allows a running process to be interrupted and replaced by another.
True or False	
6	The operating system only calls a dispatcher when it first boots up. (True/False)



3. ALGORITHMS FOR SCHEDULING

The operating system's short-term scheduler is responsible for deciding which process should receive access to the CPU next. This decision is guided by a scheduling algorithm. Each algorithm is designed to improve different aspects of system performance—some aim to ensure fair treatment of all processes, while others focus on speed, system responsiveness, or the overall number of tasks completed efficiently.

There are two primary ways the CPU can be allocated:

- In **non-preemptive scheduling**, once a process is assigned the CPU, it keeps control of it until it either finishes completely or voluntarily stops, usually by waiting for input or another event.
- In **preemptive scheduling**, the operating system has the ability to take the CPU away from a currently running process and assign it to another process, often based on priority or time constraints.

To illustrate how this works, consider two processes: P1 and P2. If P2 becomes ready to run while P1 is already executing, the system checks which one has higher priority. If P1's priority is greater than P2's, it continues to run. If P2 has a higher priority, P1 is temporarily halted, and the CPU is assigned to P2.

Let us now explore one of the most widely used and fundamental scheduling methods:

3.1. First-Come, First-Served (FCFS)

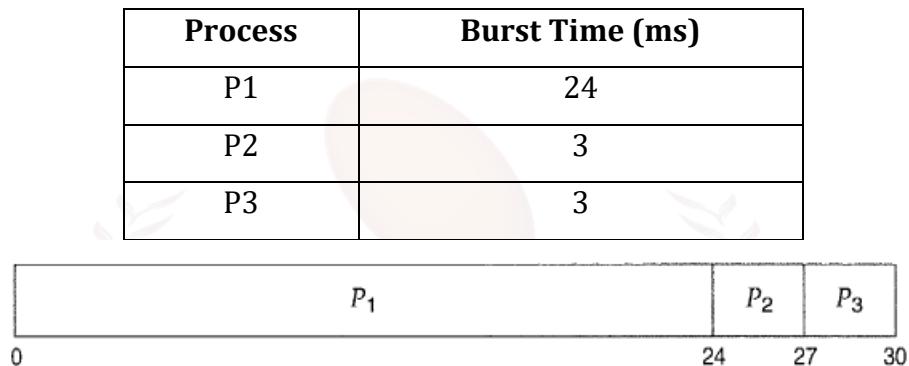
The First-Come, First-Served (FCFS) scheduling technique is the simplest form of CPU scheduling. In this method, the process that requests the CPU first is the one that receives it first. It operates much like a queue at a service counter: the first process to enter the queue is the first one served.

- This method is usually implemented using a First-In, First-Out (FIFO) queue.
- When a process enters the ready queue, its process control block (PCB) is placed at the end.
- When the CPU becomes available, it is given to the process at the front of the queue.
- Once the CPU is assigned to a process, that process runs until it finishes or performs an I/O operation. The next process in the queue then receives the CPU.

Because of its straightforward nature, the FCFS scheduling algorithm is easy to write and understand. But this simplicity comes with some limitations—particularly in how it handles waiting time.

Example 1:

For example, consider a set of three processes that all arrive at time 0, with the following CPU burst times:



Using FCFS, the waiting times are as follows:

- P1 starts first, so it waits 0 ms
- P2 must wait for P1 to finish: 24 ms
- P3 waits for both P1 and P2: 27 ms

So, the average waiting time = $(0 + 24 + 27) / 3 = 17$ ms

If the processes arrive in a different order (P2, P3, then P1), the average waiting time becomes:



- P2 starts immediately: 0 ms
- P3 waits for P2: 3 ms
- P1 waits for both P2 and P3: 6 ms

So, the average waiting time is now $(0 + 3 + 6) / 3 = 3$ ms, which is much better.

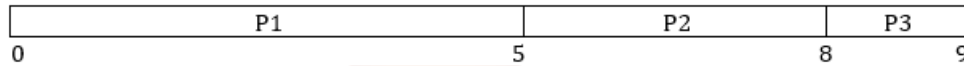
This example shows how the arrival order in FCFS can greatly affect performance.

Example 2: FCFS in Action with Arrival Times

Process	Arrival Time	Burst Time
P1	0 ms	5 ms
P2	2 ms	3 ms
P3	4 ms	1 ms

The order in which the processes are scheduled and executed:

- P1 arrives first and starts at 0 ms, finishes at 5 ms
- P2 arrives at 2 ms, starts at 5 ms, finishes at 8 ms
- P3 arrives at 4 ms, starts at 8 ms, finishes at 9 ms



Process	Arrival	Burst	Start	Completion	Waiting Time (Start - Arrival)	Turnaround Time (Completion - Arrival)
P1	0	5	0	5	0	5
P2	2	3	5	8	3	6
P3	4	1	8	9	4	5

- Average Waiting Time = $(0 + 3 + 4) / 3 = 2.33$ ms
- Average Turnaround Time = $(5 + 6 + 5) / 3 = 5.33$ ms

The Convoy Effect in FCFS Scheduling

A significant drawback of FCFS scheduling is the convoy effect. This happens when a long-running process gets the CPU first and holds it for an extended time, forcing shorter tasks to wait.

In the following scenario:

- One process performs long CPU computations (CPU-bound), and
- Several other processes perform quick tasks followed by input/output (I/O-bound).

In such a case, the CPU-bound process takes the CPU, while the I/O-bound processes wait. During this time, the I/O devices are unused. Once the CPU-bound process completes its task and switches to I/O, the I/O-bound processes quickly run and then return to waiting. Then again, the CPU-bound process dominates the CPU.

This cycle creates a bottleneck. Short processes that could have completed quickly are delayed unnecessarily, and system resources are not used efficiently.

- **Advantages:**
 - Simple to implement
 - Equitable with regard to arrival sequence

- **Disadvantages:**

- Long waiting times for short jobs behind long ones (convoy effect)
- Poor average response and turnaround time

Use Case: Suitable for batch systems with predictable job lengths

3.2. Shortest Job First

The Shortest Job First (SJF) scheduling technique is another commonly used method in CPU scheduling. As the name suggests, this algorithm gives preference to the process that requires the least amount of CPU time to complete its next burst. When the CPU becomes available, the process with the shortest upcoming CPU burst is selected for execution.

This approach differs from others like FCFS (First-Come, First-Served), as SJF is not based on the order of arrival. Instead, it depends entirely on the estimated duration of the next CPU usage period of each process. If two processes happen to require the same burst time, the tie is resolved using the FCFS approach, giving the CPU to the process that entered the queue first.

Let's explore this scheduling technique with an example to understand how it works in practice.

Example 1: Non-Preemptive SJF Scheduling

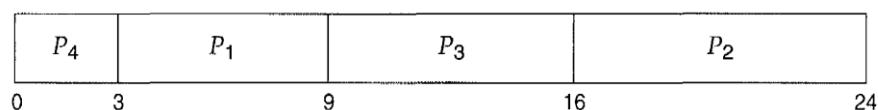
Given the following set of processes with their respective burst times:

Process	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

To apply the SJF scheduling technique:

- We begin by selecting the process with the shortest burst time, which is P4 with 3 ms.
- After P4 finishes, we choose the next shortest burst, which is P1 (6 ms), followed by P3 (7 ms), and finally P2 (8 ms).

This results in the following Gantt chart order:



Based on this order, we can compute the waiting times:

- P4 starts immediately at time 0 → waiting time = 0 ms
- P1 starts after P4 → waiting time = 3 ms
- P3 starts after P1 → waiting time = 3 + 6 = 9 ms
- P2 starts after P3 → waiting time = 3 + 6 + 7 = 16 ms

Average waiting time = $(0 + 3 + 9 + 16) / 4 = 7$ ms

This result clearly demonstrates one of the biggest strengths of SJF scheduling—it minimises the average waiting time among all processes in a batch.

This improvement is achieved because short processes are moved forward in the execution line, reducing the overall time other processes spend waiting. When shorter processes are handled earlier, they quickly leave the system, freeing up resources for others. Compared to FCFS or other scheduling methods, SJF often provides significantly lower waiting and turnaround times.

While SJF offers optimal performance in theory, applying it in practice poses challenges—particularly because the operating system usually cannot know the exact length of the next CPU burst in advance. For long-term (job-level) scheduling, especially in batch systems, users may be asked to provide an estimate of how long their job will take when submitting it. This estimate can be used to schedule jobs using the SJF principle.

Some users may intentionally enter lower values to receive quicker scheduling. But if the job runs longer than estimated, it may be terminated with a time-limit-exceeded error and must be resubmitted. This technique, although imperfect, encourages users to provide reasonably accurate estimates of burst times.

Pre-emptive SJF (Shortest Remaining Time First)

When the SJF algorithm is extended to allow preemption, it becomes known as Shortest Remaining Time First (SRTF). This version allows the scheduler to interrupt the currently running process if a newly arrived process has a shorter burst time remaining.

Let's look at an example to illustrate how this works:

Process	Burst Time (ms)	Arrival Time (ms)
P1	8	0
P2	4	1

P3	9	2
P4	5	3

Step-by-step explanation:

- At time 0, only P1 is available, so it begins execution.
- At time 1, P2 arrives. Since P2 requires only 4 ms, which is less than P1's remaining 7 ms, the CPU switches to P2.
- At time 3, P4 arrives. Since P2 is already running and needs just 1 ms more, it is allowed to complete.
- The system continues this way, always selecting the process with the shortest remaining time.

This Pre-emptive behavior leads to lower average waiting times in systems where processes arrive at different times.

Let's calculate the average waiting time for this example:

- P1 waited from time 1 to 10 → waiting time = 9 ms
- P2 started immediately at time 1 → waiting time = 0 ms
- P3 started at time 17 (arrived at 2) → waiting time = 15 ms
- P4 started at time 5 (arrived at 3) → waiting time = 2 ms

Average waiting time = $(9 + 0 + 15 + 2) / 4 = 6.5$ ms

If we used non-Pre-emptive SJF instead, the average waiting time would be 7.75 ms, showing how Pre-emptive scheduling improves performance.

Advantages:

- Efficient use of CPU
- Produces the lowest average waiting time when burst durations are known
- Improves turnaround time for smaller tasks

Disadvantages:

- Requires accurate knowledge or estimation of future CPU burst durations
- Can lead to starvation of long processes if shorter ones keep arriving

Best Suited For:

- Systems where burst times are known or can be reliably predicted (e.g., batch systems, simulations)
- Scenarios where short tasks should be handled quickly to maintain performance

3.3. Priority Scheduling

Priority Scheduling is a widely used CPU scheduling technique where each process is assigned a specific priority value. The CPU is always allocated to the process with the highest priority, ensuring that more urgent tasks are addressed first. When two or more processes share the same priority level, the system reverts to First-Come, First-Served (FCFS) scheduling to resolve the tie.

This scheduling method is a generalised form of the Shortest Job First (SJF) algorithm. In fact, SJF can be treated as a special case of priority scheduling, where the priority is inversely related to the predicted length of the next CPU burst. In this context, shorter CPU bursts are treated as higher priorities, allowing the CPU to handle shorter tasks first, reducing average waiting time.

Different operating systems use different ranges of numbers to indicate priorities. Common ranges include 0 to 7 or 0 to 4,095. The interpretation of these numbers can vary:

- Some systems treat lower numbers as higher priority
- Others treat higher numbers as higher priority

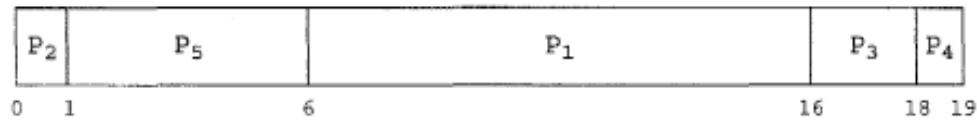
For clarity in this explanation, we will assume that smaller numbers represent higher priorities. So, a process with a priority of 1 is more important than one with a priority of 5.

Example 1: Static Priorities (All Processes Arrive at Time 0)

Process	Burst Time (ms)	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Scheduling Order Based on Priority:

We would use priority scheduling to arrange these process in accordance with the Gantt chart that follows:



- For process P₁, the waiting time is 6 milliseconds.
- for process P₂, it is 0 milliseconds.
- for process P₃, it is 16 milliseconds,
- for process P₄, it is 18 milliseconds.
- for process P₅, it is 1 milliseconds.

Therefore, $(6+0 + 16 + 18 + 1)/5 = 8.2$ milliseconds is the average waiting time.

Pre-emptive vs. Non-Pre-emptive Priority Scheduling

Priority scheduling can be implemented in two forms:

- Non-Pre-emptive Priority Scheduling:
 - Once the CPU is assigned to a process, it runs until completion or until it voluntarily gives up the CPU.
 - A new process, even with higher priority, waits until the current process completes.
- Pre-emptive Priority Scheduling:
 - If a higher-priority process arrives while another is running, the current process is interrupted and the CPU is reassigned.

Example 2: Non-Pre-emptive Priority Scheduling

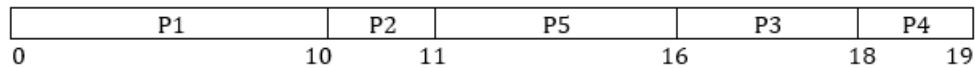
Process	Arrival Time	Burst Time	Priority
P1	0 ms	10 ms	3
P2	2 ms	1 ms	1
P3	4 ms	2 ms	4
P4	5 ms	1 ms	5
P5	6 ms	5 ms	2

Scheduling Steps:

1. At time 0, P₁ starts (only process available)
2. Other processes arrive during P₁'s execution
3. After P₁ finishes, select highest-priority from available processes (P₂)

4. Continue based on priority: P5, then P3, then P4

Gantt Chart



Calculation Table:

Process	Arrival	Burst	Priority	Start	Completion	WT	TAT
P1	0	10	3	0	10	0	10
P2	2	1	1	10	11	8	9
P5	6	5	2	11	16	5	10
P3	4	2	4	16	18	12	14
P4	5	1	5	18	19	13	14

- Average Waiting Time = $(0 + 8 + 5 + 12 + 13) / 5 = 7.6$ ms
- Average Turnaround Time = $(10 + 9 + 10 + 14 + 14) / 5 = 11.4$ ms

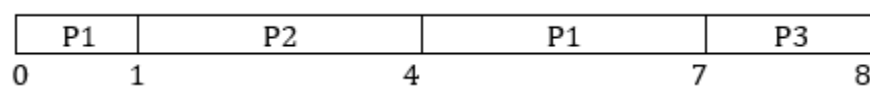
Example 3: Pre-emptive Priority Scheduling

Process	Arrival Time	Burst Time	Priority
P1	0 ms	4 ms	2
P2	1 ms	3 ms	1
P3	2 ms	1 ms	3

Execution Steps:

1. At time 0 → P1 starts
2. At time 1 → P2 arrives with higher priority → P1 is preempted
3. P2 finishes → P1 resumes
4. P3 arrives but has lower priority, waits till P1 completes

Gantt Chart:



Calculation Table:

Process	Arrival	Burst	Priority	Start	Completion	WT	TAT
---------	---------	-------	----------	-------	------------	----	-----

P1	0	4	2	0/4	7	3	7
P2	1	3	1	1	4	0	3
P3	2	1	3	7	8	5	6

- Average Waiting Time = $(3 + 0 + 5) / 3 = 2.67$ ms
- Average Turnaround Time = $(7 + 3 + 6) / 3 = 5.33$ ms

Problem of Starvation

One major drawback of priority scheduling is starvation. This occurs when low-priority processes remain in the ready queue for long periods because higher-priority processes keep arriving and preempting them. If this situation continues indefinitely, some processes may never receive CPU time, leading to inefficiencies and incomplete tasks.

Aging: A Solution to Starvation

To overcome starvation, many systems implement a technique called aging. In this method:

- A process's priority is gradually improved the longer it waits.
- For example, a system might increase a waiting process's priority by 1 every 15 minutes.
- In a system with priority values from 127 (lowest) to 0 (highest), even a process starting at 127 will eventually reach 0 if it waits long enough.
- This ensures that every process eventually gets executed, regardless of its initial priority.

Advantages:

- Prioritises important or time-sensitive tasks
- Can handle both system-level and user-defined importance

Disadvantages:

- Risk of starvation for lower-priority tasks
- Requires aging to prevent indefinite blocking

Best Use Cases:

- Real-time systems where task urgency varies
- Scenarios where certain users or processes need faster processing based on importance or cost

3.4. Round Robin Scheduling

Round Robin (RR) scheduling is a CPU scheduling approach specifically designed to support time-sharing systems, where multiple users or processes require the CPU in a way that feels equitable and responsive. This method ensures that every process gets a fair turn by allocating CPU time in fixed-size intervals, called time quanta or time slices.

The concept of fairness is central to RR scheduling. It builds upon the First-Come, First-Served (FCFS) principle, but it introduces preemption to allow the CPU to be reallocated at regular intervals, ensuring no single process dominates CPU usage for too long.

Time Quantum and Ready Queue Organisation

- A time quantum is a fixed duration (commonly between 10 to 100 milliseconds) during which a process is allowed to run on the CPU.
- The ready queue is structured like a circular FIFO queue, where each process waits its turn.
- When the CPU becomes available, the scheduler picks the first process in the queue and starts a timer.
- If the process finishes before the time quantum ends, it voluntarily relinquishes the CPU.
- If the process does not complete within the time quantum, the timer triggers an interrupt, the process is preempted, and it is placed at the end of the queue.
- The next process in line is then dispatched by the scheduler.

This rotation continues until all processes are completed.

Behaviour During CPU Burst

There are two possible outcomes when a process is running:

1. The CPU burst is shorter than the time quantum:

- The process completes or requests I/O before the time expires.
- The scheduler immediately moves on to the next process in the ready queue.

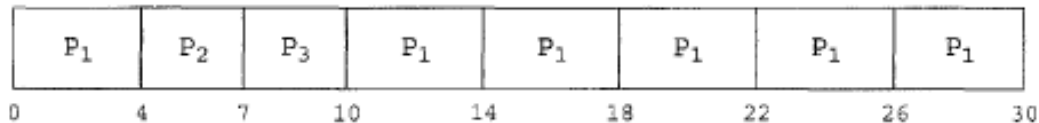
2. The CPU burst exceeds the time quantum:

- The timer stops the process.
- The operating system saves the process state.
- The process is placed at the end of the ready queue, and the CPU is reassigned.

Example 1: Simple Round Robin Execution

Process	Burst Time (ms)
P1	24
P2	3
P3	3

Time Quantum = 4 ms

**Execution Steps:**

- P1 runs first for 4 ms → remaining burst = 20
- P2 runs next and finishes within 3 ms → no re-queueing
- P3 also completes within 3 ms
- After all processes have had one turn, P1 is given the CPU again until it completes in subsequent time slices

Waiting Time Calculations:

- P1 = Total time waited before each turn: $(10 - 4) = 6$ ms
- P2 = Waited for 4 ms before execution
- P3 = Waited for 7 ms before execution
- Average Waiting Time = $(6 + 4 + 7) / 3 = 5.66$ ms

Example 2: Round Robin with Arrival Times

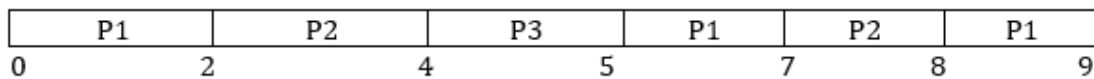
Process	Arrival Time	Burst Time
P1	0 ms	5 ms
P2	1 ms	3 ms
P3	2 ms	1 ms

Time Quantum = 2 ms

Gantt Chart Timeline:

- P1 starts at 0 and runs for 2 ms → 3 ms remaining
- P2 and P3 have now arrived
- P2 runs for 2 ms → 1 ms remaining
- P3 runs for 1 ms → completes
- P1 runs again for 2 ms → 1 ms remaining
- P2 completes its final 1 ms
- P1 completes with its final 1 ms

Gantt Chart



Completion Details:

Process	Arrival	Burst	Completion	Turnaround Time (TAT)	Waiting Time (WT)
P1	0	5	9	$9 - 0 = 9$	$9 - 5 = 4$
P2	1	3	8	$8 - 1 = 7$	$7 - 3 = 4$
P3	2	1	5	$5 - 2 = 3$	$3 - 1 = 2$

- Average Waiting Time = $(4 + 4 + 2) / 3 = 3.33$ ms
- Average Turnaround Time = $(9 + 7 + 3) / 3 = 6.33$ ms

Considerations for Choosing the Time Quantum

The effectiveness of Round Robin scheduling greatly depends on the time quantum selected:

- If the time quantum is too short:
 - Frequent context switching occurs
 - More time is spent switching between tasks than executing them
 - System overhead increases, leading to inefficiencies
- If the time quantum is too long:
 - The scheduling starts behaving similarly to FCFS
 - Processes may monopolise the CPU, leading to delays for others
 - The key benefit of responsiveness is lost

Thus, a balance must be maintained. Ideally, the time quantum should be slightly greater than the average CPU burst time of processes in the system.

Key Benefits of Round Robin Scheduling

- **Fairness:** All processes are treated equally, each getting a turn on the CPU.
- **Responsiveness:** Especially suitable for interactive and user-facing applications that require immediate CPU attention.
- **Simplicity:** Easy to implement using a circular queue.

Drawbacks of Round Robin Scheduling

- Performance varies with time quantum:
 - If not chosen wisely, it can either waste CPU cycles (due to overhead) or cause delays (if too large).
- Longer waiting time for processes with longer burst times, especially if the ready queue is heavily loaded.
- Not always optimal for minimising average waiting or turnaround time.

Suitable Scenarios

- Round Robin is well-suited for time-sharing systems, multi-user operating environments, and interactive programs.
- It ensures quick response to user inputs and prevents starvation, since all processes eventually receive CPU time.

3.5. Scheduling for Multilevel Queues

In some operating system environments, processes can be naturally grouped into distinct categories based on behavior, purpose, or resource needs. For example, interactive user processes that demand quick responses differ significantly from batch jobs that run in the background without immediate interaction. Since these groups of processes have different execution requirements, a single scheduling algorithm may not be suitable for all of them. To address this, a multilevel queue scheduling approach is adopted.

Under multilevel queue scheduling, the ready queue is divided into several distinct queues, each intended for a specific category of processes. Every process is permanently assigned to one of these queues when it enters the system, based on a defined attribute such as:

- Process type (interactive or batch)
- Memory usage
- Priority level
- Source (user-level or system-level)

Each queue has its scheduling algorithm, which matches the needs of its process category. For example:

- A foreground queue (used for interactive jobs) may apply Round Robin (RR) scheduling to ensure responsiveness.
- A background queue (used for batch processing) may use First-Come, First-Served (FCFS), since turnaround time is often more important than immediate feedback.

Inter-Queue Scheduling

Besides managing how processes within a queue are scheduled, the system must also determine which queue gets access to the CPU at any given moment. There are typically two strategies for this:

➤ Fixed-Priority Scheduling Between Queues

- Each queue is assigned a static priority.
- Higher-priority queues are always served first.
- Lower-priority queues are only considered when all higher-priority queues are empty.
- For example, a system might use the following order of priority:
 1. System processes
 2. Interactive editing jobs
 3. Regular user applications
 4. Background batch jobs
 5. Student or low-importance jobs

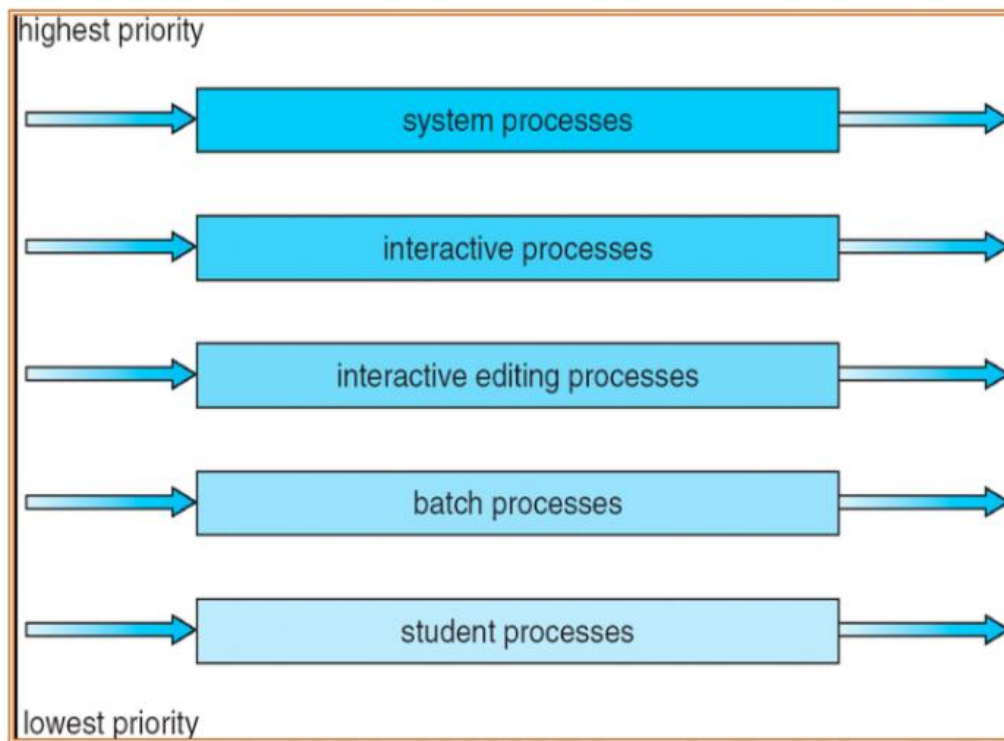


Figure 1: Scheduling for Multilevel Queues

(<https://images.velog.io/images/gabie0208/post/dc270a2f-6a8e-4f4b-ab90-f16fa411948f/%EC%8A%A4%ED%81%AC%EB%A6%B0%EC%83%B7%202021-10-16%20%EC%98%A4%ED%9B%84%208.30.49.png>)

If a new system-level process arrives while a background process is running, the background process is preempted, and the system process gets the CPU.

➤ Time-Slicing Between Queues

- The CPU time is divided across queues.
- For example, the foreground queue could be allocated 80% of CPU time (divided using Round Robin), while the background queue might receive 20% (served using FCFS).
- This ensures that no queue is completely starved, even with lower priority.

Advantages:

- The approach allows for specialisation, where different scheduling strategies can be applied to different types of workloads.
- Ensures clear separation between process types, which helps manage system responsiveness.

Disadvantages:

- Rigid structure: Once a process is placed in a queue, it cannot move to another, even if its behaviour changes.
- This lack of flexibility can lead to inefficiencies or unfair delays.
- Best Use Case:
- Systems where process categories are clearly defined, such as desktops, multi-user workstations, or servers that handle a mix of real-time and background tasks.

3.6. Multilevel Feedback-Queue Scheduling

While multilevel queue scheduling clearly categorises processes, it lacks the adaptability to handle processes that change behaviour over time. This is where the multilevel feedback-queue scheduling method becomes valuable.

This scheduling technique does not permanently fix processes to a single queue. Instead, they can move between queues based on their actual CPU usage behaviour. The system learns about the nature of a process during execution and adjusts its priority accordingly.

This dynamic arrangement improves system performance by prioritising I/O-bound and interactive processes, which tend to require shorter CPU bursts and respond to users. On the other hand, CPU-bound processes, which use the processor heavily without performing frequent I/O, are gradually moved to lower-priority queues.

Consider a setup with three queues, named Q0, Q1, and Q2:

- Q0 has the highest priority and uses Round Robin with a time quantum of 8 ms.
- Q1 is the next level, also using Round Robin but with a longer quantum, such as 16 ms.
- Q2 is the lowest priority and may use FCFS, which is reserved for long processes.

How processes move between queues:

- A new process begins in Q0.
- If it doesn't finish within the 8-ms quantum, it is moved to Q1.
- If it doesn't finish in Q1's 16 ms quantum, it is moved again to Q2.
- Once in Q2, the process is only run when Q0 and Q1 are empty and are served on a first-come, first-served basis.

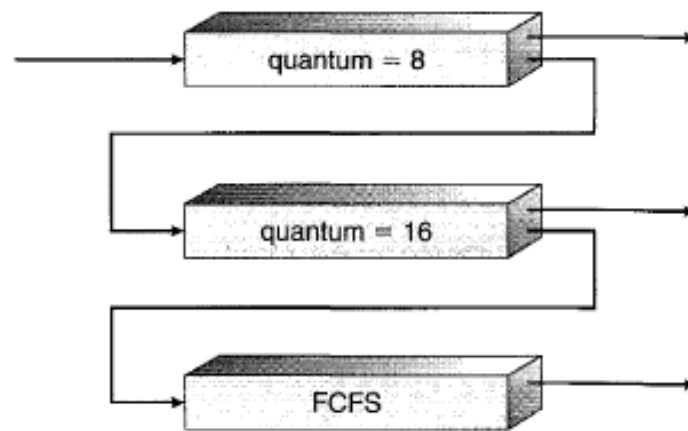


Figure 2: Multilevel feedback queue scheduling

Benefits of the Feedback Mechanism

This system brings two significant improvements:

1. **Efficiency:** Short, interactive processes are finished quickly and rarely demoted.
2. **Fairness:** Long-running processes are not starved, since aging mechanisms can eventually raise their priority again if they wait too long.

Aging ensures that no process remains stuck in low-priority queues forever. For example, if a process in Q2 waits a long time, it might be promoted to Q1, and eventually back to Q0, ensuring that it gets CPU time.

Key Parameters for Implementation

To define a multilevel feedback-queue scheduler, several parameters must be decided:

- **Number of queues:** How many levels of priority are needed?
- **Scheduling algorithm for each queue:** Should it be RR, FCFS, or something else?
- **Demotion rule:** How does the system decide when a process should move to a lower queue?
- **Promotion rule (aging):** When should a process be moved to a higher-priority queue?
- **Initial queue placement:** How does the system assign a process to a queue when it first arrives?

Example:

Imagine the following scenario:

- A process with a CPU burst of 5 ms enters Q0 and completes within one quantum. It remains in Q0.
- Another process has a burst of 20 ms. It uses up its quantum in Q0, gets demoted to Q1, and still doesn't finish. It is then moved to Q2.
- Since Q0 and Q1 get priority, the long-running process in Q2 may experience delays. But if it waits too long, the aging policy may bump it up to Q1 or Q0 again.

Advantages:

- Highly flexible and adaptable to different types of processes.
- Improves system responsiveness and avoids starvation through aging.
- Mimics real-world behavior by dynamically adjusting to process needs.

Disadvantages:

- Complex to configure: Requires careful tuning of parameters.
- Determining optimal queue transitions is difficult and may require experimentation.
- More overhead than simpler algorithms due to monitoring and queue shifting.

Best Use Case:

- Systems with a wide variety of process types and unpredictable workloads, such as general-purpose operating systems, interactive systems, and multitasking servers.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
7	Which of the following scheduling methods works best with time-sharing systems? a) FCFS b) SJF c) Priority Scheduling d) Round Robin
8	Which scheduling mechanism offers the process that arrives first access to the CPU? a) Priority Scheduling b) FCFS c) Round Robin d) Multilevel Feedback Queue

Fill in the Blanks	
9	In Shortest Job First scheduling, the process with the _____ CPU burst time is selected next.
Multiple Choice Questions	
10	What kind of scheduling algorithm is not Pre-emptive ? a) Round Robin b) Pre-emptive Priority Scheduling c) FCFS d) Multilevel Feedback Queue
True or False	
11	The convoy effect is a problem in FCFS scheduling where long processes block shorter ones.
Fill in the Blank	
12	Starvation can occur in _____ scheduling when low-priority processes are never selected.
Multiple Choice Questions	
13	Which algorithm can cause starvation if aging is not applied? a) Round Robin b) SJF c) Priority Scheduling d) FCFS
Fill in the Blank	
14	_____ is the process of progressively raising a waiting process's priority in order to avoid hunger.
Multiple Choice Questions	
15	The amount of time allotted to each procedure in round robin scheduling is known: a) Burst Time b) Turnaround Time c) Time Quantum d) Response Time

4. SUMMARY

- In this, learners have learnt that the CPU scheduler is responsible for selecting which process from the ready queue should get CPU time when the CPU becomes idle. They have also understood that processes are chosen based on scheduling strategies, and the ready queue can take various forms beyond a simple FIFO order.
- About key decision-making factors like priority, burst time, waiting time, and scheduler response time. They have also recognised that balancing these criteria is essential for achieving fairness, speed, and overall system performance.
- The FCFS scheduling assigns the CPU in the order in which processes arrive, much like a basic queue. They have also seen how this can cause long waiting times for short jobs if long ones are ahead, leading to the convoy effect.
- The SJF prioritises the process with the shortest CPU burst, reducing average waiting and turnaround times. They have also discovered that the method requires accurate estimation of burst durations, which is often difficult to predict.
- The processes are scheduled based on their priority levels, with higher-priority processes receiving CPU time first. They have also explored issues like starvation and how aging helps to prevent low-priority processes from being indefinitely ignored.
- The Round Robin scheduling assigns CPU time in equal slices (time quantum) to each process in turn, promoting fairness. They have also understood that the size of the time quantum impacts performance, where too small causes overhead and too large mimics FCFS behaviour.
- The processes are assigned to different queues based on characteristics like type or priority, and each queue has its own scheduling algorithm. They have also seen that this structure enables specialisation but lacks flexibility, as processes cannot move between queues.
- The method allows processes to move between queues based on their behaviour, improving adaptability and fairness. They have also understood how this scheduling model supports short- and long-term processes using dynamic priority adjustments and aging mechanisms.

5. GLOSSARY

CPU Scheduling	- The process of selecting which process from the ready queue should be assigned to the CPU for execution.
CPU Scheduler	- A system component that chooses one process from the ready queue whenever the CPU becomes idle.
Dispatcher	- A part of the operating system that switches the CPU from one process to another, managing context switching and starting the following process.
Dispatch Latency	- The time taken by the dispatcher to stop one process and start another.
Pre-emptive Scheduling	- A scheduling method where a running process can be interrupted and replaced by another process.
Non-Pre-emptive Scheduling	- A scheduling method is one where a running process keeps the CPU until it finishes or moves to the waiting state.
Ready Queue	- A list of processes that are waiting to be assigned to the CPU.
Burst Time (CPU Burst)	- The amount of time a process needs to use the CPU continuously before either completing or requiring I/O.
Waiting Time	- The total time a process spends in the ready queue waiting for CPU allocation.
Response Time	- The time between a process's request for CPU and the first time it is allocated the CPU.
First-Come, First-Served (FCFS)	- A simple scheduling algorithm that assigns the CPU to processes in the order they arrive.
Shortest Job First (SJF)	- A scheduling method that selects the process with the shortest CPU burst time to run next.
Priority Scheduling	- A CPU scheduling method that assigns the CPU based on process priority.

Aging	- A technique used to prevent starvation by gradually increasing the priority of waiting processes.
Round Robin (RR) Scheduling	- A time-sharing scheduling method that assigns each process a fixed time slice for CPU execution.
Time Quantum	- The fixed time duration for which a process is allowed to use the CPU in Round Robin scheduling.
Multilevel Queue Scheduling	- A scheduling method that organises processes into multiple queues based on type or priority, each with its own scheduling policy.
Multilevel Feedback-Queue Scheduling	- A flexible scheduling technique where processes can move between queues based on their CPU usage patterns.
Starvation	- A situation where low-priority processes may never get CPU time because higher-priority processes continuously occupy the CPU.
Context Switching	- The process of saving the state of a running process and loading the state of the following process selected by the scheduler.

6. TERMINAL QUESTIONS

1. What is the role of the CPU scheduler in process management?
2. State the advantages and disadvantages of First-Come, First-Served (FCFS) scheduling.
3. What is the significance of aging in priority scheduling?
4. Compare Pre-emptive and non-Pre-emptive scheduling with suitable examples.
5. Discuss the priority scheduling method with a detailed example.
6. Explain the differences between FCFS, SJF, and Round Robin scheduling in detail.
7. What is multilevel queue scheduling and how does it handle process categories?
8. Why is it challenging to implement the Shortest Job First scheduling in real systems?
9. How does the multilevel feedback-queue scheduling prevent starvation?
10. Examine the average waiting times for the following processes using FCFS, SJF, and Round Robin scheduling.

Process	Burst Time (ms)
Process 1	6
Process 2	8
Process 3	7
Process 4	3

Solve using each algorithm and compare the waiting times.

11. Solve the following using Priority Scheduling (Pre-emptive) and calculate waiting times.

Process	Arrival Time	Burst Time (ms)	Priority
P1	0	5	2
P2	1	3	1
P3	2	4	3
P4	3	2	2

Draw the Gantt chart showing process switching based on priority.

7. ANSWERS

Self-Assessment Questions

1. Context switching
2. False (Non-preemptive scheduling does not allow interruption.)
3. File Size
4. Short-term scheduler
5. Pre-emptive
6. False (The dispatcher is invoked at every process switch.)
7. Round Robin
8. FCFS
9. shortest
10. FCFS
11. True
12. Priority
13. Priority Scheduling
14. aging
15. Time Quantum

Terminal Questions Answers

1. When the CPU becomes idle, the CPU scheduler chooses one process from the ready queue to allocate the CPU to. It guarantees effective and systematic CPU utilisation.
(Refer to Section 2 CPU Scheduler)
2. FCFS is simple and fair as it serves processes in the order of their arrival. However, it may result in long waiting times for short jobs when longer jobs arrive first, leading to the convoy effect
(Refer to Section 3.1 FCFS Scheduling)
3. Aging gradually increases the priority of waiting processes to prevent starvation of low-priority tasks.
(Refer to Section 3.3 Priority Scheduling)
4. Pre-emptive scheduling interrupts running processes for higher-priority ones, while non-preemptive scheduling allows the process to finish. Windows uses Pre-emptive scheduling, but older systems like Windows 3.x used non-preemptive. (Refer to Section 2 CPU Scheduler)

5. Priority scheduling selects processes based on assigned priority values, where higher-priority processes are executed first. Example: five processes with different priorities and burst times.
(Refer to Section 3.3 Priority Scheduling)
6. Round Robin allots time slices, SJF selects the shortest work, and FCFS serves in arrival order. Waiting time, response time, and fairness are all impacted differently by each approach. (Refer to Sections 3.1, 3.2, 3.4)
7. Multilevel queue scheduling divides processes into separate queues based on their types or priorities. Each queue uses a specific scheduling policy.
(Refer to Multilevel Queue Scheduling section 3.5)
8. SJF requires accurate prediction of CPU burst time, which is difficult as the exact duration is often unknown. Approximation methods are used instead.
(Refer to Section 3.2 Shortest Job First)
9. Processes waiting in low-priority queues gradually gain priority over time, ensuring all processes eventually receive CPU attention.
(Refer to Multilevel Feedback-Queue Scheduling section 3.6)

10. Comparison of FCFS, SJF, and Round Robin (Time Quantum = 4 ms)

FCFS Waiting Time: Order: P1 -> P2 -> P3 -> P4

Average Waiting Time = $(0 + 6 + 14 + 21) / 4 = 10.25$ ms

SJF Waiting Time: Order: P4 -> P1 -> P3 -> P2

Average Waiting Time = $(0 + 3 + 9 + 16) / 4 = 7$ ms

Round Robin Waiting Time: Detailed Gantt chart required, approximate average waiting time: 9 ms

Conclusion: SJF gives the best waiting time. (Refer to Sections 3.1, 3.2, 3.4.)

11. Pre-emptive Priority Scheduling - Average Waiting Time

Gantt Chart: P1 (0-1) -> P2 (1-4) -> P1 (4-8) -> P4 (8-10) -> P3 (10-14)

Process	Completion Time	Turnaround Time	Waiting Time
P1	8 ms	8 ms	$8 - 5 = 3$ ms
P2	4 ms	3 ms	$3 - 3 = 0$ ms
P3	14 ms	12 ms	$12 - 4 = 8$ ms
P4	10 ms	7 ms	$7 - 2 = 5$ ms

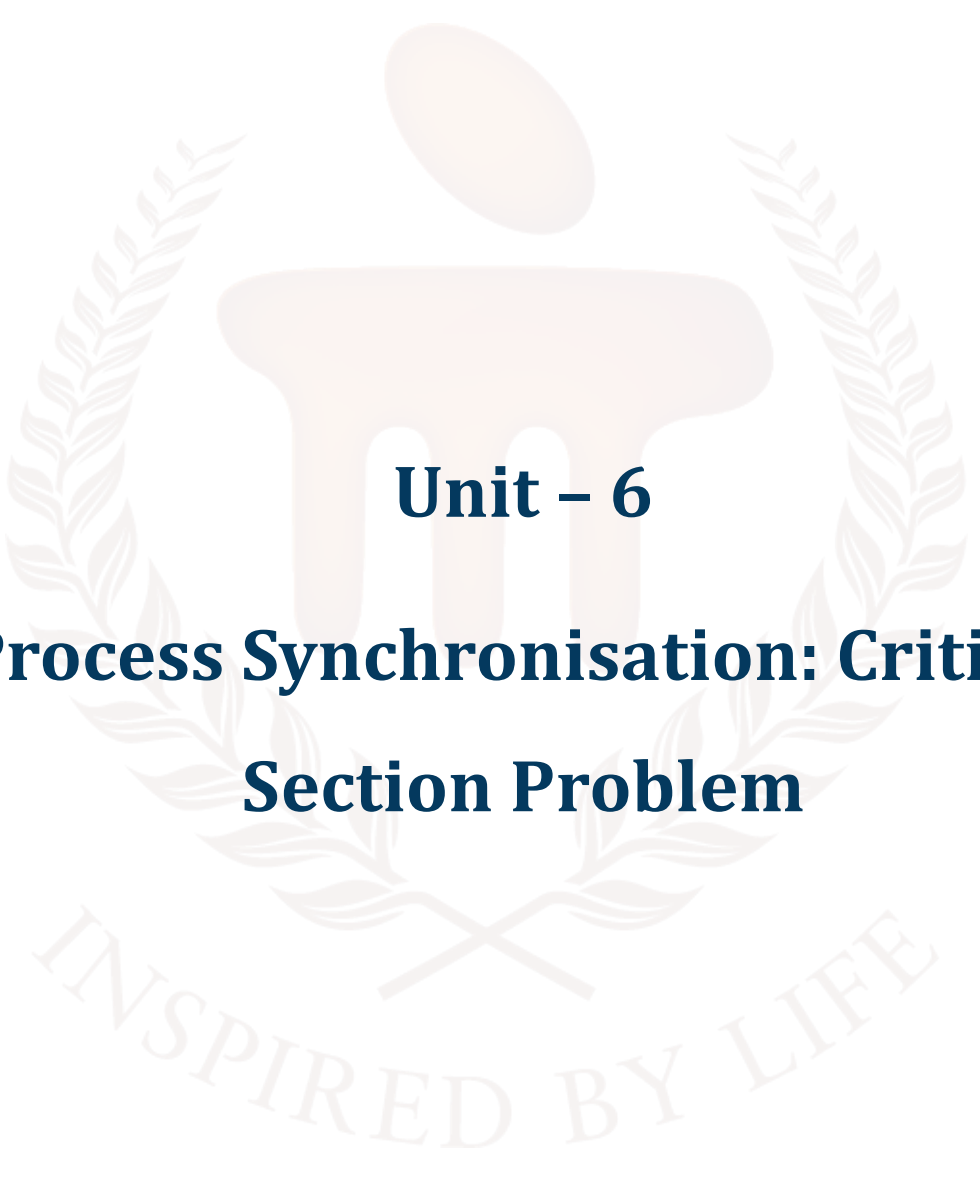
Average Waiting Time = $(3 + 0 + 8 + 5) / 4 = 4$ ms

(Refer to Section 3.3 Priority Scheduling.)

8. REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition), Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th Edition), Pearson.
3. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th Edition), Pearson.





Unit – 6

Process Synchronisation: Critical Section Problem

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Process synchronisation	-	-	6 - 13
2.1	Critical Section Problem	1	1	
3	Peterson's problem	2, 3	2	14 - 18
4	Synchronisation Hardware	4, 5, 6, 7, 8	3	19 - 26
5	Summary	-	-	27
6	Glossary	-	-	28 - 30
7	Terminal Questions	-	-	31
8	Answers	-	-	32 - 34
9	References	-	-	35

1. INTRODUCTION

In this unit, learners will understand how operating systems manage the coordination of multiple processes through process synchronisation. The unit begins with the concept of process synchronisation, which is essential for ensuring that concurrent processes or threads operate smoothly without interfering with each other. A key focus is on the critical section problem, where learners will explore how multiple processes share resources and why it's crucial to prevent race conditions when accessing shared data.

The topic then introduces Peterson's problem, a classic software-based solution to the critical section problem that demonstrates how two processes can achieve mutual exclusion using only shared variables, without relying on hardware support. This is followed by a study of synchronisation hardware, where learners will discover various hardware mechanisms and instructions—such as test-and-set and compare-and-swap—that are used to enforce mutual exclusion in modern computer systems.

Through these concepts, learners will develop the skills to analyse, design, and implement solutions for synchronising processes, understand the limitations of software-only and hardware-supported methods, and appreciate why robust synchronisation is vital for reliable and efficient system operation.

- Define key terms such as process synchronisation, critical section, race condition, deadlock, and semaphore.
- Explain the requirements that must be met by a solution to the critical section problem
- Discuss how preemptive and non-preemptive kernels differ in how they manage process synchronisation.
- Show how to solve the critical section problem by utilising synchronisation tools
- Apply Peterson's solution to a two-process synchronisation problem and trace the sequence of events leading to mutual exclusion.
- Differentiate between software-based and hardware-based synchronisation approaches.

An illustration of an open book with a glowing lightbulb above it, surrounded by various icons representing technology, business, and innovation. The lightbulb is the central focus, emitting a warm glow. Above it, a cluster of white line-art icons floats, including a rocket, a bar chart, a pie chart, a gear, a lightbulb, a dollar sign, a computer monitor, a smartphone, a magnifying glass, a target, a handshake, a lightbulb, a gear, a lightbulb, a dollar sign, a computer monitor, a smartphone, a magnifying glass, a target, a handshake, a lightbulb, a gear, a lightbulb, a dollar sign, a computer monitor, a smartphone, a magnifying glass, a target, a handshake. The background is a dark, textured surface, possibly a table or floor, with a subtle gradient of light blue and white at the top.

2. PROCESS SYNCHRONISATION

The sharing of system resources between processes in a way that manages concurrent access to shared data reduces the likelihood of inconsistent data is known as process synchronisation

To ensure data consistency, mechanisms that ensure coordinated execution of cooperating processes are required.

Uncontrolled process execution can lead to:

1. Race conditions: where the timing of events determines the system's output. When several processes access and alter the same data simultaneously and the outcome of the execution depends on the precise order in which the access takes place, this is known as a race situation. To avoid the race condition, we must ensure that only one process at a time can alter the variable counter. To achieve such a guarantee, we require some kind of process synchronisation. A race condition occurs when multiple threads or processes access shared data concurrently and the outcome depends on the sequence of access.

Example: A bank account balance is updated using two procedures:

Process A reads the balance: ₹10,000

- Process B also reads the balance: ₹10,000
- A deposits ₹1,000 → writes ₹11,000
- B withdraws ₹500 → writes ₹9,500

Expected balance: ₹10,500

Actual result: ₹9,500

Reason: This happens because the two updates were not synchronised, so the second update overwrites the first.

Solution: This can be prevented by ensuring that only one process modifies the data at a time, using synchronisation mechanisms such as mutex locks.

2. Data inconsistency: When shared resources are altered by several processes without adequate coordination, data inconsistency happens, producing erratic or distorted outcomes. This usually occurs in systems when multiple processes try to read and write data at the same time. For instance, if two processes write to the same file simultaneously, their actions may interact and cause the output

to have missing information or garbled data. Such discrepancies may jeopardise the accuracy of the data and the dependability of the system. Only one process can access crucial portions at a time thanks to synchronisation techniques like mutexes and semaphores, which guarantee atomicity and data consistency.

3. Deadlock is a condition in which a group of processes become stuck, each waiting for a resource held by another, with no process able to proceed. This results in an indefinite waiting cycle. For instance, if Process A holds Resource 1 and requests Resource 2, while Process B holds Resource 2 and requests Resource 1, both processes wait endlessly, blocking each other. Such circular dependencies halt system progress and waste resources. Systems must employ tactics such as using deadlock avoidance algorithms, keeping a strict resource allocation sequence, or putting detection and recovery procedures in place to end the waiting loop in order to prevent deadlocks.

4. Starvation: When a process is consistently refused access to a resource it needs because other processes are always taking precedence, this is known as starvation. This typically happens in scheduling systems where low-priority processes are overlooked in favour of higher-priority ones. For example, a low-priority process may never receive CPU time if the scheduler always favours high-priority tasks. Over time, this can prevent the process from making any progress. The use of equitable scheduling policies and strategies, such as priority aging, which raises a process's priority the longer it waits, can help systems avoid starvation by guaranteeing that all processes eventually receive the resources they need.

5. Busy waiting: Process synchronisation is used to solve issues that come up during concurrent process execution. It ensures orderly and efficient access to shared resources while avoiding several common issues. One such issue is busy waiting, where a process repeatedly checks if it can enter a critical section, consuming valuable CPU time even while doing no useful work. This form of polling is inefficient, especially in systems with limited processing power. The solution lies in using blocking synchronisation mechanisms, such as semaphores, which suspend a process until a specific condition is met, allowing the CPU to be allocated to other active tasks efficiently.

How is Synchronisation Achieved?

Operating systems employ a number of structures and methods to manage resource sharing and avoid conflicts:

- **Mutexes** (Mutual Exclusion Objects): To avoid concurrent access to shared data, make sure that only one process can enter a vital part at a time.

- **Semaphores:** Signalling mechanisms that manage resource availability.
 - Binary semaphores work like mutexes, allowing one process access.
 - Counting semaphores permit a certain number of processes to use a resource at once.
- **Monitors:** High-level synchronisation constructs that encapsulate shared data and the procedures that operate on it. They automatically handle mutual exclusion, simplifying concurrent programming.
- **Critical Sections:** some code segments that need to be run atomically (without interruption) in order to access common resources.

Protecting critical sections with mutexes or semaphores prevents data inconsistency and race conditions.

These synchronisation methods help the operating system maintain data integrity, avoid race conditions, and ensure orderly execution among competing processes.

2.1. Critical Section Problem

Consider a system that has n processes in it. There is a code segment in every process. This part of the code is considered essential. Take the Railway Reservation System, for example.

When two individuals from separate stations want to purchase tickets, they try to do it at the same time. The destination and the train number are same. Unfortunately, both berths are varying for the one available spot. It is also known as the critical section problem. When one process is operating in the critical area, other processes should not be allowed to enter the critical section simultaneously.

Establishing a protocol that enables the processes to cooperate is the difficult part. Each operation must request permission to enter its critical area. The portion of the code that fulfills this request is the entrance section. The important element may be followed by an exit segment. The remaining code is in the rest section.

The main challenge is to develop a protocol that ensures cooperation among processes. Each process must request permission to enter its critical section. The code's entrance section implements this request. After the crucial part, there can be an escape section. The remaining code is in the rest section.

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (1);
```

Figure 1: General Structure of a Typical Pi Process

Three crucial requirements must be met by any solution in order to properly address the critical-section problem:

1. Mutual Exclusion

The most basic prerequisite is mutual exclusion. It stipulates that only one process can run in its critical area at a time. Race situations and inconsistent data states may result from many processes entering the crucial section at the same time and altering shared resources. By enforcing reciprocal exclusion, system stability and accuracy are maintained by ensuring that shared data is accessible in a regulated and secure manner. Usually, hardware-level instructions like TestAndSet or synchronisation mechanisms like mutexes and semaphores are used to implement it.

2. Progress

When several processes are vying for access, progress makes sure the system decides fairly which process gets to enter the critical area. The system must not continuously delay the decision if no process is in its critical section at the moment and one or more processes are awaiting entry. Processes that are idle or in their non-critical (remainder) portions should not influence the decision; only those that are actively seeking entrance should. This requirement guarantees the system's responsiveness and efficiency while avoiding needless delays.

3. Bounded Waiting

Bounded waiting ensures that all processes will ultimately gain access to the critical section, thereby preventing starvation. There must be a limit on how many times other processes can access their important parts after a process has made a request before the asking process receives its turn. By preventing the situation where a lower-priority or unlucky process waits indefinitely because of ongoing preemption by others, this condition encourages fairness.

It is assumed that each process is operating at a nonzero speed. However, we cannot make any educated guesses regarding the speeds of the 11 procedures. The operating system could have a lot of kernel-mode processes active. As a result, the kernel code, which implements an operating system, is susceptible to several possible race conditions. Consider a kernel data structure that records all of the open files on the system. (either removing it from the list or adding it to it). If two processes were to open files simultaneously, separate changes to this list could result in a race condition.

Among the various kernel data structures susceptible to possible race conditions are those for memory allocation, process lists, and interrupt handling. The task of ensuring that the operating system is free of such race situations falls to kernel developers.

Multiple processes that share system resources like memory, CPU, and I/O devices must be managed by modern operating systems. In kernel mode, where processes carry out privileged instructions and have access to essential system functions, a crucial portion of this management takes place.

Preemptive kernels and nonpreemptive kernels are the two primary methods that operating systems use to control process execution in the kernel in order to guarantee system stability, responsiveness, and equity. Understanding the difference between these two kernel models is essential for appreciating how operating systems manage concurrency, responsiveness, and avoidance of race conditions and deadlocks

Preemptive Kernel

A preemptive kernel enables the suspension or interruption of an active kernel-mode process to give the CPU to another process, particularly one with a higher priority. At practically any stage of the kernel's execution, this preemption may take place.

The scheduler in a preemptive system keeps an eye on the status of the processes and can step in when:

- A higher-priority process is prepared to execute.
- The process that is executing at the moment has its time quantum expire.
- There are external occurrences, such as hardware interruptions.

The preemptive model improves CPU utilisation and ensures that important tasks are not delayed unnecessarily.

Benefits:

- **Improved Responsiveness:** A preemptive kernel is ideal for real-time systems, where timing is critical. Even if another process is running in kernel mode right now, a high-priority real-time process can take over the CPU right away.
- **Better CPU Sharing:** No single process can monopolise the CPU for an extended time, as the scheduler can intervene and distribute time fairly.
- **Support for Multitasking:** It allows smoother multitasking and more predictable responsiveness in interactive systems.

Drawbacks:

- **Increased Complexity:** It is more difficult to build and maintain preemptive kernels. Developers must use synchronisation strategies like mutexes, spinlocks, or semaphores to make sure that shared kernel data structures are adequately protected.
- **Race Conditions:** When two kernel-mode threads access the same data simultaneously (especially in multi-core systems or SMP architectures), it can lead to unpredictable behaviour.
- **Interrupt Handling Complexity:** Careful design is needed to avoid inconsistencies when interrupts cause the current process to be preempted in the middle of critical operations.

Preemptive kernels are used by operating systems such as Linux, Windows, and macOS to improve performance and real-time capabilities.

2. Non preemptive Kernel

Processes operating in kernel mode cannot be interrupted by a non-preemptive kernel. A process maintains control of the CPU after it enters the kernel until it:

- voluntarily relinquishes control (for example, by finishing a task or initiating a blocking activity).
- Enters the waiting state, such as awaiting input or output.
- Terminates.

One process at a time continues to control the kernel. Even higher priority processes have to wait for the executing process to complete its kernel-mode execution. The main draw here is the simplicity of the kernel.

Benefits:

- **Simplicity in Design:** Because only one process can run in kernel mode at a time, there's a greatly reduced risk of race conditions. Synchronisation is easier and may not require complex locking mechanisms.
- **Lower Overhead:** No need for frequent context switching within kernel mode, which saves CPU time and reduces overhead.
- **Predictable Behaviour:** Since execution order is straightforward, it can be easier to test and debug.

Drawbacks:

- **Poor Responsiveness:** A long-running kernel-mode operation can delay all other processes, including time-critical ones.
- **Risk of Starvation:** If a low-priority process holds the kernel for too long, higher-priority processes may suffer from delayed access.
- **Limited Real-Time Support:** Nonpreemptive kernels are unsuitable for real-time applications where deterministic timing is essential.
- **Blocked Resource Utilisation:** Other processes may remain idle while waiting, leading to inefficient CPU usage.

Earlier versions of operating systems like older Unix systems and some embedded Oses used non-preemptive kernels due to their simplicity and predictability.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is not a requirement for solving the critical section problem? A) Mutual Exclusion B) Progress C) Starvation D) Bounded Waiting
2	What is the main drawback of busy waiting? A) It increases memory usage B) It leads to deadlocks C) It consumes CPU cycles doing no useful work

	D) It blocks I/O devices
3	Which of the following techniques prevents starvation by increasing process priority over time? A) Race condition B) Priority inversion C) Priority aging D) Pre-emption
Fill in the Blank	
4	_____ is a classical synchronisation problem in which multiple processes try to book a single train seat at the same time.
Multiple Choice Questions	
5	Monitors provide: A) Busy waiting B) Hardware-level synchronisation C) Automatic mutual exclusion D) Starvation protection
6	Which kernel type allows a running process to be interrupted in kernel mode? A) Cooperative kernel B) Preemptive kernel C) Non-preemptive kernel D) Static kernel

3. PETERSON'S PROBLEM

The critical section problem is one of the core problems in concurrent programming. Peterson's approach provides a classical software-based solution to the critical section problem. Consider a scenario in which two processes must share a resource, such as a memory buffer, printer, or even a shared variable. However, in order to prevent data corruption or inconsistency, they shouldn't use it simultaneously. The critical section problem arises at this point.

Currently, Peterson's solution consists solely of software. This implies that no hardware-level atomic operations, such as compare-and-swap or test-and-set, are needed. It uses shared variables and is entirely logic-based. Even though it occasionally fails to function as intended due to compiler optimisations and current hardware, it remains a stunningly straightforward method of comprehending how logic alone can accomplish mutual exclusion and other features like progress and finite waiting.

Only two processes, P0 and P1, are particularly covered by Peterson's technique. We let Pj be the alternative process, where $j = 1 - i$, when we talk about process Pi (where $i = 0$ or 1). Thus, j is 1 if i is 0, and j is 0 if i is 1. As a result, the reasoning is symmetrical and simpler to write.

Two bits of information are shared by the two processes:

int turn;

boolean flag[2];

- **turn:** This specifies who gets to go into the critical part first.
- **flag[2]:** This boolean array indicates if a process wishes to proceed to its crucial phase. Therefore, if P0 wants in, flag[0] is true; if P1 wants in, flag[1] is true.

What is the Algorithm's Process?

Each process performs the following actions prior to reaching its crucial phase:

1. It indicates its interest by setting its flag to true.
2. It assigns the other process the turn variable.
3. After that, it waits in a while loop that only ends when it's their turn ($turn == i$) or the other process is not interested ($flag[j] == false$).

The code for process Pi looks like this:


```
do {  
    flag[i] = TRUE;  
    turn = j;  
    while (flag[j] && turn == j);  
    critical section  
    flag[i] = FALSE;  
    remainder section  
} while (TRUE);
```

Figure 2: Process A's structure in Peterson's solution.

Process P_i declares that if the other process wants to access the critical area, it can do so by first setting $flag[i]$ to be true and then setting $turn$ to the value j . $Turn$ will be set to both i and j at about the same moment if both processes attempt to enter at the same moment. Only one of these assignments will remain; the other will happen but be instantly replaced. The value of $turn$ ultimately determines which of the two processes can go to the critical phase first.

The correctness of this solution is now demonstrated.

In this solution, each process signals its intention to enter, and the use of the $turn$ variable ensures orderly access to the critical section. However, it waits only when the other process is interested and it's their turn.

We have to demonstrate that:

1. Mutual exclusion is ensured
2. The criteria for advancement is met.
3. The condition of bounded-waiting is satisfied.

Mutual Exclusion

Let's look at the process of mutual exclusion. If neither process is ever inside the crucial region simultaneously, we say that mutual exclusion is maintained.

Assume that P_0 and P_1 attempt to access the crucial section at the same time. Both of them have their flags set to true. Each set then switches to the other procedure. Who gets to move forward will now depend on the final value written to $turn$. Assume that P_1 is assigned $turn = 0$ last. Next, $turn == 0$.

Now examine the while loop's condition:

- For P0: $turn == 1$ and $flag[1] == true$ and $turn == 1$, so P0 must wait.
 - P1 waits if $flag[0] == true$ and $turn == 0$. Therefore, only one process can enter at a time
- This demonstrates that while the other process is stuck, only one can break the while loop and reach the vital area.

Therefore, reciprocal exclusion is maintained and neither can be in at the same time.

Progress

The property of progress comes next. This implies that the decision of who enters next cannot be put off forever if one or more processes wish to enter and no process is in the crucial area.

In Peterson's solution, the other process can move forward straight away if the first process is uninterested ($flag[j] == false$). The procedure whose turn it is will begin if both parties are interested. After the current process ends and resets its flag, the one that was not chosen will eventually have a turn. The choice is based on fair reasoning, which guarantees that the system continues to function, rather than on the indefinite hold or delay of a single process.

Bounded Waiting

This makes sure that after a certain number of times the other process reaches the critical area, a process that is waiting to enter will have a turn. To put it simply, no process should go hungry.

Let's say P_i is standing by. It will only wait because P_j is either in its vital part or is attempting to enter. P_j sets $flag[j] = false$ when it completes the essential portion. P_i can now continue. P_i does not reset turn during this period; instead, it stays in the same state as when it entered the waiting state. P_i is next after P_j finishes its turn, so there is just one waiting round required. Therefore, bounded waiting is guaranteed since P_i will enter after P_j has executed at most one critical segment.

We frequently use the following to depict the code structure of process P_i in Figure 2:

The while loop (waiting for safe entry), the critical part (doing the protected task), the exit section (setting $flag[i] = false$), the remainder section (additional regular work), the non-critical section (doing regular work), and the entry portion (setting $flag[i]$ and turn).

This structure makes it evident where each component of the reasoning is applied and reinforces the way mutual exclusion is maintained.

```

do {
    acquire lock
    critical section
    release lock
    remainder section
} while (TRUE);

```

Figure 3: Locks are used to solve the critical-section problem.

What Is Elegant About Peterson's Solution?

Because it is straightforward and purely logical; it effectively illustrates the interplay of shared memory, control flags, and order of execution; and it satisfies all three essential requirements for resolving the critical section problem—mutual exclusion, progress, and bounded waiting—Peterson's algorithm is still a textbook example even though it is no longer feasible on real processors due to problems like instruction reordering or a lack of memory visibility guarantees.

Real-World Restrictions

Compilers and CPUs in contemporary systems have the ability to rearrange instructions for efficiency. These reorderings potentially violate the presumptions Peterson's technique is based on in the absence of unique memory barriers. For this reason, hardware-supported synchronisation techniques like mutexes, semaphores, or atomic instructions are used in genuine operating systems. Nevertheless, Peterson's technique provides an effective teaching tool in a theoretical or controlled setting, such as an operating systems course or a simulation.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions

- | | |
|---|---|
| 7 | Which algorithm is a classical software-based solution to the critical section problem for two processes? |
| | A) TestAndSet |
| | B) Peterson's Algorithm |
| | C) Banker's Algorithm |
| | D) Dining Philosophers |

8	In Peterson's algorithm, what does the turn variable signify? A) Whose turn it is to finish the process B) The current lock status C) Which process is allowed to enter the critical section D) The number of processes waiting
True or False	
9	Peterson's algorithm works efficiently on modern multicore processors.
Fill in the Blanks	
10	In Peterson's algorithm, the flag array is used to indicate a process's _____ to enter the critical section.



4. SYNCHRONISATION HARDWARE

One of the main issues with concurrent processing in the context of operating systems is controlling access to shared resources. The critical-section problem occurs when several threads or processes access and modify shared data at the same time, which, if left unchecked, might produce unexpected outcomes. To prevent race situations and preserve data consistency, it is crucial to make sure that only one process uses the shared resource at a time. This is where the idea of synchronisation is useful.

Comparing Hardware and Software Solutions

Software-based approaches to this issue, including Peterson's method, which enforces mutual exclusion just by correctly sequencing code and shared variables, were previously investigated. Although these techniques offer a basic grasp of synchronisation, the intricate ways that contemporary CPUs carry out instructions—using caching, pipelining, and out-of-order execution—may make them unreliable on contemporary computer systems. Therefore, in order to guarantee reliable and effective synchronisation, real-world systems frequently rely on hardware-supported techniques.

The Basis of Synchronisation: Locks

A lock is a basic synchronisation mechanism that restricts access to the critical section, ensuring only one process enters at a time. Prior to entering the critical area, the process must acquire the lock, and upon exiting, it must release it. Race circumstances are avoided by serialising access to vital resources through the use of locks.

Implementing locks on multiprocessor systems requires hardware support to ensure that lock operations are atomic (indivisible).

Hardware's Function in Synchronisation

Locks and other synchronisation primitives can be reliably implemented thanks to the atomic instructions provided by modern hardware systems. The integrity of the shared data is maintained even in the event of interruptions or concurrent execution by several processors thanks to these atomic instructions, which function as single, uninterruptible actions.

Disabling interrupts during the execution of the critical part is a simple fix for a uniprocessor system. By turning off interrupts, the active process can finish accessing shared variables without being interrupted because it cannot be preempted. Non-preemptive kernels use this method.

The Drawbacks of Turning Off Interrupts

Disabling interrupts is not feasible for multiprocessor systems for a number of reasons, even though it works fine for single-processor systems:

1. **Disabling Broadcasting Interrupts:** Disabling interrupts on multiprocessor systems would require disabling them on all CPUs, causing delays and overhead.
2. **Timing Problems and Clock Updates:** Disabling interrupts may cause periodic operations, such as system clock updates, to be delayed, which could lead to system instability.
3. **Loss of Responsiveness:** Prolonged interrupt disabling raises the possibility of priority inversion and deadlock while also decreasing system responsiveness.

Hardware synchronisation mechanisms were created to ensure safe and effective atomic operations in light of these constraints.

Atomic Instructions for Hardware

In order to address issues with ineffective interrupt disabling and software-only synchronisation, hardware designers created atomic instructions such as TestAndSet, Swap, CompareAndSwap, and FetchAndAdd (not discussed here, but frequently used).

Without turning off interrupts, these instructions enable safe concurrent operation of shared variables.

Instructions for Test and Set

TestAndSet is one of the most basic and often used atomic instructions.

Two atomic activities are carried out by this instruction:

1. It retrieves the value from a memory location, typically a lock variable or a Boolean flag.
2. The memory location is set to TRUE.

The TestAndSet operation's pseudocode looks like this:

```
boolean TestAndSet(boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
}
```



```
    return rv;  
}
```

Figure 4: the TestAndSet () instruction.

Many modern computer systems incorporate certain hardware commands to test and modify the content of a word or to switch the contents of two words atomically, that is, as one uninterruptible unit. These special rules make it very easy to solve the critical-section problem. Instead of discussing a particular command for a single machine, we abstract the fundamental concepts that underlie various types of instructions. The TestAndSet() instruction is defined as seen in Figure 4.

- It is atomic, meaning that no other process can stop it from running.
- The process has successfully obtained the lock if the value of *target is FALSE;
- if it is TRUE, the lock has already been taken and the process needs to wait.
- Using TestAndSet for Mutual Exclusion

A shared Boolean variable lock that has been set to FALSE will be examined. Mutual exclusion is illustrated by the code structure seen in Figure 5:

```
do {  
    while (TestAndSet(&lock))  
        ; // busy wait  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```

Figure 5: Using TestAndSet for Mutual Exclusion.

In this configuration, a process iterates through a busy-wait loop until it is able to set lock to TRUE.

- It returns the lock to FALSE after completing the crucial portion, enabling access by others.

TestAndSet's drawbacks include:

- A key limitation of TestAndSet or Swap-based solutions is busy waiting: the process repeatedly checks a condition in a loop, wasting CPU cycles instead of doing useful work. The CPU is strained by this continuous testing, particularly in multiprocessor systems where several tasks may run concurrently, which results in low system performance and excessive power usage.

- Starvation is a related problem that occurs when a process keeps failing to obtain the lock because other processes keep getting it. This usually occurs in systems that lack constrained waiting or enough fairness guarantees. One process may continually obtain the lock, making other processes wait endlessly, if it executes slightly faster or has a temporal advantage. Particularly in situations with considerable competition, there is no guarantee that each process will receive an equal opportunity.
- Both starving and busy waiting make the system less responsive and equitable. Alternatives like condition variables or semaphores, which suspend processes rather than spinning them, are preferred by modern operating systems for blocking synchronisation. Particularly in multitasking contexts, this guarantees more fair access to crucial areas while saving CPU time.

Instructions for Swapping

Swap is another atomic synchronisation instruction. Swap exchanges the contents of two memory regions atomically, in contrast to TestAndSet, which changes only one variable.

```
void Swap(boolean *a, boolean *b) {  
    boolean temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Figure 6 the Swap () instruction.

Swap-based mutual exclusion

This is a typical Swap structure (Figure 7):

```
do {  
    key = TRUE;  
    while (key == TRUE)  
        Swap(&lock, &key);  
    // critical section  
    lock = FALSE;  
    // remainder section  
} while (TRUE);
```

Figure 7: Swap-based mutual exclusion.

A local variable called "key" has been set to TRUE.

- The lock and key values are swapped. The process moves into the critical part if the lock was FALSE and the key becomes FALSE.
- If not, the loop keeps going until the key == FALSE as a result of the swap.

Swap's limitations

- Busy waiting wastes CPU cycles, much like TestAndSet.
- No bound waiting: There is no limit to how long some processes can take.

Waiting Boundaries with TestAndSet

A modified algorithm (Figure 8) expands the fundamental TestAndSet implementation with more data structures to address the issue of constrained waiting (making sure each process has a turn):

Data Structures Used:

boolean waiting[n]; // array to track waiting processes

boolean lock = FALSE; // global lock

Algorithm Structure:

```
do {
    waiting[i] = TRUE;
    key = TRUE;
    while (waiting[i] && key)
        key = TestAndSet(&lock);
    waiting[i] = FALSE;

    // critical section
    j = (i + 1) % n;
    while ((j != i) && !waiting[j])
        j = (j + 1) % n;
    if (j == i)
        lock = FALSE;
    else
        waiting[j] = FALSE;

    // remainder section
```

```
} while (TRUE);
```

Figure 8: Bounded-waiting mutual exclusion with TestAndSet().

- The intention to enter the critical part is indicated by `waiting[i] = TRUE`.
- After passing the `TestAndSet` check, a process waits until it is locked.
- It determines the next eligible process by iteratively checking the waiting array after leaving the crucial region.
- To ensure fairness and bounded waiting, only one process is permitted to reset its waiting flag and continue.

We see that after a process exits its critical section, it searches the array `waiting` in the cyclic ordering $(i + 1, i + 2, \dots, n - 1, 0, \dots, i - 1)$ to demonstrate that the bounded-waiting criterion is satisfied. The process that enters the crucial section next is designated as the first one in this ordering that is in the entry section (`waiting[j] == true`). As a result, every process that is waiting to reach its critical section will do so in $n - 1$ turns. Unfortunately, it is not an easy task for hardware designers to implement atomic `TestAndSet()` instructions on multiprocessors.

Benefits:

- One important advantage is the guarantee of mutual exclusion, which prevents race conditions by limiting the number of processes that can approach the critical region at once.
- Additionally, the progress requirement is met, which means that if the critical portion is free, the system always permits some process to continue.
- Additionally, it enforces limited waiting, which ensures that no process waits endlessly and that each process receives a turn within a predictable boundary. To prevent starvation and maintain fairness, any waiting process will enter its crucial part after a maximum of $n - 1$ other entries.

Drawbacks:

- The technique uses more memory, particularly when there are a lot of processes, because it needs more room to maintain structures like the `waiting[]` array.
- Additionally, it limits adaptability in dynamic systems by relying on predetermined process IDs or the precise number of processes.
- The system must carefully monitor which processes are waiting and control cyclic ordering to ensure fairness, which further adds to the implementation's complexity. These characteristics

make constrained waiting synchronisation less appropriate for general-purpose multiprocessor systems and more appropriate for restricted settings.

Implementation Issues in Systems with Multiple Processors

Implementing atomic operations in multi-processor systems poses a number of difficulties that make synchronisation more difficult. Atomic instructions like TestAndSet and Swap are useful in single-processor systems, but it is far more difficult to guarantee their accuracy and dependability in multiprocessor settings.

- The atomicity guarantee, which stipulates that no other processor can access the memory location involved in an atomic operation until it is finished, is one of the main issues. Data corruption and race situations may result from a failure to maintain atomicity.
- Memory coherency, which deals with preserving a consistent picture of shared data across all processors, is another crucial issue. Updates made by one CPU must be immediately visible to all others since each processor may cache copies of shared variables. Inadequate coordination could result in disparate processors using outdated or inconsistent data.
- Systems use cache coherency protocols, such as MESI (Modified, Exclusive, Shared, Invalid), to overcome this issue by ensuring cache consistency. As an alternative, several architectures employ bus locking, which stops other processors from using the memory bus while crucial tasks are being performed.
- Both approaches introduce performance trade-offs: coherence protocols increase communication overhead, while bus locking increases latency. Fast and equitable synchronisation becomes more challenging as the number of processors increases, which makes hardware-based locking both essential and challenging.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Which instruction is commonly used in hardware-level synchronisation to test and set a lock atomically? A) Add and Jump B) TestAndSet C) Move and Set D) Load and Store

Fill in the Blanks

- | | |
|----|---|
| 12 | In the TestAndSet approach, the shared variable used to control access to the critical section is typically called _____. |
|----|---|

Multiple Choice Questions

- | | |
|----|---|
| 13 | Which of the following is a hardware instruction used to achieve mutual exclusion?
A) Wait
B) Signal
C) Swap
D) Fetch |
| 14 | In TestAndSet-based mutual exclusion, which condition is not guaranteed?
A) Mutual exclusion
B) Progress
C) Atomic execution
D) Bounded waiting |

Fill in the Blanks

- | | |
|----|--|
| 15 | The _____ instruction allows the contents of two variables to be exchanged atomically. |
|----|--|



5. SUMMARY

- In this topic, learners have explored the need for process synchronisation to prevent conflicts when multiple processes access shared resources. They have understood how synchronisation maintains consistency and stability in a concurrent processing environment.
- The critical section problem where multiple processes compete to access a shared resource safely. They have examined the essential requirements for any solution—mutual exclusion, progress, and bounded waiting.
- The Peterson's algorithm as a software-based method to handle the critical section problem for two processes. They have seen how shared flags and a turn variable help enforce mutual exclusion, progress, and bounded waiting.
- How hardware-level instructions like TestAndSet and Swap can be used to achieve mutual exclusion. They have recognised the trade-offs in using hardware-based approaches, especially in multiprocessor systems.
- Have learnt the operational differences between preemptive and non-preemptive kernels in managing concurrent processes. They have analysed the benefits and limitations of each approach in terms of responsiveness, complexity, and fairness.

6. GLOSSARY

Process Synchronisation	A technique used in operating systems to coordinate the execution of - processes that share resources, ensuring data consistency and avoiding conflicts.
Critical Section	A portion of code where a process accesses shared resources. Only one - process at a time should execute in its critical section to avoid race conditions.
Race Condition	A situation where multiple processes access and modify shared data - concurrently, and the final result depends on the timing of their execution.
Mutual Exclusion	A synchronisation principle that ensures only one process at a time can - enter its critical section and access shared resources.
Progress	A condition that ensures if no process is in its critical section, then a - decision must be made about which waiting process enters next, without indefinite delay.
Bounded Waiting	A requirement that limits the number of times other processes can enter - their critical sections before a waiting process is allowed to enter.
Peterson's Solution	A classical software-based algorithm for achieving mutual exclusion - between two processes using shared variables and simple control logic.
Semaphore	A synchronisation primitive that uses counters and signalling to control - access to shared resources. It can be binary (mutex) or counting.
Busy Waiting	A synchronisation approach where a process continuously checks a - condition (like a lock) in a loop, consuming CPU cycles unnecessarily.
Preemptive Kernel	A kernel that allows the currently running process to be interrupted, so - another process can be scheduled, enhancing responsiveness.
Non-Preemptive Kernel	A kernel that does not interrupt a running process in kernel mode; the - process retains CPU control until it yields or finishes.

TestAndSet Instruction	- A hardware atomic instruction that tests and sets a memory location in one step, used to implement locks in critical section handling.
Swap Instruction	- A hardware-based atomic instruction that swaps values between two variables, also used for implementing mutual exclusion.
Deadlock	- A condition where a group of processes are each waiting for a resource held by another, causing an indefinite wait.
Starvation	- A condition where a process is perpetually denied access to resources because other processes are continuously prioritised.
Monitor	- A high-level synchronisation construct that provides a mechanism to safely allow only one process at a time to execute a procedure that accesses shared data.
Atomic Operation	- An operation that is performed as a single, indivisible unit of work. It either completes entirely or not at all, preventing concurrent access issues.
Context Switching	- The process of storing the state of a currently running process and loading the state of another process. This allows multitasking and process scheduling.
Kernel Mode	- A privileged execution mode in which the operating system has unrestricted access to system resources, used for running core system functions.
User Mode	- A restricted mode where user applications run, with limited access to system resources, to protect system stability and security.
Interprocess Communication (IPC)	- Mechanisms that allow processes to communicate and synchronise their actions when executing concurrently.
Spinlock	- A lock that causes a thread trying to acquire it to simply wait in a loop ("spin") while checking if the lock is available, leading to busy waiting.
Priority Aging	- A technique to prevent starvation by gradually increasing the priority of waiting processes over time.

Fairness	- A property of scheduling and synchronisation ensures that all processes get a fair chance to access resources and execute.
Entry Section	- The code part where a process requests permission to enter its critical section.
Exit Section	- The code part executed after leaving the critical section, often used to release locks or update shared variables.
Remainder Section	- The part of a process's code that lies outside of the critical, entry, and exit sections. It does not involve shared resource access.
Blocking	- : A state where a process waits (is suspended) until a condition is met or a resource becomes available, conserving CPU cycles compared to busy waiting.
Lock Variable	- A boolean or binary variable used to control access to the critical section, often manipulated with atomic operations like TestAndSet.
Starvation-Free Algorithm	- An algorithm that guarantees every waiting process will eventually get a turn to enter the critical section.
Hardware Synchronisation Support	- Special processor instructions (like TestAndSet or Swap) used to achieve atomic operations and implement locks at the hardware level.

7. TERMINAL QUESTIONS

1. Define a race condition and explain with an example.
2. List and explain the three essential requirements to solve the critical section problem.
3. What is busy waiting? Why is it considered inefficient?
4. Explain mutual exclusion using the TestAndSet instruction.
5. What is the purpose of the 'flag' and 'turn' variables in Peterson's Algorithm?
6. Describe Peterson's Algorithm in detail with its structure and working.
7. Explain the differences between preemptive and non-preemptive kernels. Provide advantages and disadvantages of each.
8. What are the limitations of using busy waiting for mutual exclusion? Suggest alternatives.
9. Discuss how TestAndSet and Swap instructions work in achieving synchronisation. Compare their strengths and weaknesses..
10. Illustrate the use of bounded waiting with a modified TestAndSet approach.
11. Why is disabling interrupts not a viable solution in multiprocessor systems?
12. Differentiate between software-based and hardware-based synchronisation solutions with examples.
13. Explain how semaphores and monitors help in synchronising processes.
14. Describe a scenario where a deadlock may occur. How can it be avoided?
15. What challenges do hardware designers face while implementing atomic operations in multiprocessor systems?

8. ANSWERS

Self-Assessment Questions

1. Starvation
2. It consumes CPU cycles doing no useful work
3. Priority aging
4. Critical Section Problem
5. Automatic mutual exclusion
6. Pre-emptive kernel
7. Peterson's Algorithm
8. Which process is allowed to enter the critical section
9. False
10. readiness or intention
11. TestAndSet
12. Lock
13. Swap
14. Bounded waiting
15. Swap

Terminal Questions Answers

Answer 1: A race condition occurs when multiple processes access shared data concurrently, and the final outcome depends on the sequence of access. For example, simultaneous updates to a bank balance can result in incorrect totals.

Refer to Section 2 Process Synchronisation.

Answer 2: The solution must ensure mutual exclusion, progress, and bounded waiting to guarantee safe and fair resource access. These ensure only one process accesses a critical section, a fair turn is granted, and starvation is prevented.

Refer to Section 2.1 Critical Section Problem.

Answer 3: Busy waiting is a method where a process continuously checks for a condition, using CPU cycles without performing meaningful work. It leads to resource wastage in multiprocessor environments.

Refer to Section 2 Process Synchronisation.

Answer 4: TestAndSet ensures only one process enters the critical section by atomically checking and setting a lock variable. If the lock is already set, other processes wait.

Refer to Section 4 Synchronisation Hardware.

Answer 5: The flag array indicates intent to enter the critical section, and turn decides which process gets access, ensuring fairness and mutual exclusion.

Refer to Section 3 Peterson's Problem.

Answer 6: Peterson's algorithm uses two shared variables—flag and turn—to coordinate mutual exclusion between two processes. It satisfies mutual exclusion, progress, and bounded waiting conditions.

Refer to Section 3 Peterson's Problem.

Answer 7: Pre-emptive kernels allow interruptions for better responsiveness, while non-preemptive ones are simpler and safer. Pre-emptive systems suit multitasking; non-pre-emptive ones reduce race risks.

Refer to Section 2 Process Synchronisation (Pre-emptive/Non-preemptive Kernel subsection).

Answer 8: Busy waiting wastes CPU cycles and doesn't guarantee fairness. Alternatives include semaphores and monitors which suspend the process until conditions are met.

Refer to Section 2 Process Synchronisation and 4 Synchronisation Hardware.

Answer 9: TestAndSet atomically sets and checks a flag; Swap exchanges values to manage locks. Both ensure atomicity but cause busy waiting and may not ensure fairness.

Refer to Section 4 Synchronisation Hardware.

Answer 10: A waiting array tracks processes. When a process exits its critical section, it searches cyclically to grant access to the next waiting process, ensuring fairness.

Refer to Section 4 Synchronisation Hardware (Bounded-waiting mutual exclusion).

Answer 11: It leads to issues like delayed clock updates, responsiveness loss, and complex interrupt management across CPUs. Hardware-level atomic instructions are better alternatives.

Refer to Section 4 Synchronisation Hardware.

Answer 12: Software-based methods like Peterson's use logic and shared variables; hardware-based solutions like TestAndSet use atomic CPU instructions for reliability.

Refer to Section 3 and 4.

Answer 13: Semaphores use counters and signals to control resource access. Monitors encapsulate shared data and procedures, offering automatic mutual exclusion.

Refer to Section 2 Process Synchronisation.

Answer 14: When two processes each wait for a resource held by the other, deadlock occurs. Avoidance techniques include ordering resources or using detection and recovery.

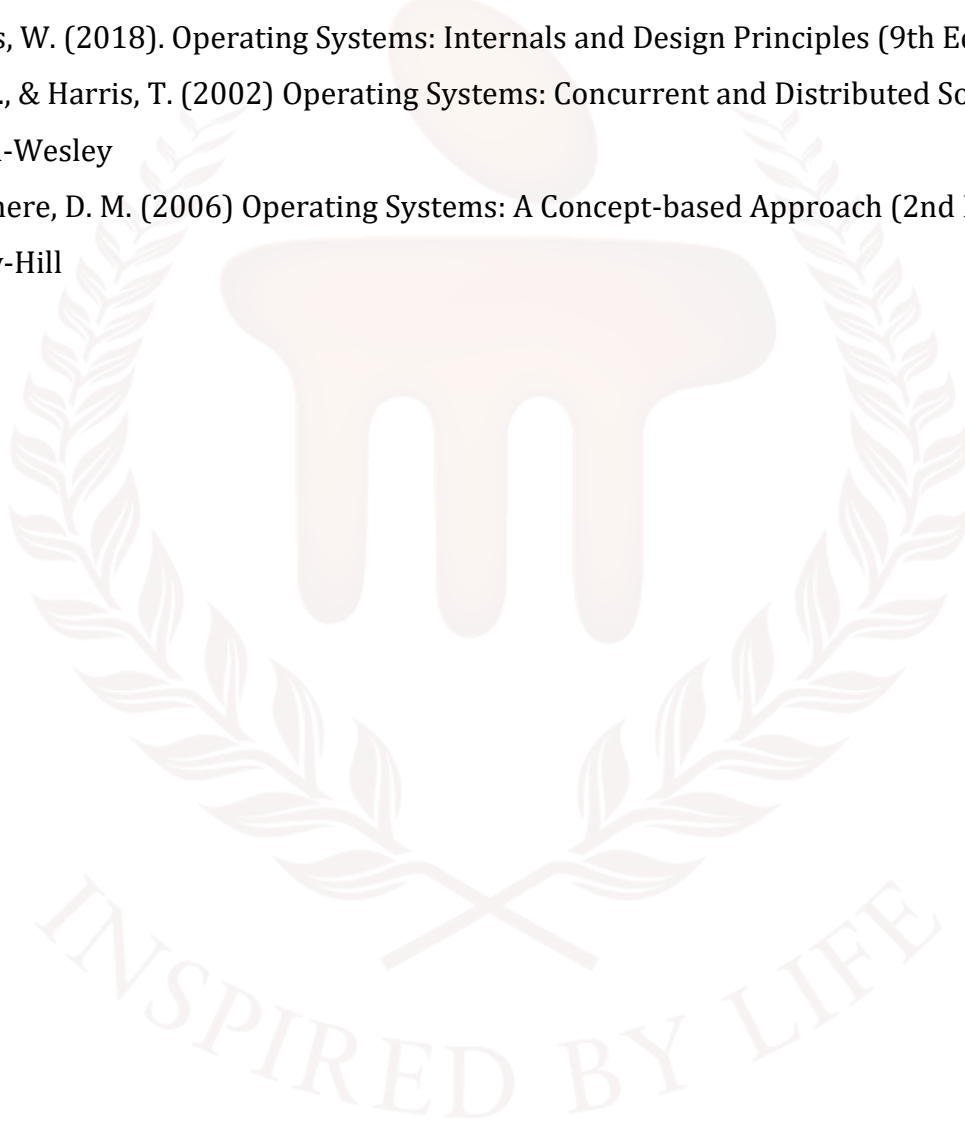
Refer to Section 2 Process Synchronisation.

Answer 15: Ensuring atomicity, cache coherency, and memory consistency are major challenges. Bus locking and coherency protocols add to complexity and performance trade-offs.

Refer to Section 4 Synchronisation Hardware.

9. REFERENCES

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition), Wiley.
2. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th Edition), Pearson.
3. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th Edition), Pearson.
4. Bacon, J., & Harris, T. (2002) Operating Systems: Concurrent and Distributed Software Design, Addison-Wesley
5. Dhamdhere, D. M. (2006) Operating Systems: A Concept-based Approach (2nd Edition), Tata McGraw-Hill





Unit – 7

Process Synchronisation: Semaphores

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Semaphore Concept	-	1	6 - 8
3	Semaphore Usage	-	2	9 - 13
4	Basics of Semaphores	-	3	14 - 24
5	Summary	-	-	25
6	Glossary	-	-	26 - 28
7	Terminal Questions	-	-	29
8	Answers	-	-	30 - 32
9	References	-	-	33

1. INTRODUCTION

In this unit, learners will study the principles and mechanisms of process synchronisation with a specific focus on semaphores, a fundamental concept in concurrent and parallel processing within operating systems. Synchronisation is necessary in environments where multiple processes or threads access shared resources. Without proper synchronisation, conflicts such as race conditions, deadlocks, and inconsistent data can arise, affecting the reliability and stability of a system. Semaphores provide a structured approach to manage such access and ensure orderly execution of concurrent processes.

Learners will begin by understanding the importance of synchronisation and how it relates to the execution of processes in multitasking and multi-user systems. Key terms such as critical section, race condition, and mutual exclusion will be introduced to explain the challenges involved when multiple processes require access to shared memory or devices. The concept of deadlock will also be discussed to highlight potential risks when synchronisation is not handled correctly.

The core part of this unit focuses on semaphores — integer variables that are used to signal and control access to resources. The two main operations associated with semaphores, namely `wait()` and `signal()`, will be explored in detail. These operations are atomic, ensuring that they are executed completely without interruption, which is critical for maintaining data consistency. Learners will study how these operations are implemented and used to control process behaviour around critical sections.

This unit will also cover the different types of semaphores: counting semaphores, which can take non-negative integer values and are used to manage multiple instances of a resource, and binary semaphores, which can take only the values 0 and 1 and are commonly used for mutual exclusion. Practical scenarios, such as the producer-consumer problem and the reader-writer problem, will help learners understand how semaphores can be applied to real-world synchronisation challenges. These examples demonstrate how semaphores coordinate producer and consumer actions in bounded buffer systems. They also manage access in shared data scenarios involving readers and writers.

Learners will study common issues associated with semaphore use, such as busy waiting, and explore improved techniques like blocking, where a process voluntarily gives up control until it is signalled to continue. The differences in implementing semaphores in single-processor and multiprocessor

systems will also be addressed, including hardware-supported atomic operations and software-level control mechanisms.

Further, this unit discusses both the advantages and drawbacks of using semaphores. Their ability to support multiple synchronisation patterns and efficient CPU usage makes them widely applicable in operating systems and real-time systems. At the same time, challenges such as deadlock, starvation, and improper usage are also examined, emphasising the importance of careful design and implementation.

By the end of the unit, learners will gain a comprehensive understanding of semaphores and their role in managing concurrency, enabling them to design and analyse solutions for process synchronisation in various computing environments.

1.1. Objectives

After studying this unit, you should be able to:

- Define key terms such as process synchronisation, critical section, semaphore, race condition, deadlock, and mutual exclusion.
- Identify the basic operations used with semaphores (`wait()` and `signal()`).
- Explain the purpose of semaphores in process synchronisation.
- Describe the difference between binary and counting semaphores.
- Illustrate how semaphores help in solving synchronisation problems like producer-consumer and reader-writer.
- Apply semaphore-based synchronisation techniques to manage access to shared resources in a multi-process system.
- Demonstrate the use of `wait()` and `signal()` operations in simple code or algorithmic examples.
- Distinguish between busy waiting and blocking mechanisms in semaphore implementations.



2. SEMAPHORE CONCEPT

The hardware-based solutions to the critical-section problem, such as those using TestAndSet() and Swap() instructions, are often complex and difficult for application programmers to use correctly. As an alternative, semaphores offer a higher-level and more manageable synchronisation mechanism.

A semaphore is an integer variable that is accessed and modified only through two atomic operations: wait() and signal() (also known historically as P and V operations).

- **P (Proberen):** Dutch for "to test" — corresponds to wait()
- **V (Verhogen):** Dutch for "to increment" — corresponds to signal()

wait() Operation

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait (no operation)  
    S--;  
}
```

- Blocks the process if the value of the semaphore S is less than or equal to 0.
- Otherwise, it decrements S and allows the process to enter the critical section.

signal() Operation

```
signal(S) {  
    S++;  
}
```

- Increments in the value of semaphore S, potentially allowing a waiting process to proceed.

Note: Both wait() and signal() must be atomic operations. No two processes can simultaneously execute these operations on the same semaphore.

Important Properties

- The evaluation ($S \leq 0$) and modification ($S--$) in wait() must be executed without interruption.
- Semaphores provide a general-purpose tool for managing synchronisation and mutual exclusion.

Features of Semaphores

Semaphores are fundamental synchronisation primitives widely used in concurrent programming to manage access to shared resources. Their key features make them a powerful tool for solving the critical-section problem and coordinating process execution in multiprogramming environments.

The important features are:

1. Machine Independence: One of the most significant features of semaphores is their machine independence. Unlike hardware-based solutions such as TestAndSet or Swap, semaphores are purely software constructs. This means that semaphores can be implemented at the operating system level or within user applications without relying on any specific hardware features. As a result, programs that use semaphores for synchronisation can run on different machines and operating systems with minimal modification, making semaphores a portable and flexible solution. This feature is particularly valuable in modern distributed and cross-platform environments.

2. Ease of Implementation: Semaphores are relatively simple to implement. A semaphore is essentially an integer variable, protected by atomic wait() and signal() operations. These operations can be implemented using low-level atomic instructions provided by the CPU, but the semaphore logic itself remains straightforward. The simplicity of semaphores allows operating system developers to provide native support, and application developers can also implement them in software if necessary. This ease of implementation makes semaphores accessible to both beginners and experienced programmers and enables rapid development of synchronisation solutions.

3. Simplicity in Correctness Checking: Correctness in concurrent programming is a challenging task due to potential issues such as race conditions, deadlocks, and resource starvation. Semaphores offer a structured and predictable behavior through their two basic operations: wait() and signal(). This limited and well-defined interface simplifies the reasoning about program behavior. Since the number of operations is small and their effects are well-understood, developers can more easily verify that their programs maintain mutual exclusion, prevent deadlocks, and ensure progress. This clarity makes semaphores a preferred choice in scenarios where correctness is critical.

4. Support for Multiple Critical Sections: In complex systems, multiple resources may require mutual exclusion independently. Semaphores support this by allowing the use of distinct semaphores for each critical section or resource. This means that programmers can fine-tune access controls and synchronisation policies for different parts of a program. For instance, one semaphore might manage

access to a printer, while another controls access to a database. This modular approach enhances code organisation, scalability, and maintainability.

5. Concurrent Resource Acquisition: Semaphores also enable coordinated access to multiple resources, allowing multiple processes to acquire and release shared resources concurrently and efficiently. For example, a counting semaphore can represent a pool of identical resources (e.g., database connections or server threads). When a process needs a resource, it performs a `wait()` operation, and when it finishes using the resource, it performs a `signal()`. This mechanism ensures that resources are allocated fairly and that no two processes use the same resource simultaneously, while also maximising overall system utilisation.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following operations is used to decrease the value of a semaphore? a) <code>signal()</code> b) <code>block()</code> c) <code>wait()</code> d) <code>wakeup()</code>
Fill in the Blank	
2	A semaphore is an _____ variable used for synchronising processes.
3	The two atomic operations associated with semaphores are _____ and _____.
True or False	
4	A semaphore can be used to ensure only one process enters the critical section at a time.
Fill in the Blanks	
5	A _____ is a software construct used to synchronise processes by signaling.
6	In semaphore terminology, the P operation stands for _____ and the V operation stands for _____.
Multiple Choice Question	
7	Which of the following problems is solved using semaphores? a) Memory fragmentation b) File corruption c) Critical section problem d) CPU scheduling

3. SEMAPHORE USAGE

There are different ways to implement semaphores, but one of the most basic—and problematic—approaches involves busy waiting. This method, though simple to understand and implement, has serious performance implications, particularly in single-processor systems. Fortunately, improvements to semaphore implementations help eliminate the inefficiencies of busy waiting by employing blocking mechanisms instead

- **Busy Waiting and Spinlocks**

The most basic semaphore implementation relies on busy waiting, where a process that cannot immediately access the critical section will enter a loop and repeatedly check whether the resource has become available. This loop is known as spinning, and the semaphore is referred to as a spinlock.

Example: Basic Spinlock Implementation

```
while (semaphore <= 0) {  
    // spin (busy wait)  
}  
semaphore--;
```

This approach is straightforward: the process keeps checking the semaphore's value, and when it's greater than 0, it decrements the value and enters the critical section. However, this method has significant drawbacks.

Problems in Single-Core Systems

In a single-core system, only one process can run at a time. When a process is busy waiting, it monopolises the CPU without doing any useful work. This prevents other processes—including the one that might release the resource—from executing. As a result, the system wastes CPU cycles, increases power consumption, and reduces overall efficiency. The busy-waiting process effectively blocks the progress of all others by occupying the CPU continuously.

Advantages in Multi-Core Systems

Despite its inefficiencies in single-core systems, spinlocks can be useful in multiprocessor environments. When multiple processors are available, one processor can spin (wait) while another processor holds the lock and eventually releases it. Because the spinning thread does not prevent other processors from progressing, this model can be efficient—but only when the lock is expected

to be held for a short duration. This avoids the overhead associated with a context switch, which involves saving and restoring process states and is typically expensive in terms of time.

Thus, spinlocks are sometimes preferable in systems with brief lock-hold times and multiple processing units, such as in low-level OS kernels or embedded systems.

- **Eliminating Busy Waiting with Blocking**

To address the inefficiencies of busy waiting, modern semaphore implementations allow processes to block themselves instead of spinning. This significantly improves system performance, particularly in single-processor environments.

Instead of looping, a process that finds the semaphore unavailable can block itself—i.e., voluntarily move to a waiting state. The operating system can then switch the CPU to another process that is ready to execute. When the semaphore becomes available, a wakeup mechanism is used to transition the blocked process back to the ready state, allowing it to be scheduled again.

Revised Semaphore Operations

wait() with Blocking

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block(); // system call to suspend the process  
    }  
}
```

signal() with Wakeup

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P); // system call to resume the process  
    }  
}
```

With this model, the semaphore maintains a queue of waiting processes. When the semaphore's value drops below zero, it reflects the number of processes currently blocked. These blocked processes are stored in a queue (usually FIFO) until a `signal()` operation makes the resource available again.

Semaphore Structure

To support blocking, semaphores must hold additional information beyond just the counter.

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

- *value*: Tracks the number of available resources (positive) or the number of waiting processes (negative).
- *list*: Points to a queue of blocked processes, typically implemented using the system's process control blocks (PCBs).

Each process in the waiting queue can be linked using pointers embedded in their PCB. The semaphore's queue management can be based on different strategies (e.g., FIFO, priority-based), though FIFO is commonly used to ensure fairness and bounded waiting.

- **Atomicity and Interrupt Handling**

One of the most critical requirements for semaphore operations is atomicity. Both `wait()` and `signal()` must execute as indivisible units, preventing race conditions where two or more processes manipulate the semaphore simultaneously, resulting in inconsistent or incorrect behavior.

Ensuring Atomicity in Single-Processor Systems

In a single-processor environment, the operating system can disable interrupts during the execution of `wait()` and `signal()` to ensure that no other process can preempt the current one:

```
disable_interrupts();  
wait(S);  
// critical section  
signal(S);  
enable_interrupts();
```

Disabling interrupts ensures that the wait() or signal() operation completes before another process can be scheduled. However, this method is only suitable for small, fast operations because disabling interrupts for extended periods can degrade system responsiveness.

Ensuring Atomicity in Multiprocessor Systems

Disabling interrupts on one processor does not prevent other processors from executing critical sections. Disabling interrupts on all processors is technically possible but not practical due to high overhead and complexity.

Instead, multiprocessor systems rely on hardware-supported atomic operations like:

- test-and-set
- compare-and-swap
- fetch-and-add

These atomic instructions are provided by most modern CPUs and can be used to build low-level spinlocks or atomic blocks of code.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
8	In a semaphore implementation with blocking, when a process executes wait() and finds the semaphore value negative, it is: a) Terminated b) Put to sleep c) Moved to ready state d) Given CPU control
True or False	
9	Busy waiting is more efficient than blocking in single-core systems.
Multiple Choice Question	
10	What does the term 'atomic operation' refer to in semaphore usage? a) Operation that requires no memory b) Operation that runs without interruption c) Operation that can be paused anytime d) Operation that needs multiple CPUs

Fill in the Blank	
11	When a process cannot proceed and is suspended until a condition becomes true, it is said to be in a _____ state.
Multiple Choice Question	
12	The operation that adds a process to the semaphore's wait queue is performed during: a) signal() b) release() c) wait() d) start()
True or False	
13	In a semaphore, the signal() operation can wake up a blocked process.
Fill in the Blank	
14	In multiprocessor systems, semaphores rely on hardware-supported _____ instructions.
True or False	
15	A blocked process using semaphores continuously consumes CPU cycles.
16	The value of a counting semaphore can be negative.



4. BASICS OF SEMAPHORE

A semaphore is essentially a variable or abstract data type used to control access to a common resource in a concurrent system such as a multitasking operating system. It provides a way to solve critical section problems and avoid race conditions by signaling between processes or threads.

The semaphore concept was introduced by Edsger Dijkstra in the 1960s as a mechanism to manage synchronisation between processes.

semaphores are classified into two major types based on the range of values they can hold and the purpose they serve:

- Counting Semaphore
- Binary Semaphore

1. Counting Semaphore

A counting semaphore is an integer variable that can take any non-negative value (0, 1, 2, ...). It is designed to count the number of available resources or the number of allowed simultaneous accesses.

- Initialised to the number of available resources.
- Each time a process wants to enter a critical section or use a resource, it performs a wait (P) operation.
- If the semaphore's value is greater than zero, it is decremented, and the process proceeds.
- If the semaphore's value is zero, it means no resource is available, and the process is blocked until a resource is freed.
- When a process finishes using the resource, it performs a signal (V) operation, incrementing the semaphore and potentially waking a blocked process.

Counting semaphores are ideal for managing access to multiple identical resources such as:

- Database connections in a connection pool.
- Number of printers available in a printing system.
- Tickets available for booking.
- Slots in a bounded buffer in producer-consumer problems.

Example: Imagine a system with 5 identical printers. Here's how a counting semaphore manages access:

- The semaphore is initialised to 5.
- When a process wants to print, it performs wait:
 - If the semaphore value is 3, it decrements to 2, and the process gets access to a printer.
 - If the semaphore value is 0, the process blocks, waiting for a printer to be free.
- When a process finishes printing, it performs signal:
 - The semaphore value increments from 0 to 1.
 - One of the waiting processes is notified to proceed.

Benefits:

- **Efficient resource management:** Semaphores can effectively handle access to a limited number of identical resources by maintaining a count.
- **Supports concurrency:** Allows multiple processes/threads to access resources concurrently, as long as resources are available.
- **Promotes fairness:** Waiting processes can be queued in a defined order (e.g., FIFO), helping to ensure fair access to resources.

Drawbacks:

- **Risk of deadlocks:** Incorrect usage or improper sequencing can lead to deadlocks, where processes are stuck waiting for resources indefinitely.
- **Lack of strict ownership:** Any process can signal (release) the semaphore, not necessarily the one that waited (acquired), which can lead to logic errors or resource misuse.

2. Binary Semaphore

A binary semaphore is a special case of counting semaphore with only two possible values: 0 and 1. It is also called a mutex semaphore or simply a mutex.

- Initialised to 1, indicating the resource is free.
- When a process enters the critical section, it performs a wait (P) operation:
 - If the semaphore is 1, it changes to 0, and the process enters.
 - If the semaphore is 0, the process blocks until it becomes 1.
- When the process leaves the critical section, it performs a signal (V) operation:
 - The semaphore is set back to 1.
 - If processes are waiting, one of them is allowed to enter.

Binary semaphores are used to enforce mutual exclusion — only one process or thread can access a critical section or shared resource at a time.

They are commonly used in:

- Protecting critical sections of code.
- Synchronising processes or threads.
- Controlling access to hardware devices.

Example: Consider a critical section where only one thread should execute at a time:

- Binary semaphore initialised to 1.
- Thread A performs wait:
 - Semaphore goes from 1 to 0.
 - Thread A enters critical section.
- Thread B performs wait:
 - Semaphore is 0, so thread B blocks.
- Thread A completes and performs signal:
 - Semaphore goes back to 1.
 - Thread B wakes and enters.

Benefits:

- **Simple and effective for mutual exclusion:** Binary semaphores (with values 0 or 1) are straightforward tools for ensuring that only one process or thread accesses a critical section at a time.
- **Prevents concurrent access to critical resources:** They block other processes until the resource is released, ensuring exclusive access.
- **Avoids race conditions:** By enforcing mutual exclusion, they help maintain data consistency and correctness in shared memory operations.

Drawbacks:

- **No ownership tracking:** Any process can signal (release) the semaphore, not necessarily the one that waited (acquired), which can lead to programming errors.
- **Deadlock risk:** If a process enters a critical section but fails to signal (release) the semaphore, other processes can be permanently blocked.

Comparison Table: Counting and Binary Semaphores

Feature	Counting Semaphore	Binary Semaphore
Values	0, 1, 2, 3, ... (any non-negative int)	Only 0 or 1
Use Case	Multiple identical resources	Single resource or critical section
Behavior	Allows multiple processes simultaneously	Allows only one process at a time
Initialisation	Number of available resources	1 (resource available)
Synonym	General semaphore	Mutex semaphore

Mutex v/s Binary Semaphore

Though binary semaphores and mutexes are often used interchangeably, there are subtle differences:

Aspect	Mutex	Binary Semaphore
Ownership	Enforces ownership — only the thread/process that locked it can unlock it.	No ownership — any thread/process can signal (V).
Primary Use	Designed strictly for mutual exclusion in critical sections.	Used for both mutual exclusion and signalling mechanisms.
Unlocking Behavior	Only the owner can release the lock.	Any thread/process can perform a signal (V).
Deadlock Potential	Safer due to ownership rules (less prone to misuse).	Higher risk if the signal is not properly managed.
System-Level Behavior	Often has priority inheritance to prevent priority inversion.	Typically does not include advanced scheduling features.

Practical Examples of Semaphores**Example 1: Producer-Consumer Problem Using Counting Semaphore**

In the Producer-Consumer (or Bounded-Buffer) problem, two types of processes share a fixed-size buffer:

- Producers generate data and insert it into the buffer.
- Consumers remove data from the buffer and process it.
- Semaphore empty: counts the number of empty slots.
- Semaphore full: counts the number of filled slots.
- Semaphore mutex: binary semaphore to protect buffer access.

Challenges

- Producers must not add data to a full buffer.
- Consumers must not remove data from an empty buffer.
- Access to the buffer must be mutually exclusive to prevent race conditions.

Operations:

- Producer waits on empty, waits on mutex, inserts item, signals mutex, signals full.
- Consumer waits on full, waits on mutex, removes item, signals mutex, signals empty.

This ensures producers do not overflow the buffer, and consumers do not consume from empty buffers.

Example 2: Reader-Writer Problem Using Semaphores

The Reader-Writer Problem is a classic synchronisation problem in concurrent programming. It deals with coordinating access to shared data between multiple reader and writer processes. This problem arises in situations where many processes need to read from a shared resource, but only a few need to write to it — such as reading a file, querying a database, or accessing shared memory.

This problem requires synchronising multiple readers and writers to access shared data.

- Binary semaphore `rw_mutex` to allow writers exclusive access.
- Counting semaphore `read_count_mutex` to protect the reader count.
- Variable `read_count` tracks the number of active readers.

To manage synchronisation, we use:

1. `rw_mutex` (binary semaphore):

- Ensures that only one writer (or no readers) accesses the data at a time.
- Blocks both readers and writers when a writer is active.

2. `read_count_mutex` (binary semaphore):

- Protects the access to the shared variable `read_count`.

3. `read_count` (integer variable):

- Tracks the number of readers currently accessing the shared data.

These semaphores ensure that multiple readers can access the shared resource concurrently, but only one writer can modify it at a time.

Reader's Process:

When a reader wants to access the shared data:

1. **Acquire** read_count_mutex – to safely update read_count.
2. **Increment** read_count.

If read_count == 1, it means this is the first reader:

It locks rw_mutex to block writers.

3. **Release** read_count_mutex.
4. **Read the shared data.**
5. After reading, acquire read_count_mutex again.
6. **Decrement read_count.**

If read_count == 0, it means this is the last reader:

It signals rw_mutex to allow writers.

7. **Release** read_count_mutex.

Writer's Process:

When a writer wants to write to the shared data:

1. **Wait on rw_mutex.**
 - This ensures exclusive access to the data.
2. **Write the data.**
3. **Signal rw_mutex** to allow others (readers or writers).

Advantages:

- **Provide a Robust Mechanism to Manage Concurrent Access**

One of the most significant advantages of semaphores is their ability to handle concurrent access to shared resources safely. When multiple processes or threads attempt to access the same resource — such as memory, files, or hardware devices — there is a risk of race conditions, where the final state depends on the unpredictable timing of the threads. Semaphores solve this by ensuring that only a permitted number of processes can access the resource at any time.

For example, a semaphore can ensure that only one thread enters a critical section, or only a limited number of threads access a pool of resources like database connections. By using atomic wait and signal operations, semaphores effectively prevent inconsistencies and corruption caused by concurrent modifications. This robustness is crucial for building reliable multitasking systems and multi-threaded applications.

- **Can Be Used to Implement Many Synchronisation Patterns**

Semaphores are highly versatile and can be adapted to implement a wide variety of synchronisation patterns beyond simple mutual exclusion. These patterns include producer-consumer, reader-writer, barrier synchronisation, and resource counting.

For instance, in the producer-consumer problem, semaphores manage the count of available buffer slots and synchronise producers and consumers efficiently. In reader-writer scenarios, semaphores help coordinate multiple readers with writers to optimise read access without compromising data integrity.

Because semaphores provide fundamental wait and signal operations, they serve as building blocks to create more complex synchronisation mechanisms, enabling developers to tailor concurrency control to specific application requirements.

- **Lightweight Compared to Busy Waiting**

Busy waiting (or spin-waiting) occurs when a process continuously checks a condition in a loop without relinquishing CPU control. While simple, busy waiting wastes CPU cycles and degrades system performance, especially in single-core or heavily loaded systems.

Semaphores eliminate this inefficiency by enabling blocking. When a process cannot proceed (e.g., resource not available), it is put to sleep or suspended by the operating system rather than spinning in a loop. This allows the CPU to perform other tasks, improving overall system efficiency. When the resource becomes available, the semaphore signals the waiting process to wake up and continue execution.

This lightweight behaviour makes semaphores ideal for real-world multitasking systems, where efficient CPU usage and responsiveness are critical.

- **Support Blocking — Processes Can Sleep While Waiting, Saving CPU Cycles**

Unlike simple flags or locks that might require active polling, semaphores use blocking mechanisms at the OS level. When a process executes a wait operation and the semaphore value indicates unavailability, the process is suspended and placed in a waiting queue.

This blocking feature saves valuable CPU cycles because the waiting process is not consuming CPU resources unnecessarily. Instead, it “sleeps” until another process releases the semaphore by performing a signal operation.

This advantage is particularly important in systems with many concurrent threads or processes, as it reduces contention and helps maintain responsiveness without wasting computational power.

- **Applicable in Multitasking Operating Systems and Real-Time Systems**

Semaphores are widely supported and implemented in modern multitasking operating systems like Linux, Windows, and macOS. They provide a standardised and proven way to coordinate access to resources among numerous processes or threads running simultaneously.

Moreover, semaphores are crucial in real-time systems, where timing and predictability are paramount. They help guarantee that critical sections or resources are accessed in a controlled manner, preventing priority inversion or unexpected delays.

In real-time applications such as embedded systems, industrial control, and telecommunications, semaphores enable the system to meet strict timing constraints while ensuring safe concurrency.

Drawbacks of Semaphores

While semaphores are powerful and widely used synchronisation tools, they come with several inherent limitations and challenges. Understanding these drawbacks is crucial for effective and safe use in concurrent programming.

- **Semaphores Are Essentially Global Variables That Are Shared**

Semaphores are typically implemented as shared variables accessible to multiple processes or threads. Because they are global in nature, this exposes them to potential misuse. When a semaphore is shared globally, any part of the program can modify its value, which increases the risk of accidental or intentional misuse.

This global visibility means that a bug or improper operation in one part of the program can have unintended consequences in another, potentially compromising synchronisation and leading to inconsistent states or corrupted data.

- **Improper Use Can Cause Deadlocks, Priority Inversion, or Starvation**

The flexibility of semaphores is a double-edged sword. Incorrect use of semaphores can introduce serious problems:

- **Deadlocks:**

It occurs when two or more processes wait indefinitely for semaphores held by each other, causing a circular wait with no progress. For example, if process A waits on semaphore S1 while holding S2, and process B waits on S2 while holding S1, both are stuck.

- **Priority Inversion:**

Happens when a higher-priority task is waiting for a semaphore held by a lower-priority task. This can cause system performance degradation or missed deadlines in real-time systems.

- **Starvation:**

If the semaphore management does not enforce fairness, some processes may be perpetually delayed if other processes continually acquire the semaphore, preventing them from accessing the resource.

These problems require careful design and additional mechanisms (like priority inheritance or deadlock detection) to mitigate.

- **Semaphores Can Be Accessed From Any Location Inside a Program**

Since semaphores are typically global or shared objects, any part of the program can perform wait (P) or signal (V) operations on them. This unrestricted access makes it difficult to track and enforce correct usage.

In large or complex programs, this can lead to race conditions or bugs where a semaphore is signaled or waited upon in inappropriate contexts. Debugging such issues is challenging because the semaphore's state can be changed unexpectedly from various code paths.

- **Appropriate Usage Is Not Controlled Nor Guaranteed**

Semaphores do not inherently enforce correct usage patterns. For example, there is no guarantee that a process performing a wait operation will later perform a corresponding signal operation. Similarly, there is no mechanism preventing a process from signaling a semaphore it has not waited on.

This lack of control can lead to synchronisation errors such as:

- Double signaling, which can falsely increase the semaphore count.
- Missing signals, which cause other processes to block indefinitely.
- Out-of-order signaling and waiting, breaking synchronisation logic.

Because semaphores are simply counters with atomic operations, enforcing proper protocol depends entirely on the programmer.

- **The Data That the Semaphore Controls Access To and the Semaphore Itself Have No Linguistic Relationship**

Semaphores are generic synchronisation primitives that do not inherently link to the specific data or resources they control. This means the semaphore is just a counter or flag unrelated to the actual data structure or resource it protects.

This separation can lead to design complexity and mistakes, such as using the wrong semaphore with the wrong resource or losing track of which semaphore protects which data. Higher-level

abstractions like monitors or mutex objects encapsulate both the data and synchronisation mechanism, making the relationship explicit and safer.

- **They Have Two Functions: Scheduling Restrictions and Mutual Exclusion**

While semaphores provide powerful control over scheduling (blocking/waking) and mutual exclusion, their dual purpose can sometimes complicate their use. Combining these two roles in a single primitive makes it easy to misuse or misunderstand their behavior.

For instance, using a binary semaphore as both a mutex and a signaling mechanism can lead to subtle bugs if the timing or ordering of wait and signal operations is incorrect. Separating concerns—using mutexes strictly for mutual exclusion and other signaling constructs for coordination—can lead to clearer and more maintainable code.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
17	A binary semaphore can hold how many values? a) 0 to N b) 0 or 1 c) 1 to N d) Only 0
18	Which of the following problems can occur due to improper use of semaphores? a) Paging b) Fragmentation c) Starvation d) Swapping
Fill in the Blanks	
19	In the producer-consumer problem, the semaphore that tracks empty slots is usually named _____.
20	A _____ semaphore is also called a mutex.
True or False	
21	Binary semaphores can manage multiple instances of a shared resource.
22	Starvation means a process gets more than its fair share of CPU cycles.
Multiple Choice Questions	
23	Which one of the following is NOT a feature of semaphores?

	a) Atomic operations b) Machine independence c) Automatic deadlock resolution d) Mutual exclusion support
24	What is the initial value of a semaphore used to manage a single resource? a) 0 b) -1 c) 1 d) Any value
25	Which mechanism is commonly used to avoid priority inversion in semaphores? a) Disabling interrupts b) Priority inheritance c) Context switching d) Semaphore chaining
Fill in the Blanks	
26	The _____ value in a counting semaphore represents the number of available resources.
27	The reader-writer problem uses a variable called _____ to track the number of active readers.
True or False	
28	Deadlock is always caused by incorrect semaphore initialisation.
29	Binary semaphores can be used in both mutual exclusion and signaling.
30	In the producer-consumer problem, mutex ensures mutual exclusion while accessing the buffer.

5. SUMMARY

- In this unit, learners have learnt the fundamental idea behind semaphores as a synchronisation tool used to manage access to shared resources in concurrent systems. They have understood how the wait () and signal () operations control process execution and ensure mutual exclusion.
- The practical aspects of implementing semaphores, including the challenges posed by busy waiting and how blocking mechanisms provide a more efficient solution. They have explored how semaphores are used in single-core and multi-core systems for maintaining synchronisation and system performance.
- Explored the two main types of semaphores—counting and binary—and their respective roles in managing multiple and single resource access. They have also studied common synchronisation problems like the producer-consumer and reader-writer problems and how semaphores help solve them.

6. GLOSSARY

Process Synchronisation	- A technique used in operating systems to coordinate the execution of processes that share resources, ensuring data consistency and avoiding conflicts.
Critical Section	- A portion of code where a process accesses shared resources. Only one process at a time should execute in its critical section to avoid race conditions.
Race Condition	- A situation where multiple processes access and modify shared data concurrently, and the final result depends on the timing of their execution.
Deadlock	- A state in which two or more processes are blocked indefinitely, each waiting for a resource that the other process holds.
Semaphore	- A synchronisation primitive used to control access to shared resources. It is an integer variable that supports atomic wait() and signal() operations.
wait() (P operation)	- A semaphore operation that decrements the semaphore value. If the result is negative, the process is blocked until the resource becomes available.
signal() (V operation)	- A semaphore operation that increments the semaphore value and may wake up a blocked process waiting on the semaphore.
Binary Semaphore	- A type of semaphore that can hold only the values 0 or 1. It is typically used to enforce mutual exclusion.
Counting Semaphore	- A type of semaphore that can hold a non-negative integer value, representing the number of available resources.
Mutual Exclusion	- A condition in which only one process at a time is allowed to access a critical section or shared resource.
Busy Waiting	- A method where a process continuously checks a condition (like a semaphore) in a loop without yielding the CPU, often leading to inefficiency.

Blocking	- A synchronisation technique where a process voluntarily stops executing and is placed in a waiting queue until it is signaled to resume.
Spinlock	- A type of busy-wait synchronisation mechanism where a process repeatedly checks a lock variable until it can acquire the lock.
Atomic Operation	- An operation that runs completely without the possibility of interruption, ensuring that shared data is accessed safely.
Priority Inversion	- A condition where a higher-priority process is waiting for a resource held by a lower-priority process, causing unintended delays.
Starvation	- Starvation occurs when a process waits indefinitely because other processes continuously acquire the required resource before it.
Producer-Consumer Problem	- A classic synchronisation problem where producers generate data and insert it into a buffer, and consumers remove and process it.
Reader-Writer Problem	- A synchronisation problem that involves coordinating multiple readers and writers to ensure safe concurrent access to shared data.
wait queue	- A data structure used by semaphores to keep track of processes that are blocked and waiting for a resource.
Test-and-Set / Compare-and-Swap	- Hardware-level atomic instructions used to implement synchronisation primitives like locks and semaphores.
Synchronisation Mechanism	- A method or tool used to coordinate concurrent process execution, ensuring correct sequence and access to shared resources.
Context Switch	- The process of storing and restoring the state of a CPU so that multiple processes can share a single CPU resource effectively.
Shared Resource	- Any hardware or software resource that can be used by more than one process or thread, such as memory, files, or devices.
Thread	- A lightweight unit of process execution that can run concurrently with other threads within the same process.
Multitasking	- An operating system capability that allows multiple processes to run at the same time by rapidly switching between them.

Concurrency	-	The execution of multiple instruction sequences at the same time, often involving synchronisation to prevent conflicts.
Blocking Semaphore	-	A semaphore implementation where processes are suspended (blocked) instead of continuously checking, thus saving CPU cycles.
Semaphore Queue	-	A queue used internally by the semaphore to manage blocked processes waiting for the semaphore to become available.
Mutual Exclusion (Mutex)	-	A property that ensures that only one thread or process can access a resource or critical section at a time.
Initialisation of Semaphore	-	The process of setting the starting value of a semaphore, which determines the number of available resources or lock state.
Kernel-Level Semaphore	-	A semaphore implemented and managed by the operating system kernel, often used for inter-process synchronisation.
User-Level Semaphore	-	A semaphore managed entirely in user space, used mainly for synchronisation within a single process across multiple threads.
Fairness	-	A property of synchronisation mechanisms that ensures no process or thread is indefinitely delayed (prevents starvation).
Bounded Buffer	-	A fixed-size buffer shared between producer and consumer processes, commonly used in synchronisation problems.
Ownership	-	In synchronisation, the principle that only the process or thread that acquires a resource (like a mutex) should release it.

7. TERMINAL QUESTIONS

1. What is a semaphore and why is it used?
2. Distinguish between binary and counting semaphores.
3. Define the critical section problem.
4. What is busy waiting? Why is it inefficient?
5. How does blocking improve semaphore performance?
6. What are the key operations of a semaphore?
7. Explain race condition with an example.
8. What are spinlocks and where are they useful?
9. What role does atomicity play in semaphore operations?
10. What is starvation in the context of semaphores?
11. Explain the producer-consumer problem using semaphores.
12. Describe the reader-writer problem and how semaphores solve it.
13. Discuss the differences between mutexes and binary semaphores.
14. Analyse the drawbacks of using semaphores.
15. How does a semaphore differ from a lock or flag?
16. How does semaphore usage vary in single-core and multi-core systems?
17. Describe how semaphores prevent race conditions.
18. Write pseudocode to implement a counting semaphore.
19. What strategies can help avoid deadlocks when using semaphores?
20. How are semaphores applied in real-time and embedded systems?

8. ANSWERS

Self-Assessment Questions

1. c) wait()
2. integer
3. wait(), signal()
4. True
5. Semaphore
6. Proberen, Verhogen
7. c) Critical section problem
8. b) Put to sleep
9. False
10. b) Operation that runs without interruption
11. blocked
12. c) wait()
13. True
14. Atomic
15. False
16. True (in the internal logic, it indicates the number of blocked processes)
17. b) 0 or 1
18. c) Starvation
19. empty
20. binary
21. False
22. False
23. c) Automatic deadlock resolution
24. c) 1
25. b) Priority inheritance
26. positive
27. read_count
28. False
29. True

30. True

Terminal Questions Answers

Answer 1: A semaphore is an integer-based synchronisation tool that controls access to shared resources in concurrent systems. It uses atomic operations like `wait()` and `signal()` to prevent race conditions. Refer to Section 2 – Semaphore Concept

Answer 2: Binary semaphores take values 0 or 1 and ensure mutual exclusion, while counting semaphores manage multiple resource instances. Refer to Section 4 – Basics of Semaphore

Answer 3: The critical section problem involves ensuring that only one process accesses a shared resource at a time to maintain consistency. Refer to Section 2 – Semaphore Concept

Answer 4: Busy waiting is when a process repeatedly checks a condition without releasing CPU control, which wastes CPU time. Refer to Section 3 – Semaphore Usage

Answer 5: Blocking puts waiting processes to sleep rather than spinning, allowing the CPU to execute other tasks and improving system efficiency. Refer to Section 3 – Eliminating Busy Waiting with Blocking

Answer 6: The two main operations are `wait()` (to decrement and possibly block) and `signal()` (to increment and possibly wake a process). Refer to Section 2 – Semaphore Concept

Answer 7: A race condition occurs when processes read and write shared data concurrently, leading to unpredictable results. Refer to Section 2

Answer 8: Spinlocks are semaphores that use busy waiting and are effective in multi-core systems for short critical sections. Refer to Section 3 – Busy Waiting and Spinlocks

Answer 9: Atomicity ensures semaphore operations are indivisible, preventing data corruption during concurrent access. Refer to Section 3 – Atomicity and Interrupt Handling

Answer 10: Starvation occurs when a process waits indefinitely because other processes keep acquiring the semaphore before it can. Refer to Section 4 – Drawbacks of Semaphores

Answer 11: This problem involves producers inserting data into a bounded buffer and consumers removing it. Semaphores manage empty slots, filled slots, and mutual exclusion. Refer to Section 4 – Practical Examples of Semaphores

Answer 12: Semaphores like `rw_mutex` and `read_count_mutex` help coordinate access to shared data so multiple readers can read while writers maintain exclusivity. Refer to Section 4 – Reader-Writer Problem Using Semaphores

Answer 13: Mutexes enforce ownership (only the locker can unlock), whereas binary semaphores lack this restriction, offering general signaling capabilities. Refer to Section 4 – Mutex vs Binary Semaphore

Answer 14: Drawbacks include complexity, potential for misuse, deadlock, and lack of ownership tracking or encapsulation with shared data. Refer to Section 4 – Drawbacks of Semaphores

Answer 15: Unlike simple locks or flags, semaphores support counting and can block/wake processes based on availability, allowing more structured control. Refer to Section 4 – Basics of Semaphore

Answer 16: Single-core systems often disable interrupts to ensure atomicity, while multi-core systems rely on hardware-supported atomic instructions. Refer to Section 3 – Ensuring Atomicity

Answer 17: Semaphores ensure that only one process accesses the critical section by using `wait()` and `signal()` operations to manage entry and exit. Refer to Section 2 – Semaphore Concept

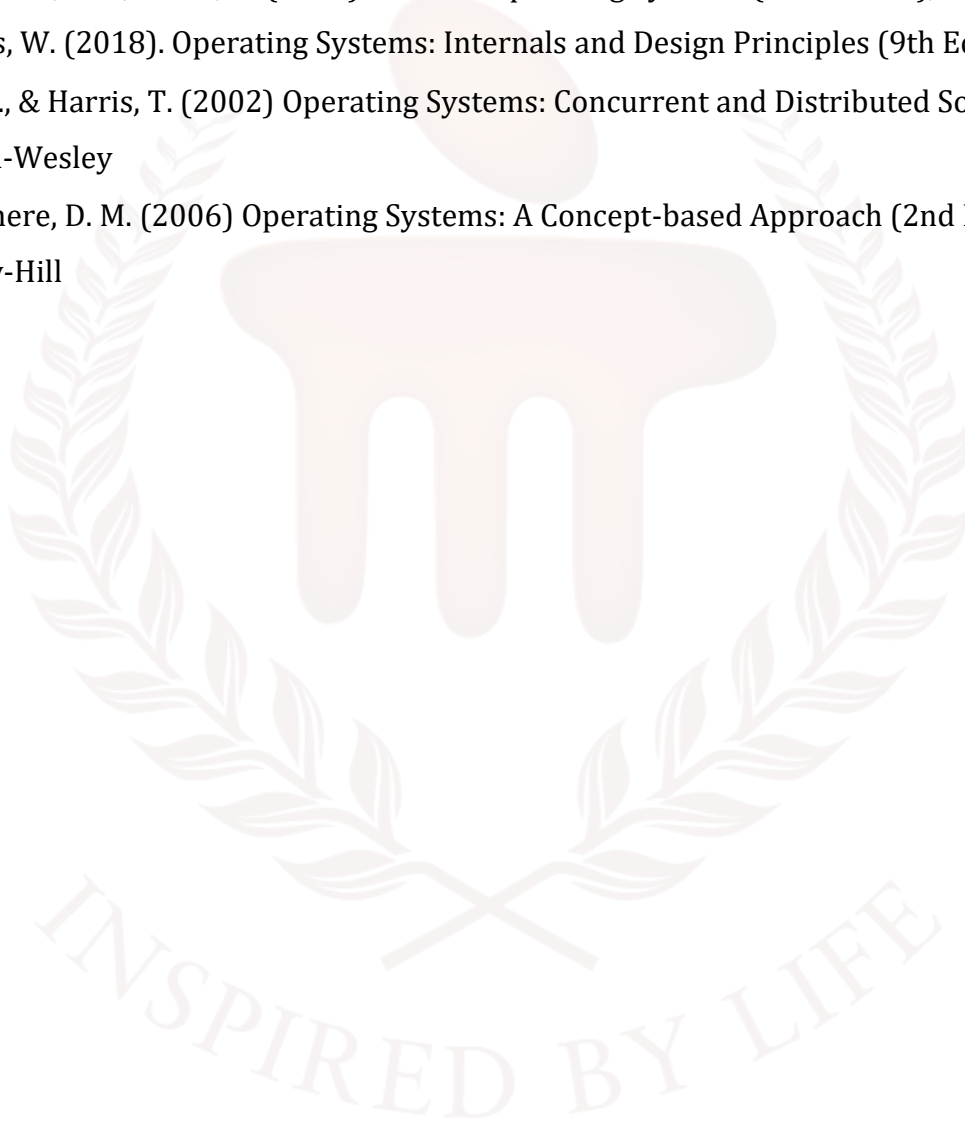
Answer 18: A counting semaphore uses `wait()` to decrement and `signal()` to increment its counter while managing blocked processes via a queue. Refer to Section 4 – Counting Semaphore

Answer 19: Avoid circular waits, enforce ordering, use timeout mechanisms, and apply techniques like resource hierarchy or wait-die schemes. Refer to Section 4 – Drawbacks of Semaphores

Answer 20: Semaphores ensure timely and predictable access to critical resources in real-time systems, maintaining strict scheduling and avoiding race conditions. Refer to Section 4 – Applicable in Real-Time Systems

9. REFERENCES

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition), Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th Edition), Pearson.
- Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th Edition), Pearson.
- Bacon, J., & Harris, T. (2002) Operating Systems: Concurrent and Distributed Software Design, Addison-Wesley
- Dhamdhere, D. M. (2006) Operating Systems: A Concept-based Approach (2nd Edition), Tata McGraw-Hill

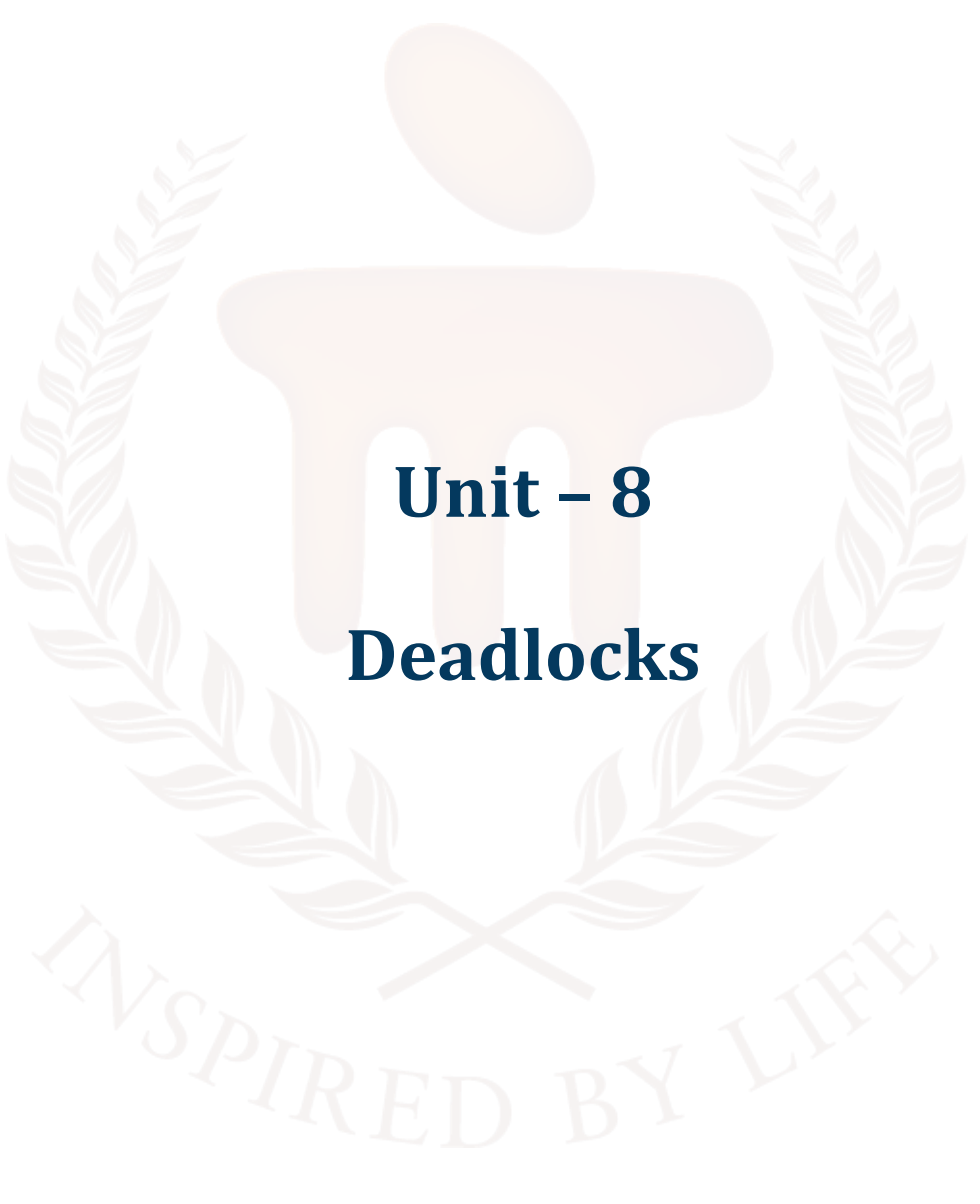


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 8

Deadlocks

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 7
1.1	Objectives	-	-	
2	Deadlock Characterisation	-	-	7 - 13
2.1	Types of Resources	-	-	
2.2	Necessary Conditions for Deadlock	1	-	
2.3	Resource-Allocation Graphs	2	-	
2.4	Deadlock Models (Single vs Multiple Instances)	3, 4	1	
3	Deadlock Prevention	-	-	14 - 20
3.1	Structuring Prevention Strategies	5	-	
3.2	Mutual Exclusion Elimination	-	-	
3.3	Hold-and-Wait Prevention	6	-	
3.4	No Pre-emption Techniques	-	-	
3.5	Circular-Wait Avoidance	7	2	
4	Deadlock Avoidance	-	-	21 - 27
4.1	Safe and Unsafe States	-	-	
4.2	Banker's Algorithm Principles	-	-	
4.3	Resource-Request Algorithm	-	-	
4.4	Implementation Challenges	-	-	
4.5	Performance Considerations	-	3	
5	Case Studies and Practical Considerations	-	-	28 - 35
5.1	Deadlock in UNIX-like Systems	-	-	
5.2	Database Transaction Deadlock	8	-	
5.3	Distributed Systems Scenarios	-	-	
5.4	Real-time System Constraints	-	4	
6	Summary	-	-	36 - 37
7	Glossary	-	-	38

8	Terminal Questions	-	-	39
9	Answers	-	-	40 - 41
10	References	-	-	42



1. INTRODUCTION

In the previous unit, we explored the fundamental concepts of process synchronisation and concurrency control in operating systems. We studied the critical section problem, various synchronisation tools like semaphores and monitors, and the challenges of coordinating multiple processes or threads accessing shared resources. This laid the groundwork for understanding the kinds of conflicts that can arise when several processes compete for limited system resources.

In this unit, we delve into the phenomenon of deadlocks a critical issue that can halt the progress of processes in a system. We begin with Deadlock Characterisation, identifying different types of resources, the four necessary conditions for deadlock, the use of Resource-Allocation Graphs, and how deadlocks manifest under different models, whether involving single or multiple instances of resources. Understanding these foundations is key to recognising when and how deadlocks arise.

Next, we cover two major system-level strategies for handling deadlocks: Deadlock Prevention and Deadlock Avoidance. Prevention techniques involve structuring resource allocation to ensure that at least one of the necessary conditions for deadlock is never satisfied whether by eliminating mutual exclusion, hold-and-wait, pre-emption, or circular-wait. In contrast, avoidance relies on careful planning and the dynamic evaluation of resource requests to ensure the system remains in a safe state, using tools like the Banker's Algorithm. We also examine the limitations, performance costs, and practical implementation concerns of these approaches.

Finally, the unit concludes with Case Studies and Practical Considerations, grounding the theory in real-world systems. We analyse deadlock behaviour in UNIX-like kernels, database transactions, distributed environments, and real-time systems, revealing the trade-offs and challenges unique to each context. These case studies highlight how theoretical deadlock strategies are adapted or bypassed entirely depending on system goals and constraints.

To study this unit effectively, begin by clearly understanding the four necessary conditions for deadlock and how they interrelate. Use visual tools like resource-allocation graphs to model scenarios, then contrast prevention and avoidance strategies, noting their system overheads and applicability. As you progress to the case studies, focus on how general principles are tailored to fit specific system architectures. Think critically about the balance between theoretical purity and practical design in deadlock management.

1.1. Objectives

After studying this unit, you should be able to:

- Classify different types of system resources and their roles in deadlock scenarios.
- Identify the conditions and models that result in deadlock occurrences.
- Apply deadlock prevention and avoidance techniques to system design problems.
- Analyse the functionality and impact of the Banker's and resource-request algorithms.
- Evaluate real-world deadlock issues in operating systems, databases, and distributed systems.



2. DEADLOCK CHARACTERISATION

Deadlock is a fundamental issue in concurrent computing. It arises when two or more processes cannot proceed because each is waiting for resources held by the others. In operating systems, understanding how deadlock occurs, how to represent it, and how different models affect detection and recovery is essential for designing robust resource-management schemes. This section explores the types of resources involved, the necessary conditions that give rise to deadlock, the graphical tools used to describe resource allocation, and the distinction between single-instance and multiple-instance deadlock models.

2.1. Types of Resources

In an operating system, a resource is any entity hardware or software that processes require for execution. Resources fall largely into two categories:

1. **Preemptable Resources:** These may be taken away from a process and reassigned without invalidating the computation. Typical examples include:
 - CPU cycles (the scheduler may pre-empt a running process).
 - Memory pages (the system can swap a page out).
2. **Non-preemptable Resources:** Once allocated to a process, these cannot be forcibly removed until the process relinquishes them. Examples include:
 - I/O devices such as printers or data tapes.
 - Files opened in exclusive mode.

A more nuanced classification recognises:

- **Consumable Resources:** Used upon allocation, then gone. For instance, semaphore signals (P/V operations) or data packets.
- **Reusable Resources:** Remain available after use. These include CPU registers, tape drives, or memory buffers.

Understanding the distinction is vital, since deadlock prevention strategies may rely on pre-emption (for preemptable resources) and are often impossible for non-preemptable ones.

2.2. Necessary Conditions for Deadlock

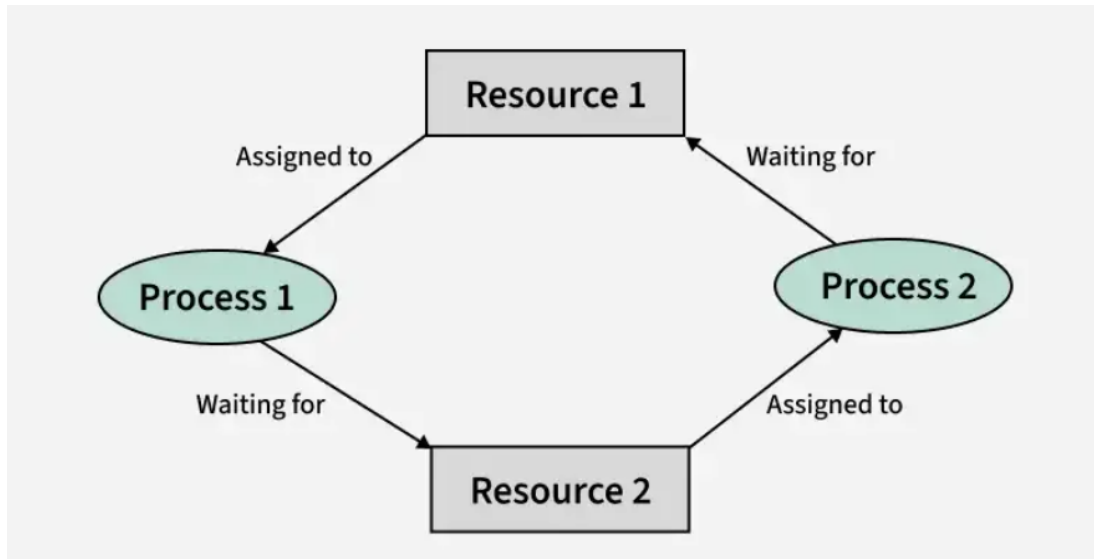


Figure 1: Occurrence of a Deadlock

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20250116142222043136/d.webp>

The figure illustrates a classic deadlock scenario involving two processes and two resources. Process 1 is assigned Resource 1 but is simultaneously waiting for Resource 2. Conversely, Process 2 is assigned Resource 2 while waiting for Resource 1. This circular dependency creates a deadlock, as neither process can proceed because each holds a resource that the other requires, and neither is willing to release its held resource. This situation is a common problem in concurrent systems, preventing the completion of tasks.

For deadlock to occur, four conditions commonly known as the Coffman conditions must hold simultaneously:

1. **Mutual Exclusion** - At least one resource must be held in a non-shareable mode. Only one process may use the resource at any given time.
2. **Hold and Wait** - A process holding at least one resource is waiting to acquire additional resources held by other processes.
3. **No Pre-emption** - Resources cannot be pre-empted; that is, they cannot be forcibly removed from the processes holding them until the resource is used to completion and released voluntarily.
4. **Circular Wait** - A set of processes $P_1, P_2, \dots, P_n$ exist such that P_1 is waiting for a resource held by P_2 , P_2 is waiting for a resource held by P_3 , and so on, with P_n waiting for a resource held by P_1 .

These conditions are both necessary and sufficient: if all four hold, a deadlock may result; breaking any one condition ensures deadlock cannot occur.

2.3. Resource–Allocation Graphs

A resource–allocation graph (RAG) provides a visual tool to detect and reason about deadlock. It is a directed bipartite graph with two types of nodes:

- Process nodes (circles) labelled $P_1, P_2, \dots$
- Resource nodes (squares) labelled $R_1, R_2, \dots$

Edges represent allocation and requests:

- Assignment edge ($R_i \rightarrow P_j$): resource R_i is currently allocated to process P_j .
- Request edge ($P_j \rightarrow R_k$): process P_j has requested an instance of resource R_k and is waiting.

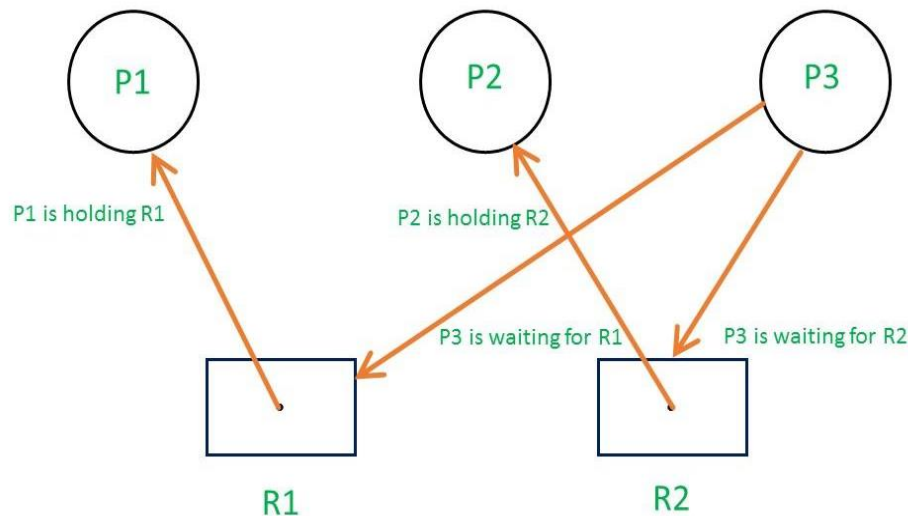


Figure 2: Resource Allocation Graph

Source: <https://media.geeksforgeeks.org/wp-content/uploads/Presentation1-1.jpg>

The figure presents a resource allocation graph illustrating a scenario without a deadlock, despite involving multiple processes and resources. Process P1 is holding Resource R1, while Process P2 is holding Resource R2. Process P3, however, is waiting for both Resource R1 and Resource R2. Importantly, there is no circular waiting dependency; although P3 is waiting, neither P1 nor P2 are waiting for a resource held by P3. This arrangement allows for the eventual completion of all processes, as resources can be released and allocated sequentially.

Detecting Deadlock via RAG

- Single instance of each resource: A cycle in the graph indicates deadlock.

- Multiple instances per resource: A more complex condition - if a cycle exists, deadlock may occur, but absence of cycle guarantees no deadlock.

Construction steps:

1. For each process that holds resources, draw assignment edges from resource nodes to that process.
2. For each waiting request, draw request edges from the process to the resource.
3. Examine the resulting graph for cycles or knots (see multiple-instance).

2.4. Deadlock Models (Single vs Multiple Instances)

Deadlock behaviour differs markedly depending on whether resources have a single instance or multiple instances.

Single-Instance Model: Each resource type has exactly one instance.

- **Deadlock detection:** A cycle in the RAG implies deadlock.

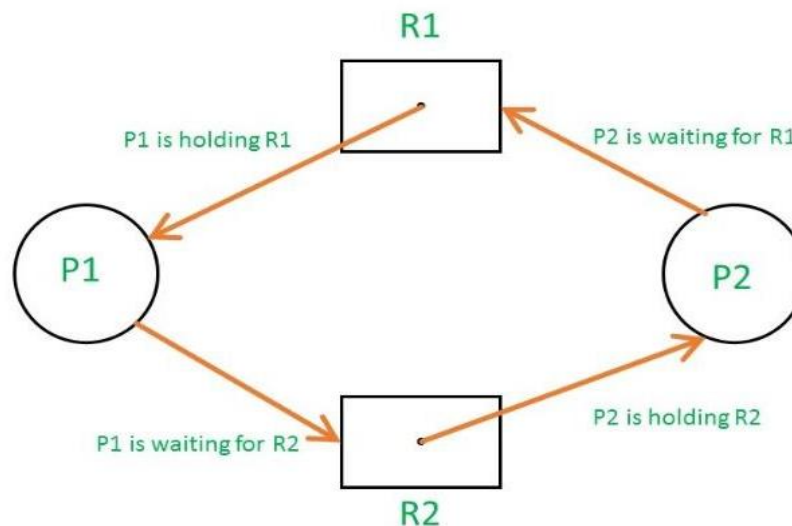


Figure 3: Single-Instance Model

Source: <https://media.geeksforgeeks.org/wp-content/uploads/Slide1.jpg>

The figure depicts a resource allocation graph demonstrating a deadlock situation with single instance resource types. Process P1 is holding Resource R1 and simultaneously waiting for Resource R2. At the same time, Process P2 is holding Resource R2 and waiting for Resource R1. This forms a cyclic dependency, where each process needs a resource held by the other, leading to a permanent blocking state. This visual representation clearly shows how the allocation and request of resources can result in a system deadlock.

- **Example scenario:** Printer device R_1 , scanner R_2 , plotter R_3 each unique. If P_1 holds R_1 and requests R_2 , while P_2 holds R_2 and requests R_1 , a cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$ emerges.

This simplicity permits straightforward detection but ignores real-world cases where, for instance, multiple identical printers exist.

Multiple-Instance Model: A resource type may have several identical instances.

- **Deadlock detection:** More complex; cycle in RAG is necessary but not sufficient.

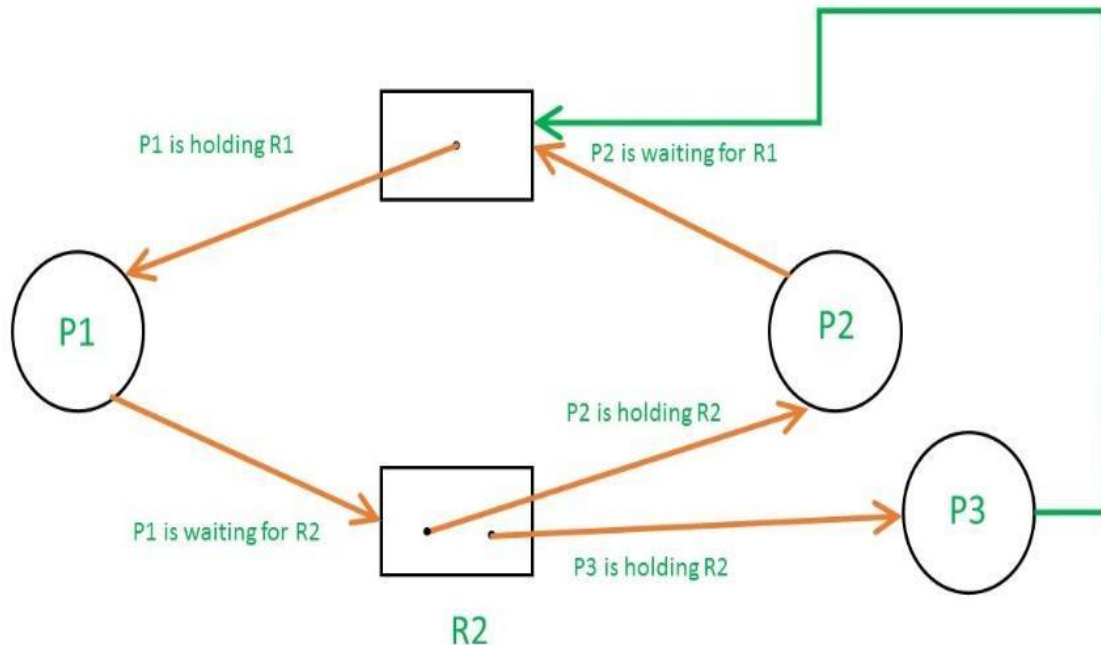


Figure 4: Multiple-Instance Model

Source: <https://media.geeksforgeeks.org/wp-content/uploads/Slide3.jpg>

The figure displays a resource allocation graph demonstrating a deadlock situation within a multi-instance resource model. Process P_1 holds an instance of Resource R_1 and is waiting for an instance of Resource R_2 . Concurrently, Process P_2 holds an instance of Resource R_2 and is waiting for an instance of Resource R_1 . Additionally, Process P_3 also holds an instance of Resource R_2 . The critical aspect leading to the deadlock is the circular wait between P_1 and P_2 for resources R_1 and R_2 , even though multiple instances of R_2 exist, as all available instances are held, creating a blocked state.

- **Wait-for graph:** A reduced graph where processes are nodes and an edge $P_i \rightarrow P_j$ exists if P_i is waiting for a resource held by P_j . Deadlock occurs if this graph contains a cycle.

Key distinctions:

Cycle versus knot: A cycle is a simple loop of waiting dependencies. A knot is a strongly connected component in the wait-for graph where no process can proceed because it is waiting for resources held within the same component.

Banker's algorithm applicability: In multiple-instance systems, the banker's algorithm calculates whether granting a request leads to an unsafe state. The request is granted only if the resulting state is safe meaning that a sequence of process completions exists without deadlock.

In systems with multiple resource instances, it is common to use safety algorithms to check for possible deadlocks before allocating resources. Since strict ordering becomes difficult to maintain in such dynamic environments, detection and recovery methods are often preferred. Although these approaches introduce some processing overhead, they offer better concurrency compared to simpler prevention techniques, making them more suitable for complex, multitasking operating systems.

Deadlock characterisation hinges upon identifying the nature of resources, understanding the four necessary conditions that allow deadlock, and employing graphical models to visualise and detect unsafe states. The distinction between single-instance and multiple-instance models further refines how an operating system must track and resolve circular waits. Mastery of these concepts underpins effective resource-management strategies, ensuring systems remain both safe and performant.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which of the following is an example of a non-preemptable resource? a) CPU cycles b) Memory pages c) A printer allocated exclusively to a process d) Semaphore signals
2	Which Coffman condition ensures that a process cannot be forcibly deprived of a resource once it has been allocated? a) Mutual exclusion b) Hold and wait c) No pre-emption d) Circular wait

3	In a resource-allocation graph with a single instance of each resource type, what does the presence of a cycle signify? a) No process is currently waiting b) Deadlock has occurred c) All resources are preemptable d) System is in a safe state
4	What is the key distinction between a cycle and a knot in a multiple-instance resource model's wait-for graph? a) A cycle involves only one process, whereas a knot involves all processes b) A cycle guarantees deadlock, whereas a knot indicates a strongly connected component of waiting processes c) A cycle occurs only with preemptable resources, whereas a knot occurs with consumable resources d) A cycle can be broken by the banker's algorithm, whereas a knot cannot
5	For which purpose is the banker's algorithm primarily used in a multiple-instance resource environment? a) To convert a resource-allocation graph into a bipartite graph b) To detect cycles in a single-instance model c) To determine whether granting a request leads to an unsafe state d) To classify resources as reusable or consumable

3. DEADLOCK PREVENTION

Deadlock prevention consists of design principles and algorithms that ensure the four necessary conditions for deadlock cannot all hold simultaneously. Rather than detecting and recovering from a deadlock once it occurs, prevention techniques impose constraints on how resources are requested and allocated. This approach sacrifices some degree of concurrency in order to guarantee system safety. In this unit, we examine the overall structuring of prevention strategies and then consider each of the four Coffman conditions in turn, showing how system design can eliminate or constrain them.

3.1. Structuring Prevention Strategies

A coherent preventive scheme must address each necessary condition: mutual exclusion, hold-and-wait, no pre-emption, and circular wait. In practice, one typically chooses one or two conditions to break, rather than attempting to eliminate all simultaneous possibilities, because complete prevention may impose prohibitive overhead or reduce system utilisation drastically.

Key elements of a structured prevention approach include:

- **Policy definition** - Establish clear rules by which processes request and release resources. These rules should be simple enough to enforce efficiently.
- **Resource ordering** - Assign a unique numerical ordering to each resource type, guiding processes to acquire resources only in ascending (or descending) order. This prevents circular wait.
- **Request aggregation** - Require processes to declare all required resources at start-up. If any cannot be allocated immediately, none are granted; the process waits until all become available, thus preventing hold-and-wait.
- **Pre-allocation versus incremental allocation** - Compare the trade-offs between granting all resources at once (which reduces concurrency but prevents hold-and-wait) and allowing incremental requests subject to resource ordering.
- **Runtime checks and roll-back** - When a process requests additional resources that would break prevention rules, the system can either delay the request or pre-empt some resources (if supported) and roll back the process to a previous safe state.

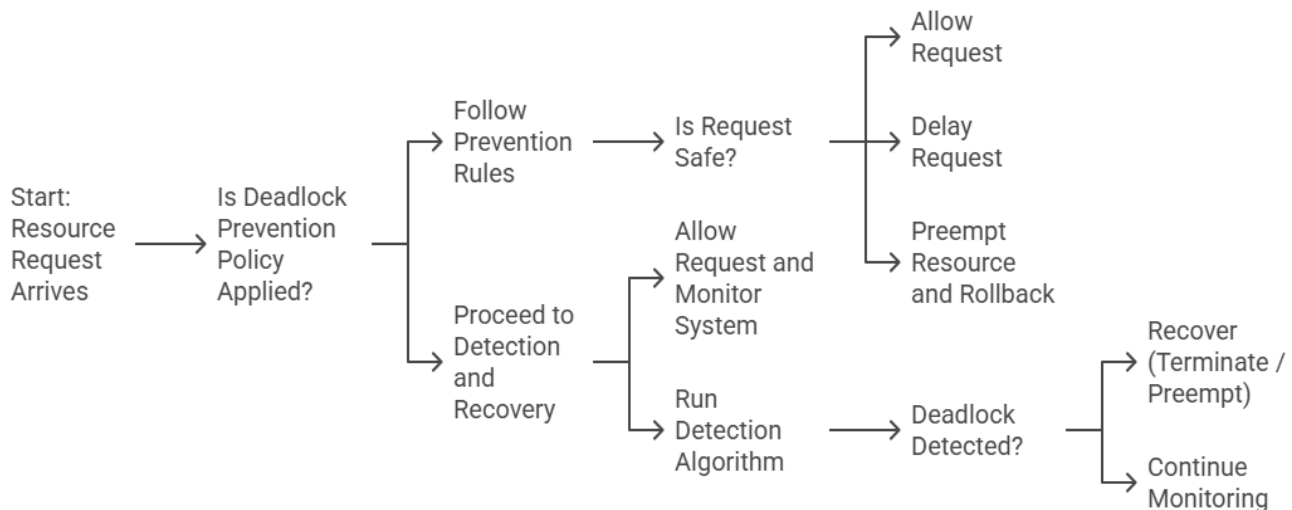


Figure 5: Resource Request Decision Path

(Image generated using an AI Tool for educational use)

The figure presents a flow chart illustrating the decision path for a resource request within an operating system. When a resource request arrives, the system first checks if a deadlock prevention policy is applied. If it is, the system follows prevention rules and assesses if the request is safe; based on this, it may allow or delay the request, or pre-empt resources and rollback. If no prevention policy is in place, the system proceeds to detection and recovery, allowing the request while monitoring the system. A detection algorithm is then run, and if a deadlock is identified, the system initiates recovery through termination or pre-emption, otherwise it continues monitoring.

3.2. Mutual Exclusion Elimination

The mutual exclusion condition requires that at least one resource type be held non-shareably. Since many resources (such as printers or exclusive files) inherently demand non-shareable access, complete elimination of mutual exclusion is rarely possible. However, where feasible, one can reduce the contention by employing techniques that permit shared or spooled access.

Techniques to reduce mutual exclusion

- a) Spooling and virtualisation:** Devices like printers or magnetic tapes may use spooling: processes write output to a shared buffer on disk, from which a dedicated spooler service interacts with the device. In effect, processes share the spool file rather than the device itself.
- b) Read-only sharing:** If multiple processes require only read access to a resource (for instance, a shared library or a configuration file), the resource can be opened in a read-only mode by all. Exclusive write access is still enforced, but read locks do not block each other.

- c) **Data replication:** Create multiple copies of frequently accessed data or files. Each process reads and writes to its local copy, reconciling changes through a consistency protocol. While this carries complexity, it can transform a non-shareable resource into several logically identical ones.
- d) **Logical resource partitioning:** Break a monolithic resource into smaller, independent parts. For example, divide a single large memory buffer into multiple buffers. Each part can then be allocated to different processes without mutual exclusion.

Even when mutual exclusion cannot be eliminated entirely, reducing the instances and scope of non-shareable resources decreases the risk of deadlock.

3.3. Hold-and-Wait Prevention

The hold-and-wait condition allows a process to hold resources while waiting for additional ones, which can lead to deadlocks. Preventing this requires ensuring that no process holds resources when requesting new ones, which can be done using several strategies.

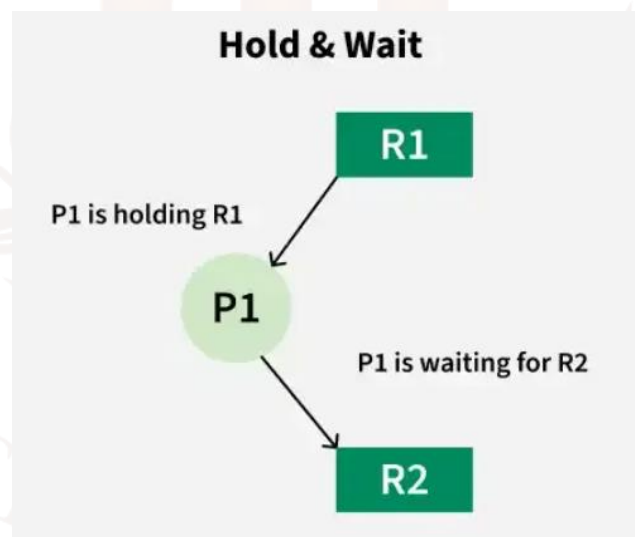


Figure 6: Hold and Wait

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20250129160805367050/HoldWait.webp>

The figure depicts a "Hold & Wait" condition, which is one of the necessary conditions for a deadlock to occur. In this illustration, Process P1 is shown to be holding Resource R1. Simultaneously, Process P1 is also waiting for Resource R2. This scenario perfectly exemplifies the "hold and wait" condition, where a process holds at least one resource while simultaneously waiting for additional resources that are currently being held by other processes.

One-step allocation: It forces processes to request all necessary resources at the beginning. If any are unavailable, the process waits without holding any, removing the possibility of hold-and-wait but often reducing efficiency.

Pre-request release: It requires a process to release all its currently held resources before asking for more. It supports gradual resource acquisition but may lead to frequent releases, increased overhead, and potential starvation if released resources are claimed by others.

Timeout and rollback: It allow a process to release all held resources if a needed one isn't granted within a set period, then retry later. This dynamic method prevents long waits but requires well-configured timeouts, or it may cause repeated retries.

Pre-emptive resource reallocation: It enables the system to forcibly take resources from waiting processes and roll them back. It eliminates hold-and-wait risks entirely but is practical only for preemptable resources and may result in high rollback costs.

3.4. No Pre-emption Techniques

The no pre-emption condition restricts the system from reclaiming resources once allocated to a process. To prevent deadlock, certain strategies introduce controlled pre-emption, allowing resources to be taken back under specific conditions.

Resource rollback involves deallocating all resources from a process if it requests additional unavailable ones, restarting it from the beginning to make resources available for others.

Partial pre-emption allows only certain resources, like memory, to be taken back, maintaining a balance between flexibility and system stability.

Pre-emptible resource classification divides resources into pre-emptible and non-pre-emptible categories during system design, so pre-emption rules apply only to the suitable types.

Checkpoint and rollback protocols let processes save their state periodically; if a conflict arises, the system rolls back the process to a previous checkpoint, releasing the held resources.

These techniques weaken the no pre-emption condition in a controlled way to support deadlock prevention. Each method brings trade-offs between system overhead, implementation complexity, and the level of protection offered against deadlocks.

3.5. Circular-Wait Avoidance

The circular wait condition describes a closed chain of processes each waiting for a resource held by the next in the chain. By imposing a strict ordering on resource types and requiring processes to request resources in order, circular waits can be precluded.

Resource ordering strategy

- A. Assign a unique numeric ordering** - Give each resource type R_i an integer value, $\text{Order}(R_i)$.
- B. Ordered request rule** - A process may request only resources with higher order values than those it already holds. If it needs a resource of lower order, it must first release all higher-ordered resources and then request both in proper sequence.
- C. Avoiding cycles** - Since all requests move strictly in one direction (increasing order), a cycle cannot form, as that would require a return to a lower-ordered resource.

Example of ordering

- Suppose resource types “printer” (Order = 1), “scanner” (Order = 2), “modem” (Order = 3).
- A process holding the scanner may request the modem (allowed), but must release the scanner before requesting the printer (lower order).

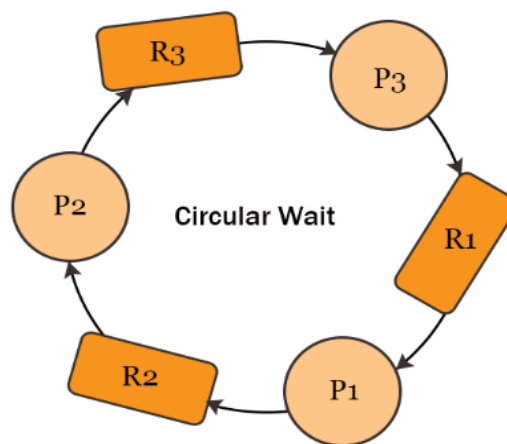


Figure 7: Circular-Wait Avoidance

Source: <https://cstaleem.com/wp-content/uploads/2020/05/circular-wait.png>

The figure illustrates the "Circular Wait" condition, which is a key requirement for a deadlock to form. In this arrangement, Process P1 is waiting for Resource R1, which is held by Process P3. Process P3 is waiting for Resource R3, held by Process P2. Finally, Process P2 is waiting for Resource R2, which is held by Process P1. This creates a closed chain of processes, each waiting for a resource held by the

next process in the cycle, thus preventing any of them from proceeding and resulting in a deadlock.

Circular wait avoidance enforces a fixed global order in which resources must be requested, preventing processes from forming circular chains. This method is simple to implement and removes the need for runtime deadlock detection. However, it can lead to overhead when processes with dynamic needs must release and reacquire resources to follow the order. It is most effective when resource types are known and stable, but less practical in systems with dynamic modules or devices.

Alternative avoidance schemes

- **Wait-for graph acyclicity:** Continuously maintain the wait-for graph and allow a request only if it does not create a cycle. Effectively the same as the banker's algorithm but without elaborate safety computations.
- **Hierarchical resource grouping:** Group resources into clusters, assign orderings to clusters rather than individual types. Within each cluster, use a finer ordering or other prevention measures. Reduces the complexity of maintaining global ordering.
- **Two-phase locking in databases:** A transaction must acquire all locks (read or write) in one phase before releasing any in the second phase. By assigning an order to data items (for instance, by their key values), circular waits in lock acquisition are avoided.

Deadlock prevention trades some concurrent performance for absolute safety, ensuring that the system never enters an unsafe state. By structuring prevention strategies around the four Coffman conditions mutual exclusion, hold-and-wait, no preemption and circular wait operating systems can maintain liveness. Each prevention technique carries its own overheads and application contexts. A well-designed resource manager will select the combination of strategies best suited to the workload, balancing throughput, resource utilisation and implementation complexity.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which prevention strategy requires a process to request all its needed resources at once before execution begins? a) Resource ordering b) One-step allocation c) Partial pre-emption d) Hierarchical grouping

7	Spooling is most directly used to address which deadlock condition? a) Hold and wait b) Mutual exclusion c) No pre-emption d) Circular wait
8	In circular-wait avoidance, what must a process do if it holds a higher-ordered resource but requires a lower-ordered one? a) Request both resources simultaneously b) Release the higher-ordered resource and then request in proper sequence c) Use timeout and rollback to obtain the lower-ordered resource d) Pre-empt the lower-ordered resource from another process
9	Which technique involves periodically saving a process's state so resources can be released and the process restarted if necessary? a) Resource ordering b) Checkpoint and rollback c) Request aggregation d) Read-only sharing
10	What is the primary purpose of request aggregation in deadlock prevention? a) To ensure processes hold resources as long as possible b) To eliminate the need for resource graphs c) To prevent hold-and-wait by granting resources only when all can be allocated d) To enforce a global resource order for all processes

4. DEADLOCK AVOIDANCE

Deadlock avoidance seeks to assign resources only when the system can guarantee that it will never enter an unsafe state. Unlike prevention, which prohibits certain usage patterns, avoidance dynamically inspects each request to determine if granting it preserves system safety. The canonical example is the banker's algorithm, which simulates allocations and checks future possibilities before committing resources. In practice, avoidance may impose substantial overhead, but it allows more concurrency than simple prevention schemes. This section examines safe and unsafe states, the principles of the banker's algorithm, the detailed resource-request algorithm, implementation challenges, and performance considerations.

4.1. Safe and Unsafe States

A safe state is one in which the system can allocate resources to each process in some order and still avoid deadlock. Conversely, an unsafe state does not necessarily imply deadlock is present, but it indicates that the system could enter a deadlock if processes request their maximum additional resources.

- **Available vector** - Indicates the number of available instances for each resource type. Denoted $A[j]$ for resource type j .
- **Maximum demand matrix** - $Max[i][j]$ specifies the maximum number of instances of resource type j that process P_i may request.
- **Allocation matrix** - $Allocation[i][j]$ records the number of instances of resource type j currently allocated to P_i .
- **Need matrix** - $Need[i][j] = Max[i][j] - Allocation[i][j]$, representing the remaining resource needs for P_i .

Safe-State Verification

To test if a state is safe, perform the following:

- Initialise a Work vector equal to Available.
- Mark all processes as unmarked.
- Find an unmarked process P_i such that $Need[i] \leq Work$. If none exists, go to step 5.
- If such P_i is found, set $Work \leftarrow Work + Allocation[i]$, mark P_i , and repeat step 3.
- If all processes are marked, the state is safe; otherwise, it is unsafe.

- **Interpretation:** The procedure simulates a sequence in which processes complete, release resources, and thereby allow remaining processes to finish.
- **Example:**

Suppose Available = [3, 2, 2], three processes P_0 , P_1 , P_2 , with Max and Allocation as follows:

Process	Allocation	Max	Need
P_0	[0, 1, 0]	[7, 5, 3]	[7, 4, 3]
P_1	[2, 0, 0]	[3, 2, 2]	[1, 2, 2]
P_2	[3, 0, 2]	[9, 0, 2]	[6, 0, 0]

- Work starts at [3, 2, 2].
- P_1 's Need [1, 2, 2] $\leq$ Work, so Work $\leftarrow$ [5, 2, 2], mark P_1 .
- P_0 's Need [7, 4, 3] $>$ Work, skip.
- P_2 's Need [6, 0, 0] $>$ Work, skip.
- No further matches, so state is unsafe if any unmarked remain.

This example shows an unsafe state, since P_0 and P_2 cannot be satisfied in sequence.

4.2. Banker's Algorithm Principles

The banker's algorithm is named by analogy with a banking system that grants loans cautiously so as never to run out of cash. It assumes that processes announce their maximum possible resource demands in advance. The algorithm grants requests only if it can ensure the system remains in a safe state.

Core Data Structures

- **Available:** vector of counts of free resource instances.
- **Max:** matrix of maximum resource demands.
- **Allocation:** matrix of resources currently assigned.
- **Need:** matrix of additional resources each process may still request; computed as Max – Allocation.

Algorithm Outline

1. **Initial Safety Check** - When the system starts, verify that the initial state (with zero allocations) is safe.
2. **Request Handling** - When process P_i requests $r[j]$ instances of resource R_j :
 - If $r[j] > \text{Need}[i][j]$, the request is invalid (over maximum claim).

- If $r[j] > \text{Available}[j]$, P_i must wait (not enough resources).
- Otherwise, pretend to allocate:
 - $\text{Available}[j] \leftarrow \text{Available}[j] - r[j]$
 - $\text{Allocation}[i][j] \leftarrow \text{Allocation}[i][j] + r[j]$
 - $\text{Need}[i][j] \leftarrow \text{Need}[i][j] - r[j]$
- Perform safe-state verification.
 - If safe, grant the request.
 - If unsafe, roll back to original values and make P_i wait.

3. Release Resources - When P_i releases resources, update:

- $\text{Available}[j] \leftarrow \text{Available}[j] + \text{release}[j]$
- $\text{Allocation}[i][j] \leftarrow \text{Allocation}[i][j] - \text{release}[j]$
- $\text{Need}[i][j] \leftarrow \text{Need}[i][j] + \text{release}[j]$

Example: Consider the system from section 4.1 with the same Available, Max and Allocation matrices.

Suppose P_0 requests $[0, 2, 0]$:

1. Check request $\leq$ Need: $[0, 2, 0] \leq [7, 4, 3]$, valid.
2. Check request $\leq$ Available: $[0, 2, 0] \leq [3, 2, 2]$, valid.
3. Pretend allocate:
 - Available becomes $[3, 0, 2]$
 - $\text{Allocation}[0]$ becomes $[0, 3, 0]$
 - $\text{Need}[0]$ becomes $[7, 2, 3]$
4. Run safety algorithm:
 - $\text{Work} = [3, 0, 2]$; mark unmarked processes whose $\text{Need} \leq \text{Work}$.
 - Only P_2 's $\text{Need} [6, 0, 0] > \text{Work}$; no process can proceed.
 - No one marked, so unsafe.
5. Roll back and have P_0 wait.

This ensures the system never enters an unsafe state, thereby preventing deadlock.

4.3. Resource-Request Algorithm

The resource-request algorithm is the practical implementation of the banker's principles at runtime. It responds to incoming requests and interacts with the safety verifier.

Algorithm Steps:

When process P_i issues a request vector $Request_i$:

1. **Validity check** - For each resource type j , if $Request_i[j] > Need[i][j]$, signal an error (over-commitment).
2. **Availability check** - If $Request_i[j] > Available[j]$ for any j , P_i must wait until resources are released.
3. **Pretend allocation**
 - $Available[j] \leftarrow Available[j] - Request_i[j]$
 - $Allocation[i][j] \leftarrow Allocation[i][j] + Request_i[j]$
 - $Need[i][j] \leftarrow Need[i][j] - Request_i[j]$
4. **Safety verification** - Invoke the safe-state algorithm described in section 4.1. If the state is safe, commit the allocation; otherwise, restore original values and have P_i wait.
5. **Grant or defer** - If safe, signal success to P_i .
 - If unsafe, P_i is blocked; the request remains pending until resources free up.

In practical systems, resource request handling involves several key elements to maintain efficiency and correctness. A blocking queue is maintained for processes whose requests cannot be immediately fulfilled, and these are re-evaluated in order when resources become available. All safety checks and resource updates must be done atomically, requiring mutual exclusion to avoid race conditions. Optimisations may include limiting safety checks to when the available resource state changes and indexing processes by their highest remaining need to speed up safety verification. These practices help ensure the algorithm remains both safe and efficient in real-time operation.

4.4. Implementation Challenges

Implementing deadlock avoidance, such as the banker's algorithm, poses several real-world challenges in operating systems.

- **Advance Declaration of Maximum Needs:** Processes must declare maximum resource needs in advance, which is manageable in batch systems but difficult in dynamic or interactive workloads. Incorrect estimates either too low or too high can cause system inefficiencies or violation of algorithm rules.
- **Computational Overhead:** Each resource request triggers a safety check that simulates allocations, leading to performance costs, especially with many processes and resource types. This complexity impacts scalability in larger systems.

- **Data Structure Management:** The algorithm relies on several matrices that consume memory and must be updated atomically. Locking these data structures ensures correctness but can reduce overall system performance if overused.
- **Dynamic Environments:** When processes enter or leave the system, matrices need resizing, and maintaining safety during these transitions is complex. The presence of dynamic devices further complicates accurate resource declaration.
- **Fault Tolerance:** System crashes or aborted processes require mechanisms to restore consistent resource states. Ensuring reliable checkpointing and partial rollbacks, particularly in distributed setups, adds another layer of complexity.

4.5. Performance Considerations

While deadlock avoidance preserves system safety more flexibly than prevention, it incurs costs that affect system performance and user experience. Striking an appropriate balance is essential.

Trade-Offs

- **Concurrency versus safety:** Strict checking reduces concurrency because many requests that would not actually cause deadlock are denied. More permissive policies increase risk of unsafe states or require heavier detection and recovery if deadlock occurs.
- **Latency of requests:** Each safety check adds delay before a request is granted. In time-sensitive applications, this may be intolerable. Batch-oriented systems can tolerate higher latencies, making avoidance more feasible.
- **Resource utilisation:** Conservative estimates of maximum demands lead to under-utilisation. Dynamic workloads, where needs fluctuate, require adaptive strategies.

To reduce overhead in deadlock avoidance, several strategies can be applied. Lazy safety checks postpone full verification until resource availability drops below certain levels or after multiple requests, allowing quicker approvals when risk is low. Request batching groups smaller demands from related processes into one check, cutting down on simulations. Priority-based granting relaxes checks for high-priority tasks, accepting some risk to improve responsiveness. Hybrid approaches combine avoidance for critical resources with detection and recovery for others, balancing safety and performance. Emerging techniques use machine learning to predict unsafe requests, guiding when full checks are necessary. Performance is measured through throughput, average response time, resource utilisation, and deadlock frequency in hybrid systems. Regular monitoring of these metrics helps adjust parameters like batching size and check frequency to optimise the system.

Deadlock avoidance offers a dynamic and flexible means of ensuring system safety, permitting greater concurrency than rigid prevention schemes. The banker's algorithm and its resource-request counterpart demonstrate how to simulate future allocations before committing resources. Yet practical implementation must address challenges of advance demand declaration, computational overhead, dynamic environments and fault tolerance. Careful performance measurement and hybrid techniques can mitigate these costs. Mastery of avoidance strategies enables systems to maintain liveness while delivering efficient resource use.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	What does the Need matrix represent in the banker's algorithm? a) The total resources available in the system b) The maximum resources each process may ever request c) The remaining resource requirements for each process d) The resources already allocated to each process
12	During the banker's algorithm request handling, what happens if a process's request exceeds the Available vector? a) The system pretends to allocate and then checks safety b) The request is invalid and an error is signalled c) The process must wait until resources are released d) The system rolls back all allocations of the process
13	Which of the following is a common technique to reduce the overhead of safety checks in deadlock avoidance? a) Increasing the size of the Available vector b) Lazy safety checks based on resource availability thresholds c) Enforcing global resource ordering d) One-step allocation of all resources at start
14	Which challenge arises from requiring advance declaration of maximum resource needs in the banker's algorithm? a) Difficulty in detecting cycles in the wait-for graph b) Excessive memory usage for the Available vector c) Incorrect estimates causing either under-utilisation or rule violation d) Inability to pre-empt non-preemptable resources

- | | |
|----|---|
| 15 | In safe-state verification, what does marking a process signify?
a) The process has been granted its maximum resources
b) The process cannot proceed due to resource shortages
c) The process can complete with the current Work vector and release its resources
d) The process has exceeded its declared maximum demand |
|----|---|



5. CASE STUDIES AND PRACTICAL CONSIDERATIONS

While theoretical models of deadlock characterisation, prevention, and avoidance provide essential foundations, real-world systems often present complications that demand nuanced handling. This section examines concrete case studies in UNIX-like operating systems, database transaction managers, distributed environments, and real-time systems. Each subtopic illustrates how deadlock manifests in practice, the strategies employed for detection, avoidance or recovery, and the trade-offs that architects must weigh.

5.1. Deadlock in UNIX-like Systems

In UNIX-like kernels, resource management spans everything from process scheduling to file-system operations and inter-process communication. Deadlock can arise within kernel subsystems often unexpectedly due to complex interactions among locks and shared data structures. Understanding typical locking patterns and common pitfalls is key to designing resilient kernels.

Kernel Locks and Concurrency

Most modern UNIX variants employ fine-grained locking to improve concurrency. Types of locks include:

- **Spinlocks** - Used in low-level code where the critical section is short; busy-waiting on multi-processor systems can be efficient.
- **Mutexes (sleep locks)** - Processes that cannot acquire the lock are put to sleep, avoiding CPU wastage at the expense of context-switch overhead.
- **Read-write locks** - Allow multiple readers to hold the lock concurrently but require exclusive access for writers.
- **Seqlocks** - Optimise read-heavy workloads by letting readers proceed without blocking writers, at the cost of possible retries.

Deadlocks within the kernel commonly arise from issues related to lock acquisition and ordering. One frequent scenario is locking recursion without proper protection, where a function holding a lock attempt to acquire the same lock again, resulting in a self-induced deadlock. Another typical cause is inconsistent lock ordering between subsystems; for example, one subsystem may acquire locks in the order A then B, while another does so in the reverse order, creating a circular wait. Additionally, complex resource chains in input/output subsystems can lead to deadlocks when processes hold file-

system locks while requesting buffers, device resources, or scheduler locks. Upgrading kernel read-write locks also poses a risk, as a thread holding a read lock may block indefinitely while trying to acquire a write lock if other readers remain active.

To manage these risks, many UNIX-like kernels avoid complex deadlock detection by enforcing strict lock-ordering policies. A global lock hierarchy defines a total order for acquiring locks, requiring all routines to follow this sequence; violations often result in diagnostic panics. Development tools may include lock validators that monitor acquisition sequences to warn of potential inversions. In safety-critical or embedded systems, watchdog timers monitor lock durations and can trigger kernel panics or scheduler pre-emptions to recover from deadlocks, ensuring system responsiveness even in adverse conditions.

Case Study: Linux's RCU and Locks

The Linux kernel's Read-Copy-Update (RCU) mechanism allows readers to proceed without locks, deferring updates until readers quiesce. However, combining RCU with conventional locks can introduce subtle deadlocks:

- A writer in an RCU-protected section might acquire a mutex and then wait for an RCU grace period, while another thread holds the mutex and tries to call an RCU read-side function that waits for mutual exclusion, producing a circular wait.

Linux addresses such scenarios via:

- **Explicit lock-dep tracing** - A debugging feature that records dynamic lock dependencies and flags potential circularities.
- **Context checks** - Forbidding RCU callbacks from contexts where blocking could violate assumptions.

5.2. Database Transaction Deadlock

Database management systems (DBMS) routinely deal with concurrent transactions acquiring locks on shared data items. Deadlock among transactions can lead to reduced throughput and must be managed automatically by the DBMS.

Two-Phase Locking Protocol

Most DBMS employ strict two-phase locking (2PL):

- A. Growing phase:** A transaction acquires all the locks (shared or exclusive) it needs, without releasing any.
- B. Shrinking phase:** Upon first release of a lock, the transaction enters the shrinking phase and cannot acquire further locks.

While 2PL ensures serialisability of transactions, it makes deadlock possible, since two transactions may hold locks the other requires.

Wait-for Graph and Cycle Detection

The DBMS builds a wait-for graph, whose nodes represent active transactions and whose directed edges denote waiting relations ($T_1 \rightarrow T_2$ if T_1 waits for a lock held by T_2). Deadlock detection proceeds by:

- Periodically scanning the graph for cycles, using depth-first search or variations of Tarjan's strongly-connected components algorithm.
- If a cycle is detected, the system must resolve it via transaction aborts.

Deadlock Resolution - Options for recovery include:

- **Victim selection:** Choose one transaction in the cycle to abort and roll back. Criteria may include:
 - **Youngest transaction** (minimises wasted work).
 - **Least partial work done** (minimises rollback cost).
 - **Lowest priority** (reflects business rules).
- **Timeouts:** Impose a maximum wait time; any transaction exceeding it is aborted. This is simple but may spur unnecessary aborts in long but legitimate waits.
- **Abort cascades:** Some systems pre-emptively abort dependent transactions to prevent cascading rollbacks, though this can reduce throughput.

Example Scenario

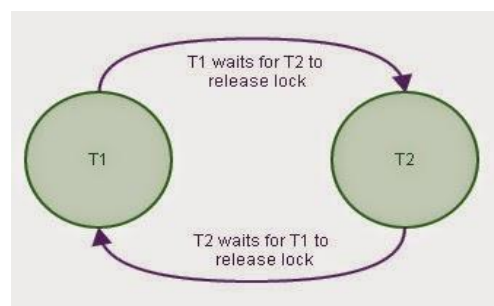


Figure 8: Wait-for Graph

Source: <https://www.exploredatabase.com/2014/04/deadlock-detection-technique-in-database.html>

Consider two transactions, T_1 and T_2 :

- T_1 holds an exclusive lock on row A and requests an exclusive lock on row B.
- T_2 holds an exclusive lock on row B and requests an exclusive lock on row A.

Wait-for graph: $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$, forming a cycle.

Resolution: abort the transaction chosen as victim say, T_2 . Its locks are released, allowing T_1 to proceed; T_2 then restarts.

5.3. Distributed Systems Scenarios

In distributed environments, resources and processes span multiple nodes. Deadlock detection and resolution becomes more complex due to partial knowledge, message delays, and potential failures.

Resource Models in Distribution: Distributed deadlock models classify resource dependencies as:

- **AND model** - A process waits until it acquires all resources in its request set.
- **OR model** - A process may proceed when it acquires any one of several alternative resources.
- **AND-OR (general) model** - Combinations of AND and OR dependencies, capturing sophisticated synchronisation patterns.

Detection Algorithms

1. **Centralised detection** - A single coordinator collects wait-for graphs from all nodes and runs cycle detection.
 - **Advantages:** Simpler algorithm.
 - **Drawbacks:** Single point of failure and potential bottleneck.
2. **Hierarchical detection** - Nodes are organised into a tree. Each level aggregates local wait-for graphs and passes condensed information upward.
 - **Advantages:** Reduces data volume and avoids a single coordinator.
 - **Drawbacks:** More complex to implement; partial delays can delay detection.
3. **Fully distributed detection** - Each node runs a probe-based algorithm. A probe message for transaction T carries the initiator's identity and traverses wait-for edges across nodes. If it returns to the originator, a cycle exists.
 - **Advantages:** No central point; scales with nodes.

- **Drawbacks:** High message overhead; risk of false positives if partial failures occur.

Recovery in Distributed Systems

After detecting a deadlock cycle, the system must choose a victim (often the transaction with least work or lowest global priority) and ensure all nodes roll back its partial changes. Distributed rollback protocols must guarantee atomicity across nodes, usually by:

- Employing two-phase commit in reverse: notifying all participants to abort rather than commit.
- Maintaining undo logs to revert changes.

Real-world Example: In a distributed file system, deadlock can occur when Client A holds a lock on Node 1 and requests a lock on Node 2, while Client B holds a lock on Node 2 and requests a lock on Node 1. This creates a cycle in the distributed wait-for graph across the two servers. A distributed probe algorithm detects this cycle by sending messages between servers until the cycle is confirmed. Recovery involves aborting one client's operation to release its locks and break the deadlock.

5.4. Real-time System Constraints

Real-time operating systems (RTOS) demand bounded response times and predictable behaviour. Deadlock resolution in such settings must respect timing constraints and often cannot tolerate undue delays or aborts.

Priority Inversion and Deadlock Interaction

In real-time systems, priority inversion occurs when a high-priority task waits on a low-priority one due to resource locking. If a medium-priority task pre-empts the low-priority task, the high-priority task may wait indefinitely behaving like a deadlock.

To mitigate this, RTOS employ protocols such as:

- **Priority inheritance** - The low-priority task holding the resource temporarily inherits the higher priority, preventing pre-emption by medium-priority tasks until it releases the resource.
- **Priority ceiling protocol (PCP)** - Each resource is assigned a ceiling equal to the highest priority of any task that may lock it. A task may lock a resource only if its priority exceeds the current system ceiling; this prevents deadlocks and bounds priority inversion.

- **Immediate ceiling protocol (ICP)** - Similar to PCP but raises the task's priority immediately to the resource's ceiling upon acquisition, potentially avoiding lock blocking by other tasks.

Deadlock Avoidance in RTOS

Because aborting or rolling back real-time tasks may violate deadlines, avoidance rather than detection is crucial:

- Static analysis of resource graphs - Compile-time checks ensure that no circular wait can arise, given known task-resource utilisation patterns.
- Non-blocking synchronisation primitives - Use lock-free or wait-free data structures (for example, ring buffers with atomic pointers) so that tasks never block waiting for others.
- Resource reservation and budgeting - Allocate time slots or resource budgets in advance, guaranteeing that each task can complete its critical section within its deadline.

Example: Aero-control Systems

In an aircraft's flight-control computer:

- Sensor-fusion tasks, guidance tasks and actuator-control tasks interact via shared data buffers and communication buses.
- Strict cyclic scheduling and on-chip memory banks reduce the need for locks; where locks remain, PCP is employed to prevent deadlock and bound priority inversion.
- Worst-case execution time (WCET) analysis ensures that even under contention, all tasks meet their deadlines.

These case studies highlight that no single deadlock strategy suits every environment. Key takeaways include:

- **UNIX-like kernels** favour strict lock ordering and development-time detection over runtime recovery, owing to performance demands and the difficulty of rolling back kernel state.
- **Database systems** balance deadlock detection with transaction abort policies tuned to minimise wasted work and maintain consistency.
- **Distributed systems** confront partial knowledge and communication delays; detection algorithms range from centralised to fully distributed, each with trade-offs in scalability and message overhead.
- **Real-time systems** must guarantee timeliness and predictability, typically avoiding deadlock altogether via static analysis, non-blocking primitives, and priority protocols.

In all cases, designers must weigh concurrency, overhead, complexity and safety. Diagrammatic representations such as wait-for graphs, resource allocation graphs, timing charts and lock-dependency hierarchies aid comprehension and validation. The choice of technique will depend on workload characteristics, system architecture and the criticality of timely execution.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
16	In UNIX-like kernels, which locking mechanism permits readers to proceed without blocking writers, potentially requiring retries? a) Spinlocks b) Mutexes c) Seqlocks d) Read–write locks
17	Under strict two-phase locking in database systems, in which phase does a transaction release its first lock and thereafter cannot acquire any more? a) Growing phase b) Shrinking phase c) Commit phase d) Validation phase
18	Which distributed deadlock detection approach avoids a single point of failure by sending probe messages among nodes? a) Centralised detection b) Hierarchical detection c) Fully distributed detection d) Hybrid detection
19	Which real-time scheduling protocol temporarily elevates the priority of a low-priority task holding a resource to prevent unbounded priority inversion? a) Priority ceiling protocol b) Immediate ceiling protocol c) Priority inheritance d) Static cyclic scheduling

20	In database deadlock recovery, what method involves selecting one transaction in the cycle to abort and roll back? a) Timeout enforcement b) Victim selection c) Abort cascade d) Lock escalation
----	--



6. SUMMARY

- Deadlock arises when processes wait indefinitely for resources held by one another, requiring analysis of resource types preemptable, non-preemptable, consumable and reusable to understand where prevention or recovery may be applied.
- The four Coffman conditions (mutual exclusion, hold-and-wait, no pre-emption and circular wait) must all be present for deadlock to occur; breaking any one condition is sufficient to prevent it.
- Resource-allocation graphs depict processes and resources as nodes with request and assignment edges; in single-instance systems, any cycle signals deadlock, while in multiple-instance systems a cycle is necessary but not sufficient.
- In single-instance models each resource type has one unit, making cycle detection straightforward, whereas multiple-instance models require wait-for graphs and safety algorithms (such as the banker's algorithm) to distinguish simple cycles from true deadlocks.
- Deadlock prevention imposes constraints on resource requests via resource ordering, request aggregation, one-step allocation or pre-allocation versus incremental allocation and may use rollback or timeout mechanisms to eliminate hold-and-wait and circular-wait conditions.
- Techniques such as spooling, read-only sharing, replication and logical partitioning can reduce or remove mutual exclusion requirements, while controlled pre-emption and checkpoint-and-rollback protocols address the no pre-emption condition.
- Circular-wait avoidance enforces a global ordering of resources; processes must request resources in increasing order or release higher-order resources before acquiring lower-order ones, thereby eliminating cycles by design.
- Deadlock avoidance dynamically evaluates each request's impact on system safety: the banker's algorithm simulates allocations using Available, Max, Allocation and Need matrices, granting requests only if the ensuing state remains safe.
- Practical implementation of avoidance methods faces challenges in advance demand declaration, computational and memory overheads, dynamic workload adaptation and fault tolerance; optimisations include lazy safety checks, request batching, priority-based granting and hybrid schemes.
- Real-world systems illustrate varied approaches: UNIX-like kernels favour strict lock hierarchies and static analysis, databases use two-phase locking with cycle detection and victim selection,

distributed systems employ centralised or probe-based algorithms, and real-time systems rely on priority protocols and non-blocking primitives to preserve timeliness.



7. GLOSSARY

Deadlock	- A situation in which two or more processes are unable to proceed because each is waiting for a resource held by another in the set.
Mutual Exclusion	- The requirement that a resource cannot be shared and is held by only one process at a time.
Hold-and-Wait	- A condition in which a process holds one resource while waiting to acquire additional resources held by other processes.
No Pre-emption	- The restriction that once a resource is allocated to a process, it cannot be forcibly taken away until the process releases it voluntarily.
Circular Wait	- A scenario where a closed chain of processes exists, each waiting for a resource held by the next process in the chain.
Resource-Allocation Graph	- A directed bipartite graph representing processes and resources, with edges denoting current allocations and pending requests.
Safe State	- A system state in which there exists at least one sequence of process executions that can complete without leading to deadlock.
Banker's Algorithm	- A deadlock-avoidance algorithm that simulates resource allocation requests and ensures that the system remains in a safe state before granting them.
Wait-For Graph	- A simplified graph in which nodes are processes and a directed edge from P_1 to P_2 indicates that P_1 is waiting for a resource held by P_2 .
Two-Phase Locking (2PL)	- A protocol for database transactions in which locks are acquired during a growing phase and released during a subsequent shrinking phase to ensure serialisability.

8. TERMINAL QUESTIONS

1. Explain the distinction between preemptable and non-preemptable resources and give two examples of each.
2. Describe the four necessary conditions for deadlock and discuss how breaking one of these conditions prevents deadlock.
3. Illustrate how a resource-allocation graph can be used to detect deadlock in a single-instance system.
4. Compare and contrast the single-instance and multiple-instance deadlock models, noting how each affects detection strategies.
5. Outline a structured prevention strategy that employs resource ordering and explain its advantages and disadvantages.
6. Discuss how spooling and data replication can reduce mutual exclusion and aid deadlock prevention.
7. Examine the banker's algorithm's approach to safe-state verification, including an example of its safety check procedure.
8. Identify the main implementation challenges of deadlock avoidance in real operating systems and suggest possible optimisations.
9. Analyse how two-phase locking and wait-for graph cycle detection operate together to manage deadlock in database transaction systems.
10. Evaluate the special considerations required for avoiding or resolving deadlock in real-time systems, with reference to priority inheritance and the priority ceiling protocol.

9. ANSWERS

Self-Assessment Questions

1. a) A printer allocated exclusively to a process
2. c) No pre-emption
3. b) Deadlock has occurred
4. b) A cycle guarantees deadlock, whereas a knot indicates a strongly connected component of waiting processes
5. c) To determine whether granting a request leads to an unsafe state
6. b) One-step allocation
7. b) Mutual exclusion
8. b) Release the higher-ordered resource and then request in proper sequence
9. b) Checkpoint and rollback
10. c) To prevent hold-and-wait by granting resources only when all can be allocated
11. c) The remaining resource requirements for each process
12. c) The process must wait until resources are released
13. b) Lazy safety checks based on resource availability thresholds
14. c) Incorrect estimates causing either under-utilisation or rule violation
15. c) The process can complete with the current Work vector and release its resources
16. c) Seqlocks
17. b) Shrinking phase
18. c) Fully distributed detection
19. c) Priority inheritance
20. b) Victim selection

Terminal Questions Answers

Answer 1: Preemptable resources can be taken from a process and reassigned without invalidating its computation (e.g., CPU cycles and memory pages). Non-preemptable resources, once allocated, cannot be forcibly removed until the process releases them (e.g., printers and files opened in exclusive mode). Refer section 2.1

Answer 2: Deadlock requires all Coffman conditions mutual exclusion, hold-and-wait, no pre-emption, and circular wait to hold; breaking any one (for example, by enforcing one-step allocation

to eliminate hold-and-wait) ensures deadlock cannot occur. Refer section 2.2

Answer 3: In a single-instance system, construct a bipartite graph of processes and resources; any directed cycle in this graph indicates a deadlock because each process in the cycle waits for a resource held by the next. Refer section 2.3

Answer 4: Single-instance models have one unit per resource type, so any cycle in the RAG guarantees deadlock, whereas multiple-instance models permit several units and require wait-for graph analysis because a cycle is necessary but not sufficient for deadlock. Refer section 2.4

Answer 5: Assign a global numerical order to resources and require processes to request them in increasing order; this prevents circular wait (no cycles) but can incur overhead when releasing and reacquiring resources if the needed order is violated. Refer section 3.1

Answer 6: Spooling allows processes to share a buffer (e.g., printer spool) rather than the device itself, and data replication gives each process its own copy of data, both transforming non-shareable resources into shareable ones and reducing mutual-exclusion locks. Refer section 3.2

Answer 7: The algorithm simulates granting a resource request by adjusting Available, Allocation, and Need matrices and then performs the safety check: if all processes can finish in some order (i.e., marked by $\text{Need} \leq \text{Work}$), the state is safe and the request is granted. Refer section 4.1

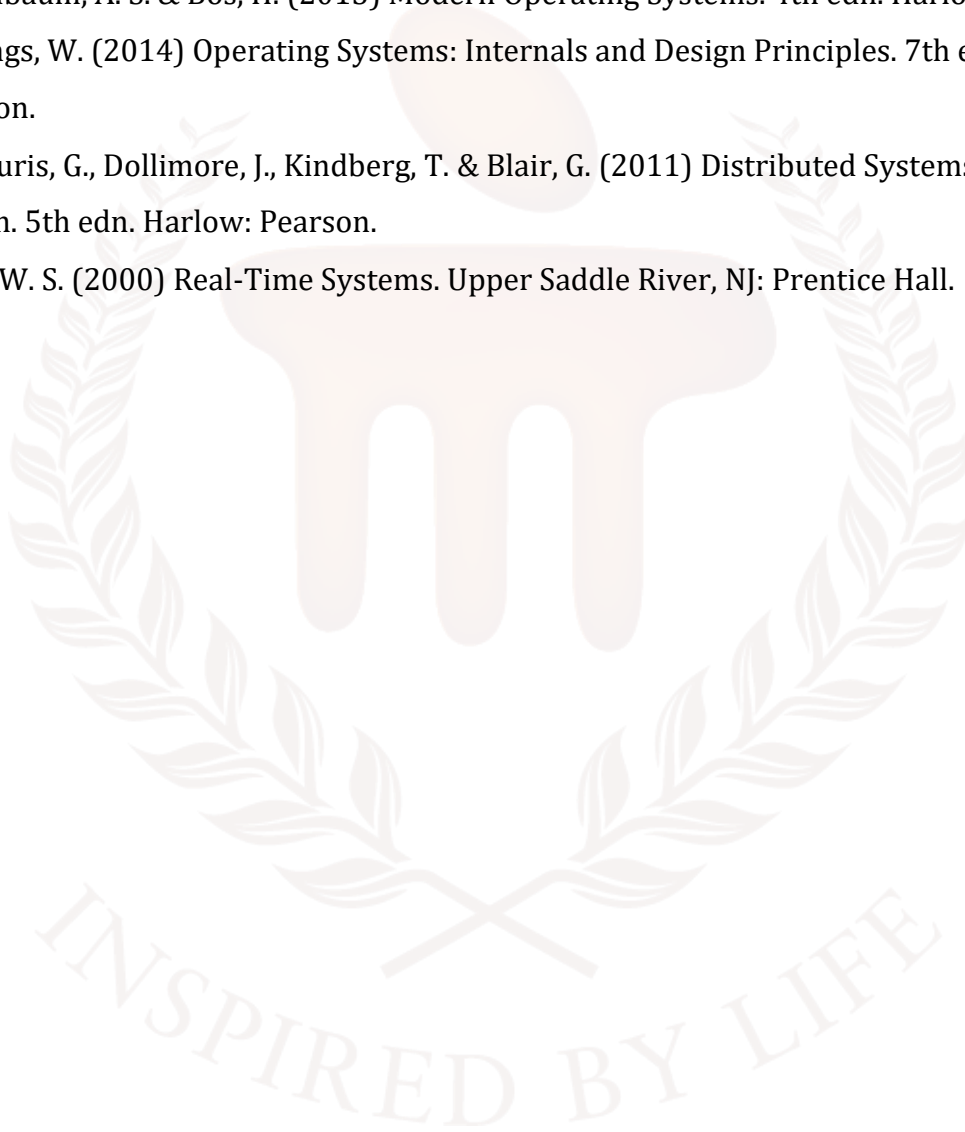
Answer 8: Major hurdles include requiring advance maximum-need declarations, computational overhead of frequent safety checks, dynamic resizing of matrices in changing workloads, and ensuring atomic updates for fault tolerance; optimisations like lazy checks and request batching can mitigate cost. Refer section 4.4

Answer 9: Strict two-phase locking ensures serialisability by a growing phase (acquire all locks) and shrinking phase (release), while the DBMS builds a wait-for graph of transactions and detects cycles to identify deadlocks, aborting a chosen victim to recover. Refer section 5.2

Answer 10: Real-time systems avoid deadlock by using priority inheritance (elevate a lock holder's priority to that of waiting tasks) or the priority ceiling protocol (prevent a task from locking if its priority doesn't exceed the system ceiling), thus bounding priority inversion and eliminating circular waits. Refer section 5.4

10. REFERENCES

- Silberschatz, A., Galvin, P. B. & Gagne, G. (2018) Operating System Concepts. 10th edn. Hoboken, NJ: Wiley.
- Tanenbaum, A. S. & Bos, H. (2015) Modern Operating Systems. 4th edn. Harlow: Pearson.
- Stallings, W. (2014) Operating Systems: Internals and Design Principles. 7th edn. Harlow: Pearson.
- Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. (2011) Distributed Systems: Concepts and Design. 5th edn. Harlow: Pearson.
- Liu, J. W. S. (2000) Real-Time Systems. Upper Saddle River, NJ: Prentice Hall.

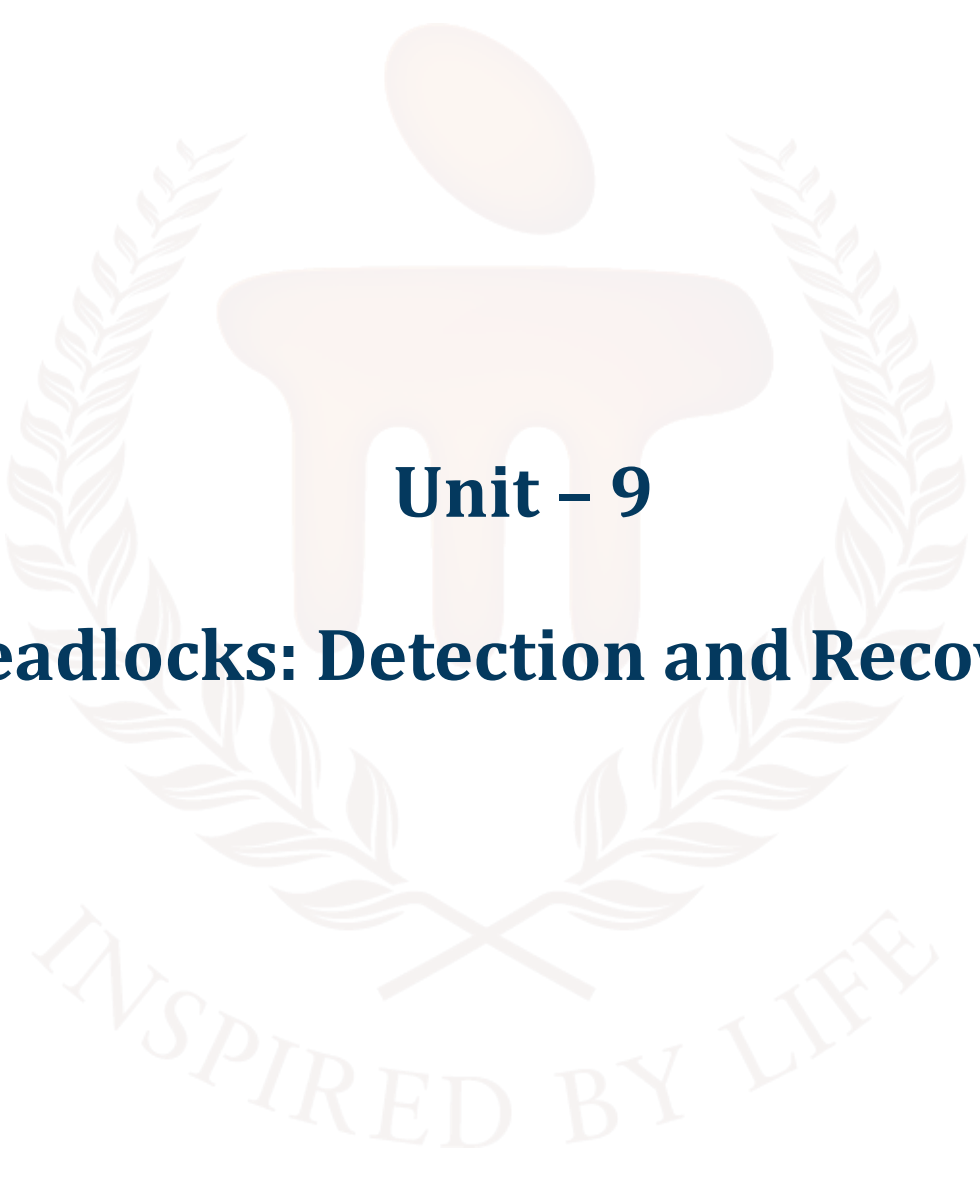


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 9

Deadlocks: Detection and Recovery

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Deadlock Detection	-	-	6 - 16
2.1	Detection in Single-Instance Systems	1	-	
2.2	Wait-for Graph Technique	2	-	
2.3	Detection Algorithm for Multiple Instances	-	-	
2.4	Frequency and Overhead of Detection	3	-	
2.5	Detection in Distributed Environments	4	1	
3	Recovery from Deadlock	-	-	17 - 24
3.1	Process Termination Strategies	-	-	
3.2	Resource Pre-emption Methods	-	-	
3.3	Checkpointing and Rollback	-	-	
3.4	Recovery Cost Analysis	5	-	
3.5	Practical Considerations	-	2	
4	Case Studies and Practical Considerations	-	-	25 - 32
4.1	Deadlock Recovery in UNIX/Linux	6	-	
4.2	Database Management Systems	-	-	
4.3	Distributed Systems Scenarios	-	-	
4.4	Real-time System Constraints	-	3	
5	Summary	-	-	33
6	Glossary	-	-	34
7	Terminal Questions	-	-	35
8	Answers	-	-	36 - 38
9	References	-	-	39

1. INTRODUCTION

In the previous unit, we examined the fundamental aspects of deadlocks within operating systems. This included understanding how deadlocks occur through the characterisation of conditions such as mutual exclusion, hold and wait, no pre-emption, and circular wait. We also explored how systems can be structured to prevent or avoid deadlocks by eliminating one or more of these necessary conditions. The discussion covered both theoretical models and practical considerations, laying the groundwork for a deeper investigation into how systems manage deadlocks after they occur.

In this unit, the focus shifts to detecting and recovering from deadlocks, a crucial area in system reliability and resource management. Deadlock detection involves identifying whether a deadlock has taken place, which requires systematic checking of resource allocation and process states. This becomes essential in systems where deadlock prevention is either impractical or overly restrictive. The ability to detect a deadlock ensures that corrective action can be taken to restore normal operation without requiring a complete system shutdown.

The recovery aspect follows detection and involves various strategies to resolve the identified deadlocks. These strategies may include terminating one or more processes involved in the deadlock or pre-empting certain resources from held processes. Each recovery approach comes with its own set of trade-offs in terms of system stability, data integrity, and operational efficiency. The unit outlines the criteria used to select appropriate recovery techniques based on the system's design and the severity of the deadlock.

While studying this unit, it is advisable to begin by revisiting the conditions that lead to deadlocks, as this will reinforce the importance of detection and recovery techniques. Understanding the distinctions between prevention and recovery will help clarify why some systems choose detection-based approaches. Diagrams such as resource allocation graphs can be particularly helpful in visualising deadlock scenarios and the detection process. Proceed step-by-step, first mastering detection algorithms before examining the recovery methods and their implications.

After studying this unit, you should be able to:

-

2. DEADLOCK DETECTION

When prevention or avoidance strategies are unsuitable or too restrictive, an operating system must detect deadlocks at runtime and then recover from them. Detection focuses on recognising when processes are stuck in a circular waiting pattern. Recovery then interrupts or rolls back one or more processes to break the cycle. This section examines detection methods for single-instance and multiple-instance resource systems, explores the wait-for graph technique, considers the cost of periodic detection, and extends strategies to distributed environments.

2.1. Detection in Single-Instance Systems

In systems where each resource type has exactly one instance, deadlock detection is relatively straightforward. The operating system can represent the current state via a Resource-Allocation Graph (RAG). Recall that in a RAG:

- Process nodes are drawn as circles ($P_1, P_2, \dots$).
- Resource nodes are drawn as squares ($R_1, R_2, \dots$).
- Assignment edges ($R \rightarrow P$) denote a resource allocated to a process.
- Request edges ($P \rightarrow R$) denote a process waiting for a resource.

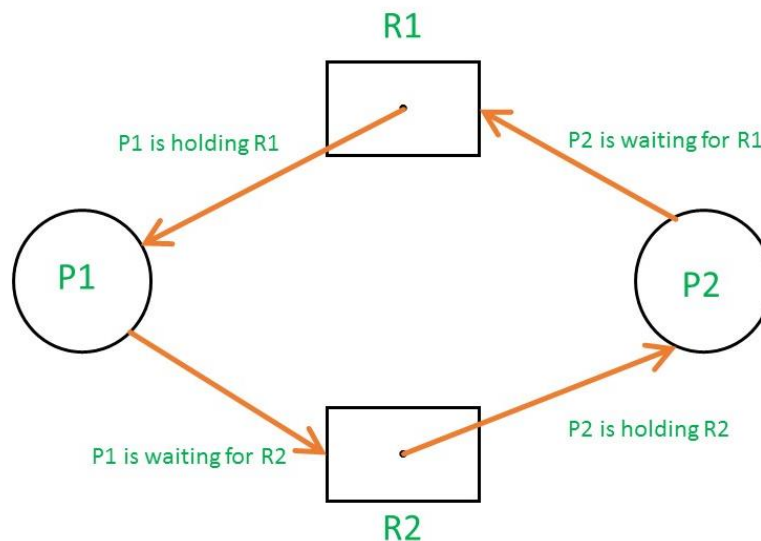


Figure 1: Resource Allocation Graph

Source: <https://media.geeksforgeeks.org/wp-content/uploads/Slide1.jpg>

The figure depicts a resource allocation graph demonstrating a deadlock situation with single instance resource types. Process P1 is holding Resource R1 and simultaneously waiting for Resource

R2. At the same time, Process P2 is holding Resource R2 and waiting for Resource R1. This forms a cyclic dependency, where each process needs a resource held by the other, leading to a permanent blocking state. This visual representation clearly shows how the allocation and request of resources can result in a system deadlock.

Simple Cycle Detection

Since each resource has one instance, a cycle in the RAG implies deadlock. The detection algorithm proceeds as follows:

1. Construct or update the RAG whenever a process requests or releases a resource.
2. Search for cycles using depth-first search (DFS) or another graph-traversal method.
3. If a cycle is found, mark all processes in the cycle as deadlocked.

In single-instance resource systems, deadlock detection operates with a clear advantage in accuracy, as any cycle present in the wait-for graph directly corresponds to a real deadlock situation. This ensures that the system does not generate false positives during its checks. Detection is immediate, allowing the system to recognise a deadlock as soon as a cycle forms, which aids in prompt resolution. However, this accuracy and immediacy come at a computational cost, as detecting cycles in a graph requires traversing through its nodes and edges, which can be resource-intensive. Particularly, the cycle detection process involves an algorithm with time complexity proportional to the number of nodes and edges, making frequent checks costly in terms of performance if not optimised.

Example

Consider three processes P_1, P_2, P_3 and three resources R_1, R_2, R_3 , each with one instance:

- P_1 holds R_1 and requests R_2 .
- P_2 holds R_2 and requests R_3 .
- P_3 holds R_3 and requests R_1 .

The RAG has edges $P_1 \rightarrow R_2, R_2 \rightarrow P_2, P_2 \rightarrow R_3, R_3 \rightarrow P_3, P_3 \rightarrow R_1, R_1 \rightarrow P_1$, forming a cycle. Detection flags P_1, P_2 and P_3 as deadlocked.

2.2. Wait-for Graph Technique

The wait-for graph (WFG) is a simplified structure derived from the RAG. In a WFG:

- Nodes represent processes only.
- Directed edge $P_1 \rightarrow P_2$ exists if process P_1 is waiting for a resource held by process P_2 .

By collapsing resources out of the graph, the WFG reduces complexity.

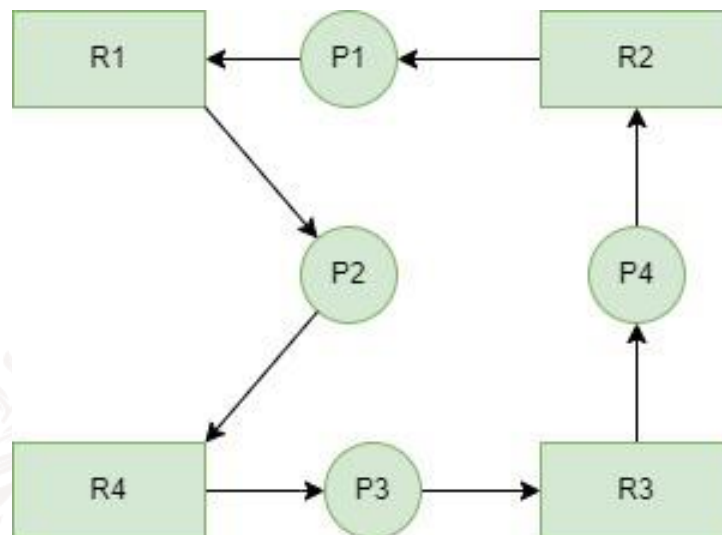


Figure 2: Wait-for Graph

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20220428122737/WaitForGraph2.jpg>

The figure displays a "wait-for graph", a common representation used to detect deadlocks in systems where resources have single instances. In this graph, an edge from Process P_i to Process P_j signifies that P_i is waiting for a resource currently held by P_j . This specific graph shows that P_1 is waiting for P_2 , and P_4 is waiting for P_3 . Crucially, the absence of any cycles within this graph indicates that the system is currently free from deadlocks, as there is no set of processes each waiting for another in a closed loop.

Construction of a Wait-for Graph

1. For each request edge $P_i \rightarrow R_j$ in the RAG, identify the process P_k such that $R_j \rightarrow P_k$ is an assignment edge.
2. Add edge $P_i \rightarrow P_k$ in the WFG.
3. Repeat for all request edges.

In single-instance systems, a cycle in the WFG indicates deadlock directly.

The wait-for graph offers several advantages over the full resource-allocation graph, particularly in systems with single instances of resources. By omitting resource nodes, it reduces the overall number of nodes and edges, leading to a more compact and manageable representation. This streamlined structure improves the efficiency of cycle detection, as the graph only reflects process-to-process dependencies, eliminating the need to trace through intermediate resource nodes. As a result, the

system can more quickly identify deadlocks and make informed decisions regarding recovery strategies. The simplified layout also aids in maintaining a clearer view of which processes are directly dependent on one another, enhancing the responsiveness and clarity of deadlock management.

Cycle Detection in the WFG

Use standard graph algorithms:

- **Depth-First Search (DFS):** mark nodes as unvisited, visiting and completed. Encountering a back-edge to a currently visiting node indicates a cycle.
- **Tarjan's Algorithm:** finds strongly connected components; any component of size greater than one (or a self-loop) signifies deadlock.

2.3. Detection Algorithm for Multiple Instances

When resources have multiple instances, neither a cycle in the RAG nor in the WFG is sufficient to guarantee deadlock; it is merely a necessary condition. A more general detection algorithm, akin to the one used in the banker's approach, must be applied periodically.

Data Structures

- Available vector: $A[j]$ = number of instances of resource j currently unallocated.
- Allocation matrix: $\text{Allocation}[i][j]$ = instances of resource j held by process i .
- Request (or Need) matrix: $\text{Request}[i][j]$ = instances of resource j that process i is waiting to acquire.

Detection Procedure

1. Initialise a Work vector $\leftarrow$ Available; mark all processes as unexamined.
2. Search for an unexamined process P_i such that $\text{Request}[i] \leq \text{Work}$.
 - If found, assume P_i can finish:
 - $\text{Work} \leftarrow \text{Work} + \text{Allocation}[i]$
 - Mark P_i as safe (or examined).
 - Repeat step 2 until no such P_i exists.
3. Deadlocked processes
 - Any process not marked safe at the end is deadlocked.

Example:

Consider resources R_1 (2 instances), R_2 (1 instance), and processes P_1, P_2 :

- Available = [1, 0]
- Allocation P_1 = [1, 0], Request P_1 = [1, 0]
- Allocation P_2 = [1, 0], Request P_2 = [0, 1]

Run the detection:

- Work = [1, 0].
- P_1 's request [1, 0] $\leq$ Work $\rightarrow$ Work $\leftarrow$ [2, 0]; mark P_1 safe.
- P_2 's request [0, 1] $\nless$ Work $\rightarrow$ cannot mark.
- No further processes can be marked; P_2 is deadlocked.

2.4. Frequency and Overhead of Detection

Deadlock detection imposes computational and performance costs. The operating system must balance timely detection against the overhead of running detection algorithms.

Detection Frequency

- **Continuous Detection** - Run the detection algorithm on every resource-request operation.
 - Advantage: Immediate identification of deadlock.
 - Disadvantage: High CPU overhead, especially in systems with frequent resource requests.
- **Periodic Detection** - Schedule detection at regular intervals (for example, every T milliseconds or after N allocations).
 - Advantage: Controls overhead by tuning frequency.
 - Disadvantage: Delays detection, prolonging deadlocked states and postponing recovery.
- **Threshold-based Triggering** - Invoke detection when certain thresholds are exceeded, such as:
 - Available instances drop below a low-water mark.
 - A process has been waiting longer than a specified timeout.

Overhead Considerations

- **Time Complexity**

- Single-instance cycle detection: $O(n + e)$.
- Multiple-instance detection: $O(m \times n^2)$, where n is number of processes and m number of resource types.
- **Memory Usage**
 - Matrices for Allocation and Request require $O(m \times n)$ space.
 - Graph representations also consume memory, though often modest relative to overall OS footprint.
- **Impact on Throughput**
 - Frequent detection reduces CPU cycles available for user tasks.
 - Periodic or threshold-based schemes mitigate impact but risk longer deadlock resolution times.
- **Starvation Risk**
 - Processes repeatedly chosen as deadlock victims may starve; care must be taken in victim selection policies to avoid bias.

Effective deadlock detection benefits from the implementation of well-considered practices that adapt to the system's operational environment. Allowing adjustable parameters enables administrators to tailor detection intervals or sensitivity thresholds based on specific workload patterns, ensuring neither performance nor reliability is compromised. Incorporating adaptive strategies further enhances efficiency, where the system can increase the frequency of detection during periods of high resource contention and reduce it when utilisation is low, conserving computational resources. A balanced approach often proves most effective, where critical resources are safeguarded through prevention or avoidance techniques, while detection and recovery methods are applied to less critical components. This tiered strategy helps maintain overall system stability without unnecessary performance overhead.

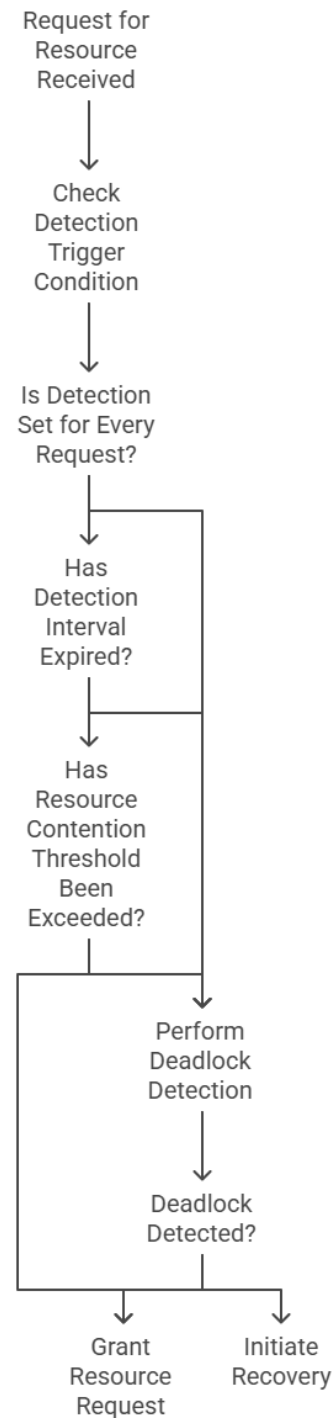


Figure 3: Deadlock Detection Decision Flow

The figure presents a decision diagram illustrating how an operating system determines when to trigger deadlock detection. Upon receiving a resource request, the system first checks the detection trigger condition. It then assesses if detection is set for every request. If not, it checks if a detection interval has expired or if a resource contention threshold has been exceeded. If any of these

conditions are met, deadlock detection is performed. If a deadlock is detected, recovery is initiated; otherwise, or if no detection was triggered, the resource request is granted.

2.5. Detection in Distributed Environments

Detecting deadlock in distributed systems is more challenging due to the absence of a centralised global state and the presence of network delays and failures. Several algorithms have been devised to identify distributed deadlocks.

Centralised versus Distributed Detection

Centralised Detection - A designated coordinator collects wait-for information from all nodes and runs a global detection algorithm.

- Pros: Simple to implement.
- Cons: Single point of failure; potential communication bottleneck.

Distributed Detection- Nodes collaborate to detect cycles using message passing. No single coordinator; each node participates equally.

- Pros: No single point of failure; scales with the number of nodes.
- Cons: Increased message complexity; more complex protocol design.

Probe (Path-Pushing) Algorithm

One popular distributed algorithm is the Chandy–Misra–Haas probe technique:

- Initiation:** When process P_i at node N_i finds it cannot proceed, it sends a probe message (i, i, j) along the wait-for edge to process P_j at node N_j .
- Forwarding:** Upon receiving a probe (h, i, j) , process P_j examines its own waits. For each outgoing wait-for edge $P_j \rightarrow P_k$, it forwards the probe as (h, i, k) to the node of P_k .
- Detection**

If a probe returns to its origin ($h == k$), a cycle exists, indicating a distributed deadlock involving all processes on the probe's path.

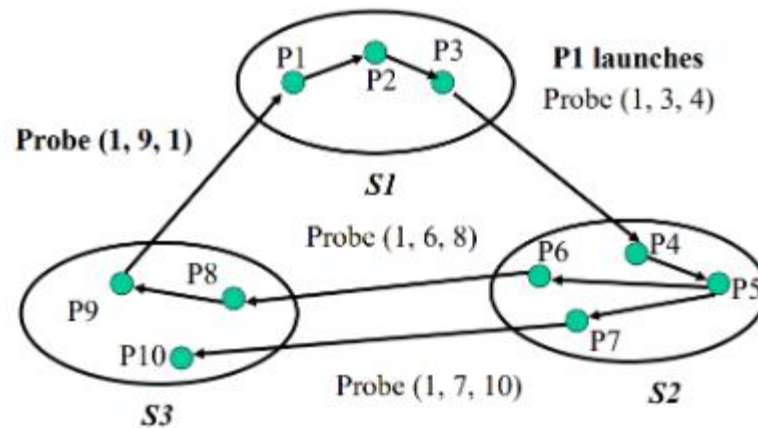


Figure 4: Chandy-Misra-Haas Probe Technique

Source: [https://www.researchgate.net/profile/Mega-Putra-](https://www.researchgate.net/profile/Mega-Putra-2/publication/348249157/figure/fig4/AS:976720660275220@1609879760926/Chandy-Misra-Haas-probe-based-WFG.ppm)

[2/publication/348249157/figure/fig4/AS:976720660275220@1609879760926/Chandy-Misra-Haas-probe-based-WFG.ppm](https://www.researchgate.net/profile/Mega-Putra-2/publication/348249157/figure/fig4/AS:976720660275220@1609879760926/Chandy-Misra-Haas-probe-based-WFG.ppm)

The figure illustrates the Chandy-Misra-Haas probe technique for distributed deadlock detection across multiple sites. This method involves a probe, represented by a triplet (i, j, k) , where 'i' is the initiator, 'j' is the process that sent the probe, and 'k' is the process to which the probe is sent. Here, P1 launches a probe $(1, 3, 4)$, which propagates through the system, crossing site boundaries from S1 to S2 and S3, with different probe values reflecting the path taken. This distributed approach allows for deadlock detection without a central controller, as probes circulate through the system to identify cycles of waiting processes.

The probe or path-pushing algorithm is a distributed deadlock detection technique where probes are sent along the wait-for paths to identify cycles indicative of a deadlock. This method must be equipped to handle common issues in distributed systems such as message delays and node failures. To avoid unmanageable message build-up, timeouts are applied so that any probe not returned within a set interval is discarded. Each node uses identifiers in the form of tuples to track which probes it has already encountered, ensuring that duplicates are not forwarded unnecessarily. In cases of node or communication failures, the algorithm must be designed to either restart the detection process or safely conclude that the affected components are not involved in a deadlock, thereby maintaining system consistency and reliability.

Apart from path-pushing, several alternative approaches offer varied advantages depending on system structure. Edge-chasing algorithms, for instance, allow messages to travel in both directions along the wait-for graph, enabling deadlock detection when two messages intersect, marking the

presence of a cycle. Hierarchical detection strategies divide the system into regions, each performing local detection and then communicating summarised results to a central authority, reducing the load on individual nodes and improving scalability. Snapshot-based detection methods involve taking a consistent global snapshot of the system state, using techniques like the Chandy–Lamport algorithm, and then performing centralised analysis to detect deadlocks. These methods are particularly useful in asynchronous environments where global state observation is otherwise complex.

Example - In a distributed database:

- Transaction T_1 at Node A waits for a lock held by T_2 at Node B.
- T_2 waits for a lock held by T_3 at Node C.
- T_3 waits for a lock held by T_1 at Node A.

Each node forwards probes, and when the probe from A to B to C returns to A, the system concludes that T_1 , T_2 and T_3 are deadlocked across the network.

Deadlock detection is essential for systems that cannot prevent every deadlock scenario through static policies. In single-instance systems, cycles in the RAG or WFG provide immediate signals of deadlock. When multiple instances are present, matrix-based detection algorithms identify processes that cannot complete. Detection imposes overhead in CPU time, memory and potential latency; thus, systems tune detection frequency or combine detection with prevention. In distributed environments, the absence of centralised state demands message-based algorithms, such as probe-chasing techniques. Mastery of these methods enables reliable detection and paves the way for effective recovery strategies.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	In a resource-allocation graph for single-instance systems, what does an assignment edge signify? a) A process waiting for a resource b) A resource allocated to a process c) A process terminating its use of a resource d) A resource being released
2	Which graph simplifies deadlock detection by representing only processes and their dependencies?

	a) Resource-Allocation Graph b) Dependency Matrix c) Wait-for Graph d) Allocation Tree
3	In the multiple-instance detection algorithm, what does the Work vector initially contain? a) Total instances of all resources in the system b) Number of instances currently unallocated for each resource type c) Instances allocated to each process d) Processes marked as deadlocked
4	What is the primary disadvantage of continuous deadlock detection on every resource request? a) Delayed identification of deadlocks b) Increased risk of starvation c) High CPU overhead d) Complex victim selection
5	In the Chandy–Misra–Haas probe technique for distributed detection, a deadlock is confirmed when: a) Probes circulate without ever returning b) A probe returns to its origin process c) All probes are discarded due to timeout d) A coordinator collects all wait-for information

3. RECOVERY FROM DEADLOCK

Once the operating system has detected a deadlock, it must employ recovery techniques to break the circular waiting and restore normal operation. Recovery methods fall into two principal categories: terminating processes to release held resources, and pre-empting resources from processes to satisfy others. A combination of these approaches, supported by checkpointing and rollback, can minimise wasted work and ensure system consistency. This section examines process termination strategies, resource pre-emption methods, the role of checkpointing and rollback, cost analysis of recovery actions, and practical considerations for deploying these techniques.

3.1. Process Termination Strategies

Terminating processes is the most direct way to resolve deadlock: by ending one or more processes within the deadlocked set, the system recovers resources and allows the remaining processes to continue. However, process termination carries the penalty of lost computation, potential user data loss and possible violation of application semantics.

Single-process termination

- **Select a single victim** - Identify one process within the deadlocked cycle to terminate. The choice typically aims to minimise overall cost.
- **Criteria for selection**
 - Least CPU time consumed: reducing wasted effort.
 - Fewest resources held: maximising resource release.
 - Lowest priority or importance: minimising user impact.
 - Shortest remaining execution time: approximated by historical run times.
- **Procedure**
 - Abort the chosen process.
 - Release all resources it holds.
 - Resume detection and recovery if necessary until no deadlock remains.

Multiple-process termination

- **Abort all deadlocked processes:** Guarantees resolution but wastes maximum work.
- **Abort processes one at a time:** Iteratively select and terminate victims, checking after each termination whether deadlock persists.

- **Trade-off analysis**

- Aborting more processes releases more resources, potentially saving repeated cycles.
- Aborting fewer processes reduces computation loss but may require additional detection cycles.

Terminating processes to recover from deadlock offers a straightforward solution that is relatively easy to implement. One of the primary advantages of this strategy is its ability to quickly reclaim the resources held by the affected processes, enabling the system to resume normal operation without the need for complex reallocation or rollback procedures. This method is particularly useful in scenarios where immediate recovery is prioritised and the affected processes do not carry critical data or state.

However, process termination carries several drawbacks that limit its suitability in more sensitive or stateful environments. The most significant concern is the potential loss of valuable computation, especially if the terminated process was running for an extended period or handling complex tasks. There is also a risk of compromising system consistency, particularly if the process had performed non-recoverable operations or made changes to shared data. In cases where user applications or services are involved, abrupt termination may lead to data corruption or user dissatisfaction, especially when the process is not designed to handle such interruptions gracefully. Therefore, while termination can resolve deadlocks efficiently, it must be applied judiciously with consideration for system integrity and user impact.

3.2. Resource Pre-emption Methods

Rather than aborting entire processes, the operating system can pre-empt specific resources from a process and allocate them to another. This approach recovers resources more surgically but must ensure that the deprived process can later resume safely.

Pre-emption policy considerations

- A. Resource pre-emptability** - Only pre-emptible resources (such as CPU, memory pages) can be taken. Non-preemptable resources (I/O devices, locks on critical sections) remain problematic.
- B. Rollback capability** - Upon pre-emption, the process's state may be inconsistent; checkpointing or state-saving is necessary to restore a valid state.

C. Granularity of pre-emption

- **Fine-grained:** reclaim individual resources (for example, one memory page or one semaphore count).
- **Coarse-grained:** reclaim all resources of one type or all resources held by the process.

D. Cost of pre-emption - Context switch overhead. Memory copy or page-in/page-out delays. Re-initialisation of device drivers or communication channels.

Pre-emption techniques

- **Memory page eviction:** Swap out part of a process's address space to disk, freeing frames for others. The process is suspended until pages are swapped back in.
- **Priority-based resource steal:** Temporarily boost the priority of the victim so that it completes critical section swiftly and releases resources.
- **Token or permit revocation:** In systems using capability or token passing, forcibly remove the token and reassign it to the requesting process. The previous holder must request a new token when resumed.
- **Lock striping:** Break large locks into smaller ones, allowing some locks to be pre-empted without aborting the process entirely. The process then re-acquires locks in a consistent order.

Pre-emption algorithm outline

1. Identify processes in the deadlock cycle.
2. For each candidate process, compute pre-emption cost for each resource type.
3. Select resource(s) to pre-empt that yield the greatest benefit at lowest cost.
4. Save process state and deallocate the chosen resources.
5. Resume victim after recovery actions are complete.

3.3. Checkpointing and Rollback

Checkpointing underpins both process termination and resource pre-emption strategies. By periodically saving a process's state, the system ensures that it can revert to a known good snapshot if forced to release resources or upon abnormal termination.

Checkpoint strategies

- **Time-based checkpointing** - Save state at fixed time intervals. Suitable for long-running processes.

- **Event-based checkpointing** - Trigger checkpoints on significant events such as resource allocation, completion of a critical phase, or just prior to I/O operations.
- **Incremental checkpointing** - Only record changes since the last checkpoint, reducing storage and time overhead.

Rollback mechanism

- A. State restoration** - Reload the process's saved memory image, register values and open file descriptors.
- B. Resource reacquisition** - Re-request resources in the correct order; may block if resources remain unavailable.
- C. Consistency checks** - Validate that I/O operations or external interactions are consistent; may involve compensating actions.

In an interactive data processing task, checkpoint and rollback mechanisms play a crucial role in maintaining progress despite disruptions such as deadlock. The system saves a checkpoint after each significant calculation stage, preserving the process state at that moment. If a deadlock arises and certain memory pages need to be pre-empted to resolve it, the affected process can revert to the most recent checkpoint. Once the required memory pages are restored, the process resumes by re-executing the calculations from that point onward. This approach ensures that the system can recover without starting the task from the beginning, reducing wasted computation and preserving the logical integrity of the process.

3.4. Recovery Cost Analysis

Evaluating the cost of recovery actions is essential for making informed decisions about which process or resources to sacrifice. The goal is to minimise total system overhead and service disruption.

Cost factors

- **Computation loss** - Amount of CPU time wasted due to aborted work or rollback.
- **Time to recovery** - Wall-clock time from detection to full resumption of affected processes.
- **Resource consumption** - Additional memory, disk I/O and network traffic generated by checkpointing, rollback and re-execution.
- **User impact** - Application-level consequences such as long-running transactions being aborted, loss of data or degraded response times.

- **Complexity overhead** - Development and maintenance burdens introduced by sophisticated recovery code.

Recovery from deadlock involves weighing various cost-benefit trade-offs to determine the most efficient and fair resolution strategy. Aborting a long-running process can release a large quantity of resources, potentially resolving the deadlock, but it also results in the loss of significant computational effort. In contrast, pre-empting a small number of memory pages may appear less costly, yet it might prove ineffective if other critical resources continue to be held.

Introducing frequent checkpoints helps reduce rollback effort when recovery is required, though it adds ongoing performance overhead during normal execution. A common strategy to minimise wasted computation is to terminate the youngest process, as it has performed the least amount of work. However, consistently targeting newer processes can unfairly affect batch or background jobs, leading to imbalance and potential inefficiency in long-running systems. These trade-offs highlight the importance of tailoring recovery strategies to specific system workloads and operational goals.

Example:

Suppose three deadlocked processes P_1 , P_2 and P_3 have consumed 1200 s, 800 s and 300 s of CPU time respectively. They hold resources worth 200 MB, 150 MB and 50 MB. If rollback cost per MB is 0.5 s and aborting a process loses all CPU time, compute:

- Cost of abort P_1 : 1200 s lost, 200 MB freed (100 s rollback avoided).
- Cost of abort P_2 : 800 s lost, 150 MB freed (75 s rollback avoided).
- Cost of abort P_3 : 300 s lost, 50 MB freed (25 s rollback avoided).

Net cost = computation lost minus rollback saved:

- P_1 : 1200 s – 100 s = 1100 s
- P_2 : 800 s – 75 s = 725 s
- P_3 : 300 s – 25 s = 275 s

Table 1: Cost Analysis of Process Termination for Deadlock Recovery

Process	CPU Time Consumed (s)	Resources Held (MB)	Rollback Saving (s)	Net Cost (s)
P_1	1200	200	100	1100
P_2	800	150	75	725

P ₃	300	50	25	275
----------------	-----	----	----	-----

Conclusion: Terminating P₃ results in the lowest net cost of 275 seconds, making it the most suitable candidate for termination in this scenario.

3.5. Practical Considerations

In real operating systems, recovery strategies must integrate with overall process management, I/O subsystems and user expectations. Several pragmatic factors influence design choices.

- **Integration with Process Control Block (PCB)** - The PCB should be extended to include fields for storing checkpoint data, rollback logs, and details of any resource pre-emption. All updates involving rollback or resource changes must occur atomically to prevent inconsistencies in the system state.
- **Coordination with I/O and Filesystems** - File systems that support journalling can help revert incomplete file operations during rollback, preserving data integrity. However, some device drivers cannot be rolled back easily, making pre-emption of such hardware resources difficult or requiring complex handling.
- **User and Application Transparency** - When a process is aborted, the system should notify the user and return specific error codes so applications can recover. Some operating systems also provide interfaces for applications to manage their own checkpoints and recovery processes.
- **Fairness and Starvation** - To avoid repeatedly affecting the same process, the system should rotate victims using a history log. It may also temporarily boost the priority of a process that has been rolled back or lost resources, helping it finish more quickly in future scheduling cycles.
- **Hybrid Recovery Schemes** - Many systems use a mix of strategies. Lightweight resources are usually pre-empted first, while more persistent deadlocks involving non-preemptable resources may lead to termination. Long-running tasks may be checkpointed selectively, and in distributed systems, coordinated rollback is handled using global commit and abort mechanisms.
- **Evaluation and Tuning** - Operating systems should monitor recovery metrics such as recovery time and frequency of rollbacks to assess effectiveness. Based on this data, administrators can fine-tune system parameters like checkpoint intervals, deadlock detection thresholds, and the criteria for choosing victims.

Recovery from deadlock involves careful trade-offs between wasted computation, resource reclamation speed and overall system complexity. Process termination provides a blunt but effective

tool, while resource pre-emption and checkpointing enable more nuanced recovery with reduced overhead. Accurate cost analysis guides victim selection, ensuring that the system expends minimal effort to return to a liveness state. Practical realisations must integrate with process control structures, I/O mechanisms and user interfaces, maintaining fairness and transparency. By blending multiple strategies and continuously tuning parameters, operating systems can manage deadlocks robustly without unduly compromising performance.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which recovery strategy involves ending one or more processes in the deadlocked set to release resources? a) Resource pre-emption b) Checkpointing c) Process termination d) Rollback
7	When selecting a single process to abort in deadlock recovery, which criterion aims to reduce lost computation? a) Highest priority b) Fewest resources held c) Shortest remaining execution time d) Largest memory footprint
8	What is the main advantage of resource pre-emption over process termination? a) It eliminates the need for checkpoints b) It recovers resources without aborting entire processes c) It ensures processes never starve d) It avoids the use of rollback mechanisms
9	In checkpointing schemes, what does incremental checkpointing record? a) The full process state at each interval b) Only changes since the last checkpoint c) Resource allocation snapshots only d) User interactions since process start

10	In cost analysis for process termination, net cost is calculated as computation lost minus what? a) Time to recovery b) Rollback saving c) Resource consumption d) Starvation risk
----	---



4. CASE STUDIES AND PRACTICAL CONSIDERATIONS

Real-world systems rarely resemble the neat models presented in theory. Each environment—from monolithic kernels through transaction-oriented databases to geographically dispersed clusters and hard-real-time control systems—imposes its own constraints and demands distinct deadlock-recovery tactics. This section explores how recovery techniques are applied in UNIX/Linux kernels, database management systems, distributed platforms and real-time operating systems, highlighting practical trade-offs and architectural adaptations.

4.1. Deadlock Recovery in UNIX/Linux

In UNIX and Linux kernels, deadlock recovery is typically shunned in favour of strict avoidance and prevention, since rolling back kernel state is hazardous. Yet, certain subsystems incorporate lightweight detection and limited recovery mechanisms to mitigate rare deadlock conditions.

Kernel Locking Discipline

Before considering recovery, most kernels enforce a rigid lock hierarchy:

- **Global lock ordering** - All spinlocks and mutexes carry a rank; code that grabs multiple locks must do so in ascending order of rank.
- **Lock chaining checks** - During development, lock-dependency validators record all acquisition sequences, flagging inversions that could yield deadlock.

This design minimises the incidence of cycles, reducing the need for runtime recovery.

Watchdog and Panic

When a deadlock slips through, the kernel cannot safely roll back arbitrary operations. Instead, it relies on:

- **Watchdog timers** - If a CPU hangs holding a spinlock beyond a threshold, a watchdog process may force a kernel panic or a soft-reboot.
- **Kernel panic dump** - The system writes a core image to disk (or network) for post-mortem analysis. This does not restore service automatically but aids developers in locating the locking flaw.

Limited Recovery: OOM Killer

Although not a general deadlock remedy, the Out-Of-Memory (OOM) killer in Linux can free resources when memory exhaustion stalls the system:

- **Victim selection** - Using heuristics such as memory footprint and process priorities, it terminates one or more user processes.
- **Resource reclamation** - By freeing page tables and caches, it may unstick kernel subsystems waiting for memory.

While this addresses resource exhaustion rather than true circular locks, it exemplifies a fail-safe measure when avoidance proves insufficient.

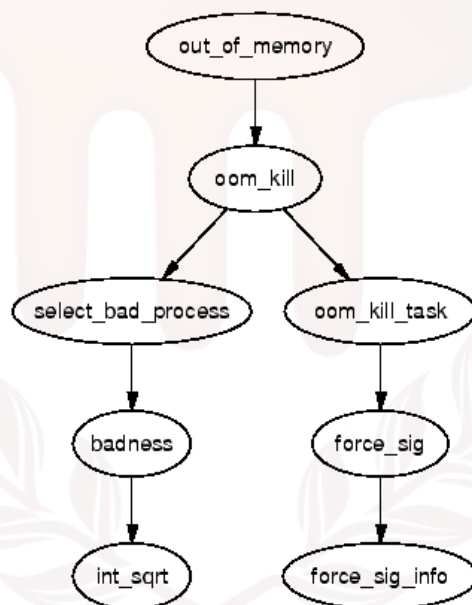


Figure 5: OOM Call Graph

Source: <https://www.kernel.org/doc/gorman/html/understand/understand-html077.png>

The figure presents a simplified flow diagram of the Linux OOM (Out Of Memory) killer's operation when the system runs out of memory. Upon an 'out_of_memory' event, the 'oom_kill' function is invoked, which then branches into two main paths: either 'select_bad_process' or 'oom_kill_task'. The 'select_bad_process' path involves calculating a 'badness' score, potentially using functions like 'int_sqrt', to identify the most suitable process to terminate. The 'oom_kill_task' path directly proceeds to forcefully signal the selected process, using 'force_sig' and 'force_sig_info', to reclaim system resources and restore stability.

Filesystem Journalling

To guard against inconsistencies in the event of abrupt process kills or panics:

- **Journalling filesystems** - Record metadata changes before committing them; if the system halts mid-update, the journal can roll forward or back to a consistent state.
- **Ordered mode** - Ensures data writes occur before metadata commits, further reducing the risk of corruption during forced recovery actions.

4.2. Database Management Systems

Database systems frequently encounter deadlocks due to concurrent transaction processing but must still uphold data integrity and sustain performance levels. Recovery strategies in these systems typically involve prompt deadlock detection, strategic transaction abort decisions, and carefully executed rollback procedures that aim to minimise data loss and system disruption.

- **Two-Phase Locking and Deadlock Detection** - Relational databases often rely on strict two-phase locking (2PL), a method that enforces serialisability by ensuring that transactions acquire all required locks before releasing any. This protocol helps maintain data consistency but does not prevent deadlock situations. The database system constructs and maintains a wait-for graph that maps dependencies between transactions. This graph is periodically scanned for cycles, each representing a deadlock. When such a cycle is detected, the database system must identify and roll back one of the involved transactions to break the cycle and allow others to proceed.
- **Victim Selection and Rollback** - Selecting the appropriate transaction to abort is a critical part of deadlock recovery and involves evaluating several criteria. Typically, younger transactions are preferred for termination as they have performed less work, and aborting them causes less disruption. Transactions that hold fewer locks are also considered better candidates, as their termination has a smaller impact on system resources. Some systems use cost estimation models to gauge the effort required to undo a transaction's effects. Once a victim is chosen, the system uses undo logs to revert changes, clears all the transaction's locks, and permits other blocked transactions to continue.
- **Cascading Rollbacks and Autonomous Transactions** - To avoid cascading rollbacks, where one transaction's termination leads to others also needing to be undone, many databases enforce strict 2PL. This ensures that no transaction can access uncommitted changes made by another, thereby isolating failures. Some databases implement autonomous transactions, which are independent

sub-units that can commit separately. This can help contain the scope of deadlocks and limit their effects on the broader system.

- **Checkpointing and Recovery** - Databases periodically perform checkpointing, where committed data in memory is written to disk to provide a consistent system state. In the event of a crash, the system can recover by replaying only the logs recorded after the last checkpoint, thus speeding up the restart process. While checkpointing does not directly affect deadlock detection or avoidance, it reduces the cost of recovery by limiting the amount of reprocessing required after a long-running transaction is aborted.

4.3. Distributed Systems Scenarios

Distributed systems, whether implemented as computing clusters, grid infrastructures or microservice-based architectures, present unique challenges for deadlock detection and recovery. These environments are characterised by the absence of a centralised state, communication delays, and the possibility of partial failures. As a result, traditional centralised techniques are not effective and must be adapted or replaced with decentralised approaches that can operate correctly and efficiently across multiple nodes.

- **Distributed Deadlock Detection** - To identify deadlocks in distributed settings, systems often rely on probe-based algorithms such as the Chandy–Misra–Haas method, or edge-chasing techniques where messages traverse wait-for dependencies across nodes to detect cycles. Once a deadlock is detected, the resolution can follow one of two approaches: either a designated coordinator makes a global decision to select a victim transaction, or all involved nodes communicate and agree on which transaction to abort. The chosen transaction's operations on each node are then rolled back in a coordinated manner, typically using communication protocols similar in concept to a two-phase commit, but operating in reverse.
- **Recovery Challenges** - A key difficulty in distributed recovery lies in the possibility of partial rollback. In some cases, a transaction may have already committed updates on certain nodes while others remain pending. Ensuring consistency in such situations demands coordinated rollback or compensating actions across all affected nodes. Reliable message delivery is another critical concern. Abort signals or compensatory instructions must be idempotent and correctly ordered, so they do not result in corrupted state when duplicated or delayed. Additionally, the system must cope with dynamic membership, as nodes may join or leave the network during detection or recovery, potentially altering wait-for graphs and leading to lost or misdirected probe messages.

- **Examples in Practice** - One practical instance is the implementation of distributed file locking in networked file systems such as NFS or clustered file systems. These systems often rely on lease-based mechanisms, where locks automatically expire after a defined period. This approach allows the system to recover from situations where a client becomes unreachable, thus preventing indefinite deadlock. In microservice environments, strict locking is typically avoided altogether. Instead, distributed transactions are often managed using sagas, a design pattern where each step has a corresponding compensating action. If a transaction step fails, previous steps are reversed by invoking these compensating operations, allowing the system to proceed without permanent locks.
- **Adaptive Recovery Policies** - Some distributed systems implement timeout-based policies, where operations that exceed a specified maximum duration are automatically aborted and followed by compensatory steps. This approach helps release resources and maintain system responsiveness. Another common policy is back-off and retry, where clients facing resource contention wait for a progressively longer period before trying again. This technique reduces the likelihood of persistent deadlock and helps to balance overall system load during periods of high contention.

In summary, deadlock management in distributed systems demands more than just detection; it requires coordination, flexibility, and fault tolerance. By blending decentralised detection algorithms with smart recovery policies, modern distributed architectures are able to manage deadlocks without compromising stability or performance.

4.4. Real-time System Constraints

Hard real-time systems—such as avionics, medical devices and industrial controllers—demand guaranteed response times. Deadlock recovery by abort or rollback is often unacceptable, since it may breach timing constraints and undermine system safety. Hence, these systems must preclude deadlock at design time or employ specialised run-time protocols that bound blocking times.

Priority Ceiling and Inheritance Protocols

To forestall deadlock and limit priority inversion:

- **Priority inheritance protocol (PIP)** - A lower-priority task holding a resource inherits the priority of the highest-priority task waiting for that resource.

- **Priority ceiling protocol (PCP)** - Each resource has a ceiling equal to the highest priority of tasks that may lock it. A task may only lock a resource if its priority exceeds the current system ceiling; this prevents circular waits at the cost of possible blocking before resource acquisition.

These protocols ensure that blocking durations are bounded and that circular waiting cannot arise.

Non-blocking Synchronisation

Where even bounded blocking is intolerable, systems use:

- **Lock-free algorithms** - Data structures manipulated with atomic compare-and-swap operations; tasks may retry but never block waiting for others.
- **Wait-free algorithms** - Guarantee that every operation completes within a finite number of steps, irrespective of other tasks' actions.

By avoiding locks altogether, deadlock is eradicated, though at the expense of algorithmic complexity and potential performance overhead.

Static Analysis and Scheduling

Compile-time tools and schedulable analysis play a key role:

- **Resource usage profiles** - The developer specifies maximum resource hold times and access patterns; automated analysis verifies absence of circular dependencies.
- **Cyclic scheduling** - Strict time slots are allocated to tasks for resource access; if a task cannot acquire a resource in its slot, it yields and tries in the next cycle—ensuring predictability.

Case Example: Avionics Flight Control

In an avionics flight control system, managing shared resources is critical to ensure reliable and timely operation. Periodic tasks access common data buffers, communication buses, and hardware registers, requiring careful coordination to avoid delays or deadlocks. The Priority Ceiling Protocol (PCP) is employed to guarantee that high-priority sensor-fusion tasks are never blocked indefinitely by lower-priority tasks, such as logging. To further minimise the risk of deadlock, lock-free ring buffers are used for handling telemetry data, allowing data exchange without blocking. Additionally, thorough static analysis is conducted before deployment to verify that no sequence of resource requests can create a cycle, providing strong assurance against deadlock during operation.

Recovery from deadlock in practical systems is rarely a matter of applying a single generic algorithm. In UNIX/Linux kernels, fully fledged rollback is impractical, so strict avoidance prevails and, in

extremis, the system opts for panic or out-of-memory processes. Database systems handle deadlock routinely through cycle detection, victim selection and undo-log rollbacks, preserving transactional integrity. Distributed platforms contend with partial failures and network delays via probe-based detection and coordinated abort protocols, often supplemented by compensation patterns. Hard real-time environments eschew recovery altogether, instead enforcing protocols that guarantee bounded blocking or employing lock-free designs. Across all cases, architects must weigh system complexity, performance overhead, determinism requirements and failure modes to choose the appropriate blend of prevention, detection and recovery.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	In UNIX/Linux kernels, which mechanism handles situations where a deadlock slips through prevention? a) Two-phase locking b) Kernel panic triggered by a watchdog timer c) Distributed probe algorithm d) Priority ceiling enforcement
12	Which component in Linux terminates user processes when the system runs out of memory? a) Watchdog timer b) Out-Of-Memory (OOM) killer c) Lock dependency validator d) Kernel panic dump
13	What strategy do database systems employ to prevent cascading rollbacks during deadlock recovery? a) Priority ceiling protocol b) Autonomous transactions c) Strict two-phase locking d) Edge-chasing algorithms
14	In microservice architectures, which pattern uses compensating actions to reverse steps without strict locks? a) Two-phase commit b) Saga c) Path-pushing probe

	d) Journalling
15	Which protocol in real-time systems forbids a task from locking a resource unless its priority exceeds a predefined ceiling? a) Priority inheritance protocol b) Priority ceiling protocol c) Non-blocking synchronisation d) Lock-free algorithm



5. SUMMARY

- Deadlock detection identifies cycles of resource requests and allocations in single-instance and multiple-instance systems, enabling the operating system to recognise blocked process sets.
- In single-instance systems, a Resource-Allocation Graph (RAG) or its simplified Wait-for Graph (WFG) reveals deadlock directly through cycle detection using depth-first search or Tarjan's algorithm.
- For multiple-instance resource types, a matrix-based detection algorithm mimics the banker's method by comparing request vectors against available resources and marking safe processes.
- Detection frequency can be continuous, periodic or threshold-based, each option balancing immediacy of deadlock identification with CPU and memory overhead.
- The cost of detection grows with the number of processes, resources and detection interval, affecting throughput and risking starvation if victims are chosen poorly.
- In distributed environments, deadlock detection requires message-passing schemes such as the Chandy–Misra–Haas probe algorithm, edge-chasing or hierarchical snapshot methods to manage global cycles without centralised state.
- Process termination recovers from deadlock by aborting one or more victim processes based on criteria such as CPU usage, resources held or execution time, at the expense of lost computation.
- Resource pre-emption reclaims specific allocatable resources from victim processes, supported by checkpoint and rollback mechanisms to restore process state and minimise wasted work.
- Recovery cost analysis weighs computation loss, rollback savings, resource consumption and user impact to select the most efficient victim or combination of recovery actions.
- Real-world systems tailor deadlock recovery—UNIX/Linux favour prevention and panic on rare deadlocks, databases use two-phase locking with cycle detection and rollbacks, distributed platforms employ decentralised probes and sagas, and hard real-time systems preclude deadlock through priority protocols and non-blocking algorithms.

6. GLOSSARY

Resource-Allocation Graph	- A directed graph representing processes, resources, allocation edges and request edges, used to detect deadlock in single-instance systems.
Wait-for Graph	- A condensed graph showing only processes as nodes, with edges indicating which process is waiting for another, simplifying cycle detection.
Available Vector	- An array indicating the number of currently unallocated instances for each resource type in multiple-instance detection.
Allocation Matrix	- A two-dimensional array recording how many instances of each resource are held by each process.
Request (Need) Matrix	- A table listing the number of additional resource instances each process requires to complete.
Chandy–Misra–Haas Probe Algorithm	- A distributed deadlock detection method sending probes along wait-for edges to identify cycles across nodes.
Checkpointing	- The act of saving a process's state at points in time or upon events to enable rollback after pre-emption or abortion.
Rollback	- Restoring a process to its last checkpointed state and re-issuing resource requests to continue execution.

7. TERMINAL QUESTIONS

1. Explain the difference between deadlock prevention and deadlock detection, and discuss scenarios where detection is preferable.
2. Describe how a Resource-Allocation Graph and a Wait-for Graph differ in structure and utility for single-instance systems.
3. Outline the matrix-based deadlock detection algorithm for multiple-instance resources and analyse its time and space complexity.
4. Evaluate the trade-offs between continuous, periodic and threshold-based detection frequencies in terms of system performance and responsiveness.
5. Discuss the challenges of deadlock detection in distributed environments and compare the Chandy–Misra–Haas probe technique with edge-chasing methods.
6. Critically assess the merits and drawbacks of process termination as a deadlock recovery strategy, citing criteria used for victim selection.
7. Explain how resource pre-emption works, and evaluate its reliance on checkpointing and rollback mechanisms to maintain system consistency.
8. Given the cost factors involved, propose a method for performing recovery cost analysis and justify how it guides victim selection.
9. Compare deadlock recovery approaches in UNIX/Linux kernels, database management systems and hard real-time operating systems, highlighting why each environment favours certain techniques.
10. Design a tiered deadlock management strategy for a distributed microservice architecture, incorporating detection, recovery and prevention measures.

8. ANSWERS

Self-Assessment Questions

1. b) A resource allocated to a process
2. c) Wait-for Graph
3. b) Number of instances currently unallocated for each resource type
4. c) High CPU overhead
5. b) A probe returns to its origin process
6. c) Process termination
7. c) Shortest remaining execution time
8. b) It recovers resources without aborting entire processes
9. b) Only changes since the last checkpoint
10. b) Rollback saving
11. b) Kernel panic triggered by a watchdog timer
12. b) Out-Of-Memory (OOM) killer
13. c) Strict two-phase locking
14. b) Saga
15. b) Priority ceiling protocol

Terminal Questions Answers

Answers 1: Deadlock prevention works by structurally eliminating one or more of the necessary conditions for deadlock before they occur, whereas deadlock detection allows the system to enter deadlock and then identifies cycles at runtime to trigger recovery. Detection is preferable in environments where prevention would impose excessive resource constraints or severely limit concurrency, such as systems with dynamic and unpredictable resource demands. Refer section 2.

Answers 2: A Resource-Allocation Graph (RAG) represents both processes and resource instances with request and assignment edges, making it suitable for visualising exact allocations in single-instance systems. In contrast, a Wait-for Graph (WFG) collapses resource nodes and shows only process-to-process wait dependencies, simplifying cycle detection and reducing graph complexity. Refer sections 2.1 and 2.2.

Answers 3: The matrix-based detection algorithm maintains Available, Allocation, and Request matrices, iteratively marking processes as safe if their outstanding requests can be satisfied by the

current Work vector, and finally identifying unmarked processes as deadlocked. It runs in $O(m \times n^2)$ time and uses $O(m \times n)$ space, where n is the number of processes and m the number of resource types. Refer section 2.3.

Answers 4: Continuous detection triggers checks on every resource request, offering immediate deadlock identification but incurring high CPU overhead; periodic detection balances overhead by running at set intervals but may delay recovery; threshold-based triggering fires only when specific resource or wait-time limits are exceeded, reducing routine cost at the risk of overlooking emerging deadlocks. Each strategy trades off timeliness of recovery against computational and memory impact on system throughput. Refer section 2.4.

Answers 5: Distributed deadlock detection must cope with lack of a global state, network latency and node failures, making centralised approaches prone to bottlenecks or single points of failure. The Chandy–Misra–Haas probe technique sends and tracks probes along wait-for edges to detect cycles without a coordinator, while edge-chasing methods propagate messages that intersect when a cycle exists; probes can incur high message overhead and require timeouts, whereas edge-chasing can be simpler but may miss complex cycles without careful coordination. Refer section 2.5.

Answers 6: Process termination resolves deadlocks by aborting one or more victim processes to free resources, with selection criteria such as least CPU time consumed, fewest resources held, or lowest priority guiding victim choice. While straightforward and guaranteed to break cycles, it risks significant computation loss, potential data corruption, and user disruption, making it suitable only when other recovery strategies are impractical. Refer section 3.1.

Answers 7: Resource pre-emption reclaims specific allocatable resources from deadlocked processes without full termination, relying on checkpointing to save process state and rollback mechanisms to restore consistency upon resumption. This approach minimizes wasted work by only rolling back to the last safe checkpoint and re-executing from there, but incurs overhead for state-saving, context switches, and potential re-computation. Refer sections 3.2 and 3.3.

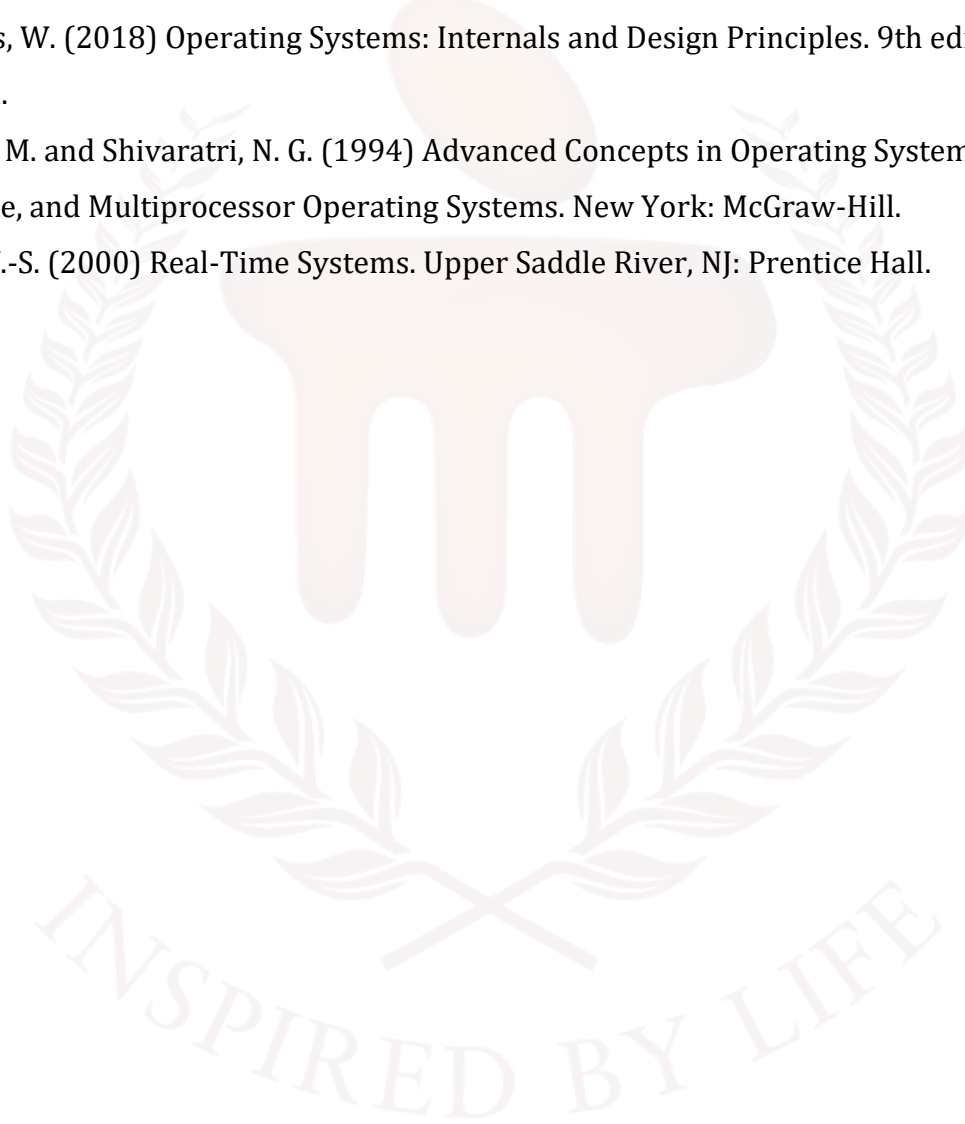
Answers 8: Recovery cost analysis quantifies computation loss, rollback savings, additional resource consumption, and user-impact to compute a net cost for each potential victim, guiding selection toward the action that minimizes total system overhead. By comparing lost CPU time against rollback savings and other factors, the system can select the lowest-net-cost victim, ensuring efficient deadlock resolution. Refer section 3.4.

Answers 9: UNIX/Linux kernels favor strict avoidance and, in extremis, kernel panic or OOM-killer heuristics rather than rollback, prioritising system stability over fine-grained recovery. Database systems routinely detect deadlocks via wait-for graphs, choose victims based on transaction age or lock count, and use undo logs for precise rollback, preserving transactional integrity. Hard real-time OSes preclude deadlock through priority ceiling or inheritance protocols and non-blocking algorithms, as runtime recovery via abort or rollback violates deterministic timing constraints. Refer sections 4.1, 4.2 and 4.4.

Answers 10: A tiered strategy in a distributed microservice architecture might employ lightweight prevention (e.g., priority ceilings or lock-free data structures) for critical services, periodic or threshold-based detection using decentralized probes for inter-service dependencies, and automated victim selection via cost analysis combined with compensating transaction sagas to recover without global rollback. This layered approach adapts detection frequency and recovery granularity to service criticality, balancing responsiveness, performance overhead, and consistency guarantees. Refer sections 2.5, 3 and 4.3.

9. REFERENCES

1. Silberschatz, A., Galvin, P. B. and Gagne, G. (2018) Operating System Concepts. 10th edn. Hoboken, NJ: Wiley.
2. Tanenbaum, A. S. and Bos, H. (2015) Modern Operating Systems. 4th edn. Harlow: Pearson.
3. Stallings, W. (2018) Operating Systems: Internals and Design Principles. 9th edn. Harlow: Pearson.
4. Singhal, M. and Shivaratri, N. G. (1994) Advanced Concepts in Operating Systems: Distributed, Database, and Multiprocessor Operating Systems. New York: McGraw-Hill.
5. Liu, J. W.-S. (2000) Real-Time Systems. Upper Saddle River, NJ: Prentice Hall.

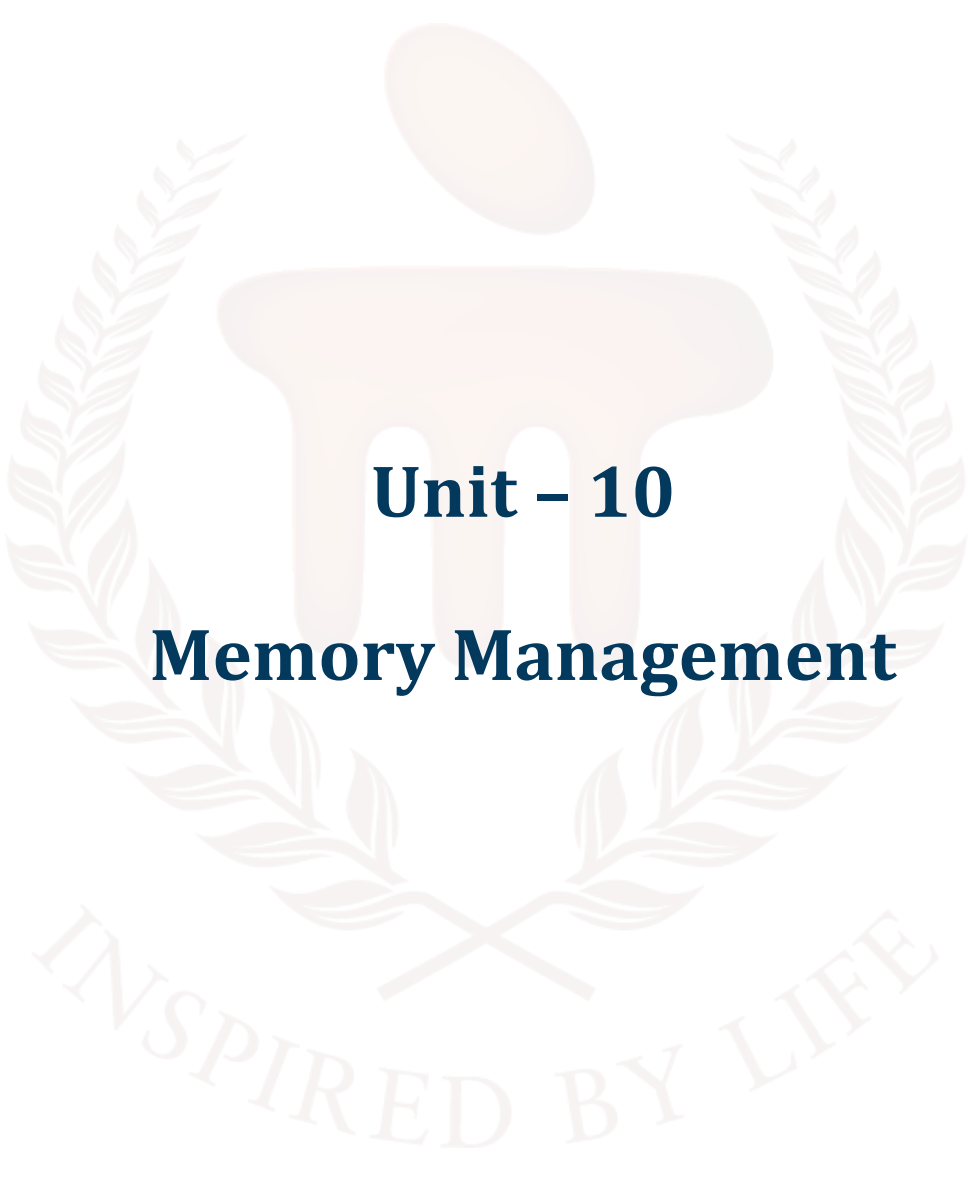


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 10

Memory Management

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Logical versus Physical Address Space	-	-	7 - 14
2.1	Definition and Distinction	1	-	
2.2	Address Mapping Mechanisms	-	-	
2.3	Protection and Isolation	-	-	
2.4	Hardware Support Requirements	2	-	
2.5	Illustrative Examples	-	1	
3	Swapping	-	-	15 - 22
3.1	Concept and Purpose	3	-	
3.2	Swap Space Management	-	-	
3.3	Performance Implications	-	-	
3.4	Thrashing and Its Prevention	4	-	
3.5	Operating System Support	-	2	
4	Contiguous Allocation	-	-	23 - 30
4.1	Basic Strategy	5	-	
4.2	Static versus Dynamic Allocation	-	-	
4.3	Fragmentation: Internal and External	6, 7	-	
4.4	Compaction Techniques	-	-	
4.5	Placement Algorithms	8	3	
5	Implementation and Performance Considerations	-	-	31 - 37
5.1	Memory Management in UNIX-like Systems	-	-	
5.2	Embedded System Constraints	-	-	
5.3	Real-Time Operating System Requirements	-	-	
5.4	Performance Evaluation Metrics	-	4	
6	Summary	-	-	38

7	Glossary	-	-	39
8	Terminal Questions	-	-	40
9	Answers	-	-	41 - 43
10	References	-	-	44



1. INTRODUCTION

In the previous unit, we examined the concept of deadlocks, focusing on how operating systems detect and recover from such situations. Deadlock detection techniques were studied both in single-instance and multiple-instance systems using tools like wait-for graphs. Various detection algorithms were explored alongside their performance overheads and frequency of use. The unit concluded with methods of recovery from deadlocks, including process termination strategies, resource pre-emption techniques, and system-specific considerations in distributed, UNIX/Linux, and real-time environments.

Building on that foundation, this unit turns to one of the most vital responsibilities of an operating system—memory management. It begins by exploring the distinction between logical and physical address space, a fundamental concept that underpins the entire structure of process execution. Understanding how these address spaces are defined and translated is essential to comprehending how programs are loaded, stored, and executed efficiently and safely within a system.

The unit then delves into two key techniques used to handle memory: swapping and contiguous memory allocation. Swapping introduces the method of temporarily moving processes in and out of main memory to maximise resource utilisation, with discussions on swap space management, its influence on performance, and how operating systems respond to problems such as thrashing. Contiguous allocation, on the other hand, addresses the strategy of allocating memory in a single continuous block. The section explains the difference between static and dynamic allocation, identifies the challenges of internal and external fragmentation, and discusses techniques such as compaction and various placement algorithms to mitigate these issues.

To effectively study this unit, it is recommended that readers adopt a step-by-step approach. Begin by establishing a firm grasp of the concepts of logical and physical address space, as this underpins the memory operations discussed later. Use diagrams to visualise address mapping, and follow process execution through the memory hierarchy. When progressing to swapping and contiguous allocation, focus on understanding the trade-offs of each technique. Make note of real-world implications, particularly the challenges they pose to performance and system stability. Relating each concept back to process management and deadlock recovery will also help integrate this material with previously learned topics.

1.1. Objectives

After studying this unit, you should be able to:

- Differentiate between logical and physical address spaces in memory systems.
- Analyse the process of swapping and its effect on system performance.
- Evaluate memory fragmentation and propose compaction strategies.
- Apply placement algorithms in contiguous memory allocation scenarios.
- Assess memory management techniques in terms of efficiency and reliability.



2. LOGICAL VERSUS PHYSICAL ADDRESS SPACE

2.1. Definition and Distinction

In a computer system, memory management plays a central role in facilitating the interaction between software and hardware. The concepts of logical and physical address spaces are fundamental to understanding how memory is accessed, managed, and protected during program execution.

A logical address refers to the address generated by the CPU during a program's execution. It is sometimes also termed a virtual address. This address does not correspond directly to a physical location in the memory hardware. Instead, it is translated into a physical address by the system's memory management unit.

A physical address is the actual location in the main memory unit (RAM) where data or instructions are stored. The physical address is used by the memory hardware to fetch or store data.

Key distinctions between the two types of address spaces are as follows:

- The logical address is visible to and used by programs and processes, whereas the physical address is not directly visible to the program.
- The logical address space refers to the set of addresses generated by a program, while the physical address space refers to the actual set of addresses available in RAM.
- Logical addresses allow for abstraction and flexibility, supporting techniques such as paging, segmentation, and virtual memory.

This separation enhances both memory protection and efficient allocation, enabling multiple processes to coexist without interfering with one another's memory.

2.2. Address Mapping Mechanisms

Since the logical address space is distinct from the physical address space, there must be a mechanism to translate logical addresses to physical ones. This translation is essential for memory abstraction and plays a vital role in memory protection and process isolation.

The key address mapping techniques are:

- **Compile-Time Binding:** In this approach, if the memory location of a process is known at the time of compilation, the compiler generates absolute code. The logical address and the physical

address are the same in this case. This method does not allow for relocation or flexibility after compilation and is seldom used in modern systems.

- **Load-Time Binding:** When memory addresses are not known at compile-time but are determined at load time, the operating system can adjust addresses to the location allocated in memory. The compiler generates re-locatable code, and the logical addresses are converted to physical ones during the loading phase.
- **Execution-Time Binding:** Modern systems usually employ execution-time binding with the help of a hardware memory management unit. In this method, the addresses are generated dynamically during program execution. Logical addresses are mapped to physical ones using a combination of base and limit registers or page tables.

Mechanism of Translation

The address translation is handled by the memory management unit (MMU), which performs the following:

- Receives the logical address from the CPU.
- Translates it into a corresponding physical address.
- Sends the physical address to the memory hardware.

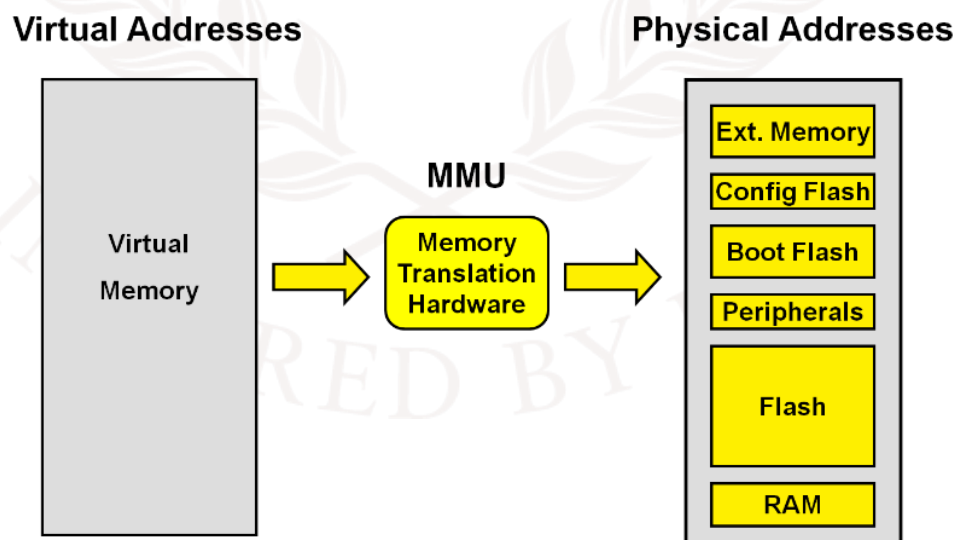


Figure 1: Address Translation Process

Source: https://s3.amazonaws.com/thinkific/file_uploads/132972/images/c5a/65b/749/1553191632042.jpg

The figure presents a simplified block diagram illustrating the process of address translation from virtual to physical addresses, a fundamental concept in modern operating systems. The CPU generates

virtual addresses, which are then sent to the Memory Management Unit (MMU). The MMU, containing memory translation hardware, is responsible for converting these virtual addresses into corresponding physical addresses. These physical addresses then map to various physical memory components, such as external memory, configuration flash, boot flash, peripherals, and RAM, allowing programs to access memory indirectly and safely.

2.3. Protection and Isolation

A major benefit of maintaining separate logical and physical address spaces is the ability to enforce protection and isolation among processes.

Protection ensures that a process cannot access memory locations outside its authorised range. If logical and physical addresses were the same, it would be difficult to restrict access to particular areas of memory.

To achieve protection, systems implement:

- **Base and Limit Registers:** The base register holds the starting physical address allocated to a process, and the limit register specifies the range. Every logical address is added to the base register before accessing physical memory, and the result is checked against the limit.
- **Page Table Mechanism:** In paged memory systems, the operating system maintains a page table for each process. This table maps logical page numbers to physical frame numbers. Each page table entry may contain additional flags to control read/write permissions and validity.

Isolation ensures that processes do not interfere with each other's memory, either accidentally or maliciously. Logical addressing, when combined with proper memory management, creates independent address spaces for each process. This isolation is essential for maintaining the stability and security of the system.

Examples of Violations Prevented by Logical Addressing:

- A faulty program writing data to another process's memory area.
- An unauthorised attempt to read sensitive information from the system memory.
- Accidental modification of operating system kernel data.

In modern operating systems, protection and isolation are fundamental to maintaining system stability and security. The use of logical addresses ensures that each process can only access its own designated memory region, preventing accidental or malicious interference with other processes. This separation is enforced by hardware mechanisms within the memory management unit, which

translate logical addresses to physical ones using controlled mappings such as page tables or segment tables. These translation boundaries act as a protective barrier, ensuring that a process cannot read or write outside its allocated space. As a result, every process operates within an isolated address space, making it possible for multiple applications to run simultaneously without compromising each other's data or execution environment. This isolation is vital for safe multitasking and system reliability.

2.4. Hardware Support Requirements

The implementation of logical and physical address separation requires dedicated hardware support. This support ensures that the translation is efficient, secure, and transparent to the running process.

The primary hardware components involved include:

A. Memory Management Unit (MMU)

The MMU is the central hardware component responsible for translating logical addresses into physical ones. It often uses a page table to map logical pages to physical frames. This translation occurs every time the CPU references memory.

Functions of the MMU:

- Performs address translation
- Enforces access protection
- Supports paging and segmentation
- Interacts with the TLB (Translation Lookaside Buffer) for speed

B. Base and Limit Registers

Used in systems with contiguous memory allocation. These registers store the starting and ending physical addresses for the memory allocated to a process. Every memory access is validated using these registers to ensure the address is within bounds.

C. Translation Lookaside Buffer (TLB)

To improve performance, the MMU uses a cache called the TLB. It stores recent translations from logical to physical addresses. Since memory translation is required for every access, the TLB significantly reduces overhead by caching page table entries.

D. Page Tables and Segment Tables

Depending on the memory management scheme, systems may use page tables (for paging) or segment tables (for segmentation). These tables are often stored in main memory, and special registers hold pointers to their locations.

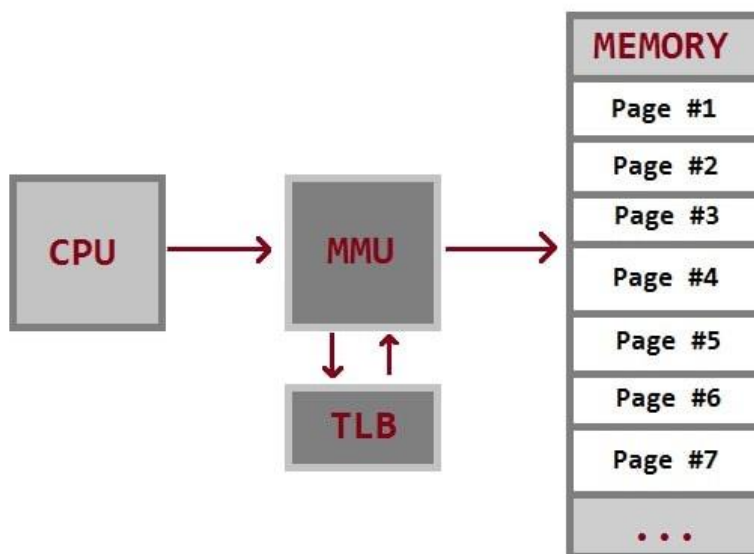


Figure 2: Memory Management Architecture

Source: https://www.techbaz.org/blogs/_data/memory-management-min.jpg

The figure displays a layered architectural diagram illustrating the process of address translation in a system utilising virtual memory. The Central Processing Unit (CPU) initially generates a virtual address, which is then sent to the Memory Management Unit (MMU). The MMU attempts to translate this address, often first consulting the Translation Lookaside Buffer (TLB), a cache for recent translations, to speed up the process. If the translation is found in the TLB, the corresponding physical address in memory is directly accessed. If not, the MMU accesses page tables (not explicitly shown but implied by the MMU's role) in memory to perform the translation before accessing the correct page in physical memory.

2.5. Illustrative Examples

To clarify the operation and usefulness of logical and physical address spaces, the following examples are presented.

Example 1: Contiguous Memory Allocation

Suppose a process is allocated memory starting from physical address 1000, with a size of 400 bytes. The base register is set to 1000, and the limit register is set to 400.

- Logical address 0 maps to physical address 1000
- Logical address 50 maps to physical address 1050
- If the process attempts to access logical address 450, a trap occurs as it exceeds the limit

This simple setup illustrates how base and limit registers isolate process memory.

Example 2: Paging System

Let us assume:

- Page size: 1 KB
- Logical address: 2049 (page number 2, offset 1)
- Page table entry for page 2: frame number 5

The MMU calculates the physical address as:

- Physical address = (frame number × page size) + offset
- = $(5 \times 1024) + 1 = 5121$

Here, logical address 2049 is mapped to physical address 5121. The process need not know where its pages are stored in physical memory.

Example 3: TLB Usage

Consider that a program accesses memory repeatedly at logical address 3000. The first access results in a TLB miss, and the MMU retrieves the mapping from the page table. This mapping is then stored in the TLB.

On subsequent accesses, the TLB supplies the physical frame directly, avoiding page table access. This speeds up execution and demonstrates the role of hardware support in address translation.

Example 4: Process Isolation

Assume two processes, A and B, are running concurrently. Both processes use logical address 1000 to reference their own data. Due to separate page tables:

- Process A's logical address 1000 maps to frame 4 (physical address 4096)
- Process B's logical address 1000 maps to frame 10 (physical address 10240)

Even though both use the same logical address, they access different physical memory. This shows how the system isolates process memory.

The separation between logical and physical address space underpins modern operating systems' ability to execute multiple processes securely and efficiently. Logical addresses offer an abstraction that separates program memory from physical hardware, allowing systems to manage memory more flexibly. Physical addresses refer to actual locations in RAM, and the mapping between the two ensures protection, isolation, and relocation. This translation is handled by hardware units like the MMU and often supported by the TLB for efficiency. Techniques such as paging and segmentation make it possible to run multiple processes securely and efficiently within the system.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which term is sometimes used interchangeably with "logical address"? a. Physical address b. Virtual address c. Base address d. Frame address
2	In which binding method are addresses fixed when a programme is loaded into memory? a. Compile-time binding b. Load-time binding c. Execution-time binding d. Static binding
3	Which hardware component is primarily responsible for translating logical addresses into physical addresses? a. Central Processing Unit b. Memory Management Unit c. Translation Lookaside Buffer d. Page table
4	Which mechanism employs base and limit registers to ensure that a process does not access memory outside its allocated range? a. Segmentation b. Paging

	c. Contiguous allocation d. Virtual memory
5	In a paging system with a page size of 1 KB, if the logical address is 2049 (page number 2, offset 1) and the page table entry for page 2 gives frame number 5, what is the corresponding physical address? a. 2049 b. 3073 c. 5121 d. 6145



3. SWAPPING

Swapping is a fundamental memory management technique employed by operating systems to manage limited main memory resources while supporting the execution of multiple processes. This mechanism plays a significant role in systems where physical memory is insufficient to hold all active processes simultaneously. By temporarily transferring processes between main memory and secondary storage, swapping facilitates multitasking and ensures that critical processes receive the memory they require.

3.1. Concept and Purpose

Swapping is the process of moving an entire process from main memory to a backing store (typically disk storage), and later bringing it back into memory for continued execution. The central aim of this mechanism is to maximise CPU utilisation by ensuring that ready processes can be accommodated in memory, even when memory is scarce.

When multiple programs run concurrently, the main memory may become fully occupied. To free up space for higher-priority or ready-to-execute processes, the operating system selects one or more processes that are in a waiting state and transfers them to the swap space on the disk. These processes are said to be swapped out. When the system decides to resume their execution, they are swapped in back into memory.

Swapping enables the operating system to:

- Maintain a pool of ready processes
- Improve CPU utilisation by avoiding idle periods
- Allow execution of processes larger than physical memory (with the help of virtual memory techniques)
- Implement process scheduling more effectively by dynamically reallocating memory

Although modern systems often rely more heavily on paging and virtual memory, swapping remains an essential concept, especially in embedded systems, mobile devices, and older computing models.

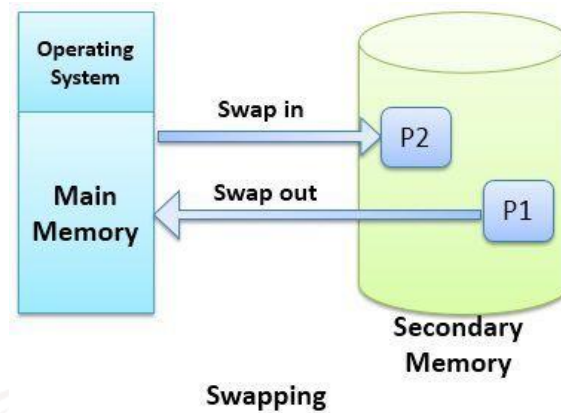


Figure 3: Swapping

Source: <https://techdifferences.com/wp-content/uploads/2017/01/Swapping.jpg>

The figure illustrates the concept of swapping in an operating system, a memory management technique. This process involves moving processes between main memory and secondary memory. As shown, Process P1 is "swapped out" from the main memory to secondary memory, typically a hard disk, to free up space in main memory for other processes. Conversely, Process P2 is "swapped in" from secondary memory to main memory, allowing it to resume execution. Swapping is employed to provide virtual memory capabilities and enable multiple processes to run concurrently, even if their combined memory requirements exceed the available physical main memory.

3.2. Swap Space Management

Swap space refers to the area of the disk designated for holding process images during swapping. The management of this space is crucial for the effective performance and reliability of the system.

Characteristics of Swap Space:

- It is located either in a separate partition or within a swap file on a standard file system.
- Swap space must be large enough to hold the maximum number of processes that may be swapped out simultaneously.
- Efficient access patterns are necessary, as disk I/O operations are significantly slower than memory accesses.

Swap-In and Swap-Out Operations:

When a process is swapped out - The operating system records its current state, including memory contents, register values, and program counter. This state is written to a reserved swap space on the disk. The physical memory previously used by the process is marked as free.

When a process is swapped in - The saved image is retrieved from the swap space. Memory is allocated to accommodate the process. The process state is restored, allowing it to resume execution seamlessly.

Swap Space Allocation Strategies:

1. **Fixed Allocation:** A fixed amount of disk space is reserved for swapping. This method is simple but may be inflexible in dynamic environments.
2. **Dynamic Allocation:** The swap area grows or shrinks based on system demand. It is more adaptable but requires careful tracking to avoid fragmentation.
3. **Priority-Based Swapping:** Processes are swapped out based on their priority or memory usage patterns, optimising swap space usage.

3.3. Performance Implications

While swapping enhances flexibility and memory availability, it introduces performance overhead due to the time required to transfer processes between memory and disk.

Factors Affecting Performance:

- **Disk I/O Time:** Accessing disk storage is orders of magnitude slower than accessing RAM. Frequent swapping leads to increased latency.
- **Process Size:** Larger processes take longer to swap, increasing the delay before execution can resume.
- **Swapping Frequency:** Excessive swapping, also known as thrashing, can severely degrade system performance.

Impact on CPU Utilisation: Swapping enables the CPU to remain active by ensuring there is always at least one ready process in memory. However, if too much time is spent swapping instead of executing instructions, the CPU utilisation may drop due to I/O bottlenecks.

Impact on User Experience: Applications may appear unresponsive when they are swapped out and take time to swap back in. System responsiveness declines if multiple processes compete for limited memory and swap space simultaneously.

Swapping, while useful for extending available memory, can lead to significant performance issues if not managed carefully. Frequent swapping in and out of processes increases disk I/O, leading to slower system response times. To address this, operating systems employ efficient page replacement algorithms that minimise redundant swaps, particularly when paging is also used. Controlling the

number of active processes through multiprogramming limits helps reduce the strain on memory, thereby lowering the frequency of swaps. In some systems, prefetching and predictive techniques are applied to anticipate which processes will soon require memory access, allowing the system to prepare in advance and reduce waiting times. These strategies collectively help maintain system performance despite the overhead introduced by swapping.

3.4. Thrashing and Its Prevention

Thrashing occurs when a system spends more time swapping processes in and out of memory than executing them. This condition results in significant performance degradation and is typically caused by an imbalance between memory demand and available physical memory.

Causes of Thrashing:

- High degree of multiprogramming leading to frequent context switching
- Processes with large working sets competing for limited memory
- Poorly configured swap space or memory allocation policies

The symptoms of thrashing are typically easy to observe. Continuous and intense disk activity indicates that the system is constantly reading from and writing to the swap space. This excessive swapping leads to very low CPU utilisation, as the processor remains idle while waiting for memory operations to complete. As a result, the response time for all running processes increases noticeably, and overall system performance deteriorates. In extreme cases, programmes may make little to no progress, leaving the system in a near-frozen state.

Prevention Techniques:

1. **Working Set Model:** Monitor the set of memory pages a process actively uses. Processes are only allowed in memory if their entire working set fits, reducing page faults and swap activity.
2. **Page Fault Frequency (PFF) Control:** If a process experiences frequent page faults, it may be swapped out to reduce memory pressure.
3. **Reducing Degree of Multiprogramming:** Temporarily suspending some processes prevents overload.
4. **Use of Virtual Memory Techniques:** Allows for better memory abstraction and delayed loading of process parts, preventing full process swapping.
5. **Memory Reservation:** Critical system processes may have reserved memory to avoid being swapped, ensuring stability.

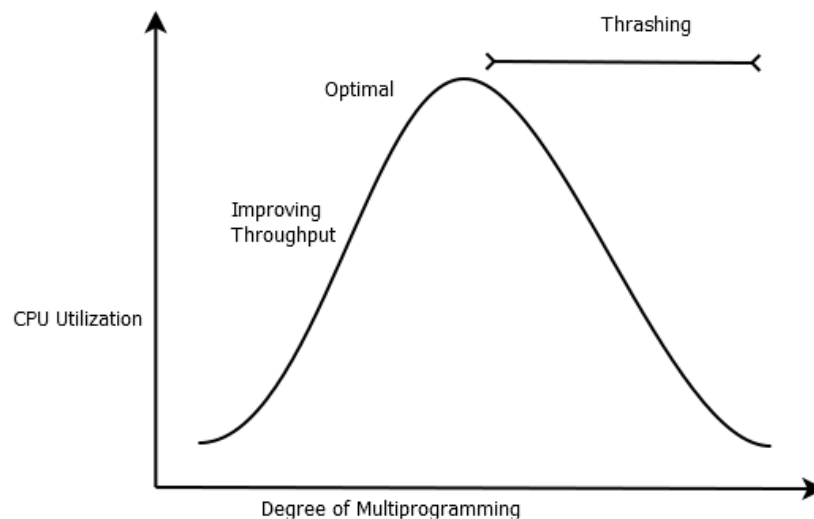


Figure 4: Thrashing

Source: <https://static.takeuforward.org/wp/uploads/2022/04/image-21.png>

The figure presents a graph illustrating the relationship between CPU utilisation and the degree of multiprogramming, clearly depicting the phenomenon of thrashing. Initially, as the degree of multiprogramming increases, CPU utilisation improves and throughput rises, reaching an optimal point. However, beyond this optimum, further increases in the degree of multiprogramming cause CPU utilisation to sharply decline. This drop signifies thrashing, where the system spends excessive time swapping pages between main and secondary memory rather than executing useful work, thus drastically reducing overall system performance.

3.5. Operating System Support

Modern operating systems implement and manage swapping through a variety of tools, policies, and subsystems. While the direct use of swapping may be limited in systems with abundant RAM, the underlying concepts remain integral to memory management.

Features Provided by Operating Systems:

1. Swap Space Management Tools:

- Allocation and monitoring of swap partitions and files
- Dynamic resizing or creation of swap files based on system load

2. Process Scheduler Integration:

- The scheduler collaborates with the memory manager to select candidates for swapping based on priority, memory usage, and I/O activity.

3. Swapping Policies:

- Algorithms define which processes to swap out, when to swap them back in, and how long to retain swapped-out processes.

4. Integration with Virtual Memory:

- Swapping often works alongside virtual memory systems. Operating systems load only required parts of a process into memory while retaining the rest in swap space.

5. Swapping Indicators and Controls:

- Most operating systems provide tools to monitor swap usage. For example, the `top`, `vmstat`, and `free` commands in UNIX-like systems.

Examples from Popular Operating Systems:

Linux uses both swap files and partitions to manage memory. It supports a `swappiness` parameter that controls how aggressively the system resorts to swapping. Tools such as `mkswap` and `swapon/off` are provided for creating and managing swap space.

Windows implements swapping through a special file known as `pagefile.sys`. It allows users to customise the size and location of this pagefile based on system requirements. The operating system also includes tools like Resource Monitor to track swap usage and overall memory performance.

macOS utilises dynamic swap files that are managed within the APFS file system. Its swapping mechanism is closely integrated with virtual memory and memory compression features to ensure efficient use of available resources.

Benefits of OS-Level Swapping Support:

- Enables systems to run large applications on limited RAM.
- Enhances process management through memory overcommitment.
- Supports multitasking and better memory utilisation.

Swapping is an essential memory management mechanism, particularly relevant in systems with limited physical memory. It allows operating systems to maintain process concurrency and manage memory dynamically. However, its use introduces challenges, especially in terms of performance and system responsiveness.

The key components of swapping include the management of swap space, coordination with the scheduler, and hardware-level efficiency in disk I/O. When implemented properly, swapping allows systems to handle more processes than would otherwise be possible. On the other hand, poor

management can lead to thrashing and severe system slowdown. By understanding how swapping works and how modern operating systems manage it, one gains deeper insight into the trade-offs involved in memory management. In a broader context, swapping illustrates the continuous balancing act between memory availability, process execution, and system performance.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	What is the principal aim of swapping in an operating system? a. To increase the size of physical memory b. To transfer entire processes between main memory and secondary storage c. To speed up CPU clock cycles d. To compress data in main memory
7	Which of the following is not a recognised swap-space allocation strategy? a. Fixed allocation b. Dynamic allocation c. Priority-based swapping d. First-fit allocation
8	Thrashing is characterised by: a. Excessive page-fault handling that degrades performance b. Complete absence of disk I/O activity c. Processes running without interruption d. Static allocation of swap space
9	Which prevention technique involves monitoring the set of pages a process actively uses and ensuring they fit in memory? a. Page-fault frequency control b. Working set model c. Memory reservation d. Priority-based swapping
10	In Linux, which parameter determines how aggressively the kernel will swap processes out of physical memory? a. vm.max_map_count b. kernel.shmmax

	c. vm.swappiness
	d. fs.file-max



4. CONTIGUOUS ALLOCATION

Contiguous memory allocation is one of the oldest and most straightforward memory management techniques used by operating systems. Under this scheme, each process is allocated a single continuous block of physical memory. While conceptually simple and easy to implement, contiguous allocation introduces several challenges related to space utilisation and memory fragmentation. This section explores the working principles of contiguous allocation, the differences between static and dynamic allocation, types of fragmentation, methods of compaction, and algorithms used to place processes in memory effectively.

4.1. Basic Strategy

The fundamental idea behind contiguous allocation is that a process is assigned a single, continuous segment of memory large enough to hold all its instructions and data. Once allocated, the process occupies the block without any interruptions, which simplifies the process of address translation.

In such systems, memory is typically divided into two portions:

- Operating system memory space (usually fixed and located at the lower or upper end of the memory)
- User space available for process allocation

When a process is created, the memory manager scans the available memory to locate a contiguous block large enough to hold the process. Once found, the entire process is loaded into that block. Upon termination, the memory is released and made available for future allocations.

Advantages of Contiguous Allocation:

- Fast and simple address computation due to continuous physical memory
- Easy implementation using base and limit registers
- Minimal overhead in tracking process memory

Disadvantages:

- Prone to fragmentation (especially external)
- Inefficient use of memory in systems with variable-sized processes
- Requires relocation if a suitable block is not found

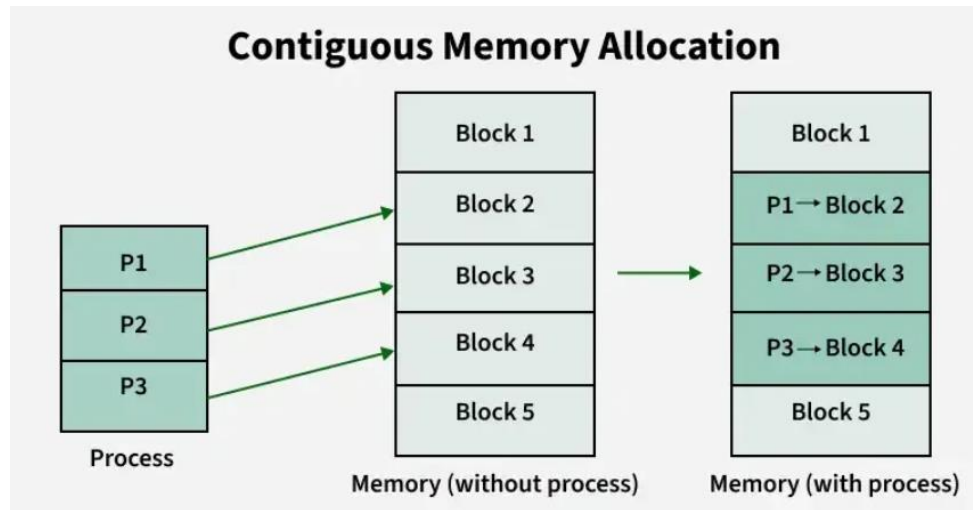


Figure 5: Contiguous Memory Allocation

Source: [https://media.geeksforgeeks.org/wp-](https://media.geeksforgeeks.org/wp-content/uploads/20250113101849093610/contiguous_memory_allocation.webp)

[content/uploads/20250113101849093610/contiguous_memory_allocation.webp](https://media.geeksforgeeks.org/wp-content/uploads/20250113101849093610/contiguous_memory_allocation.webp)

The figure demonstrates contiguous memory allocation, a method where each process is assigned a single, unbroken block of memory. On the left, individual processes P1, P2, and P3 are shown. In the centre, a conceptual view of memory blocks without processes is presented. The right side illustrates how these processes are then loaded into contiguous blocks: P1 is allocated Block 2, P2 takes Block 3, and P3 occupies Block 4. This ensures that all parts of a given process reside in a single, continuous section of physical memory, with Block 1 and Block 5 remaining as unallocated or reserved space.

4.2. Static versus Dynamic Allocation

The method of allocating contiguous memory can be classified into two primary types based on the timing and flexibility of memory assignment: static allocation and dynamic allocation.

Static Allocation: In static allocation, memory is assigned to processes at compile time or when the process is initially loaded into memory. Once allocated, the memory size remains fixed for the lifetime of the process.

- The process is expected to run within the assigned memory limits
- No memory growth is allowed during execution
- Simple and predictable; often used in embedded and real-time systems

Static memory allocation can lead to inefficiencies, as it often reserves more memory than a process actually uses, resulting in wastage. It also lacks flexibility, since the fixed allocation cannot respond to

changes in a process's needs during execution, potentially causing performance issues or resource underuse.

Dynamic Allocation: Dynamic allocation provides greater flexibility by assigning memory during the execution of the program. The process requests memory from the operating system as needed, and the OS allocates it from the pool of available free contiguous blocks.

- Allows processes to grow or shrink during execution
- Better memory usage for programs with unpredictable memory needs
- More complex to implement and manage

Dynamic memory allocation, while flexible, has certain drawbacks. It can increase the risk of fragmentation as memory blocks are allocated and freed at different times. If suitable contiguous space is not available for new allocations, the system may need to perform relocation or compaction, which can add overhead and affect performance.

4.3. Fragmentation: Internal and External

Fragmentation is one of the central concerns in contiguous memory allocation. It leads to inefficient memory usage and limits the ability of the system to allocate memory to new processes even if sufficient total memory is available.

Internal Fragmentation - Internal fragmentation occurs when the allocated memory block is larger than the memory actually required by the process. The unused portion within the allocated block remains unused and inaccessible to other processes.

Causes:

- Fixed-size memory partitions
- Allocation rounding up to meet alignment requirements

Example:

If a process needs 18 KB of memory but is placed in a 20 KB block, the remaining 2 KB goes to waste as internal fragmentation.

Solutions:

- Use variable-sized partitions instead of fixed ones
- Match process sizes more closely to available memory chunks

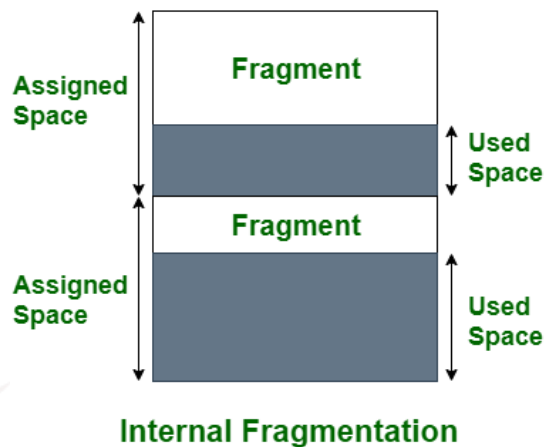


Figure 6: Internal Fragmentation

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20190924115421/Untitled-Diagram-146.png>

The figure visually represents internal fragmentation, a phenomenon in memory allocation where allocated memory blocks are larger than the actual memory required by the process. As depicted, within each "Assigned Space," there is a "Used Space" and a leftover "Fragment" of unused memory. This unused portion within an allocated block is inaccessible to other processes, leading to wasted memory. This type of fragmentation occurs when memory is divided into fixed-size blocks or pages, and a process is assigned a block that is not an exact fit for its memory demands.

External Fragmentation - External fragmentation arises when free memory is scattered across many small blocks. Although the total free memory may be sufficient to accommodate a process, it is not available in a single contiguous block.

Causes:

- Repeated allocation and deallocation over time
- Variable-sized processes occupying and releasing memory

Example:

A system may have 100 KB of free memory scattered in 10 blocks of 10 KB each, but a new process requesting 50 KB cannot be accommodated.

Solutions:

- Compaction (rearranging processes to consolidate free memory)
- Placement algorithms to optimise block selection
- Using paging or segmentation to avoid contiguous constraints

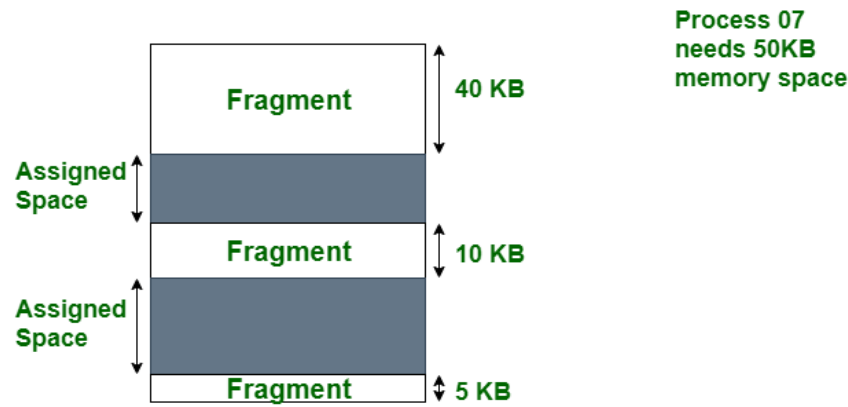


Figure 7: External Fragmentation

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20200729172413/2581.png>

The figure illustrates external fragmentation, a condition in memory management where there is enough total free space in memory to satisfy a request, but this space is not contiguous. Here, Process 07 requires 50KB of memory. Although there are three separate "Fragment" blocks of 40KB, 10KB, and 5KB, totalling 55KB of free space, none of these individual fragments are large enough on their own to accommodate the 50KB request. This dispersed free space prevents the allocation of contiguous memory to the new process, despite sufficient overall capacity.

4.4. Compaction Techniques

Compaction is a technique used to counteract external fragmentation by rearranging the contents of memory so that all free spaces are consolidated into one large block. This increases the chances of accommodating larger processes.

Working of Compaction

- The memory manager moves all active processes towards one end of the memory
- All free blocks are pushed to the opposite end, creating a large contiguous space
- Base addresses of processes are updated to reflect new locations

Compaction is generally resource-intensive and time-consuming, as it requires updating memory references, moving large amounts of data, and ensuring that processes are not executing during the operation.

When is Compaction Performed?

- When a process requires a large block and no sufficient contiguous space is available
- Periodically, during low system load

- Manually triggered in some systems

Limitations of Compaction:

- High overhead due to process relocation
- Not suitable for real-time systems where timing constraints are strict
- May interfere with ongoing operations if not synchronised properly

4.5. Placement Algorithms

To reduce fragmentation and improve memory utilisation, the operating system employs various algorithms to decide where a new process should be placed within the available memory. These are known as placement or fit algorithms.

1. First Fit

- Scans memory from the beginning and places the process in the first available block large enough to accommodate it
- Simple and fast, as it makes the first acceptable match

The first fit allocation strategy is efficient in terms of search time, as it assigns the first available block that meets the memory requirement. It works well in systems where memory usage is low. However, it often creates small unused gaps near the beginning of memory, which can accumulate and lead to fragmentation over time, reducing overall memory efficiency.

2. Best Fit

- Searches the entire memory for the smallest block that is large enough
- Minimises leftover space, thus reducing internal fragmentation

The best fit strategy aims to use memory blocks more efficiently by selecting the smallest block that fits the requirement. While this can reduce immediate wastage, it is often time-consuming as it requires scanning the entire memory. Over time, it may also lead to increased external fragmentation by leaving behind small, unusable gaps.

3. Worst Fit

- Allocates the largest available block to the process
- Idea is to leave large enough leftover space for future allocations

The worst fit strategy can be beneficial when large processes are expected, as it allocates memory from the largest available block, preserving smaller blocks for later use. However, it generally performs poorly in real-world conditions and often leads to large blocks being broken into many small, unusable fragments, reducing overall memory efficiency.

4. Next Fit

- Variant of first fit that continues the search from the last allocation point
- Reduces repeated scanning of the same memory regions

The next fit allocation method offers a balanced distribution by continuing the search from the last allocated position, which helps avoid clustering near the start of memory. This approach spreads allocations more evenly across the memory space. However, it may overlook suitable blocks that exist earlier, potentially leading to inefficient use of available memory.

Table 1: Comparison of Memory Allocation Algorithms

Algorithm	Search Time	Fragmentation	Memory Utilisation
First Fit	Fast	Moderate	Moderate
Best Fit	Slow	Low Internal, High External	Moderate
Worst Fit	Slow	High	Low
Next Fit	Moderate	Moderate	Moderate

Contiguous memory allocation offers simplicity in design and implementation but comes at the cost of flexibility and space efficiency. Static allocation is predictable but rigid, while dynamic allocation supports process growth but introduces complexity. Fragmentation, both internal and external, remains the primary challenge associated with contiguous allocation.

The operating system must carefully manage memory to reduce fragmentation, either by adopting smart allocation strategies or by performing compaction when necessary. The choice of placement algorithm also plays a crucial role in system performance and memory usage. Although modern systems often rely on advanced techniques such as paging and segmentation, the principles of contiguous allocation remain relevant, particularly in embedded systems, real-time applications, and educational contexts where resource constraints demand simplicity.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	A process allocated memory at compile time with no allowance for growth during execution exemplifies which allocation type? a. Dynamic allocation b. Static allocation c. Paging d. Segmentation
12	Which form of fragmentation occurs when an allocated block exceeds the process's actual memory requirement? a. External fragmentation b. Internal fragmentation c. Compaction fragmentation d. Alignment fragmentation
13	Which placement algorithm begins its search at the start of memory and assigns the first block that fits the process's size? a. Best fit b. Next fit c. First fit d. Worst fit
14	What is the primary objective of compaction in contiguous memory allocation? a. To split large free blocks into smaller ones b. To relocate processes and consolidate free memory into one block c. To reduce the size of process memory requirements d. To increase internal fragmentation
15	Which allocation strategy deliberately chooses the largest available free block for a process, aiming to preserve smaller blocks? a. First fit b. Next fit c. Best fit d. Worst fit

5. IMPLEMENTATION AND PERFORMANCE CONSIDERATIONS

Efficient memory management is central to the functioning of any modern operating system. Its design must balance performance, flexibility, and hardware constraints. Different environments such as general-purpose UNIX systems, embedded platforms, and real-time operating systems pose unique requirements and challenges. This section explores the practical implementation of memory management in UNIX-like systems, the specialised constraints faced in embedded applications, real-time systems' deterministic needs, and the evaluation metrics used to assess the performance of memory management strategies.

5.1. Memory Management in UNIX-like Systems

UNIX-like operating systems employ a combination of memory management techniques, such as paging, segmentation, and demand paging, to efficiently utilise system memory and ensure process isolation.

Virtual Memory Abstraction - UNIX systems treat physical memory as a finite resource that can be abstracted through the use of virtual memory. Each process is given its own virtual address space, allowing it to operate independently of other processes.

- The virtual address space is divided into segments for code, data, stack, and heap
- Logical to physical address translation is handled by the memory management unit (MMU)
- Page tables are used to map virtual pages to physical frames

Demand Paging and Lazy Loading - Demand paging is a key feature in UNIX systems that helps conserve memory. Pages are loaded into physical memory only when they are actually referenced by a process.

- Reduces initial loading time of large programs
- Minimises unnecessary memory usage
- Page faults trigger the OS to fetch the required page from secondary storage

Swapping and Page Replacement - When physical memory becomes scarce, UNIX systems may move entire processes or individual pages to the swap space.

- Swap space is a reserved area on the disk used as an extension of RAM

- Page replacement algorithms like Least Recently Used (LRU), Clock, or NRU (Not Recently Used) determine which pages to evict

Memory Protection - UNIX systems enforce memory protection through:

- Per-page access permissions (read, write, execute)
- Use of privilege levels to prevent user-mode processes from accessing kernel space
- Segregation of user space and kernel space

Shared Memory and Inter-process Communication - Shared memory is often used for efficient inter-process communication.

- Memory segments are mapped into the virtual address spaces of multiple processes
- Synchronisation mechanisms ensure consistency

5.2. Embedded System Constraints

Embedded systems operate under strict resource limitations, and their memory management requirements differ significantly from general-purpose systems. These systems often run on hardware with limited RAM, no secondary storage, and real-time performance demands.

Characteristics of Embedded Memory Management

- Memory is usually statically allocated at compile time
- Systems often operate without virtual memory or paging
- Simplicity and predictability are prioritised over flexibility

Static versus Dynamic Allocation - Static memory allocation is commonly used in embedded environments due to its deterministic behaviour.

- Memory regions are predefined and fixed at compile time
- No runtime allocation or deallocation overhead
- Reduces fragmentation risks

However, some systems require dynamic allocation for handling variable workloads or multiple tasks. Lightweight memory allocators and fixed-size memory pools are used in such cases.

Memory Mapping and Addressing - Memory-mapped I/O is used to control hardware peripherals. Direct addressing is preferred to avoid the complexity of virtual address translation.

Reliability and Safety - Safety-critical embedded systems, such as those in automotive or medical devices, impose stringent requirements:

- No memory leaks or unbounded growth of memory usage
- Use of memory protection units (MPUs) in place of full MMUs
- Certification against safety standards like ISO 26262 or IEC 61508

Example - Consider a simple microcontroller-based thermostat. The entire program is loaded at once, using statically allocated buffers for input, processing, and display. Memory usage is calculated at compile time to ensure predictability and stability.

5.3. Real-Time Operating System Requirements

Real-time operating systems (RTOS) differ from general-purpose systems by focusing on time-deterministic behaviour. Memory management in such systems must support predictable execution, timely task scheduling, and minimal latency.

Determinism in Memory Management

RTOS environments often avoid dynamic memory allocation due to its unpredictability. Memory management strategies in these systems are geared towards ensuring that every task meets its deadline.

- No paging or swapping is permitted
- Static memory allocation is preferred
- Memory is pre-partitioned and assigned to specific tasks or functions

Memory Protection and Isolation

While predictability is crucial, RTOS must also ensure that one task does not interfere with the memory of another. Lightweight protection mechanisms include:

- Use of MPUs to enforce access boundaries
- Partitioning of memory for tasks and inter-process communication
- No overcommitment of memory resources

Real-Time Memory Allocators

Where dynamic memory is essential, special real-time allocators are used:

- Allocators with constant-time behaviour ($O(1)$)

- Memory pools of fixed-size blocks
- Predictable fragmentation handling strategies

Avoidance of Fragmentation

Fragmentation is a serious concern in real-time systems. To mitigate it:

- Allocators avoid splitting memory blocks unnecessarily
- Tasks use buffers from pre-allocated pools
- Allocation and deallocation patterns are carefully analysed at design time

Case Example: In avionics systems, multiple real-time tasks may control flight instruments, each with pre-assigned memory segments. No process is allowed to overstep its boundaries, and the entire system must guarantee bounded execution time, even under stress conditions.

5.4. Performance Evaluation Metrics

Assessing the effectiveness of a memory management strategy requires measurable performance metrics. These metrics help system designers and operating system developers identify bottlenecks and optimise performance based on specific system goals.

A. Memory Utilisation

This metric evaluates how efficiently the available physical memory is being used.

- Defined as the ratio of used memory to total available memory
- Higher memory utilisation indicates less wastage
- In fragmented systems, high memory usage may not always imply high performance

B. Page Fault Rate

For systems using virtual memory, page fault rate is critical.

- Defined as the number of page faults per unit time or per memory access
- Lower page fault rates indicate more effective memory loading and better performance
- Excessive page faults may lead to thrashing

C. Allocation Time

This measures how quickly memory is allocated to processes or tasks.

- Shorter allocation times improve system responsiveness
- Crucial in real-time systems where delays cannot be tolerated

D. Fragmentation Metrics

Fragmentation is assessed both internally and externally:

- Internal fragmentation measures wasted space within allocated blocks
- External fragmentation refers to the inability to allocate memory despite sufficient total free space

Fragmentation is often quantified as a percentage of wasted memory.

E. Turnaround Time and Throughput

- Turnaround time measures the time from process submission to completion, including waiting for memory
- Throughput refers to the number of processes completed per unit time

Efficient memory management directly impacts both metrics by reducing allocation delays and preventing memory contention.

F. Latency and Jitter

In real-time systems:

- Latency is the time taken to respond to an event or interrupt
- Jitter refers to the variability in latency over time

Low and predictable latency is essential for real-time performance.

G. Swap and I/O Overhead

In systems with virtual memory:

- Swap overhead refers to the time and resources used in moving data between main memory and secondary storage
- Excessive swapping increases I/O load and degrades performance

Example: Consider two systems: one using first-fit memory allocation and the other using best-fit. While the first may allocate memory faster, the second might use memory more efficiently but incur higher fragmentation and allocation time. Performance metrics help decide which is more appropriate for a given context.

Memory management is not merely a theoretical concept but a crucial component of operating system implementation, significantly affecting performance, reliability, and suitability for specific

environments. In UNIX-like systems, virtual memory and paging enable multitasking and resource efficiency. Embedded systems prioritise simplicity and determinism, often avoiding complex memory abstraction. Real-time operating systems demand strict timing guarantees and safe memory access patterns.

Performance evaluation metrics such as memory utilisation, page fault rate, and allocation time serve as essential tools to assess and refine memory management techniques. The diverse requirements across platforms highlight the need for adaptable memory strategies tailored to the operational context. Designing and implementing effective memory management remains a fundamental challenge that balances system goals, hardware constraints, and user demands.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
16	In UNIX-like systems, which mechanism conserves memory by loading pages into physical memory only when they are referenced by a process? a. Demand paging b. Static allocation c. Swapping d. Compaction
17	In safety-critical embedded systems, which hardware unit is commonly used instead of a full MMU to enforce access boundaries? a. Translation lookaside buffer b. Memory protection unit c. Base and limit registers d. Page table
18	Which performance metric is defined as the number of page faults per unit time or per memory access? a. Allocation time b. Throughput c. Page fault rate d. Memory utilisation

19	In real-time operating systems, what term describes the variability in response time to an event or interrupt? a. Latency b. Throughput c. Jitter d. Fragmentation
20	What does swap overhead refer to in systems using virtual memory? a. The ratio of used memory to total available memory b. The time and resources required to move data between main memory and secondary storage c. The number of processes completed per unit time d. The variability in memory allocation time



6. SUMMARY

- The logical address, produced by the CPU, is translated into a physical address by the memory management unit, enabling programmes to run without direct knowledge of actual RAM locations.
- Address mapping can occur at compile time, load time or execution time, with modern systems favouring execution-time binding via hardware support such as page tables and base-and-limit registers.
- Separation of logical and physical spaces enforces protection and isolation: base-and-limit checks or per-page permissions prevent processes from accessing unauthorised memory regions.
- Efficient translation requires hardware components including the MMU, translation lookaside buffer, base and limit registers, and page or segment tables.
- Swapping moves entire processes between main memory and disk to maintain a pool of ready tasks, though it incurs disk I/O overhead and may degrade responsiveness if overused.
- Swap space may be allocated in fixed or dynamic partitions or managed by priority, with careful tracking needed to balance flexibility against fragmentation and performance.
- Thrashing, where excessive swapping outweighs useful computation, can be prevented by controlling the degree of multiprogramming, monitoring page-fault frequency or applying the working-set model.
- Contiguous allocation assigns each process a single block of physical memory; static schemes fix sizes at compile time, while dynamic schemes allow growth but risk fragmentation.
- Internal fragmentation wastes space within allocated blocks; external fragmentation scatters small free segments, both mitigated by compaction or by choosing first-fit, best-fit, worst-fit or next-fit placement algorithms.
- Real-world implementations vary: UNIX-like systems use demand paging and sophisticated replacement policies, embedded platforms favour static allocation and MPUs, and real-time systems demand predictable, low-latency memory operations evaluated by metrics such as page-fault rate, allocation time and latency jitter.

7. GLOSSARY

Logical address	-	the address generated by a programme during execution, which is translated by the MMU into a physical address.
Physical address	-	the actual location in main memory (RAM) where data or instructions reside.
Memory Management Unit (MMU)	-	the hardware component responsible for translating logical addresses to physical addresses and enforcing access permissions.
Swapping	-	the process of moving an entire process between main memory and disk to balance memory demand and CPU utilisation.
Thrashing	-	a state in which excessive swapping predominates, causing the system to spend more time moving processes than executing them.
Internal fragmentation	-	wasted space within an allocated memory block when the block exceeds the process's actual requirements.
External fragmentation	-	a condition where free memory is broken into small, non-contiguous blocks that cannot satisfy a large allocation request.
Contiguous allocation	-	a memory management scheme in which each process occupies a single, unbroken block of physical memory.
Translation Lookaside Buffer (TLB)	-	a cache that stores recent logical-to-physical address translations to accelerate memory access.
Page-fault rate	-	the frequency at which a system must load pages from secondary storage due to absent or invalid pages in physical memory.

8. TERMINAL QUESTIONS

1. Explain the key differences between logical and physical address spaces, and discuss how this separation benefits system security.
2. Describe the three address-binding mechanisms and evaluate the advantages and disadvantages of each.
3. Outline the role of the memory management unit and translation lookaside buffer in efficient address translation.
4. Discuss the concept of swapping, including its purpose, swap-space management strategies and performance implications.
5. Analyse the causes of thrashing and propose three techniques an operating system might use to prevent it.
6. Compare static and dynamic contiguous allocation, highlighting scenarios in which each method is most appropriate.
7. Define internal and external fragmentation, and explain how compaction addresses external fragmentation.
8. Evaluate the first-fit, best-fit, worst-fit and next-fit placement algorithms in terms of search time and fragmentation.
9. Contrast the memory-management requirements of UNIX-like systems, embedded systems and real-time operating systems.
10. Select three performance evaluation metrics for memory management and justify their importance in assessing a given system's effectiveness.

9. ANSWERS

Self-Assessment Questions

1. b) Virtual address
2. b) Load-time binding
3. b) Memory Management Unit
4. c) Contiguous allocation
5. c) 5121
6. b) To transfer entire processes between main memory and secondary storage
7. d) First-fit allocation
8. a) Excessive page-fault handling that degrades performance
9. b) Working set model
10. c) vm.swappiness
11. b) Static allocation
12. b) Internal fragmentation
13. c) First fit
14. b) To relocate processes and consolidate free memory into one block
15. d) Worst fit
16. a) Demand paging
17. b) Memory protection unit
18. c) Page fault rate
19. c) Jitter
20. b) The time and resources required to move data between main memory and secondary storage

Terminal Questions Answers

Answers 1: Logical addresses are the CPU-generated references used by programmes, while physical addresses correspond to actual RAM locations; this abstraction prevents direct access to hardware memory and allows processes to run in isolated spaces. By keeping these spaces separate, the system can enforce bounds checking and per-page permissions, thwarting unauthorised memory access and enhancing security. Refer to section 2.1 Definition and Distinction.

Answers 2: Compile-time binding fixes addresses at compilation, offering simplicity but no relocation; load-time binding adjusts relocatable code when a programme is loaded, striking a balance between

flexibility and overhead; execution-time binding uses the MMU to translate addresses dynamically, maximising relocation and protection at the cost of hardware complexity. Each method trades off between efficiency, flexibility and protection. Refer to section 2.2 Address Mapping Mechanisms.

Answers 3: The Memory Management Unit (MMU) translates each logical address into its corresponding physical frame, enforcing access rights, while the Translation Lookaside Buffer (TLB) caches recent translations to avoid repeated page-table lookups and accelerate memory access. Together they provide rapid, hardware-assisted mapping and protection for every memory reference. Refer to section 2.4 Hardware Support Requirements.

Answers 4: Swapping moves entire processes between main memory and disk to ensure that ready tasks can execute despite limited RAM, using a reserved swap area managed in fixed or dynamically sized partitions. While it maintains CPU utilisation, the disk I/O overhead of swapping can harm responsiveness and lead to delays if overused. Refer to sections 3.1 Concept and Purpose and 3.2 Swap Space Management.

Answers 5: Thrashing arises when excessive swapping consumes most CPU time, typically due to high multiprogramming levels or large working sets. Preventive techniques include applying the working-set model to limit in-memory pages, controlling page-fault frequency to swap out faulting processes, and reducing the degree of multiprogramming to relieve memory pressure. Refer to section 3.4 Thrashing and Its Prevention.

Answers 6: Static contiguous allocation fixes a process's memory block at load time, ensuring predictability but risking wasted space, whereas dynamic allocation grants or releases blocks during execution, better fitting variable demands but increasing fragmentation and management overhead. Static schemes suit embedded or real-time contexts, while dynamic schemes benefit general-purpose systems with fluctuating workloads. Refer to section 4.2 Static versus Dynamic Allocation.

Answers 7: Internal fragmentation occurs when a process's block exceeds its needs, leaving unused space within the allocation; external fragmentation results from many small free segments scattered through memory. Compaction combats external fragmentation by relocating active processes to one end of memory, consolidating free space into a single contiguous block. Refer to sections 4.3 Fragmentation: Internal and External and 4.4 Compaction Techniques.

Answers 8: First-fit scans from the start and allocates the first adequate block (fast search, moderate fragmentation), best-fit searches all blocks for the smallest match (slow, low internal but high external fragmentation), worst-fit uses the largest block (slow, often inefficient), and next-fit

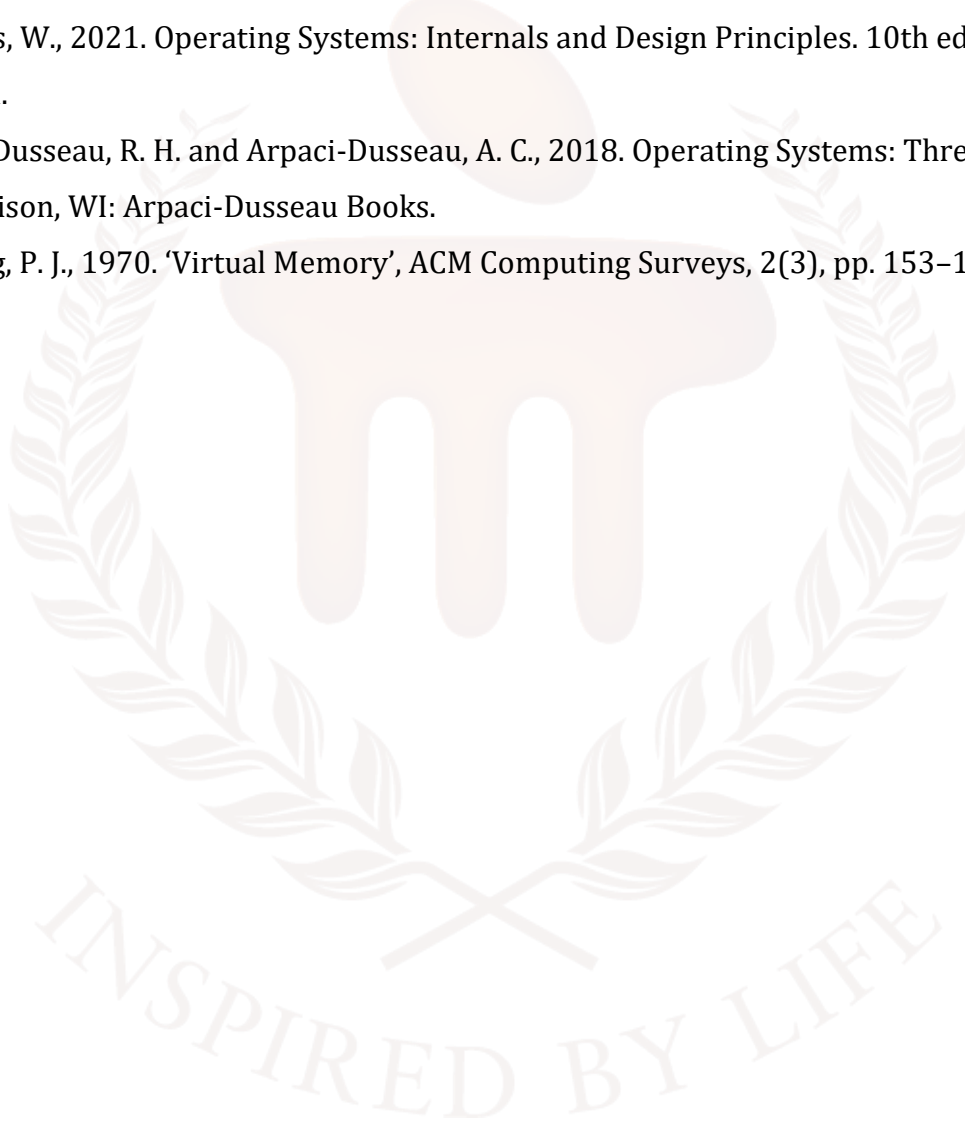
continues from the last allocation point (moderate speed and fragmentation). Each algorithm balances search time against fragmentation differently. Refer to section 4.5 Placement Algorithms.

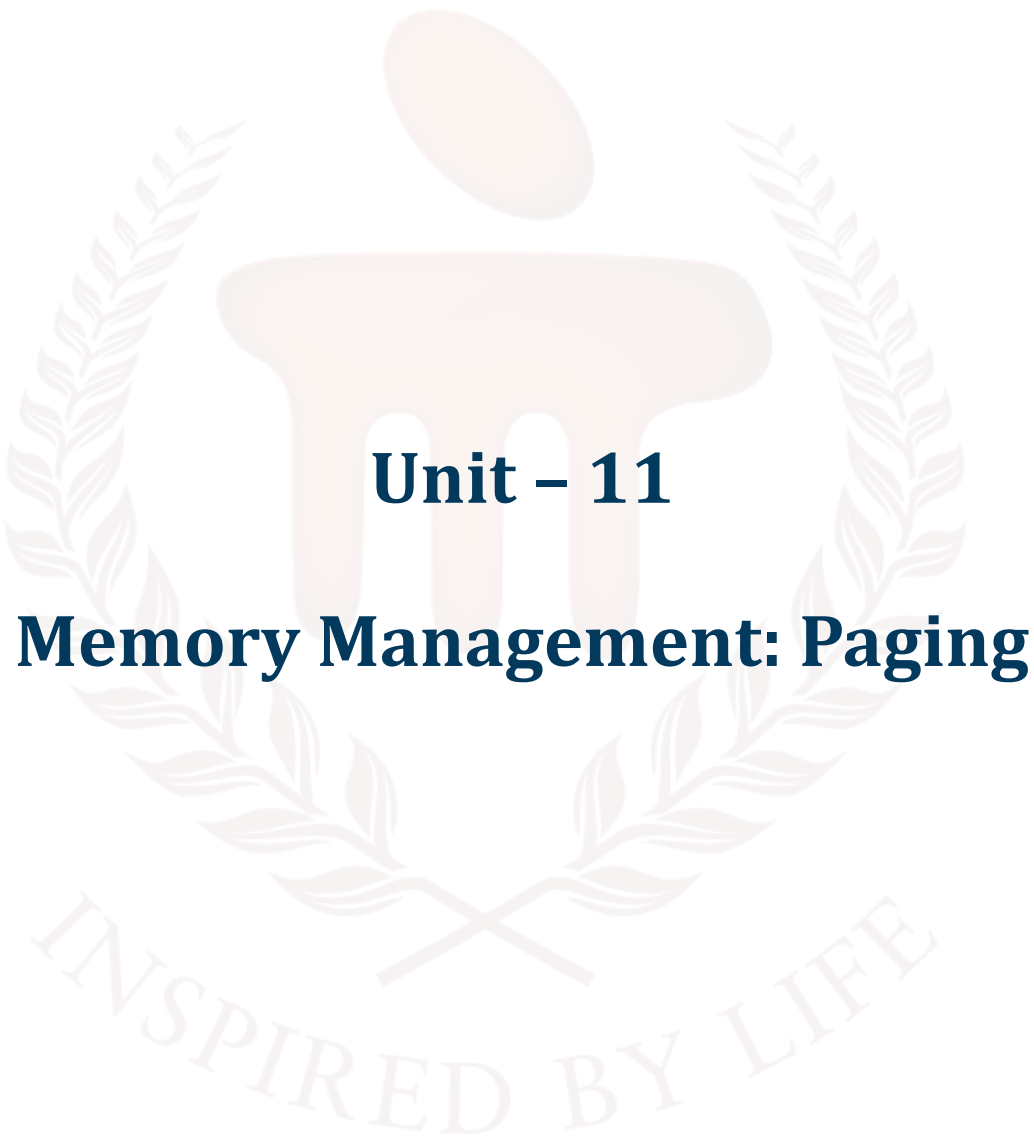
Answers 9: UNIX-like systems rely on demand paging, swap space and replacement algorithms for flexible virtual memory; embedded platforms favour static, compile-time allocation with minimal hardware and often use MPUs instead of full MMUs; real-time systems demand deterministic, pre-partitioned memory with no paging or swapping and use fixed-time allocators to guarantee deadlines. Refer to sections 5.1 Memory Management in UNIX-like Systems, 5.2 Embedded System Constraints and 5.3 Real-Time Operating System Requirements.

Answers 10: Key metrics include memory utilisation (ratio of used to total memory, indicating efficiency), page-fault rate (faults per time or access, revealing load effectiveness and risk of thrashing) and allocation time (latency of granting memory, critical for responsiveness and real-time predictability). These measures help compare and tune memory strategies for different system goals. Refer to section 5.4 Performance Evaluation Metrics.

10. REFERENCES

1. Silberschatz, A., Galvin, P. B. and Gagne, G., 2020. Operating System Concepts. 10th ed. Hoboken, NJ: Wiley.
2. Tanenbaum, A. S. and Bos, H., 2015. Modern Operating Systems. 4th ed. Harlow: Pearson.
3. Stallings, W., 2021. Operating Systems: Internals and Design Principles. 10th ed. Harlow: Pearson.
4. Arpaci-Dusseau, R. H. and Arpaci-Dusseau, A. C., 2018. Operating Systems: Three Easy Pieces. 1st ed. Madison, WI: Arpaci-Dusseau Books.
5. Denning, P. J., 1970. 'Virtual Memory', ACM Computing Surveys, 2(3), pp. 153–189.





Unit – 11

Memory Management: Paging

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Concept of Paging	-	-	7 - 17
2.1	Virtual Memory and Paging Overview	1	-	
2.2	Paging Mechanism	2	-	
2.3	Page Table Structures (Single-Level, Multi-Level)	3, 4	-	
2.4	Inverted Page Table	5	-	
2.5	Illustrative Examples	-	1	
3	Hardware Aspects of Paging	-	-	18 - 24
3.1	Memory Management Unit (MMU)	-	-	
3.2	Translation Lookaside Buffer (TLB)	-	-	
3.3	Hardware Support for Multi-Level Paging	-	-	
3.4	Address Translation Flow	6	-	
3.5	Performance Impact of Hardware Features	-	2	
4	Software Aspects of Paging	-	-	25 - 31
4.1	Operating System Page-Table Management	-	-	
4.2	Page Replacement Policies	8	-	
4.3	Shared Pages and Copy-on-Write	9	-	
4.4	Page-Fault Handling	-	-	
4.5	Software Optimisations	-	3	
5	Implementation and Performance Considerations	-	-	32 - 37
5.1	Overhead Analysis	-	-	
5.2	TLB Miss Penalties	-	-	
5.3	Memory Fragmentation Issues	10	-	
5.4	Scalability with Large Address Spaces	-	4	
6	Summary	-	-	38

7	Glossary	-	-	39
8	Terminal Questions	-	-	40
9	Answers	-	-	41 - 43
10	References	-	-	44



1. INTRODUCTION

In the previous unit, we examined the fundamental aspects of memory management, focusing on the distinction between logical and physical address spaces, the mechanism of swapping, and the strategy of contiguous memory allocation. Logical address space was understood as the address generated by the CPU, while the physical address space referred to the actual location in main memory. The concept of swapping involved transferring entire processes between main memory and disk storage, thus allowing the system to handle more processes than the physical memory could accommodate at any given time. Contiguous allocation, though simple to implement, introduced limitations such as external fragmentation and difficulties in dynamic memory expansion.

This unit introduces a more refined and efficient memory management scheme known as paging. Paging eliminates the problems associated with contiguous allocation by allowing non-contiguous allocation of physical memory. It divides both the logical and physical address spaces into fixed-size blocks called pages and frames respectively. This mapping facilitates better utilisation of memory and supports the execution of larger programs without requiring them to occupy continuous memory blocks. The core idea is to break memory into manageable units, thereby simplifying allocation, improving protection, and supporting process isolation.

The unit is organised into three primary sections: the concept of paging, hardware aspects of paging, and software aspects of paging. The concept section explains the structure and working of the paging system, including page tables, virtual memory integration, and illustrative models. The hardware section delves into essential components like the Memory Management Unit (MMU), Translation Lookaside Buffer (TLB), and hardware-level address translation. On the software side, the focus shifts to operating system responsibilities, including page-table management, page replacement algorithms, handling of shared pages, page faults, and related software-level optimisations. These elements work together to deliver a practical, robust memory management approach that is used in most modern operating systems.

To study this unit effectively, it is advised to start by understanding the basic idea of dividing memory into pages and how logical addresses are translated to physical ones. Diagrams should be used wherever possible to reinforce the understanding of address translation and the paging mechanism. Following this, one should move on to the hardware features that support paging, paying close attention to the role of the TLB and MMU in improving performance. Finally, the software strategies used by operating systems to manage paging efficiently, especially in the context of page replacement

and fault handling, should be explored in detail. Each section builds on the preceding one, so a sequential and concept-focused approach will help grasp the practical implications and theoretical principles behind paging.

1.1. Objectives

After studying this unit, you should be able to:

- Explain how paging resolves memory fragmentation issues.
- Compare the roles of MMU and TLB in address translation.
- Apply page replacement strategies to optimise memory usage.
- Analyse the impact of TLB misses on system performance.
- Discuss a logical-to-physical address mapping using multi-level page tables.



2. CONCEPT OF PAGING

2.1. Virtual Memory and Paging Overview

Modern computer systems face the constant challenge of managing memory resources efficiently. Processes often require more memory than what is physically available, leading to the necessity of a technique that can abstract this limitation. Virtual memory offers a solution by providing the illusion of a large, continuous address space to every process, irrespective of the actual physical memory size.

Virtual memory separates user logical memory from physical memory, enabling a process to be executed even if not completely loaded in the main memory. This abstraction allows for efficient multitasking, safe memory allocation, and simplified program execution. It also ensures that the memory allocated to one process is isolated from another, enhancing both protection and stability.

To implement virtual memory effectively, paging is used as the core mechanism. Paging divides both the physical and logical memory into fixed-sized blocks, simplifying memory allocation and avoiding issues such as external fragmentation.

Key characteristics of virtual memory:

- Expands usable memory beyond physical constraints
- Allows non-contiguous memory allocation
- Facilitates process isolation
- Improves memory utilisation through sharing and swapping

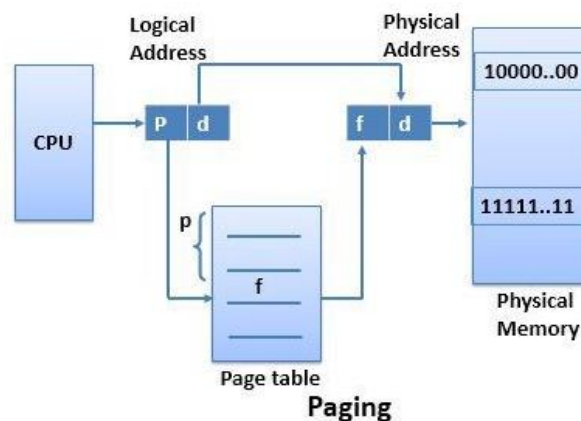


Figure 1: Paging in Memory Management

Source: https://miro.medium.com/v2/resize:fit:720/format:webp/1*TmfipZWCKQr2XcVGh6tSkA.png

png

The figure illustrates the concept of paging in memory management, demonstrating how a Central Processing Unit (CPU) accesses physical memory through a two-step address translation process. A logical address generated by the CPU is divided into a page number 'p' and an offset 'd'. The page number 'p' is then used to index a page table, which contains frame numbers 'f'. This frame number 'f', combined with the original offset 'd', forms the physical address, allowing the CPU to locate data within the physical memory. This mechanism efficiently maps a larger logical address space to a potentially smaller physical address space.

2.2. Paging Mechanism

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. In this mechanism, both logical memory (also known as virtual memory) and physical memory are divided into blocks of the same size. These blocks are known as pages in the logical memory and frames in the physical memory.

Each page is mapped to a frame through a page table, which maintains the frame number for every page number. When a process accesses a memory address, the CPU generates a logical address that is translated into a physical address using the page table.

Logical Address Structure: - A logical address generated by the CPU consists of two parts:

- Page number (p) – used to index the page table
- Page offset (d) – represents the specific location within the page

Translation Process:

1. The CPU generates a logical address.
2. The page number is used to look up the frame number in the page table.
3. The frame number is combined with the offset to form the physical address.
4. The physical address is then used to access the required location in the main memory.

Paging offers several key advantages in memory management. It completely eliminates external fragmentation, as memory is divided into fixed-size pages and frames, ensuring efficient utilisation of available space. It simplifies allocation by removing the need to find contiguous memory blocks, making it easier for the operating system to manage memory. Furthermore, paging supports virtual memory, allowing processes to exceed physical memory limits by storing parts of them on disk and loading them as needed. This enables better multitasking and process isolation.

However, paging also introduces some drawbacks. One major issue is internal fragmentation, where the last page of a process may not fully use the allocated frame, leading to wasted space. Additionally, each process requires a page table to track the mapping between virtual and physical pages, which consumes extra memory. The necessity of referencing the page table during address translation also adds overhead, potentially reducing performance compared to systems without paging.

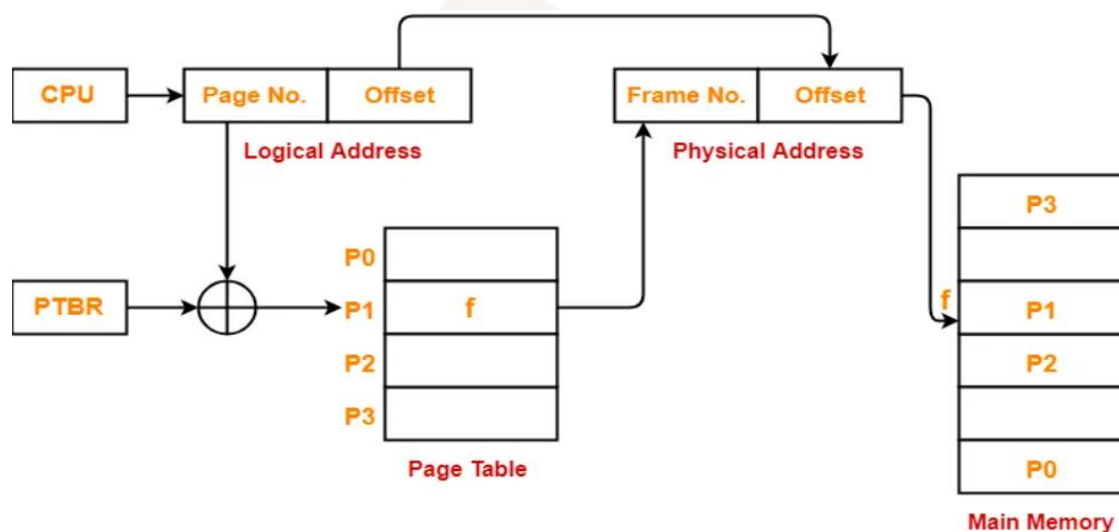


Figure 2: Translating Logical Address into Physical Address

Source: <https://www.gatevidyalay.com/wp-content/uploads/2018/11/Translating-Logical-Address-into-Physical-Address-Diagram.png>

The figure presents a flowchart detailing the conversion of a logical address into a physical address using a page table within a computer's memory management unit. When the CPU generates a logical address, it is split into a page number and an offset. The Page Table Base Register (PTBR) holds the base address of the page table. The page number is added to the PTBR's content to find the entry in the page table that contains the corresponding frame number 'f'. This frame number is then combined with the original offset to form the physical address, which points to the exact location in the main memory where the desired data resides. This process ensures efficient and organised memory access.

2.3. Page Table Structures (Single-Level, Multi-Level)

The page table is a crucial data structure in paging, storing the mapping between page numbers and frame numbers. There are several types of page table organisations, each with its own set of advantages and limitations.

Single-Level Page Table: The simplest form of a page table is the single-level table, which consists of a linear array where each index corresponds to a page number and each entry stores the frame number.

Structure:

- Page number → Index into page table
- Entry at index → Corresponding frame number

Single-level paging, while straightforward in design, faces significant limitations as system demands grow. When the logical address space is large, the page table itself becomes correspondingly large, potentially exceeding the available memory. This results in increased memory consumption solely for maintaining the mapping, leaving less space for actual process data.

Moreover, single-level paging scales poorly in environments with many processes. Each process requires its own page table, and maintaining a large number of these tables consumes considerable memory and management resources. This makes the approach less practical for modern systems with high process concurrency and extensive address space requirements.

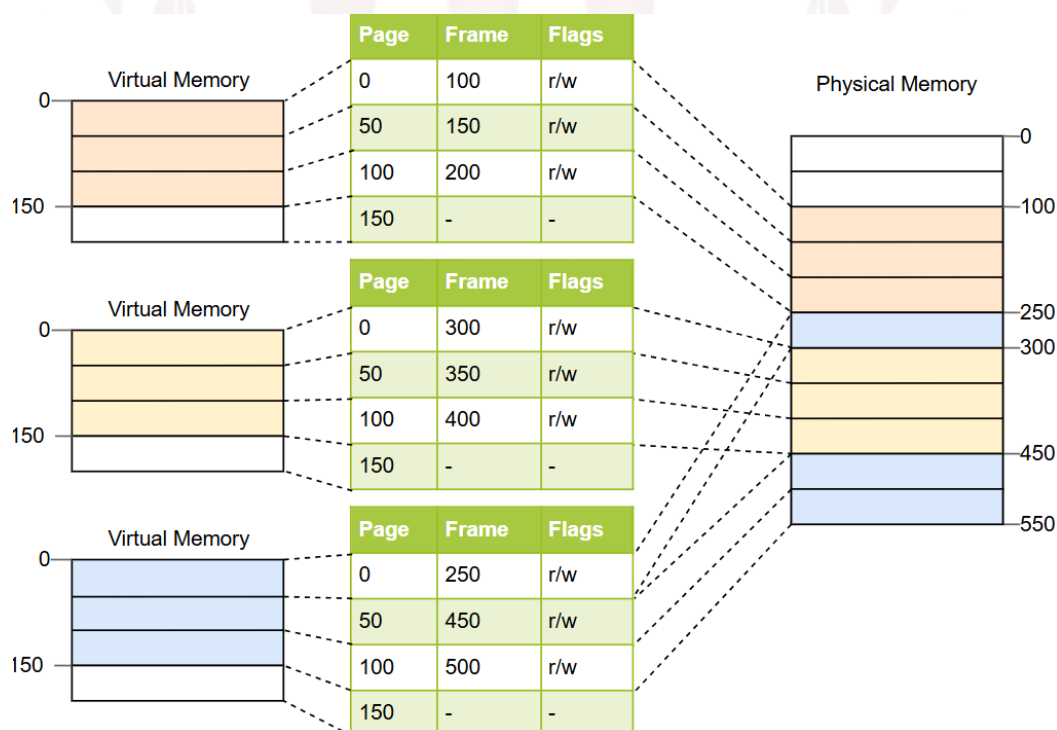


Figure 3: Single-Level Paging Table

Source: <https://os.phil-opp.com/paging-introduction/paging-page-tables.svg>

The figure illustrates how a single-level paging system maps virtual memory addresses to physical memory addresses for multiple processes. Each virtual memory block, representing a process's view

of memory, has a corresponding page table. This page table contains entries that link virtual pages (identified by 'Page' numbers) to specific 'Frame' numbers in physical memory. The 'Flags' column typically indicates permissions, such as read/write access. This setup allows different virtual pages from various processes to be scattered across non-contiguous frames in physical memory, providing efficient memory utilisation and isolation between processes.

Multi-Level Page Table

To overcome the scalability issues of single-level tables, multi-level page tables are employed. These break the page table into smaller segments, each of which can be paged again.

Two-Level Page Table:

- The logical address is divided into three parts:
 - Outer page number
 - Inner page number
 - Offset
- The outer page number indexes the outer page table.
- The inner page table pointed to by the outer table contains the frame numbers.

Multilevel paging is particularly beneficial for systems with large virtual address spaces. It reduces memory overhead by avoiding the need to allocate a single, large contiguous page table; instead, only the necessary portions of the table are created as required. This makes the approach more efficient and scalable, especially for sparse address spaces where not all entries are used.

However, multilevel paging introduces additional complexity in address translation. Each memory access requires multiple lookups—one for each level of the page table—which increases the time taken to resolve virtual addresses to physical addresses. This added latency can affect overall system performance, particularly if translation lookups are not well optimised or cached.

General Concept: An n-level page table results in n memory accesses to resolve a single virtual address, potentially increasing access time.

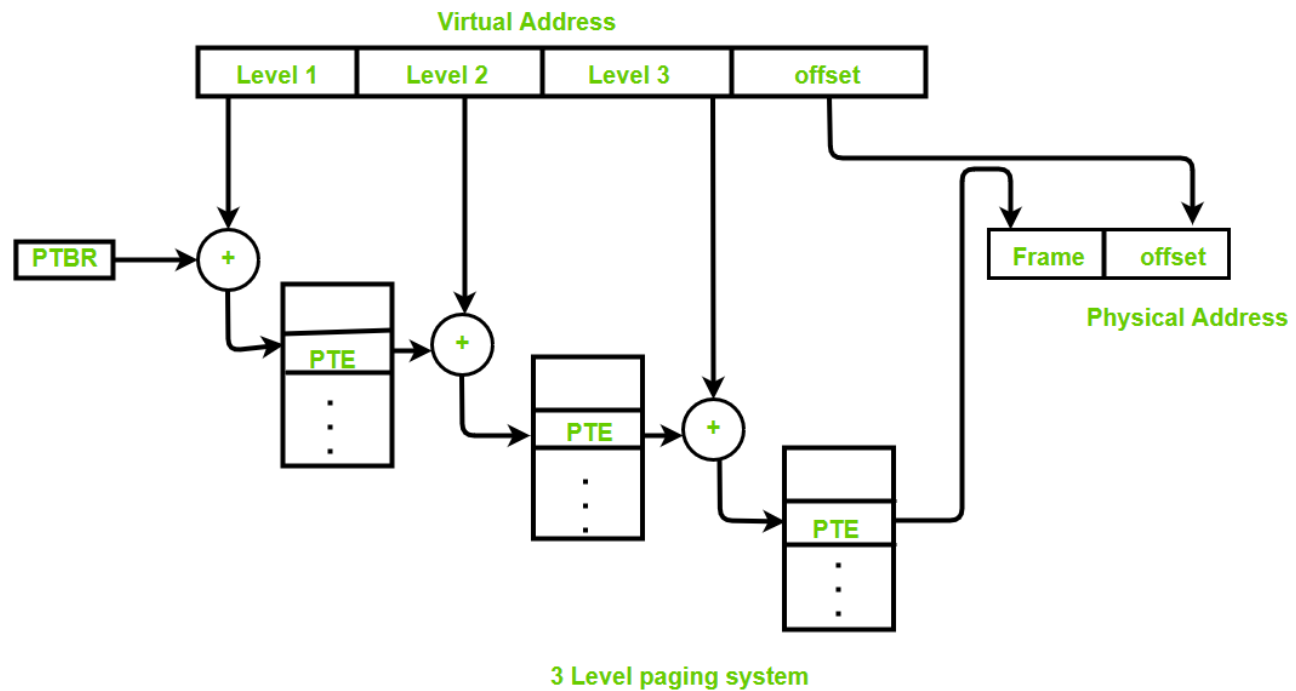


Figure 4: Multi-Level Paging Table

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20190608174704/multilevel.png>

The figure illustrates the process of address translation in a multilevel paging system, which is employed to manage large virtual address spaces more effectively. A virtual address is segmented into multiple levels – in this instance, Level 1, Level 2, Level 3, and an offset. The Page Table Base Register (PTBR) provides the starting point for the first level page table. Each level's page table entry (PTE) points to the next level's page table, until the final level's PTE yields the frame number. This frame number is then combined with the original offset to construct the physical address, allowing the system to locate data in physical memory while reducing the memory footprint of the page tables themselves.

2.4. Inverted Page Table

An inverted page table provides an alternative approach to managing memory by reversing the traditional page-to-frame mapping. Instead of maintaining a separate page table for each process, the system maintains a single global page table indexed by frame number.

Structure:

- One entry per physical frame
- Each entry contains:
 - The virtual page number stored in that frame

- The process identifier (PID)
- Control bits such as valid, dirty, or access

Address Translation Process:

1. CPU generates a logical address.
2. The system searches the inverted page table to find the matching virtual page number and PID.
3. If found, the corresponding frame is identified.
4. If not found, a page fault occurs.

The inverted page table approach offers a compact and memory-efficient solution for systems managing large virtual address spaces. Unlike traditional methods that maintain separate page tables for each process, it uses a single global table, significantly reducing memory usage. This design is well-suited for environments with many processes or extensive virtual memory demands, as it avoids the duplication of page tables.

However, this efficiency comes with trade-offs. Address translation can be slower because the system must search through the table to find the corresponding entry for a given virtual address. To overcome this delay, modern systems often employ associative memory (such as TLBs) or hashing mechanisms to speed up the lookup process. Even so, the additional complexity in managing and searching the table can introduce performance overheads.

Use of Hashing:

- Many systems employ a hash function to index into a hash table that points to the inverted page table.
- This reduces search time and improves efficiency.

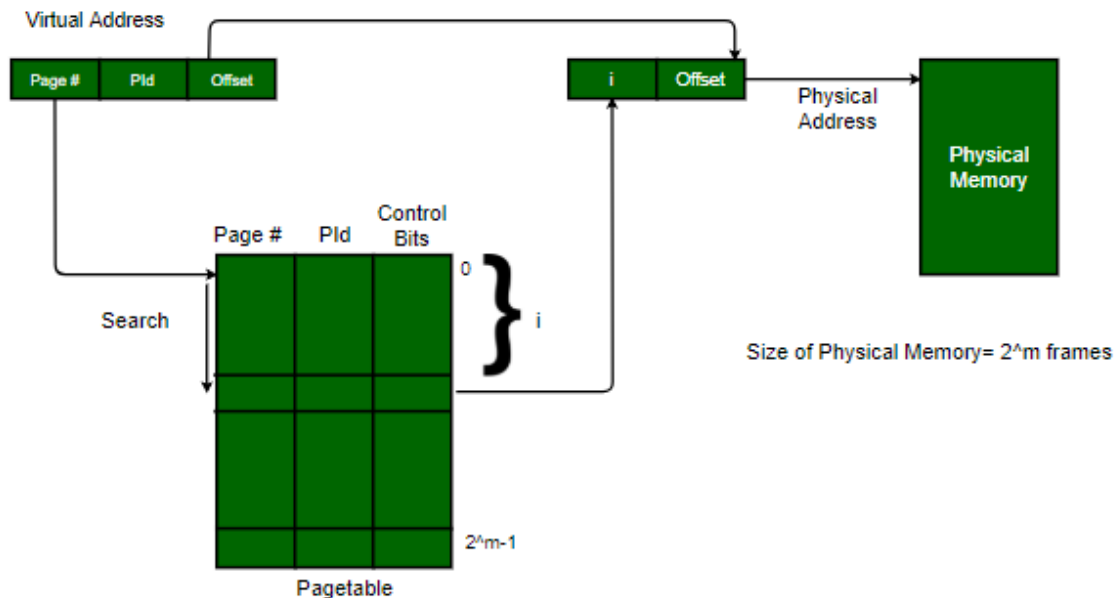


Figure 5: Inverted Page Table

Source: <https://media.geeksforgeeks.org/wp-content/uploads/33-6.png>

The figure displays the organisation of an inverted page table, which offers an alternative approach to memory management compared to traditional page tables. Unlike conventional page tables that have one entry per virtual page, an inverted page table has one entry for each physical frame. When a virtual address is generated, it is broken down into a page number, a process ID (PID), and an offset. The system then searches the inverted page table for an entry that matches both the page number and the process ID, potentially using the 'i' index to pinpoint the correct entry. Once found, this entry provides the 'i' value, which corresponds to the physical frame. This 'i' value is combined with the original offset to form the physical address, directing the system to the correct location in physical memory.

2.5. Illustrative Examples

Understanding paging mechanisms and page table structures becomes easier when supported with examples. Below are simplified scenarios to reinforce key concepts.

Example 1: Single-Level Paging

Given:

- Logical address space = 16 pages
- Page size = 1 KB

- Physical memory = 32 frames

Suppose a process requests access to logical address 8195.

Step-by-step:

1. Page size = 1024 bytes $\rightarrow$ page number = $8195 \div 1024 = 8$, offset = $8195 \bmod 1024 = 3$
2. Page table entry for page 8 = frame 12
3. Physical address = $(12 \times 1024) + 3 = 12291$

Conclusion: Logical address 8195 maps to physical address 12291.

Example 2: Two-Level Paging

Given:

- Logical address: 32-bit
- Page size: 4 KB
- Outer table: 10 bits, Inner table: 10 bits, Offset: 12 bits

Steps:

1. Outer page number = first 10 bits
2. Inner page number = next 10 bits
3. Offset = last 12 bits
4. Use outer page number to index outer table
5. Outer table points to inner page table
6. Inner table provides frame number
7. Final physical address = frame base + offset

Conclusion: Two-level paging breaks the translation into manageable steps, reducing memory waste.

Example 3: Inverted Page Table Lookup

Scenario:

- PID: 1001, Page number: 5
- System uses hash-based inverted page table

Steps:

1. Hash function processes PID and page number
2. Index into hash table

3. Match entry for PID = 1001 and page = 5
4. Retrieve physical frame

Conclusion: Inverted tables focus on physical memory frames, requiring hash-based or associative search methods.

Paging is foundational to memory management in operating systems, enabling efficient use of resources and supporting complex multitasking. While single-level paging is suitable for smaller systems, multi-level and inverted page tables offer scalability and efficiency for larger, modern systems.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the main advantage of virtual memory in modern computer systems? a) It increases the physical size of RAM b) It provides the illusion of a large, continuous address space to each process c) It ensures all processes remain entirely in main memory d) It reduces the need for a CPU cache
2	Which mechanism divides both logical and physical memory into fixed-sized blocks? a) Segmentation b) Paging c) Swapping d) Partitioning
3	Paging eliminates which type of fragmentation? a) Internal fragmentation b) External fragmentation c) Temporal fragmentation d) Spatial fragmentation
4	In the logical address structure used by paging, what does the offset specify? a) The index into the page table b) The memory protection level c) The exact location within a page d) The physical frame number
5	What does a single-level page table entry directly store?

- | | |
|--|--|
| | <ul style="list-style-type: none">a) The process identifier and offsetb) The frame number corresponding to a page numberc) The hash of the virtual addressd) The base address of the next-level table |
|--|--|



3. HARDWARE ASPECTS OF PAGING

3.1. Memory Management Unit (MMU)

The Memory Management Unit (MMU) is a hardware component responsible for handling the translation of logical addresses (generated by the CPU) into physical addresses (used to access main memory). It plays a pivotal role in the implementation of paging by managing address mapping efficiently at runtime without software intervention.

Key Functions of the MMU:

- Translates logical addresses into physical addresses using page tables.
- Provides protection by ensuring that processes do not access memory outside their assigned region.
- Facilitates support for virtual memory systems.
- Implements mechanisms for page fault detection and exception handling.

The MMU works seamlessly with the paging mechanism. When a program issues an instruction involving memory access, the CPU sends a logical address to the MMU. The MMU then uses the page number and offset to locate the corresponding physical memory location.

MMU Registers Typically Include:

- **Base register:** Points to the beginning of the page table for a process.
- **Limit register:** Indicates the size or number of entries in the page table.
- **Control bits:** Flags for valid, dirty, or accessed pages.

3.2. Translation Lookaside Buffer (TLB)

To speed up address translation, modern systems use a special cache called the Translation Lookaside Buffer (TLB). This is a small, fast memory inside or very close to the MMU that stores recent translations of logical to physical addresses. Accessing the page table from main memory for every memory reference is inefficient and introduces significant delay. The TLB reduces this delay by caching recent translations.

Structure and Operation:

- Each TLB entry holds a mapping from a virtual page number to a physical frame number.
- When a memory reference is issued:

- The page number is checked against entries in the TLB.
- If found (TLB hit), the frame number is retrieved directly.
- If not found (TLB miss), the page table is accessed from memory and the result is stored in the TLB for future use.

Types of TLBs:

- **Fully associative:** Any entry can be placed in any location. Most flexible but expensive.
- **Set associative:** A compromise, where entries are grouped into sets.
- **Direct mapped:** Each virtual page maps to one fixed TLB location. Fast but prone to collisions.

TLB Entries Include: Virtual page number, Frame number, Access control bits (read/write/execute permissions) and Validity bits.

The TLB offers several advantages to the system. It reduces the average memory access time by avoiding repeated page table lookups for frequently accessed pages. This leads to improved system performance and better responsiveness, especially during routine memory operations. It also supports faster context switching when process-specific entries are handled efficiently.

Despite its benefits, the TLB has limitations. Its size is restricted, meaning only a small number of page entries can be stored at any time. This makes it necessary to carefully manage entries, particularly during context switches. In such cases, the TLB may need to be flushed to remove outdated mappings, which can momentarily affect performance.

3.3. Hardware Support for Multi-Level Paging

Modern operating systems, especially those supporting large address spaces, implement multi-level paging to reduce memory overhead caused by large single-level page tables. Hardware must be capable of supporting this structure efficiently.

Concept:

- Instead of one flat page table, the address translation involves multiple levels of page tables.
- For example, in a two-level scheme, the logical address is divided into three parts:
 - Outer page number
 - Inner page number
 - Offset

To support this mechanism, hardware components such as the Memory Management Unit (MMU) must perform multiple memory accesses during each address translation. Special registers hold the base address of the top-level table, while translation lookaside buffers (TLBs) are extended to cache entries from various levels. Some advanced systems also employ a page-walk cache to temporarily store intermediate results from the page table traversal, improving performance and reducing latency.

Illustration:

In a 32-bit system using 4 KB pages:

- 10 bits for outer page table (1024 entries)
- 10 bits for inner page table (1024 entries)
- 12 bits for offset (4 KB page size)

Steps for Hardware Translation:

1. Extract outer page index from the logical address.
2. Access outer page table to find inner table base.
3. Extract inner index and access the inner page table.
4. Retrieve frame number and add offset to form physical address.

Hardware support for multilevel paging helps reduce the memory needed for storing page tables and allows only required parts to be loaded dynamically, making it well suited for sparse memory use. Despite these benefits, it introduces higher latency due to several memory accesses during translation and demands a more complex TLB design to manage multi-level mappings efficiently.

3.4. Address Translation Flow

Address translation is the core function of paging, and the hardware must execute this process rapidly and accurately to maintain system performance.

Translation Flow with TLB:

1. Logical address issued by CPU.
2. MMU divides the address into page number and offset.
3. Page number checked in the TLB.
 - If found, physical frame number retrieved (TLB hit).
 - If not, page table accessed (TLB miss).
4. Frame number combined with offset to form physical address.

5. Memory accessed using the physical address.

Translation Flow in Multi-Level Paging:

1. Logical address divided into multiple page indexes and offset.
2. Hardware walks through outer and inner page tables.
3. Final frame number obtained.
4. Offset applied to get physical address.

When a requested page is not available in physical memory, a page fault is triggered. The operating system responds by fetching the missing page from the disk and loading it into memory. Once the page is loaded, the page table is updated to reflect the new mapping, and the TLB may also be refreshed to include the recent entry for faster future access.

Special Hardware Instructions May Include:

- Page walk logic for table traversals
- TLB management instructions (invalidate, flush)
- Exception handling support for page faults

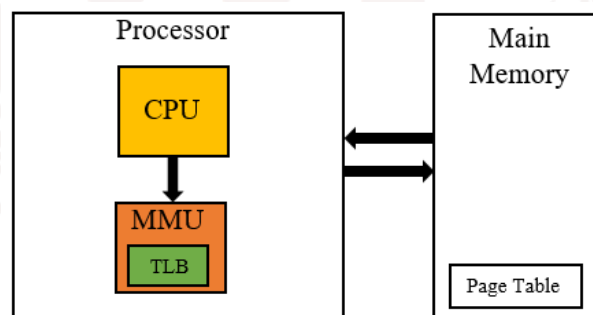


Figure 6: Address Translation Process

Source: <https://www.researchgate.net/publication/385756135/figure/fig1/AS:11431281290079200@1731497356902/Main-modules-in-the-address-translation-process.ppm>

The figure illustrates the key components involved in translating virtual addresses generated by the CPU into physical addresses for accessing main memory. Within the processor, the Central Processing Unit (CPU) issues memory requests, which are then handled by the Memory Management Unit (MMU). Crucially, the MMU incorporates a Translation Lookaside Buffer (TLB), a high-speed cache that stores recent address translations to expedite the process. If the translation is not found in the TLB, the MMU accesses the page table residing in main memory to complete the translation, before the data can be retrieved from or stored in the main memory.

3.5. Performance Impact of Hardware Features

The design and efficiency of hardware components directly influence the performance of memory access in paging systems. The speed of address translation, the frequency of TLB hits and misses, and the cost of handling page faults all contribute to system behaviour.

Key Metrics to Consider:

- **Memory access time:** The average time taken to access memory, factoring in translation overhead.
- **TLB hit ratio:** Percentage of memory accesses served directly by the TLB.
- **Page fault rate:** Number of times the page required is not present in physical memory.

Example Analysis:

- TLB access time: 10 ns
- Memory access time: 100 ns
- TLB hit ratio: 90%

Impact of Hardware Support:

- **With TLB:** Reduces the need for frequent page table lookups.
- **With multi-level paging:** Reduces size of page tables but increases lookup depth.
- **With page walk cache:** Speeds up multi-level page access.
- **With larger page sizes:** Reduces table entries, fewer TLB misses.

The performance of systems using multilevel paging is shaped by several hardware trade-offs. Increasing the number of paging levels reduces memory use for page tables but leads to longer address translation times. Larger TLBs improve hit rates and reduce lookup delays but require more expensive hardware. Associative TLBs can speed up address matching, though they introduce added circuit complexity.

Designing efficient systems requires careful balance. The size and associativity of the TLB must be optimised to deliver speed without excessive cost. Features like address space identifiers help manage quick context switches by avoiding unnecessary TLB flushing. Some architectures also employ hardware prefetching during page walks to reduce delays caused by multiple memory accesses.

The hardware aspects of paging form the backbone of efficient virtual memory systems. The Memory Management Unit orchestrates the address translation process, while the Translation Lookaside Buffer ensures that recent translations are reused rapidly. Multi-level paging introduces scalability and is supported by hardware logic to handle layered address translation.

Understanding the entire address translation flow helps in appreciating how modern operating systems achieve memory protection, isolation, and performance. Each component, from TLBs to multi-level tables, plays a role in striking a balance between speed, resource usage, and complexity.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which hardware component is chiefly responsible for translating logical addresses into physical addresses at runtime? a) Arithmetic Logic Unit b) Memory Management Unit c) Control Unit d) Cache Controller
7	In the context of the MMU, which register holds the starting address of the current process's page table? a) Limit register b) Base register c) Frame register d) Page fault register
8	What is stored in each entry of a Translation Lookaside Buffer (TLB)? a) Virtual page number and physical frame number b) Complete page table for a process c) Swap space location for a page d) Process identifier and page size
9	Which TLB organisation permits any entry to reside in any location within the buffer? a) Direct mapped b) Set associative c) Fully associative d) Hash mapped

- | | |
|----|--|
| 10 | When using a two-level paging scheme in hardware, how many memory accesses are typically required to translate a single logical address (ignoring TLB)?
a) One
b) Two
c) Three
d) Four |
|----|--|



4. SOFTWARE ASPECTS OF PAGING

4.1. Operating System Page-Table Management

The operating system is central to the management of page tables in a virtual memory environment. Each process maintains its own page table, mapping virtual pages to physical frames. While hardware (such as the MMU) performs translation at runtime, it is the operating system that creates, modifies, and maintains the page tables throughout a process's lifecycle.

Responsibilities of the Operating System:

- Allocate and initialise page tables for each process.
- Maintain metadata such as valid bits, dirty bits, protection bits, and reference counters.
- Ensure isolation between processes by preventing access to unauthorised frames.
- Manage physical frame allocation from a global free frame list or frame pool.
- Update entries upon context switches or when processes grow/shrink in memory usage.

Multi-Process Management:

- Every process has a separate page table to ensure independent address spaces.
- On a context switch, the OS updates hardware registers (e.g., page-table base register) to reflect the page table of the new process.

Page Table Entry Fields Often Include: Frame number, Valid/invalid bit, Read/write permission, Dirty bit (modified) and Referenced bit (for replacement algorithms).

4.2. Page Replacement Policies

When physical memory is full and a new page needs to be loaded, the operating system must decide which existing page to remove. This decision is governed by the page replacement policy. Efficient policies reduce page faults and improve system performance.

General Approach:

- A. Detect that a page fault has occurred.
- B. Identify that no free frame is available.
- C. Apply the page replacement algorithm to choose a victim page.
- D. If the victim page is dirty, write it to disk.
- E. Load the required page and update the page table.

Common Page Replacement Algorithms:

a) First-In, First-Out (FIFO)

- Removes the oldest page in memory.
- Simple but may lead to poor performance (e.g., Belady's anomaly).

b) Optimal Page Replacement

- Replaces the page that will not be used for the longest time in future.
- Theoretically best but impractical, as future references are unknown.

c) Least Recently Used (LRU)

- Replaces the page that has not been used for the longest time.
- Better performance, but requires tracking access time or ordering.

d) Clock (Second-Chance) Algorithm

- Circular buffer-like structure with a reference bit.
- Gives a second chance to pages that have been recently used.

e) Least Frequently Used (LFU)

- Tracks how often pages are accessed.
- Favourable for processes with recurring memory patterns.

Table 1: Comparison between Page Replacement Algorithms

Algorithm	Replacement Approach	Tracking Required	Efficiency and Notes
First-In, First-Out (FIFO)	Removes the oldest page in memory	Queue to maintain order	Simple but can suffer from Belady's anomaly
Optimal	Replaces page not used for longest time in future	Needs knowledge of future accesses	Theoretical best; not practical for real systems
Least Recently Used (LRU)	Removes the least recently accessed page	Time stamp or stack-based tracking	Performs well but requires precise tracking
Clock (Second-Chance)	Gives a second chance based on a reference bit	Reference bits and circular buffer	Approximates LRU with reduced overhead
Least Frequently Used (LFU)	Removes page with lowest access frequency	Counter for each page	Good for patterns with repeated use; may not adapt quickly to changes

4.3. Shared Pages and Copy-on-Write

Paging facilitates memory sharing between processes in an efficient and controlled manner. Shared pages reduce memory usage and support inter-process communication.

Shared Pages: Shared pages allow multiple processes to use the same physical memory for identical code, such as standard libraries, without duplication. The operating system maps the same physical frame into each process's page table with read-only access, preserving data integrity. This approach is efficient, as it reduces memory usage while maintaining separate data segments for each process.

Copy-on-Write (COW): Copy-on-Write is a memory management strategy that delays the duplication of pages until a write operation occurs. When a process is forked, both the parent and child initially share the same memory pages, which are marked as read-only. If either process attempts to modify a shared page, a separate copy is created, ensuring changes remain isolated. This approach is particularly effective during process creation, as it avoids unnecessary duplication of memory, reduces overall memory usage, and improves performance, especially when the shared pages remain unmodified.

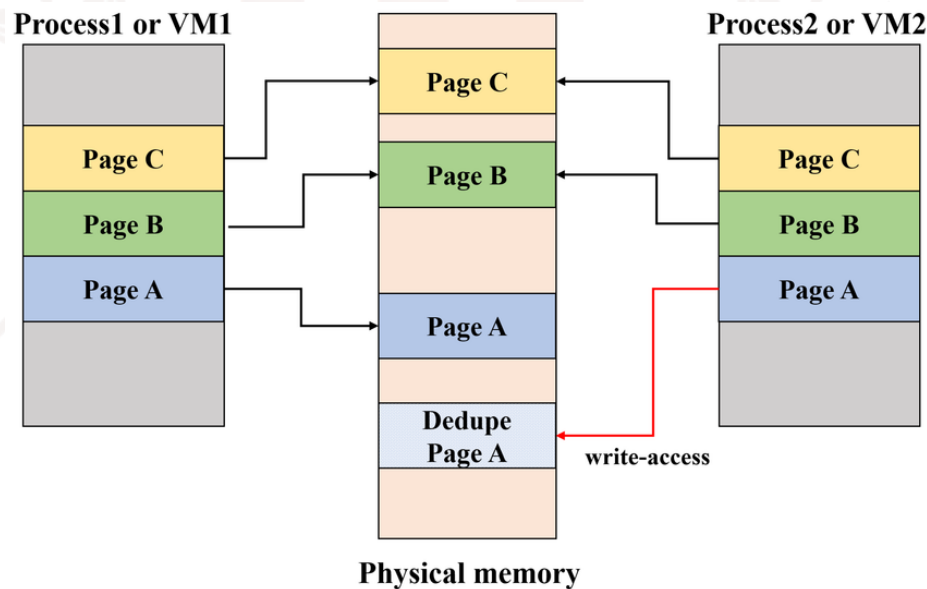


Figure 7: Copy-on-Write

Source:

<https://www.researchgate.net/publication/354394371/figure/fig2/AS:11431281415851849@1746045300539/Copy-on-Write-CoW-mechanism.tif>

The figure illustrates a memory management technique where two processes, Process1 and Process2, or two virtual machines (VM1 and VM2), initially share common pages in physical memory to

conserve space. Both processes are shown accessing 'Page A', 'Page B', and 'Page C', which all map to single copies in physical memory. When one of the processes, for example Process2, attempts to write to a shared page like 'Page A', a "Copy-on-Write" (COW) event is triggered. This event results in a duplicate of 'Page A' being created in physical memory, labelled 'Dedupe Page A', and Process2's memory mapping is updated to point to this new, independent copy, thus ensuring data integrity for both processes.

4.4. Page-Fault Handling

A page fault occurs when a process accesses a page that is not currently in main memory. Handling such faults is the responsibility of the operating system.

Types of Page Faults:

- **Minor page fault:** Page is in memory but not mapped in the current page table (e.g., due to COW).
- **Major page fault:** Page is not present in memory and must be loaded from secondary storage.

Steps in Page-Fault Handling:

1. Hardware detects the page fault and triggers a trap to the OS.
2. OS identifies the missing page and checks if the access is valid.
3. If invalid, the OS sends a segmentation fault to the process.
4. If valid, the OS selects a victim frame using a replacement policy.
5. If the victim frame is dirty, write it back to disk.
6. Load the required page from disk into the frame.
7. Update the page table and TLB.
8. Resume process execution from the faulting instruction.

Handling a page fault involves fetching the required page from disk, which is far slower than accessing memory and can impact system performance. To reduce the frequency of such faults, techniques like prefetching or clustering are used to load multiple relevant pages together. Maintaining performance also depends on choosing effective replacement algorithms that minimise delays and keep the system responsive.

Optimising the page-fault rate involves adapting to the working set of each process to ensure frequently used pages remain in memory. Efficient use of TLBs and intelligent preloading can help

reduce the number of faults. It is also important to avoid needless invalidation of pages during context switches to maintain continuity and minimise delays.

4.5. Software Optimisations

To make paging more efficient, various software-level optimisations are employed by operating systems. These are designed to reduce overhead, improve fault handling, and better utilise system memory.

Common Optimisation Techniques for Reducing Page Faults

Working Set Model - This technique monitors the set of pages a process actively uses over a recent period. By keeping only these pages in memory, the system aims to maintain efficiency and reduce unnecessary page faults.

Page Buffering - In this approach, a pool of recently evicted pages is maintained. If a page is needed again shortly after being removed, it can be quickly restored without accessing the disk, thus saving time and improving performance.

Clustering - Clustering involves grouping related pages together and loading them at once during fault handling. This reduces the number of I/O operations needed and is particularly helpful when consecutive pages are likely to be used.

Pre-Paging - Pre-paging predictively loads pages into memory before they are explicitly requested by the process. It relies on patterns of previous access and aims to lower the rate of page faults during execution.

Memory-Mapped Files - This method allows files to be accessed directly in memory rather than through standard file I/O. It reduces overhead by enabling direct manipulation of file contents via pointers and minimises the need for data copying.

TLB-aware Allocation - By arranging memory so that frequently accessed pages are more likely to remain in the TLB, this technique improves the likelihood of a TLB hit. This results in quicker address translation and better overall performance.

Use of Huge Pages - For processes that require large amounts of memory, using larger page sizes helps reduce the number of TLB entries and page faults. This is especially beneficial in environments such as databases and virtual machines.

Trade-offs and Considerations:

- Aggressive prefetching may waste bandwidth.
- Large pages improve performance but lead to internal fragmentation.
- Monitoring working sets adds computational cost.

Case Study Note:

- Linux uses the concept of “swappiness” to balance paging with swapping to disk.

Windows employs a working set trimming mechanism based on process activity.

The software aspects of paging encompass all the mechanisms and strategies employed by the operating system to manage memory effectively under the paging scheme. From page table maintenance to intelligent fault handling, the OS ensures that applications get the required memory access while maintaining performance and security.

Shared memory and techniques like copy-on-write reduce duplication. Effective page replacement algorithms, supported by proper handling of page faults and software-level optimisations, ensure that limited physical memory is used to its fullest potential. The cooperation between hardware and software in paging allows for the seamless illusion of large, private address spaces for every process. It is this software layer that enables virtual memory to be practical, performant, and scalable.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Which field in a page table entry indicates whether a page has been modified since it was loaded into memory? a) Valid bit b) Dirty bit c) Reference bit d) Protection bit
12	Which page replacement algorithm may suffer from Belady's anomaly? a) Optimal b) Least Recently Used (LRU) c) First-In, First-Out (FIFO) d) Clock

13	In a copy-on-write scheme, when is a duplicate of a shared page created? a) When the page is first loaded into memory b) When any process reads from the page c) When a process attempts to write to the page d) When the page is swapped out to disk
14	What is the first action taken by the operating system upon detecting a major page fault? a) Write the victim page to disk b) Select a victim frame using a replacement policy c) Trigger a segmentation fault d) Load the missing page from secondary storage
15	Which software optimisation predicts and loads pages into memory before they are explicitly requested? a) Page buffering b) Working set model c) Clustering d) Pre-paging

5. IMPLEMENTATION AND PERFORMANCE CONSIDERATIONS

5.1. Overhead Analysis

Paging provides flexibility and protection in memory management, yet it also introduces certain overheads. The operating system and hardware must coordinate to maintain page tables, manage memory references, and handle faults, all of which consume system resources.

Types of Overhead:

a) Storage Overhead:

- Page tables occupy significant memory, especially in systems with large virtual address spaces.
- A 32-bit address space with 4KB page size requires 1 million entries per process.
- Multi-level or inverted page tables mitigate this but introduce lookup complexity.

b) Computational Overhead:

- Every memory access requires address translation, involving page table lookup and, optionally, TLB checks.
- Context switching adds to overhead, as the page-table pointer and TLB must be updated.

c) Page Fault Overhead:

- Handling a page fault involves switching to kernel mode, checking access rights, and possibly disk I/O.
- The time consumed in resolving a fault is several orders of magnitude higher than a regular memory access.

d) Copying Overhead:

- In copy-on-write scenarios, a page must be duplicated on write access, temporarily increasing load.
- Memory management operations such as zeroing out a page before allocation also contribute to overhead.

Overhead in paging systems often arises from less visible costs such as the need to maintain metadata, including dirty and reference bits, which require constant tracking. Internal fragmentation is another issue, where unused portions of fixed-size pages lead to wasted memory. There are performance

trade-offs in choosing page sizes: larger pages help reduce the size of page tables but increase the risk of fragmentation, while smaller pages use memory more efficiently but expand the page table and may result in more frequent TLB misses.

5.2. TLB Miss Penalties

The Translation Lookaside Buffer (TLB) acts as a fast cache that holds recent virtual-to-physical address translations. While TLB improves memory access efficiency, it does not eliminate all translation delays. A miss in the TLB forces the system to retrieve the mapping from the page table in memory, which is significantly slower.

What Happens During a TLB Miss:

1. The MMU detects that the required virtual address is not in the TLB.
2. A page table lookup is initiated in memory, often involving multi-level traversals.
3. The resulting physical address is returned and entered into the TLB for future use.
4. The instruction is restarted after the address is resolved.

Impact of TLB Misses:

- A TLB miss may take 100–200 clock cycles, compared to 1–2 cycles for a TLB hit.
- Miss penalties are particularly high when page tables reside in slower memory tiers.

Categories of TLB Misses:

- **Compulsory:** First-time access to a new page.
- **Capacity:** TLB cannot hold all active entries.
- **Conflict:** Limited associativity causes eviction of entries still in use.

To minimise TLB misses and improve memory access speed, several strategies can be employed. Using larger pages, such as huge pages, allows each TLB entry to cover a greater range of memory, reducing the total number of entries needed. Operating systems can implement TLB-aware memory allocation, placing frequently accessed pages close together to improve locality. Increasing the associativity of the TLB helps lower the chance of conflicts when multiple virtual pages map to the same set. In some cases, software can preload important TLB entries in advance, especially for memory-critical regions, to avoid delays during execution.

Multi-Level Paging and TLB Misses:

- Each TLB miss may result in multiple memory accesses due to layered page tables.

- Systems implement hardware page walkers to accelerate such lookups.

TLB Miss Metrics:

- TLB hit ratio: Percentage of accesses resolved using the TLB.
- Effective memory access time can be calculated using hit and miss rates, helping in system performance tuning.

5.3. Memory Fragmentation Issues

Fragmentation refers to inefficient use of memory resources due to allocation and layout patterns. While paging largely avoids external fragmentation, it is not entirely free from memory inefficiency.

Types of Fragmentation:

- Internal Fragmentation:** Occurs when a process does not fully utilise the allocated page. For instance, a 4KB page holding only 2KB of data wastes half the space.
- External Fragmentation:** Paging avoids this by allocating fixed-size blocks. However, fragmentation may still occur at the secondary storage level (e.g., swap space).
- Fragmentation in Multi-Page Allocations:** Large data structures or memory-mapped files that span several pages may result in uneven usage across pages.

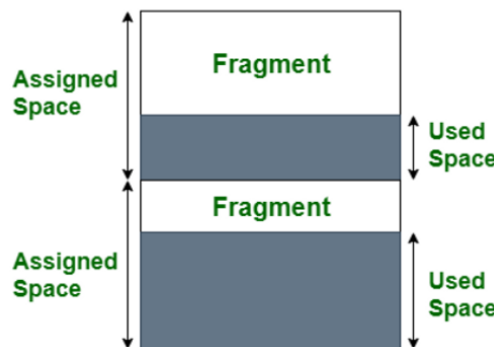


Figure 8: Memory Fragmentation

Source: <https://www.thetechplatform.com/post/fragmentation-in-os-internal-fragmentation-and-external-fragmentation>

The figure illustrates the concept of internal fragmentation within allocated memory blocks. It depicts two assigned spaces, each containing a 'Used Space' area, represented by the shaded region, and an 'Unused Space' area, labelled as 'Fragment'. This 'Fragment' represents the wasted memory within an allocated block that is not being utilised by the process. This phenomenon occurs when memory is allocated in fixed-size blocks, and the process only requires a portion of that block, leaving the

remainder as unusable fragmented space. This internal fragmentation can lead to inefficient memory usage across the system.

Managing Fragmentation in Paging and Swapping Systems

Minimising Internal Fragmentation: Reducing page size decreases wasted space per page but increases page table size. Adaptive page sizing is considered in some modern OS kernels.

Fragmentation in Swapping: The swap area may develop holes due to varied process sizes and lifetimes. Efficient page replacement policies and defragmentation tools are used to counter this.

Impact on Performance: Fragmentation can lead to increased I/O, as unused space is still managed. More memory may be consumed overall, limiting the number of concurrent processes.

OS-Level Solutions: Periodic page compaction (though rarely practical in paged systems). Delayed allocation strategies, where actual page assignment is postponed until data is written. Awareness in file systems (e.g., extent-based storage) to reduce fragmentation at disk level.

5.4. Scalability with Large Address Spaces

With the advent of 64-bit architectures, address spaces have become vast. Supporting these requires thoughtful paging implementations that scale well in terms of both performance and memory overhead.

Scaling paging systems to support large address spaces presents several challenges. Linear page tables become impractical due to their excessive size, while even multi-level tables can slow down memory access with multiple lookups. The limited capacity of the TLB leads to more frequent misses as the number of pages grows. Additionally, larger working sets put greater strain on available memory and demand more efficient replacement strategies to maintain performance.

Scaling Techniques

- **Multi-Level Paging** - This method divides the page table into smaller, more manageable parts, allowing only the necessary sections to be loaded into memory. It is commonly used in 64-bit systems, where four or five levels of paging help manage the vast address space efficiently.
- **Inverted Page Tables** - In this approach, a single global page table is indexed by frame number instead of page number, which significantly reduces the overall size of the table. However, it requires additional mechanisms such as hashing or searching to locate pages.

- **Huge Pages** - Huge pages use a larger page size than standard ones, such as 2MB or 1GB, reducing the number of TLB entries and the depth of the page table. This technique is particularly effective for memory-intensive applications like databases and scientific computing.
- **Segmented Paging** - Segmented paging combines the concepts of segmentation and paging, grouping related pages together. This approach simplifies access to large, structured memory spaces and is useful in systems where logical grouping of memory is beneficial.

Impact on System Performance

As page tables become deeper to handle larger address spaces, the time required for address translation also increases, which can slow down memory access. TLB misses become more costly because each miss triggers multiple lookups through the levels of the page table. Without optimisations such as demand paging and sparse memory mappings, overall memory usage can rise significantly, potentially reducing the number of processes the system can efficiently support.

Operating System Adaptations:

- Support for demand paging with partial page-table population.
- Use of page clustering and batching to amortise page fault costs.
- Lazy allocation and copy-on-write to reduce unnecessary memory consumption.

In Linux systems running on x86-64 architecture, a four-level paging structure is used, with each level containing 512 entries. To conserve memory, only the parts of the paging hierarchy that are actively needed for current address mappings are kept resident in memory, allowing efficient use of space while managing large address ranges.

Efficient implementation of paging requires a thorough understanding of the associated overheads, trade-offs, and performance bottlenecks. While paging offers memory protection and process isolation, it introduces costs in terms of time and memory usage.

The Translation Lookaside Buffer helps mitigate access delays, but its size and associativity limit its effectiveness. TLB misses and deep page-table hierarchies can substantially degrade performance, particularly in systems with large address spaces. Fragmentation—though less severe than in contiguous allocation schemes—still persists, especially as internal fragmentation. Addressing this requires careful design of page size, allocation strategy, and swap space management.

Scalability remains a significant concern for modern systems. With larger address spaces and more complex applications, operating systems employ multi-level paging, huge pages, and smarter

replacement policies to maintain performance. Future systems may move towards adaptive, context-aware memory management that continuously evaluates workload behaviour to optimise memory handling dynamically. Such evolutions in paging underline the importance of balancing system simplicity with execution efficiency.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions	
16	What is a primary cause of storage overhead in paging systems? a) The size of the TLB b) Repeated page faults c) Memory used by page tables d) High CPU clock speeds
17	Which type of overhead occurs when a page is duplicated due to a write operation in copy-on-write scenarios? a) Context-switching overhead b) Copying overhead c) TLB miss overhead d) Storage overhead
18	What is one key disadvantage of larger page sizes in paging systems? a) Increased number of TLB entries b) Lower fragmentation c) Higher internal fragmentation d) Reduced page table size
19	What type of TLB miss happens when the TLB cannot store all currently used entries? a) Compulsory miss b) Conflict miss c) Capacity miss d) Segment miss
20	How do huge pages help in reducing TLB miss rates? a) By increasing the number of page faults b) By reducing page size c) By allowing each entry to cover more memory d) By decreasing CPU usage

6. SUMMARY

- Virtual memory provides each process with the impression of a large, continuous address space by mapping logical addresses to physical frames through paging.
- Paging divides both logical and physical memory into fixed-size pages and frames, simplifying allocation and isolation of processes.
- Single-level page tables offer straightforward lookup but consume large amounts of memory, whereas multi-level tables reduce storage by loading only needed portions.
- An inverted page table indexes entries by physical frame rather than virtual page number, cutting down on overall table size at the expense of more complex searches.
- The memory management unit handles on-the-fly translation of logical to physical addresses, with built in support for multi level paging in modern processors.
- A translation lookaside buffer caches recent page-table entries to speed up address translation, though its limited capacity means misses still incur costly table walks.
- The operating system is responsible for maintaining page tables, choosing frame replacement policies and updating metadata bits to track page usage.
- Shared pages and copy on write techniques allow processes to share common code or data until one writes, at which point a private copy is made.
- When a page fault occurs, the kernel must identify a victim frame, possibly write it back to disk, load the required page, and update the tables, all of which affect performance.
- Practical paging systems must weigh storage and computational overhead, TLB miss penalties, fragmentation issues and the challenges of scaling to vast 64 bit address spaces.

7. GLOSSARY

Virtual memory	- A memory management technique that gives each process the impression of a large, contiguous address space by mapping its logical addresses to physical frames as needed.
Page	- A fixed size block of logical memory, typically ranging from 4 KB to several megabytes, which the operating system moves between physical memory and secondary storage.
Frame	- A fixed size block of physical memory that corresponds in size to a page; frames store the contents of pages when they are resident in RAM.
Page table	- A data structure used by the operating system to record the mapping from virtual page numbers to physical frame numbers for each process.
Inverted page table	- A global page table indexed by frame number rather than by virtual page number, reducing overall memory overhead at the cost of more complex lookup mechanisms.
Memory Management Unit (MMU)	- A hardware component that translates virtual addresses to physical addresses at runtime, handling page table lookups and raising exceptions on faults.
Translation Lookaside Buffer (TLB)	- A small, fast cache within the MMU that holds recent virtual to physical address translations to speed up memory access.
Page fault	- An event triggered when a process accesses a page not currently loaded into physical memory, prompting the operating system to retrieve the page from secondary storage.
Copy on write	- An optimisation that allows multiple processes to share the same physical page until one alters it, at which point a separate copy is made for the writing process.
Fragmentation	- Wasted memory resulting from fixed size allocation; internal fragmentation refers to unused space within a page, while external fragmentation affects larger memory pools.

8. TERMINAL QUESTIONS

1. Explain how virtual memory and paging work together to provide process isolation and efficient memory use.
2. Compare single level and multi level page table structures, outlining their advantages and disadvantages.
3. Describe the role of the translation lookaside buffer in address translation and discuss how TLB misses affect system performance.
4. Outline the steps taken by the operating system when handling a page fault, from fault detection to page table update.
5. Analyse the causes and effects of internal and external fragmentation in paging systems.
6. Discuss the inverted page table approach and how it differs from conventional page tables in terms of memory overhead and lookup complexity.
7. Evaluate the impact of page size selection on page table storage overhead, TLB efficiency and fragmentation.
8. Explain copy on write and shared pages, including how they conserve memory and when a private copy is created.
9. Assess the challenges of scaling paging mechanisms to 64 bit address spaces and describe techniques used to address these challenges.
10. Propose software and hardware optimisations that can reduce paging overheads and improve overall memory management performance.

9. ANSWERS

Self-Assessment Questions

1. b) It provides the illusion of a large, continuous address space to each process
2. b) Paging
3. b) External fragmentation
4. c) The exact location within a page
5. b) The frame number corresponding to a page number
6. b) Memory Management Unit
7. b) Base register
8. a) Virtual page number and physical frame number
9. c) Fully associative
10. c) Three
11. b) Dirty bit
12. c) First-In, First-Out (FIFO)
13. c) When a process attempts to write to the page
14. b) Select a victim frame using a replacement policy
15. d) Pre-paging
16. c) Memory used by page tables
17. b) Copying overhead
18. c) Higher internal fragmentation
19. c) Capacity miss
20. c) By allowing each entry to cover more memory

Terminal Questions Answers

Answers 1: Virtual memory and paging work together by abstracting physical memory into fixed size pages, which are mapped from a process's logical address space to physical frames only when needed. This enables each process to run in its own isolated address space, preventing unauthorised access and making efficient use of RAM by swapping pages to disk as required. Refer to section 2.1.

Answers 2: Single level page tables use a simple linear array to map pages to frames, offering direct lookups but consuming large amounts of memory for big address spaces. Multi level tables break the

mapping into smaller parts, loading only required segments to save space, at the cost of extra lookups per address translation. Refer to section 2.3.

Answers 3: The translation lookaside buffer (TLB) caches recent virtual to physical mappings to speed up address translation and reduce page table accesses. However, when a TLB miss occurs, the system must walk the page tables—which may span multiple levels—significantly increasing memory access latency. Refer to section 3.2.

Answers 4: On a page fault, the MMU traps into the kernel, which first verifies the access is valid before selecting a victim frame via the chosen replacement policy. If the victim is dirty it is written back to disk, then the needed page is read in, the page table and TLB are updated, and the faulting instruction is retried. Refer to section 4.4.

Answers 5: Internal fragmentation arises when fixed size pages are only partly used, wasting space within frames, while external fragmentation can occur in swap storage due to holes between allocated segments. Both forms of inefficiency increase I/O and limit the number of concurrent processes by misusing available memory. Refer to section 5.3.

Answers 6: An inverted page table maintains one entry per physical frame, indexed by frame number and storing the owning process's page number and PID, drastically cutting table size. Lookup complexity increases since entries must be searched or hashed, making hardware support such as associative searches or hash tables essential. Refer to section 2.4.

Answers 7: Larger page sizes reduce the total number of page table entries and TLB misses but raise internal fragmentation, while smaller pages improve granularity at the expense of bigger tables and potentially more TLB misses. Choosing an optimal page size thus involves balancing table storage overhead, cache efficiency, and wasted memory. Refer to section 5.1.

Answers 8: Shared pages allow identical code or data to be mapped read only into multiple processes, conserving memory, while copy on write delays duplication until a write occurs. At the first write attempt, the OS makes a private copy of the page, ensuring isolation without unnecessary copies during process creation. Refer to section 4.3.

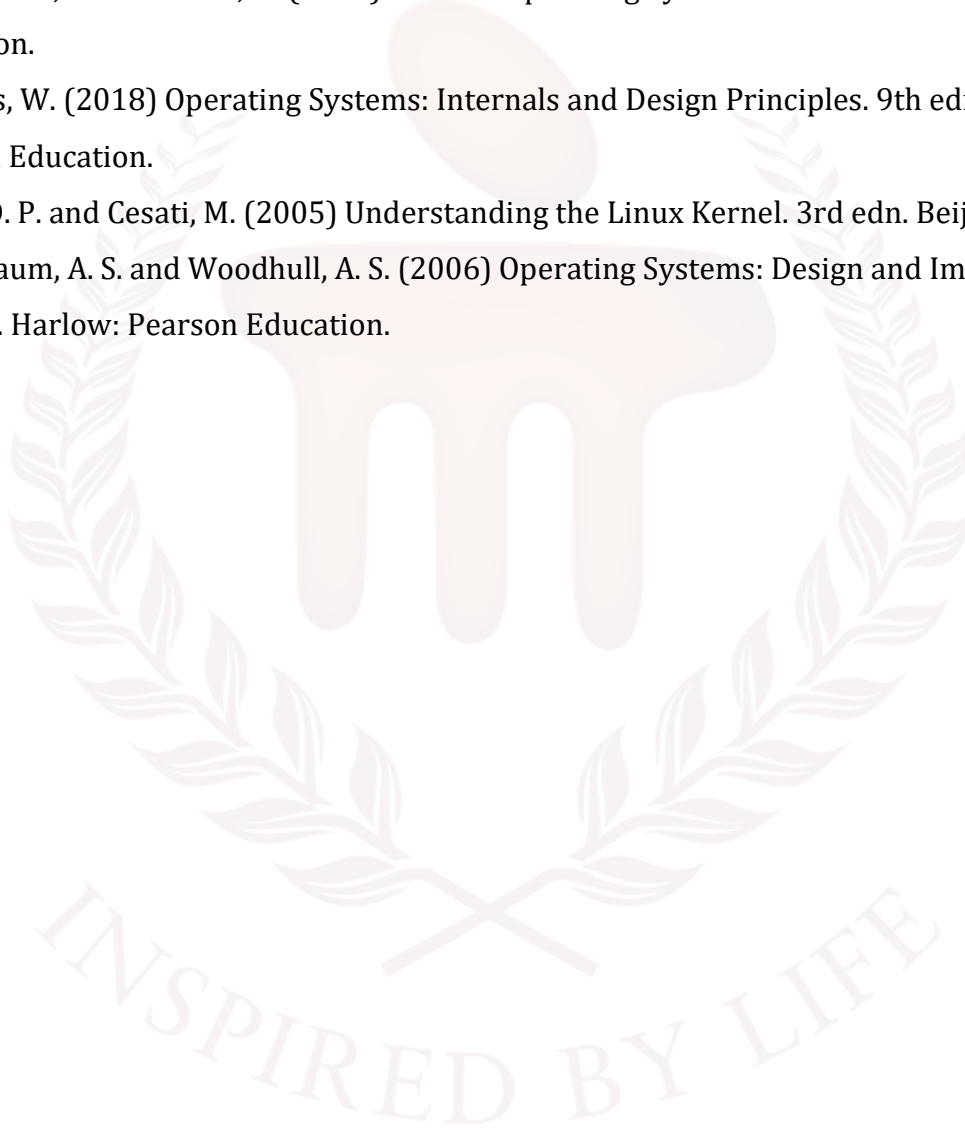
Answers 9 : Scaling paging to 64 bit address spaces challenges table size and TLB capacity, as deeper multi level tables increase lookup overhead and larger working sets raise miss rates. Techniques such as four or five level paging, huge pages, and inverted tables help manage space and reduce translation steps. Refer to section 5.4.

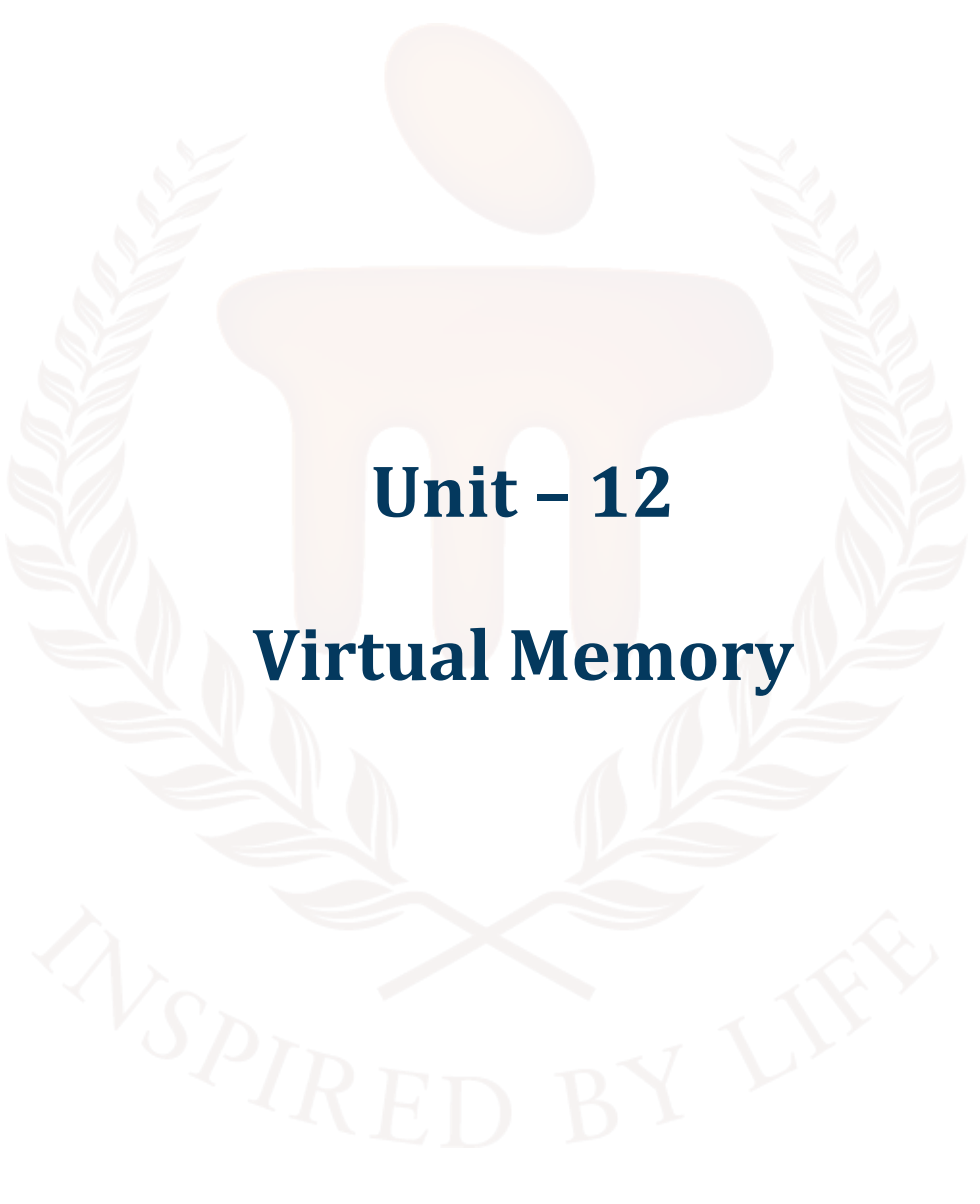
Answers 10: Software optimisations like working set tracking, pre paging and page buffering reduce faults and I/O, while hardware enhancements such as larger or set associative TLBs, page walk caches and support for huge pages cut translation latency. Together, these strategies lower both software and hardware overheads, improving overall memory management performance. Refer to sections 4.5 and 3.3.



10. REFERENCES

1. Silberschatz, A., Galvin, P. B. and Gagne, G. (2021) Operating System Concepts. 10th edn. Hoboken, NJ: Wiley.
2. Tanenbaum, A. S. and Bos, H. (2015) Modern Operating Systems. 4th edn. Harlow: Pearson Education.
3. Stallings, W. (2018) Operating Systems: Internals and Design Principles. 9th edn. Harlow: Pearson Education.
4. Bovet, D. P. and Cesati, M. (2005) Understanding the Linux Kernel. 3rd edn. Beijing: O'Reilly.
5. Tanenbaum, A. S. and Woodhull, A. S. (2006) Operating Systems: Design and Implementation. 3rd edn. Harlow: Pearson Education.





Unit – 12

Virtual Memory

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Demand Paging	1	-	6 - 14
2.1	Page Faults and Handling Mechanism	2	-	
2.2	Valid-Invalid Bit and Protection	3	-	
2.3	Swap Space Management	4	-	
2.4	I/O Overhead and Performance Impact	-	-	
2.5	Optimisations (Prepaging, Clustering)	5	1	
3	Benefits of Virtual Memory	6	-	15 - 22
3.1	Increased Memory Utilisation	-	-	
3.2	Support for Large and Sparse Address Spaces	-	-	
3.3	Isolation and Process Protection	7	-	
3.4	Simplified Program Development	-	2	
4	Implementation Considerations	-	-	23 - 30
4.1	Hardware Support Requirements	8	-	
4.2	Operating System Responsibilities	-	-	
4.3	Performance Metrics and Trade-offs	9	3	
5	Summary	-	-	31 - 32
6	Glossary	-	-	33
7	Terminal Questions	-	-	34
8	Answers	-	-	35 - 37
9	References	-	-	38

1. INTRODUCTION

In the previous chapter, we explored the concept of paging as a memory management technique. The discussion covered how logical memory is divided into pages and mapped to physical memory frames, allowing for non-contiguous allocation. We also examined the hardware components required for paging, such as the Memory Management Unit (MMU), and the software mechanisms involved in managing page tables, page faults, and address translation. These concepts laid the groundwork for understanding how memory is organised and accessed efficiently in modern systems.

Building on this foundation, the current chapter introduces the concept of virtual memory. Virtual memory extends the idea of paging by allowing processes to execute even if not all their pages are loaded in physical memory. It provides the illusion of a large, continuous memory space to each process by using both physical memory and secondary storage. This capability enables systems to handle larger programs, support multitasking more effectively, and increase overall resource utilisation.

A key mechanism in virtual memory is demand paging, where pages are loaded into memory only when they are needed. This approach minimises memory usage and reduces the time required to load programs. The chapter also explores the advantages of virtual memory, including improved system responsiveness, efficient process isolation, and better use of available hardware. These benefits are essential in both single-user environments and large-scale multi-user systems.

To study this chapter effectively, start by reviewing how paging works and how address translation is handled. Then move on to the concept of demand paging, focusing on how it determines when and how pages are loaded. Use practical examples to understand the process of page faults and their handling. Diagrams illustrating virtual address space, page tables, and memory mapping can help clarify the abstract concepts. Practising scenario-based problems will further aid in reinforcing the ideas covered.

1.1. Objectives

After studying this unit, you should be able to:

- Explain the purpose of virtual memory in modern operating systems.
- Illustrate how demand paging manages memory efficiently.
- Analyse the benefits of using virtual memory in process execution.
- Demonstrate how page faults occur and how they are handled.
- Compare the impact of virtual memory in different system scenarios.



2. DEMAND PAGING

In contemporary operating systems, demand paging is an essential technique in virtual memory management. Unlike conventional memory allocation strategies that load entire processes into memory before execution, demand paging allows processes to begin execution even if only a part of their pages are loaded into physical memory. This strategy increases memory efficiency and enables systems to support more active processes simultaneously.

Demand paging operates on the principle that not all parts of a program are needed immediately. Pages are brought into memory only when a specific instruction or data item within them is accessed. This contrasts with anticipatory methods, which might load unnecessary pages, leading to wasted memory space.

This approach is especially important in systems with limited physical memory or where the working set of a process is significantly smaller than its total address space. While demand paging brings about efficient memory utilisation, it introduces challenges such as page faults, handling latency, and performance bottlenecks due to I/O delays. The operating system must handle these gracefully through well-structured mechanisms and optimisations.

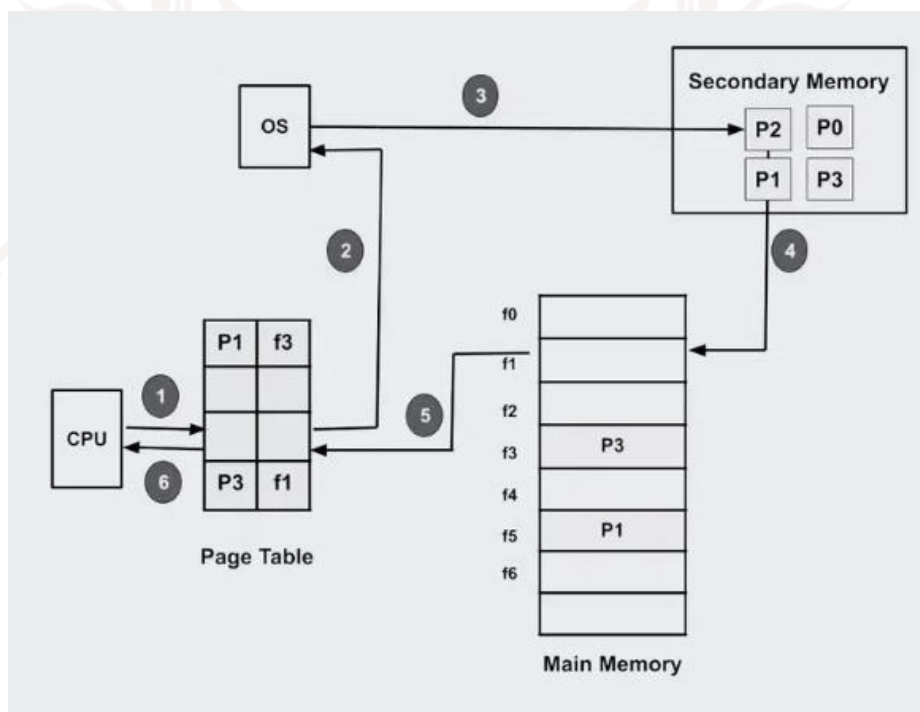


Figure 1: Demand Paging

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20230908065723/page1-768.webp>

The figure, "Demand Paging", illustrates the process of demand paging, a memory management technique that loads pages into main memory only when they are needed. The sequence begins with the CPU attempting to access a page (e.g., P3) that is not currently in the 'Page Table' (step 1), resulting in a page fault. The Operating System (OS) then identifies this fault (step 2) and initiates the retrieval of the required page from 'Secondary Memory' (step 3). The missing page (P3) is then loaded into an available frame in 'Main Memory' (step 4). The page table is subsequently updated with the new frame mapping for P3 (step 5), and finally, the CPU can successfully access the desired page (step 6).

2.1. Page Faults and Handling Mechanism

A central event in demand paging is the page fault. A page fault occurs when a process tries to access a memory page that is not currently in physical memory. This triggers the memory management unit (MMU) to signal the operating system, which then takes steps to bring the required page from secondary storage (usually disk) into memory.

Page faults are not considered errors; rather, they are expected events in demand paging systems. The performance of a demand paging system depends heavily on how efficiently it handles page faults.

Handling a Page Fault: Steps

The process of handling a page fault is a multi-step operation involving both hardware and software intervention.

- 1. Trap to the Operating System** - The CPU traps to the operating system when the memory reference cannot be fulfilled because the page is absent from RAM.
- 2. Validation Check** - The OS checks whether the page reference is valid, i.e., within the logical address space of the process. An invalid reference results in a segmentation fault or process termination.
- 3. Location Resolution** - If the reference is valid, the operating system determines where the page is stored on disk.
- 4. Free Frame Search** - A free frame in physical memory is located. If none is available, a page replacement policy is invoked.
- 5. Page-In Operation** - The desired page is read from disk into the selected frame.
- 6. Page Table Update** - The process's page table is updated to reflect the new frame address and mark the page as valid.

7. Instruction Restart - The faulting instruction is restarted now that the required page is in memory.

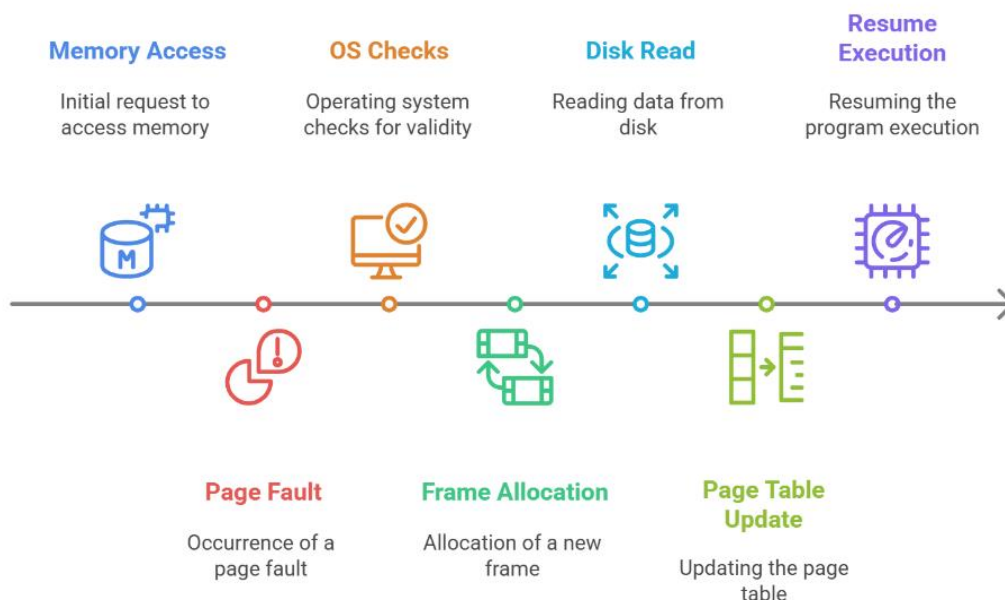


Figure 2: Memory Access and Page Fault Handling

The figure outlines the detailed sequence of events that occur during a page fault in a memory management system. The process begins with a 'Memory Access', an initial request from a program to access data in memory. If the requested page is not present, a 'Page Fault' occurs, signifying a memory access violation. The 'OS Checks' then commence, where the operating system validates the request and determines the cause of the fault. Following this, 'Frame Allocation' takes place, where a free memory frame is assigned to hold the required page. A 'Disk Read' operation then retrieves the necessary data from secondary storage. Once the data is in memory, a 'Page Table Update' occurs, where the page table is modified to reflect the new mapping. Finally, 'Resume Execution' allows the program to continue its operation from where it was interrupted.

2.2. Valid-Invalid Bit and Protection

To support demand paging effectively, the system must have a mechanism to distinguish between pages that are present in memory and those that are not. This is achieved using the valid-invalid bit, a flag associated with each page table entry.

Use of Valid-Invalid Bit

- **Valid Bit (1):** Indicates that the page is currently in physical memory and can be accessed.

- **Invalid Bit (0):** Shows that the page is not in memory. Any attempt to access such a page results in a page fault.

This bit acts as a simple but crucial hardware support feature for memory protection and fault detection.

Protection Mechanism

Demand paging systems must also ensure that memory access is controlled and secure. Protection bits are often added to page table entries alongside the valid-invalid bit.

Common Protection Attributes:

- **Read-only:** The page can be read but not modified.
- **Execute-only:** The page contains executable instructions but cannot be read or written to.
- **Read-Write:** Full access is granted.

Such control bits prevent unauthorised access, such as writing to code segments or executing data segments.

Table 1: Protection and Status Bits

Bit Name	Purpose
Valid-Invalid Bit	Indicates presence of page in memory
Read/Write Bit	Controls write permissions
Execute Bit	Controls executable access
Dirty Bit	Flags whether the page has been modified

2.3. Swap Space Management

When a process requires a page that is not in physical memory and no free frame is available, the system must choose an existing page to be evicted. The chosen page is then written to disk if it has been modified, and the new page is loaded into its place. This disk area is referred to as swap space.

Role of Swap Space

Swap space is an extension of physical memory used to store pages temporarily when RAM is full. It is a dedicated portion of the secondary storage, often pre-allocated during system setup.

Characteristics of Swap Space:

- Usually located on the system disk but may be stored on faster SSDs for performance.

- Fixed or dynamically allocated in size.
- Accessed using optimised strategies to reduce latency.

Swap In and Swap Out Operations

- **Swap In:** Loading a required page from swap space into main memory.
- **Swap Out:** Writing a page from memory into swap space to make room for other pages.

The performance of a system can degrade significantly if swap space usage becomes excessive—a condition often referred to as thrashing, where the system spends more time swapping than executing processes.

Swap Space Management Strategies

1. **Predefined Partitioning:** A fixed swap space is allocated during OS installation.
2. **Dynamic Allocation:** Swap files are created on demand to extend capacity.
3. **Page Compression:** Reducing page size before swapping to save disk bandwidth.

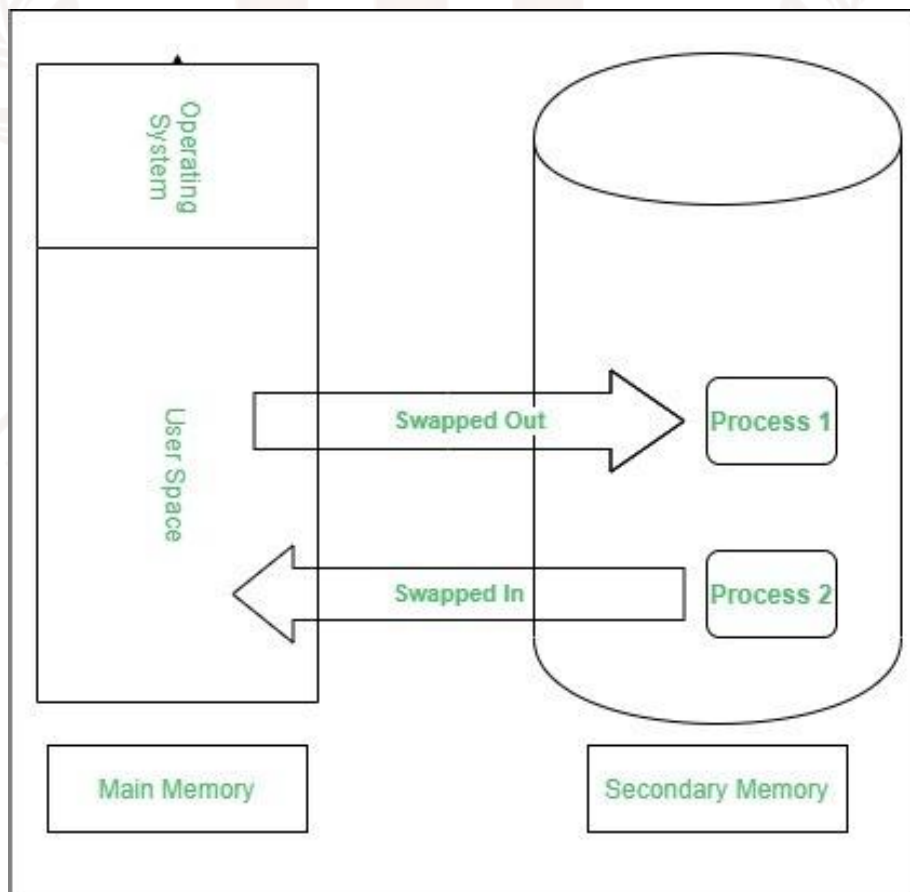


Figure 3: Swap Space Management

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20200406111356/Untitled-Diagram66-3.jpg>

The figure illustrates the concept of swapping, a memory management technique used by operating systems to temporarily move processes between main memory and secondary memory. On the left, 'Main Memory' is divided into an 'Operating System' area and 'User Space'. On the right, 'Secondary Memory' is depicted as a larger storage unit. The diagram shows 'Process 1' being 'Swapped Out' from user space in main memory to secondary memory, typically to free up main memory for other processes. Conversely, 'Process 2' is shown being 'Swapped In' from secondary memory back into the user space of main memory, allowing it to resume execution. This dynamic movement enables the system to handle processes larger than available physical memory.

2.4. I/O Overhead and Performance Impact

Demand paging, while efficient in memory utilisation, incurs a significant I/O overhead during page faults. Retrieving a page from disk is several orders of magnitude slower than accessing data from RAM.

Sources of Overhead

- **Disk Latency:** Mechanical disks introduce seek and rotation delays.
- **Page Table Management:** Frequent updates to page tables add to CPU overhead.
- **Context Switching:** The process may be suspended while the required page is being loaded, reducing CPU utilisation.

Performance Impact of Page Faults

Page faults cause the CPU to idle or switch tasks while waiting for the page to be loaded. In environments where page faults are frequent, the system performance may degrade substantially.

Factors Affecting Page Fault Frequency:

- **Process Working Set Size:** Larger working sets tend to cause more faults if RAM is insufficient.
- **Locality of Reference:** Better spatial and temporal locality reduces fault rates.
- **Page Replacement Policy:** Efficient algorithms (e.g., LRU, Clock) reduce unnecessary swaps.

Input/output overhead becomes a significant concern in systems that rely heavily on virtual memory. Each time a page fault occurs and the required page is not in memory, the operating system must pause the running process and fetch the missing page from disk. This disk access introduces latency, often several orders of magnitude slower than accessing data from RAM. As the frequency of page faults increases, these delays accumulate, causing the overall execution time of processes to grow.

The performance of CPU-bound tasks is particularly affected, as they are forced to wait idly for memory operations to complete, thereby underutilising processing resources.

To mitigate the performance impact of I/O overhead, systems can adopt various strategies. Increasing the amount of physical memory reduces the likelihood of page faults, ensuring that more pages remain resident in RAM. Careful optimisation of how data and code are arranged within memory can also help reduce non-sequential access patterns, thereby improving cache usage and lowering fault rates. Efficient page replacement algorithms such as Least Recently Used (LRU) or its approximations help retain the most useful pages in memory, minimising disruptive disk accesses. If these issues are not properly managed, the system may enter a state of thrashing, where constant page swapping prevents meaningful progress.

2.5. Optimisations (Prepaging, Clustering)

To minimise the latency associated with demand paging, operating systems implement several optimisation techniques that anticipate memory needs or reduce the cost of page transfers.

Prepaging – This refers to the strategy of loading multiple pages at once during a page fault, rather than fetching a single requested page. This method relies on the principle of spatial locality.

Benefits of Prepaging:

- Reduces the number of future page faults.
- Takes advantage of bulk disk reads, which are more efficient than multiple small reads.

Limitations:

- May result in unnecessary loading of unused pages.
- Wastes memory if the prediction is inaccurate.

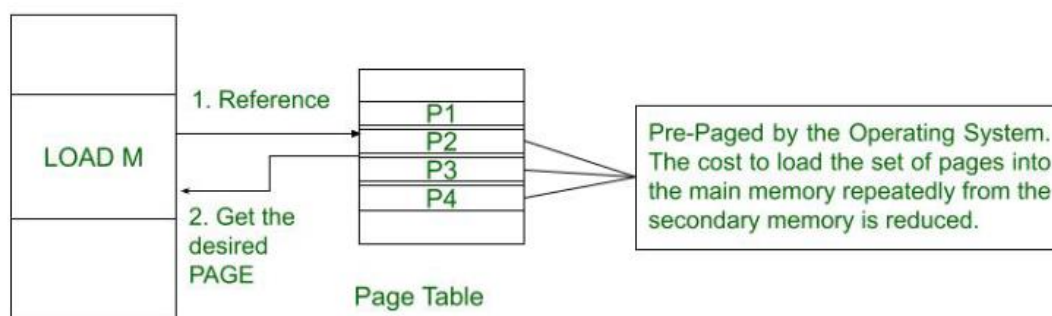


Figure 4: Prepaging

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20210611144803/1.PNG>

The figure illustrates a memory management technique aimed at reducing the overhead associated with page faults by anticipating future page requirements. When a program needs to 'LOAD M', it makes a 'Reference' (step 1) which indicates a need for a block of pages. Instead of waiting for individual page faults, the operating system 'Pre-Pages' a set of pages (P1, P2, P3, P4) into the 'Page Table' and main memory beforehand. This proactive approach ensures that when the program later requests a specific page, such as 'Get the desired PAGE' (step 2), it is already available in memory. The accompanying text explains that this reduces the cost of repeatedly loading sets of pages from secondary memory, thereby improving system performance.

Clustering - This involves grouping together logically related pages and managing them as a unit. It is especially effective in file-backed mappings and program segments such as code and stack.

Clustering Techniques:

- **Disk Clustering:** Pages stored contiguously on disk are loaded together.
- **Logical Clustering:** Pages referenced in proximity are prefetched based on past access patterns.

Hybrid Optimisations

Modern operating systems may combine prepaging and clustering with adaptive strategies. Machine learning models and heuristics can further improve prediction accuracy based on usage trends.

Demand paging is a sophisticated and integral aspect of modern operating systems. It enables efficient use of memory by allowing processes to execute without loading all their pages into memory at once. The demand paging mechanism hinges on the handling of page faults, protection using status bits, swap space management, and minimisation of performance overheads. While it offers significant advantages in terms of flexibility and memory efficiency, it also necessitates careful tuning and optimisations to maintain system performance.

Proper handling mechanisms, such as valid-invalid bits, well-managed swap space, and intelligent page replacement and prefetching strategies, help reduce the impact of faults and I/O delays. When implemented correctly, demand paging allows systems to scale efficiently, support multitasking, and maintain responsiveness under varied workloads.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the primary advantage of demand paging over loading an entire process into memory before execution? a) It eliminates page faults entirely b) It allows processes to begin execution with only a subset of their pages in memory c) It guarantees zero I/O latency d) It requires no page-replacement policy
2	Which event directly triggers a page fault in a demand-paging system? a) Accessing a page with the valid bit set to 1 b) Accessing a page marked as read-only c) Accessing a page not present in physical memory d) Accessing a page in the swap space
3	In a page-table entry, what does a valid-invalid bit value of 0 signify? a) The page is present in physical memory b) The page has been modified since it was loaded c) The page is not currently in physical memory d) The page is protected against writes
4	Which of the following best describes swap-out in swap-space management? a) Loading a required page from disk into RAM b) Writing a page from RAM to swap space to free a frame c) Checking the protection bits of a page table entry d) Updating the page table after a page-fault handling
5	What is the main goal of prepaging as an optimisation technique in demand paging? a) To reduce the size of the working set b) To load multiple pages at once based on spatial locality c) To compress pages before writing them to swap space d) To increase the swap-in and swap-out frequency

3. BENEFITS OF VIRTUAL MEMORY

Virtual memory is one of the most impactful innovations in the field of memory management. It provides an abstraction that allows programs to perceive they have access to a vast and contiguous memory space, even when the physical memory available is significantly smaller. Through mechanisms such as paging and segmentation, the operating system creates a layer between the logical address space seen by processes and the actual physical memory in the hardware.

This abstraction allows multiple processes to co-exist and execute without direct knowledge of the underlying memory limitations. As a result, virtual memory not only increases efficiency but also strengthens system reliability, scalability, and ease of development.

The key advantages of virtual memory extend across various dimensions, such as better memory utilisation, support for large and sparse address spaces, enhanced security through process isolation, and simplifying the software development process. Each of these is discussed in detail below.

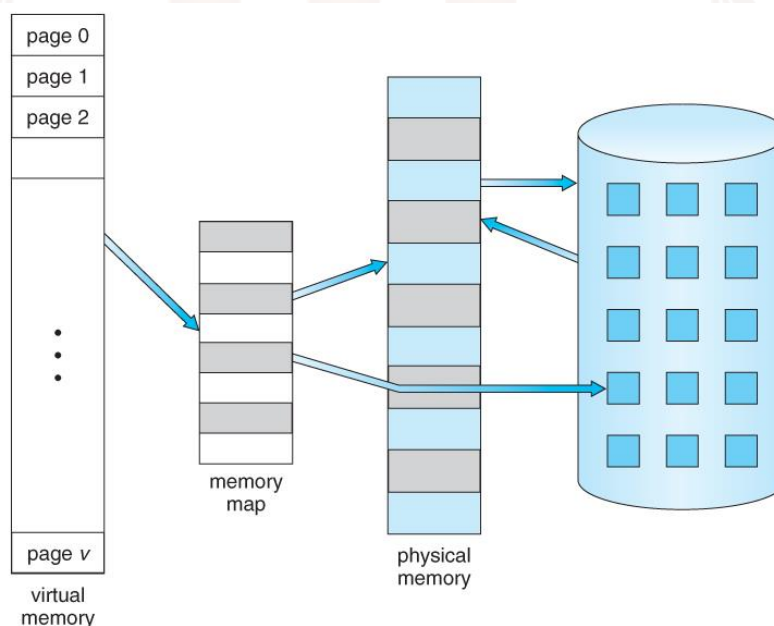


Figure 5: Memory Usage in Virtual Memory

Source: https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/images/Chapter9/9_01_VirtualMemoryLarger.jpg

The figure illustrates how virtual memory is mapped to physical memory and secondary storage. It begins with 'virtual memory', which is logically divided into pages. These virtual pages are then mapped through a 'memory map' to frames in 'physical memory'. Importantly, not all virtual pages reside in physical memory at any given time. Some pages that are not currently in active use or have

been swapped out are stored in 'secondary memory' (typically a hard drive), represented by the large cylindrical icon. This mapping mechanism allows a system to execute programs larger than the available physical memory by only loading the necessary pages into RAM, effectively expanding the apparent memory capacity.

3.1. Increased Memory Utilisation

In traditional systems without virtual memory, processes must be fully loaded into physical memory before execution. This often results in underutilisation of memory resources, as large parts of a program might remain unused during its execution. Virtual memory addresses this inefficiency by enabling on-demand memory allocation, thus making more effective use of available memory.

How Virtual Memory Enhances Utilisation

Virtual memory systems use techniques such as paging and demand loading to ensure that only the required portions of a program are loaded into physical memory. Pages are fetched as and when needed, which drastically reduces memory consumption.

Example Scenario: A program with a 100 MB logical size may actively use only 30 MB at a given time. In a system without virtual memory, the entire 100 MB would occupy space in RAM. With virtual memory, only the 30 MB in use is resident in memory, freeing up space for other processes.

Key Features That Improve Utilisation:

- **Demand Paging:** Pages are loaded only when needed, reducing waste.
- **Page Replacement:** Infrequently used pages are swapped out to accommodate new ones.
- **Sharing:** Common pages, such as library code, can be shared among processes.

3.2. Support for Large and Sparse Address Spaces

Another significant benefit of virtual memory is its ability to support large logical address spaces, even when the physical memory is limited. This is particularly useful in modern applications that require handling of large data sets or memory-intensive operations.

Large Address Space Support - Operating systems can present each process with a large virtual address space, often much larger than the actual physical memory. For example, a 64-bit processor can theoretically support up to 2^{64} bytes of addressable space per process. Virtual memory enables this by mapping virtual addresses to physical memory selectively.

Use Case Examples:

- Applications such as databases and scientific simulations often require gigabytes or even terabytes of address space.
- Systems supporting multiple concurrent users can isolate their sessions using separate virtual address spaces, irrespective of available RAM.

Sparse Address Spaces - In addition to being large, address spaces can also be sparse, meaning there can be large unused sections between active memory regions. Virtual memory makes this possible by mapping only the active parts of an address space to physical memory.

Advantages of Sparse Allocation:

- Enables use of dynamic data structures such as large arrays or matrices without allocating all memory upfront.
- Useful in recursive programs and applications where memory is allocated in non-contiguous blocks.

Virtual memory extends the usable memory space of a system beyond the physical limitations imposed by hardware. By using disk storage as a temporary extension of RAM, the operating system allows applications to access a larger address space than what is physically present. This enables processes to manage complex and expansive memory layouts, supporting the execution of large-scale programmes without being constrained by available memory. Furthermore, virtual memory handles sparse memory usage efficiently by not allocating physical memory to unused or empty regions. This avoids unnecessary allocation and ensures that physical memory is reserved only for the parts of a process that are actively in use.

3.3. Isolation and Process Protection

One of the foundational roles of virtual memory is to protect and isolate processes from each other. In a multi-tasking environment, where multiple programs run concurrently, it is vital that one process cannot interfere with another's data or code. Virtual memory ensures this isolation through controlled address translation and protection mechanisms.

Mechanisms for Isolation - Each process operates within its own virtual address space. The mapping from virtual to physical addresses is managed by the page table, which is unique to each process. As a result, processes are unaware of each other's memory layout.

Key Protection Features:

- **No Direct Access:** Processes cannot access memory outside their own address space.
- **Access Permissions:** Page table entries include protection bits specifying read, write, or execute rights.
- **Segmentation Faults:** Attempts to access unauthorised memory trigger faults and may terminate the offending process.

Benefits of Isolation

1. **Security:** Malicious or buggy programs cannot read or overwrite memory belonging to other processes.
2. **Stability:** One process crashing or misbehaving does not affect others.
3. **Controlled Sharing:** When sharing is necessary (e.g., shared libraries), the OS can map specific pages into multiple address spaces with defined permissions.

Virtual memory plays a key role in maintaining process isolation, which is essential for the security and stability of a system. By ensuring that each process operates within its own virtual address space, the operating system prevents unauthorised access to memory used by other processes. This isolation protects against accidental or malicious corruption of data, maintaining the integrity of running programmes.

To enforce strict access controls, the page table uses protection bits that define the permissions for each memory page, such as read, write or execute. These permissions are checked during every memory access, and any violation results in a trap to the operating system. As a result, the system retains full control over which pages are accessible to each process, allowing fine-grained regulation of memory use. This mechanism strengthens both process security and overall system reliability.

Table 2: Example Page Table Entries with Protection Settings and Access Outcomes

Virtual Page Number	Frame Number	Protection Bits (R/W/X)	Access Attempt	Outcome
0x01	0x0A	R W	Read	Allowed
0x02	0x0B	R W	Write	Allowed
0x03	0x0C	R	Write	Denied – Protection Fault
0x04	0x0D	R X	Execute	Allowed

0x05	0x0E	R	Execute	Denied – Protection Fault
0x06	0x0F	—	Any Access	Denied – Invalid Page
0x07	0x10	R W	Read	Allowed
0x08	0x11	R W X	Execute	Allowed

The table titled "Example Page Table Entries with Protection Settings and Access Outcomes" illustrates how different combinations of protection bits assigned to virtual memory pages determine the result of attempted memory accesses. Each entry maps a virtual page to a physical frame and specifies access permissions using read, write, and execute bits. When a process attempts to access a page, the operating system checks these bits to decide whether the action is permitted. If the access type aligns with the page's protection settings, the operation proceeds; otherwise, a protection fault occurs. Entries without valid permissions or mappings result in access denial, demonstrating how the page table enforces memory protection and process isolation.

3.4. Simplified Program Development

From a software development perspective, virtual memory simplifies many aspects of writing, compiling, and executing programs. Developers no longer need to manage memory usage manually or account for specific physical memory constraints, as virtual memory abstracts those details.

Advantages for Developers

- **No Manual Allocation:** Programmers do not need to handle memory location assignments or relocation during execution.
- **Dynamic Allocation Support:** Virtual memory allows on-the-fly memory allocation (e.g., via `malloc()` or similar constructs), making applications more flexible.
- **Simplified Linking and Loading:** Programs can be compiled assuming they start at address zero. The operating system relocates them to the actual memory space during execution.
- **Ease of Multitasking:** Developers do not need to worry about address conflicts between concurrently running programs.

Virtual memory simplifies software development by providing a structured and predictable memory environment for each process. In large software systems, modules can be developed independently with the assurance that they will operate in isolated memory spaces, reducing the likelihood of

unintended interactions or memory conflicts. This separation allows teams to work on different parts of a programme without requiring intricate coordination over memory use.

Furthermore, the debugging process benefits from this clarity, as developers can reason about program behaviour within a consistent memory model. Complex programming techniques, such as recursion or the use of linked data structures, become more straightforward to implement, since the risk of fragmentation or manual memory allocation errors is significantly reduced. This abstraction ultimately leads to more robust and maintainable codebases.

Support for Memory Abstractions

Virtual memory allows the creation of higher-level memory abstractions that simplify programming further:

- **Stacks and Heaps:** Managed by the operating system, with automatic expansion.
- **Memory Mapping:** Files can be mapped directly into virtual memory, simplifying file I/O.
- **Shared Libraries:** Load once into memory and mapped into multiple processes without duplication.

Virtual memory enables developers to concentrate on programme logic rather than the intricacies of memory layout. By abstracting the underlying physical memory, it supports essential programming principles such as modularity, recursion, and abstraction, allowing for more organised and scalable software design. The system automatically manages memory allocation and protection, reducing the likelihood of issues like fragmentation and overlapping memory regions. This removes many of the low-level concerns traditionally associated with memory management, leading to cleaner and more reliable code development.

Virtual memory is not merely a memory extension technique—it transforms how processes are executed, protected, and developed. By allowing efficient and controlled access to memory, it enables operating systems to run multiple large and complex processes concurrently, even when physical memory is limited.

The benefits of virtual memory span from enhanced memory utilisation, where systems run only the necessary portions of programs, to supporting vast and non-contiguous address spaces required by modern applications. Its protection mechanisms form the foundation for process security, ensuring that programs cannot tamper with one another, thereby upholding system stability and integrity. For developers, virtual memory simplifies code by removing the need to handle complex memory allocation tasks manually, thus fostering productivity and modularity.

As systems grow in complexity and scale, the role of virtual memory becomes increasingly vital—not only as a performance booster but also as a means to ensure secure, manageable, and scalable execution environments. Understanding its benefits lays the groundwork for exploring more advanced memory management topics, such as shared memory systems, memory-mapped I/O, and hybrid memory models.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	What mechanism allows virtual memory systems to load only the required portions of a programme into physical memory? a) Whole-process swapping b) Demand loading c) Static allocation d) Direct mapping
7	Which feature of virtual memory supports programmes with very large but sparsely used address spaces? a) Direct physical addressing b) Sparse allocation c) Fixed partitioning d) Contiguous allocation
8	How does virtual memory enforce isolation between processes? a) By copying data between processes on each access b) Through shared global address spaces c) By providing each process with its own page table d) Via fixed memory offsets
9	Which development advantage derives from programmes being able to assume they start at address zero? a) Reduced page-fault frequency b) Simplified linking and loading c) Increased physical memory use d) Mandatory manual relocation

10	What is a key benefit of allowing file contents to be mapped directly into a process's address space? a) It eliminates the need for page replacement algorithms b) It simplifies file input/output operations c) It forces all pages to remain in physical memory d) It disables protection bits on those pages
----	--



4. IMPLEMENTATION CONSIDERATIONS

Implementing virtual memory in a modern operating system is not merely a theoretical exercise. It involves the careful coordination of both hardware capabilities and software strategies. The effectiveness of virtual memory hinges upon the successful integration of specialised hardware components such as the memory management unit (MMU), translation lookaside buffer (TLB), and various protection mechanisms. On the software side, the operating system plays a pivotal role in memory allocation, page replacement decisions, fault handling, and performance optimisation.

Furthermore, understanding the trade-offs involved—between speed, memory consumption, system complexity, and cost—is essential for designing an efficient virtual memory system. This chapter delves into the practical implementation considerations, categorised under hardware requirements, software responsibilities, and performance-related metrics.

4.1. Hardware Support Requirements

Efficient support for virtual memory begins with appropriate hardware design. Hardware plays a critical role in ensuring address translation, access protection, and rapid memory operations, which are central to the operation of a virtual memory system.

Essential Hardware Components

1. Memory Management Unit (MMU): The MMU is a specialised hardware module integrated into the processor that performs real-time translation of virtual addresses into physical addresses. It is the cornerstone of virtual memory systems.

- Takes virtual addresses issued by the CPU and maps them to physical memory locations.
- Relies on page tables maintained by the operating system to perform translations.

2. Translation Lookaside Buffer (TLB): A small, fast, associative cache used to store recently accessed virtual-to-physical address mappings.

- Reduces the latency of address translation.
- Entries include virtual page numbers, physical frame numbers, access permissions, and validity bits.

3. Address Translation Logic: Embedded within the CPU, this logic automates the two-step process:

- Virtual address $\rightarrow$ Page number + offset

- Page number → Physical frame (via page table lookup or TLB)

4. Protection Bits in Page Tables: Each page table entry typically includes control bits that define access rights.

- Read, write, and execute permissions.
- These bits are checked by the MMU during memory access.

5. Trap Handling and Interrupts: When a page fault occurs, the CPU must generate a trap (or interrupt) to transfer control to the operating system.

- The interrupt system must support hardware-level signalling of invalid access or missing pages.

Optional but Recommended Features

- **Support for Multilevel Page Tables:** Enables the system to manage large virtual address spaces efficiently.
- **Cache Management Instructions:** Certain processors offer commands to invalidate or flush cache lines and TLB entries.
- **Support for Huge Pages:** Allows mapping of larger memory pages, reducing overhead and TLB pressure.

The implementation of virtual memory requires strong hardware support, particularly from components such as the Memory Management Unit (MMU), Translation Lookaside Buffer (TLB), and interrupt handling mechanisms. The MMU is responsible for translating virtual addresses into physical ones, while the TLB improves performance by caching recent translations to reduce lookup time. Interrupt support is essential for managing page faults, allowing the operating system to intervene when a page is not present in memory. Page tables play a central role in this system and must include access control bits to enforce protection rules, such as read or write permissions. The overall performance of virtual memory systems is closely tied to the speed and efficiency of these hardware components, making their design and integration critical to the system's effectiveness.

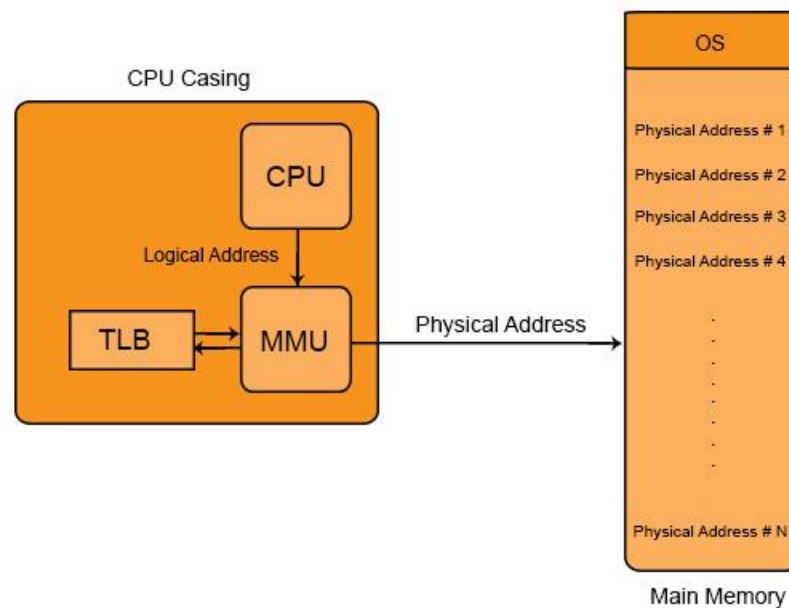


Figure 6: Address Translation with TLB

Source: <https://cstaleem.com/wp-content/uploads/2020/05/MMU.jpg>

The figure presents a labelled diagram illustrating the components involved in the address translation process, including the CPU, Memory Management Unit (MMU), Translation Lookaside Buffer (TLB), and main memory. The CPU generates a 'Logical Address', which is then passed to the MMU. The MMU is responsible for converting this logical address into a 'Physical Address'. To expedite this conversion, the MMU first consults the TLB, a high-speed cache that stores recently used address translations. If the translation is found in the TLB, the physical address is quickly resolved. If not, the MMU accesses the page table (not explicitly shown as a separate block but implicitly part of the MMU's interaction with main memory) to perform the translation, before finally accessing the desired location in 'Main Memory'.

4.2. Operating System Responsibilities

The operating system serves as the primary manager of the virtual memory system. It must not only initialise and maintain memory mappings but also handle page faults, optimise memory usage, and enforce protection policies.

1. Page Table Management

- **Creation and Initialisation:** On process creation, the OS sets up an empty page table, mapping virtual addresses to physical memory or indicating non-residency.

- **Context Switching:** During a context switch, the OS must update the MMU to point to the correct page table base address.
- **Multilevel Tables:** In systems using multi-level page tables, the OS manages all levels and allocates them as needed.

2. Page Replacement and Swapping

- The OS monitors memory usage and chooses which pages to evict when space is required.
- Implements algorithms such as LRU (Least Recently Used), FIFO, or Clock to determine which pages to replace.
- Manages a swap space on secondary storage, where evicted pages are stored temporarily.

3. Page Fault Handling

- When a process accesses a page not present in physical memory, the MMU raises a page fault.
- The OS must:
 - Identify the missing page.
 - Retrieve it from disk (swap space or executable file).
 - Allocate a frame in physical memory.
 - Update the page table and resume process execution.

4. Protection and Access Control

- The OS sets protection bits in page table entries based on process privileges.
- Enforces policies to prevent one process from reading or modifying another's memory.
- Detects and handles illegal memory accesses through fault traps.

5. Memory Allocation

- Allocates physical frames to processes and kernel subsystems.
- Manages free frame lists or frame allocation algorithms to ensure fairness and efficiency.
- Supports dynamic memory allocation via heap and stack growth.

6. Copy-on-Write (COW)

- A technique used when a process is forked.
- Shared pages are marked read-only; copies are made only when one of the processes attempts to modify them.

The operating system is responsible for managing the relationship between physical and virtual memory, ensuring that processes have the necessary memory resources while maintaining system stability and security. It handles tasks such as maintaining page tables, responding to page faults by retrieving the required pages from secondary storage, and enforcing protection rules to prevent unauthorised access. The operating system also oversees memory allocation and deallocation, deciding which pages to keep in physical memory and which to swap out. The effectiveness of these decisions, particularly in page replacement and allocation strategies, has a direct impact on overall system performance, influencing both responsiveness and resource utilisation.

4.3. Performance Metrics and Trade-offs

While virtual memory offers significant benefits, it introduces several performance considerations. The efficiency of the system is determined by how well it balances memory usage, speed, and complexity.

1. TLB Hit Ratio

- The percentage of address translations found in the TLB.
- A high hit ratio reduces access time significantly.
- Influenced by page size, process behaviour, and TLB capacity.

Trade-off:

- Larger TLBs reduce misses but consume more chip area and power.

2. Page Fault Rate

- Indicates how often page faults occur during execution.
- High fault rates degrade performance due to frequent disk access.

Optimisation Techniques:

- Prepaging: Loading a few additional pages that are likely to be used.
- Working Set Model: Maintaining only the actively used pages in memory.

3. Memory Access Time

Total access time includes:

- Time to search TLB.
- Time to access page table (if needed).

- Time to fetch page from disk (in case of page fault).

Average Memory Access Time (AMAT) Formula:

$$\text{AMAT} = (\text{TLB Hit Ratio} \times \text{TLB Access Time}) +$$

$$(\text{TLB Miss Ratio} \times (\text{Page Table Lookup Time} + \text{Memory Access Time})) +$$

$$(\text{Page Fault Rate} \times \text{Disk I/O Time})$$

4. Swap Space Usage

- Excessive swapping can lead to thrashing, where the CPU spends more time swapping pages in and out than executing instructions.
- Must be carefully managed using:
 - Fair allocation of physical frames.
 - Smart page replacement policies.

5. Overhead vs Simplicity

- Multilevel paging reduces memory wastage but increases lookup overhead.
- Larger pages reduce page table size but may cause internal fragmentation.

Table 3: Trade-offs Summary

Feature	Benefit	Trade-off
Larger Page Size	Fewer TLB misses	Higher internal fragmentation
Multi-level Paging	Supports large address space	Slower access, more complex
High TLB Hit Rate	Faster access	Requires intelligent TLB management
Copy-on-Write	Saves memory on fork	Extra page faults on write

Implementing virtual memory is a complex interplay of hardware design, operating system logic, and performance optimisation. Hardware must offer essential features such as address translation, TLB support, and trap handling. The operating system is responsible for translating these hardware capabilities into usable abstractions, enforcing protection, managing page tables, handling faults, and optimising memory usage.

The performance of a virtual memory system is not solely defined by its raw speed but by how effectively it manages the balance between resource usage and process demands. Proper evaluation of metrics such as TLB hit ratio, page fault rate, and swap activity allows designers to tune the system appropriately. By carefully designing both hardware and software aspects and analysing the

associated trade-offs, systems can achieve high efficiency, robust process isolation, and seamless scalability—making virtual memory a cornerstone of modern computing.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Which hardware component is primarily responsible for translating virtual addresses into physical addresses in real time? a) Translation Lookaside Buffer b) Memory Management Unit c) Central Processing Unit d) Cache Controller
12	Which optional hardware feature helps reduce translation lookaside buffer pressure by using larger page mappings? a) Multilevel page tables b) Huge pages c) Cache management instructions d) Trap handling support
13	Within the operating system's virtual memory management, which responsibility involves deciding which pages to evict when no free frames are available? a) Page table initialisation b) Context switching c) Page replacement d) Protection enforcement
14	What performance metric reflects the percentage of address translations found within the translation lookaside buffer? a) Page fault rate b) Working set size c) TLB hit ratio d) Swap-space usage

15	Which trade-off is associated with using larger page sizes in a virtual memory system? a) Increased TLB misses and reduced internal fragmentation b) Fewer TLB misses but higher internal fragmentation c) Faster page-fault handling but lower protection granularity d) Lower power consumption but greater lookup overhead
----	---



5. SUMMARY

- Demand Paging enables memory efficiency by loading pages only when required, reducing the need to keep the entire process in physical memory. This mechanism relies on detecting page faults, retrieving the missing page from secondary storage, and updating the page table accordingly.
- Page faults trigger a sequence of operations that involve trap handling, page retrieval, and memory updates. The OS responds to these faults by allocating a free frame, reading the required page from disk (or swap space), and resuming the interrupted process.
- The valid-invalid bit is used to mark whether a page is present in physical memory or not. This control mechanism, along with protection bits, prevents unauthorised access and enforces process-level memory isolation.
- Swap space serves as an overflow area for evicted pages, allowing memory to be reused dynamically. The OS manages this space to store non-resident pages, using it as a temporary holding area during page replacement or demand fetches.
- Demand paging introduces I/O overhead due to frequent disk access, which can impact system responsiveness. Excessive page faults or poor locality of reference may result in thrashing, severely degrading system performance.
- Optimisations such as prepaging and clustering help reduce page fault rates. Prepaging anticipates future page accesses by loading adjacent pages, while clustering groups related pages to improve locality and reduce I/O operations.
- Virtual memory significantly improves memory utilisation by allowing processes to use more memory than physically available. It enables overcommitment and sharing, ensuring better usage of RAM across multiple processes.
- Large and sparse address spaces are supported by virtual memory through techniques like demand allocation and multi-level page tables. This allows efficient handling of modern applications with dynamic memory requirements.
- Hardware components such as the MMU and TLB are critical for implementing virtual memory. The MMU performs address translation, while the TLB accelerates it by caching recent translations. Page table entries include protection bits to enforce security.

- Operating system responsibilities include managing page tables, handling faults, enforcing protection, and optimising memory usage. Key performance metrics such as TLB hit ratio, page fault rate, and swap space usage help evaluate and refine virtual memory efficiency.



6. GLOSSARY

Demand Paging	A memory management scheme where pages are loaded into physical memory only when they are needed during program execution, rather than loading the entire process at once.
Page Fault	An event triggered when a process tries to access a page not currently present in physical memory, causing the operating system to fetch the page from secondary storage and update the page table.
Valid-Invalid Bit	A bit in each page table entry that indicates whether the corresponding page is currently in physical memory (valid) or not (invalid). It helps prevent illegal memory access and supports demand paging.
Swap Space	A designated area on secondary storage (usually a hard disk or SSD) used by the operating system to temporarily store pages that have been evicted from physical memory.
Memory Management Unit (MMU)	A specialised hardware component that translates virtual addresses generated by the CPU into physical addresses used in RAM, enforcing access control and coordinating with the page table.
Translation Lookaside Buffer (TLB)	A small, high-speed cache that stores recent virtual-to-physical address translations to reduce the time needed for address resolution and improve memory access performance.
Copy-on-Write (COW)	A technique used during process duplication (e.g., fork) where parent and child processes share pages marked as read-only until one attempts a write, triggering the OS to create a separate copy.
Prepaging	An optimisation strategy where the operating system loads additional pages in advance, based on the assumption that nearby pages will likely be used soon, thereby reducing page faults.
Page Replacement Algorithm	A method used by the operating system to decide which memory pages to swap out when free frames are unavailable. Common algorithms include Least Recently Used (LRU), FIFO, and Clock.
Thrashing	A severe performance condition where excessive page faults cause the system to spend more time swapping pages in and out of memory than executing actual processes, leading to reduced throughput.

7. TERMINAL QUESTIONS

1. Explain the mechanism of demand paging and describe the steps involved when a page fault occurs.
2. Discuss the role of the valid-invalid bit in memory protection. How does it assist in enforcing access control and detecting illegal memory access?
3. Describe the structure and purpose of swap space. How does the operating system use swap space to manage virtual memory efficiently during page replacement?
4. What are the primary sources of I/O overhead in demand paging, and how do they impact overall system performance? Suggest methods to mitigate these overheads.
5. Compare and contrast the techniques of prepaging and clustering. How do these optimisations improve the effectiveness of virtual memory systems?
6. List and explain at least three benefits of virtual memory. How do these benefits support scalability, program development, and resource management in modern systems?
7. Identify and explain the hardware components essential for supporting virtual memory. How do the MMU and TLB work together during address translation?
8. Outline the key responsibilities of the operating system in managing virtual memory. How does the OS handle page replacement, protection, and memory allocation?
9. Define the term 'TLB hit ratio'. How does this metric influence memory access time, and what trade-offs are involved in TLB design?
10. Evaluate the trade-offs between performance, complexity, and memory usage in implementing virtual memory. Use examples such as multilevel paging, large page sizes, and copy-on-write to support your explanation.

8. ANSWERS

Self-Assessment Questions

1. b) It allows processes to begin execution with only a subset of their pages in memory
2. c) Accessing a page not present in physical memory
3. c) The page is not currently in physical memory
4. b) Writing a page from RAM to swap space to free a frame
5. b) To load multiple pages at once based on spatial locality
6. b) Demand loading
7. b) Sparse allocation
8. c) By providing each process with its own page table
9. b) Simplified linking and loading
10. b) It simplifies file input/output operations
11. b) Memory Management Unit
12. b) Huge pages
13. c) Page replacement
14. c) TLB hit ratio
15. b) Fewer TLB misses but higher internal fragmentation

Terminal Questions Answers

Answers 1: Demand paging is a technique whereby pages are loaded into physical memory only when they are actually needed by a running process. When a process accesses a page not present in RAM, a page fault occurs; the operating system traps to the fault handler, locates the page on disk, allocates or frees a frame as necessary, reads the page into that frame, updates the page table, and then resumes the interrupted instruction. This mechanism minimises initial load time and memory usage by avoiding unnecessary page transfers. Refer section 2.1 in the document for more details.

Answers 2: The valid-invalid bit in each page table entry indicates whether the corresponding page resides in physical memory (valid) or not (invalid). On each memory access, the hardware checks this bit: if it is invalid, a page fault is raised, preventing access to non resident or unauthorised pages and thereby enforcing basic memory protection. Refer section 2.2 in the document for more details.

Answers 3: Swap space is a reserved area on secondary storage used to hold pages evicted from RAM when physical frames are full. During page replacement, the operating system may write modified

pages out to swap space (swap out) and later read them back into memory (swap in) as needed, thus extending effective memory capacity. Refer section 2.3 in the document for more details.

Answers 4: The principal sources of I/O overhead in demand paging are disk latency (seek and rotational delays), frequent page table updates, and context switches while waiting for page transfers. These delays can cause the CPU to idle or switch tasks excessively, degrading throughput, especially under poor locality. Mitigations include increasing physical RAM, using efficient page replacement policies (such as LRU), and organising data to improve spatial and temporal locality. Refer section 2.4 in the document for more details.

Answers 5: Prepaging loads a block of pages in anticipation of subsequent accesses, reducing future page faults by exploiting spatial locality, whereas clustering groups and fetches logically related pages—often contiguous on disk—as a unit. Both techniques aim to amortise disk access overhead by performing larger, more efficient transfers and by prefetching likely needed pages. Refer section 2.5 in the document for more details.

Answers 6: Virtual memory improves utilisation by loading only active pages (demand paging), supports very large or sparsely used address spaces via selective mapping, and enforces isolation and controlled sharing through per process page tables with protection bits. These benefits allow systems to run larger applications concurrently, simplify memory management for developers, and enhance security and stability. Refer sections 3.1–3.4 in the document for more details.

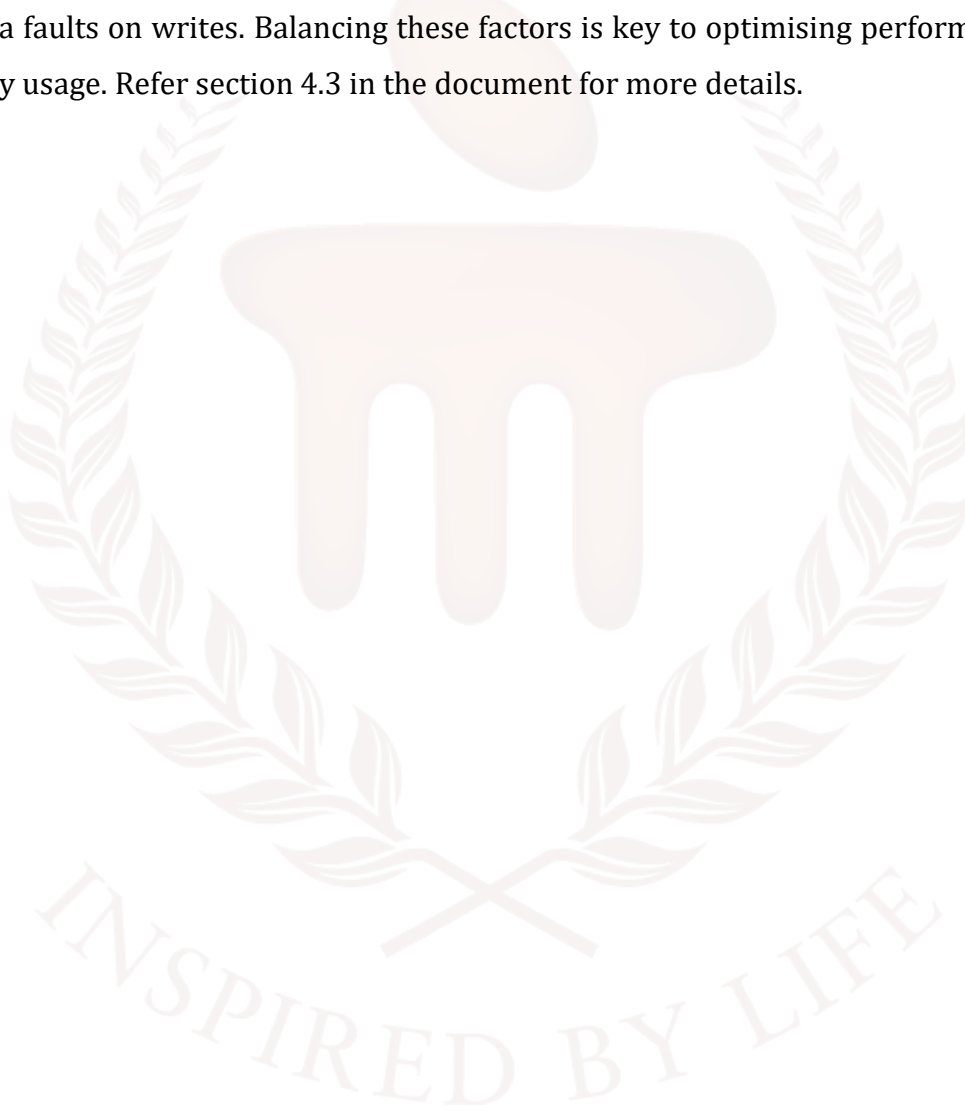
Answers 7: The Memory Management Unit (MMU) performs the translation of virtual addresses to physical addresses using the page table, while the Translation Lookaside Buffer (TLB) caches recent virtual to physical mappings to accelerate this translation. On a TLB miss, the MMU walks the page table or consults memory to resolve the address, then updates the TLB for future accesses. Refer section 4.1 in the document for more details.

Answers 8: The operating system is responsible for creating and maintaining page tables, handling page faults (identifying, loading, and mapping pages), managing swap space, enforcing protection via control bits, allocating and freeing frames, and implementing page replacement algorithms. These tasks ensure efficient memory use, process isolation, and system stability. Refer section 4.2 in the document for more details.

Answers 9: The TLB hit ratio is the fraction of memory references for which the required translation is found in the TLB. A high hit ratio dramatically reduces average memory access time, but enlarging

the TLB to improve hits increases hardware cost and power consumption. Refer section 4.3 in the document for more details.

Answers 10: Implementing virtual memory involves trade offs such as using multilevel paging, which reduces page table size at the cost of longer lookup times; larger page sizes, which lower TLB misses but increase internal fragmentation; and copy on write, which saves memory during forks but can trigger extra faults on writes. Balancing these factors is key to optimising performance, complexity, and memory usage. Refer section 4.3 in the document for more details.



9. REFERENCES

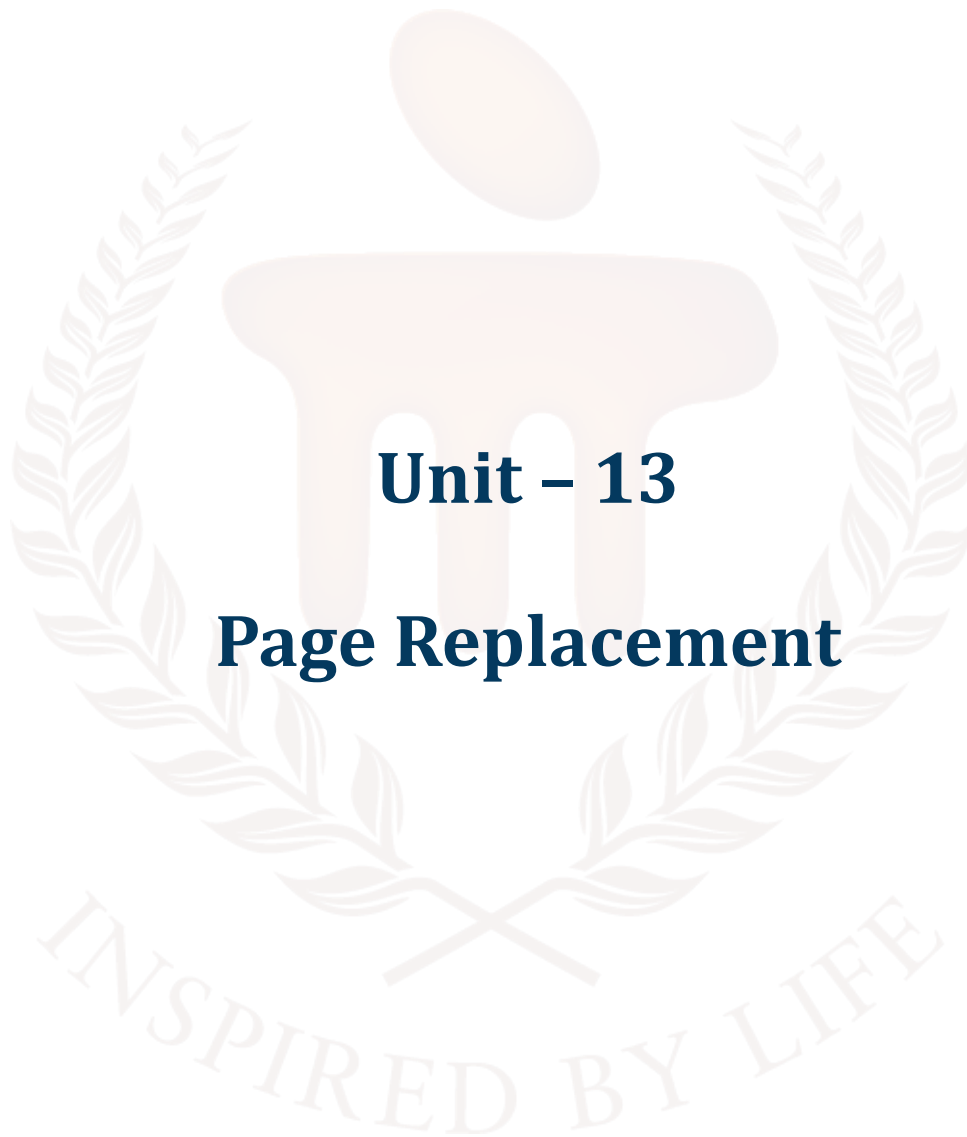
1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Hoboken, NJ: John Wiley & Sons.
2. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Harlow, England: Pearson Education.
3. Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Boston, MA: Pearson.
4. Bovet, D. P., & Cesati, M. (2005). Understanding the Linux Kernel (3rd ed.). Sebastopol, CA: O'Reilly Media.
5. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces (Version 1.00). Arpaci-Dusseau Books.

BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 13

Page Replacement

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4 - 5
1.1	Objectives	-	-	
2	Page Replacement	-	-	6 - 13
2.1	Page Fault and Replacement Need	1	-	
2.2	Frame Allocation Strategies	-	-	
2.3	Locality of Reference and Working Set	2	-	
2.4	Performance Metrics (Fault Rate, Throughput)	3	-	
2.5	Impact on System Performance	-	1	
3	Page-replacement Algorithms	-	-	14 - 23
3.1	First-In First-Out (FIFO)	4	-	
3.2	Optimal (Belady's) Algorithm	5	-	
3.3	Least Recently Used (LRU)	6	-	
3.4	Clock (Second-Chance)	7	-	
3.5	LFU and Other Variants	-	2	
4	Implementation and Performance Considerations	-	-	24 - 32
4.1	Algorithm Overhead and Data Structures	-	-	
4.2	Belady's Anomaly	8	-	
4.3	Thrashing and Its Mitigation	-	-	
4.4	Hardware versus Software Support	-	3	
5	Summary	-	-	33 - 34
6	Glossary	-	-	35
7	Terminal Questions	-	-	36
8	Answers	-	-	37 - 38
9	References	-	-	39

1. INTRODUCTION

In the previous chapter, we studied the concept of virtual memory, focusing on demand paging and its benefits. The idea of loading pages into memory only when they are required helps conserve physical memory and improves system efficiency. We also examined how virtual memory enables the execution of large programs by providing the illusion of a much larger memory space, allowing systems to run more processes concurrently with better isolation and control.

This chapter builds upon that foundation by introducing the concept of page replacement, which becomes essential when the number of pages needed exceeds the available physical frames. In such situations, the system must decide which pages to remove to make space for new ones. This decision directly affects system performance and efficiency. We explore the need for page replacement and how it fits into the broader memory management framework.

The core focus of this chapter is on the algorithms used to select pages for replacement. These include First-In-First-Out (FIFO), Least Recently Used (LRU), Optimal Page Replacement, and others. Each method has its own advantages and trade-offs, and their performance can vary based on workload patterns. The chapter also covers how these algorithms are evaluated using metrics like page fault rate and how they relate to the principle of locality.

To study this chapter effectively, begin by revisiting the basics of demand paging and how page faults occur. Then examine each algorithm individually, focusing on how it selects pages for replacement and under what conditions it performs best. Use diagrams and examples to track the decision-making process of each strategy. Practice solving problems that require calculation of page faults using different algorithms, as this will reinforce conceptual understanding through practical application.

1.1. Objectives

After studying this unit, you should be able to:

- Identify the purpose and need for page replacement in virtual memory systems.
- Compare various page-replacement algorithms and their decision-making logic.
- Apply page-replacement algorithms to compute page fault counts for given reference strings.
- Evaluate the performance of different algorithms based on workload patterns.
- Select appropriate page-replacement strategies for specific system requirements.



2. PAGE REPLACEMENT

In a virtual memory system, the illusion of a large, continuous address space is created by allowing processes to run even when not all of their pages reside in physical memory. When a program references a page that has been moved to secondary storage, the hardware generates a page fault. The operating system then steps in to resolve the fault by loading the required page into a free frame. If no free frames are available, a resident page must be selected for eviction and written back to disk if it has been modified. This cycle of detecting faults, choosing victims and transferring pages between memory and storage is at the heart of page replacement.

Effective page replacement is vital to maintaining system performance. Poorly chosen replacement policies can lead to frequent faults, stalling the CPU as it waits on disk operations, and may even trigger thrashing, where the system spends more time swapping pages than executing actual work. Conversely, carefully designed algorithms can anticipate usage patterns, honour the principles of temporal and spatial locality, and allocate frames in a manner that keeps active working sets resident in memory. The following subsections explore the causes of page faults, the strategies for frame allocation, the role of locality and working sets, and the metrics by which replacement policies are judged.

2.1. Page Fault and Replacement Need

When a process requests a page that is not currently resident in physical memory, the hardware raises a page fault. This mechanism enables a program to execute as though it has access to an address space larger than the available physical memory. The operating system traps this fault, checks whether the access is legitimate, and then arranges to bring the required page into memory from secondary storage.

A page fault triggers several steps:

- The CPU issues a fault interrupt.
- The operating system examines the faulting address to confirm that the reference is valid.
- If a free frame exists, it is assigned to the faulting page.
- If no free frame is available, the system must select a victim page to evict.
- Should the victim page have been modified since it was loaded, the OS writes it back to disk.
- The requested page is loaded into the now available frame.
- Page tables and relevant registers are updated to reflect the change.

- Control returns to the user process at the point of interruption.

While this process appears straightforward, the cost associated with page faults can be considerable. Disk access times vastly exceed main memory access times, so frequent faults can degrade performance severely. Consequently, one of the primary goals of page replacement algorithms is to minimise the number of page faults without imposing undue overhead on the system.

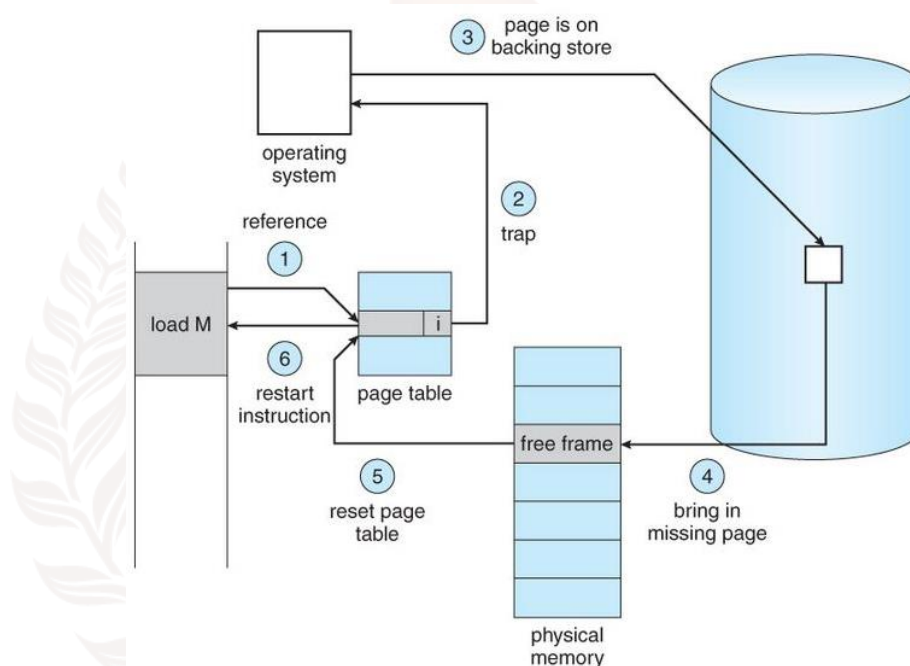


Figure 1: Page Fault Handling Process

Source: <https://media.geeksforgeeks.org/wp-content/uploads/121-1.png>

The figure illustrates the sequence of events that unfold when a page fault occurs and the subsequent process of bringing the required page into physical memory. Initially, a program attempts to 'load M' (step 1), generating a 'reference' to a page that is not present in the 'page table'. This absence triggers a 'trap' to the 'operating system' (step 2). The operating system determines that the 'page is on backing store' (step 3), indicating it resides in secondary memory. It then proceeds to 'bring in missing page' (step 4) from the backing store into a 'free frame' in 'physical memory'. Following this, the 'page table' is 'reset' (step 5) to reflect the new page-to-frame mapping. Finally, the interrupted instruction is 'restart instruction' (step 6), allowing the program to continue execution.

2.2. Frame Allocation Strategies

Frame allocation determines how many physical frames are assigned to each process. The chosen strategy directly affects paging behaviour and overall system performance.

Equal Allocation - Each process receives the same fixed number of frames. This approach is simple to implement and enforces a form of fairness, yet it fails to account for varying process sizes and working sets. A large process may suffer excessive faults, while a small one might waste frames.

Proportional Allocation - Frames are divided among processes in proportion to their allocated virtual pages. If a process holds 25 per cent of the total virtual pages in the system, it receives 25 per cent of the available frames. This strategy better matches allocation to demand but requires knowledge of each process's size and may still misjudge working set requirements.

Priority Based Allocation - Processes are assigned priorities according to their importance or urgency. Higher priority processes may receive a greater share of frames, ensuring they run smoothly. However, this can lead to starvation of lower priority processes if resources remain perpetually skewed.

Global versus Local Allocation

- **Global Allocation** permits any process to select a replacement victim from the entire pool of frames. This flexibility can reduce overall page faults, but one process's behaviour may adversely affect another's performance.
- **Local Allocation** confines page replacement candidates to the frames already allocated to the faulting process. While this isolates the impact of replacement decisions, it may cause high fault rates when a process's working set grows suddenly.

Points to consider when choosing an allocation strategy:

- Variability in process working set sizes.
- System policy on fairness versus throughput.
- Overhead of tracking process demands.

2.3. Locality of Reference and Working Set

The effectiveness of any replacement algorithm hinges on the locality principle. Programs exhibit two primary kinds of locality:

- **Temporal Locality:** An instruction or data item accessed recently is likely to be accessed again in the near future.
- **Spatial Locality:** Items whose addresses are close to recently accessed addresses tend to be referenced soon.

To harness these tendencies, the concept of a working set was introduced. The working set of a process at time t , denoted $W(t, \Delta)$, comprises the pages referenced during the interval $[t - \Delta, t]$. A suitable choice of Δ captures the pages actively in use, helping the operating system to decide how many frames to allocate so that page faults are minimised.

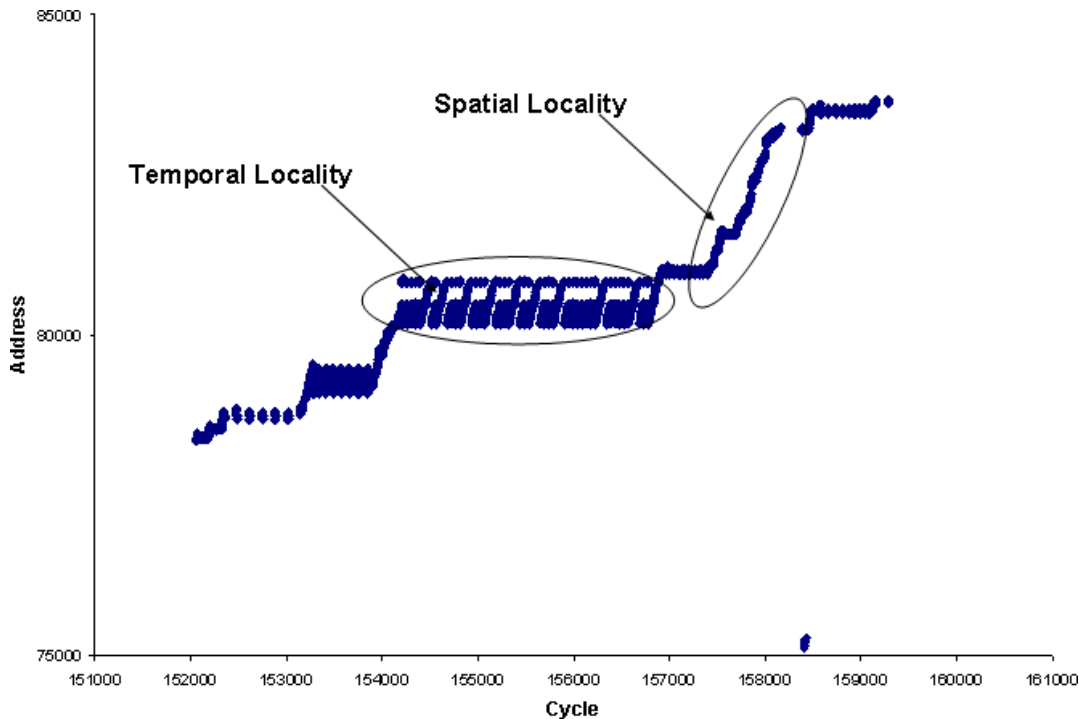


Figure 2: Locality of Reference

Source: <https://i.sstatic.net/uwMoy.gif>

The figure, "Locality of Reference", illustrates two key principles of memory access patterns: temporal locality and spatial locality. The graph plots memory 'Address' accessed over time, represented by 'Cycle' numbers. 'Temporal Locality' is evidenced by the clustered access to a relatively small range of addresses over a period, indicating that recently accessed items are likely to be accessed again soon. This is shown by the horizontal spread of accesses within a confined address range. 'Spatial Locality' is demonstrated by accesses to addresses that are numerically close to each other, often immediately after one another, suggesting that if a memory location is accessed, nearby locations are likely to be accessed next. This is visible where the address rapidly increases, indicating movement through contiguous or near-contiguous memory blocks.

Key observations about working sets:

- The working set size can grow or shrink dynamically as a process enters different phases of execution.
- Underestimation of the working set size leads to frequent page faults, known as thrashing.

- Overestimation wastes physical memory on pages that are seldom used.

Operating systems often track reference bits at regular intervals to approximate a process's working set without imposing excessive overhead. An efficient approach is the windowed clock algorithm, which maintains a circular buffer of pages and their recent usage history to simulate the working set with minimal complexity. By analysing this information, the operating system can adjust frame allocations dynamically—granting additional frames as the working set grows and reclaiming them when it shrinks—thereby optimising memory use and reducing page fault rates.

2.4. Performance Metrics (Fault Rate, Throughput)

To compare and tune replacement strategies, precise metrics are necessary.

Page Fault Rate - Calculated as the ratio of page faults to total memory references. A lower rate signifies better performance, though it must be interpreted alongside other metrics.

Throughput - Denotes the number of processes completed per unit time. An algorithm that reduces page faults but introduces excessive computational overhead may actually lower throughput.

CPU Utilisation - Indicates the proportion of time the CPU spends executing user or kernel code as opposed to waiting for I/O. Frequent page faults reduce CPU utilisation.

Response Time - The interval from when a request is submitted until the result is produced. Systems serving interactive workloads require low response times; excessive paging can cause user visible delays.

Thrashing - Occurs when the system spends more time swapping pages in and out than executing processes. Thrashing can be detected when a rise in the degree of multiprogramming causes a fall in throughput.

Table 1: Effects of Page Fault Rate on Key System Performance Metrics

Metric	Influence of High Fault Rate	Influence of Low Fault Rate
Throughput	Tends to decrease	Tends to increase
CPU Utilisation	Substantially reduced	Near optimal
Response Time	Significantly prolonged	Quickly responds to requests
Risk of Thrashing	Very high	Very low

When evaluating page replacement algorithms, it is important to strike a balance between reducing the fault rate and the computational overhead introduced by the algorithm itself. The chosen method must also be appropriate for the intended workload—whether batch, interactive or real-time—and capable of delivering consistent performance under those conditions. Finally, the algorithm should scale effectively as the number of processes grows and as address spaces become larger, maintaining efficiency without imposing prohibitive resource demands.

2.5. Impact on System Performance

The choice of replacement algorithm and allocation policy has wide ranging effects on overall system behaviour.

Inefficient page replacement can severely disrupt system operation. When page fault rates rise, processes are halted frequently while pages are fetched from disk, and valuable CPU cycles are consumed in fault handling instead of productive execution. At the same time, heavy disk I/O can contend with other input/output-bound tasks, further delaying system activities. As a result, overall throughput falls and user processes experience erratic delays, undermining both responsiveness and reliability.

By contrast, a well-designed replacement algorithm keeps fault rates low for typical access patterns while adding little computational or memory overhead. It adjusts automatically to changes in working set size, reducing the need for manual tuning, and allocates frames in a manner that honours fairness policies unless system priorities demand otherwise. Such adaptability and efficiency ensure that processes run smoothly, maximising resource utilisation and maintaining consistent performance even under varying workloads.

Common page replacement algorithms illustrate these trade offs:

1. **First In, First Out (FIFO)** - Simple implementation but prone to remove frequently used pages if they arrived early.
2. **Least Recently Used (LRU)** - Approximates ideal behaviour by keeping track of recency. More complex, requiring hardware or software support.
3. **Optimal (Belady's Algorithm)** - Evicts the page not needed for the longest future interval. Unimplementable in general but serves as a benchmark.
4. **Clock (Second Chance)** - Uses a circular buffer and a reference bit to give pages a second chance before eviction.

Points on hybrid and adaptive methods:

- **Adaptive Replacement Cache (ARC):** Balances between recency and frequency by maintaining two lists.
- **Working Set Clock:** Integrates working set tracking into a clock algorithm to approximate ideal allocation.

Page replacement lies at the heart of virtual memory management. Fault handling, frame allocation, exploitation of locality through working sets, and rigorous performance evaluation together determine the efficiency and responsiveness of an operating system. By selecting appropriate strategies and fine tuning their parameters, system designers can minimise page faults, maintain high throughput and avoid the pitfalls of thrashing. Diagrams such as flowcharts for fault processing, graphs for performance comparison and charts for allocation strategies serve to clarify these multifaceted concepts in a textbook setting.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	What is the very first step the operating system takes when handling a page fault? a) Assign a free frame to the faulting page b) Examine the faulting address to confirm the reference is valid c) Select a victim page for eviction d) Load the requested page from secondary storage
2	Which frame allocation strategy divides frames among processes in proportion to their virtual page count? a) Equal Allocation b) Priority Based Allocation c) Proportional Allocation d) Local Allocation
3	Which principle of locality is demonstrated when items whose addresses are numerically close are accessed in sequence? a) Temporal Locality b) Spatial Locality c) Working Set Locality d) Reference Locality

4	Which performance metric measures the proportion of time the CPU spends executing user or kernel code rather than waiting for I/O? a) Throughput b) CPU Utilisation c) Response Time d) Page Fault Rate
5	Which page replacement algorithm employs a circular buffer and a reference bit to grant pages a second chance before eviction? a) First In, First Out b) Clock (Second Chance) c) Least Recently Used d) Optimal (Belady's Algorithm)



3. PAGE-REPLACEMENT ALGORITHMS

Once the system must evict a page to bring in a new one, the choice of which page to remove becomes critical. A poor decision can lead to frequent page faults, excessive disk I/O and ultimately degraded performance, whereas an informed choice minimises faults and optimises throughput. Various page-replacement algorithms have been developed, each with its own strengths, weaknesses and implementation complexity. This chapter examines five core strategies: First-In First-Out (FIFO), Optimal (Belady's) Algorithm, Least Recently Used (LRU), Clock (Second-Chance), and LFU alongside other notable variants.

Understanding these algorithms requires considering both theoretical performance and practical constraints. While some approaches offer ideal behaviour under perfect foresight, others approximate that ideal in ways that are implementable with reasonable overhead. Factors such as hardware support, additional bookkeeping, and sensitivity to workload characteristics play a decisive role in selecting an appropriate algorithm for a given operating system.

3.1. First-In First-Out (FIFO)

The First-In First-Out algorithm is perhaps the simplest page-replacement strategy. Frames are organised in a queue, and when replacement is required, the page that has been in memory the longest is chosen for eviction.

Working Principle

When a page fault occurs and no free frame is available, the oldest loaded page is removed:

1. A pointer (often called the “victim pointer”) identifies the next frame in the queue.
2. The pointed-to page is evicted, regardless of how frequently or recently it was accessed.
3. The new page is loaded into the vacated frame, and the pointer advances to the next frame in cyclic order.

First in, first out (FIFO) is prized for its straightforwardness. Implementation requires only a circular queue or a simple index pointer to track the order in which pages were loaded. Because it does not maintain any additional history of page usage beyond insertion order, overhead remains minimal, making FIFO an attractive choice for systems with limited resources or where simplicity is paramount.

Despite its ease of use, FIFO overlooks the principle of locality. A page that is accessed frequently may nonetheless be evicted solely because it was among the earliest to enter memory, leading to

unnecessary faults. Furthermore, FIFO is subject to Belady's anomaly, whereby increasing the number of available frames can, counterintuitively, increase the total number of page faults under certain access patterns. This peculiarity underscores that simplicity does not always equate to optimal performance.

Performance Considerations

- FIFO's performance varies widely with workload. Simple or uniform access streams may yield acceptable results, but more complex patterns often expose FIFO's naivety.
- The anomaly makes FIFO unsuitable in systems where predictable scaling is essential.

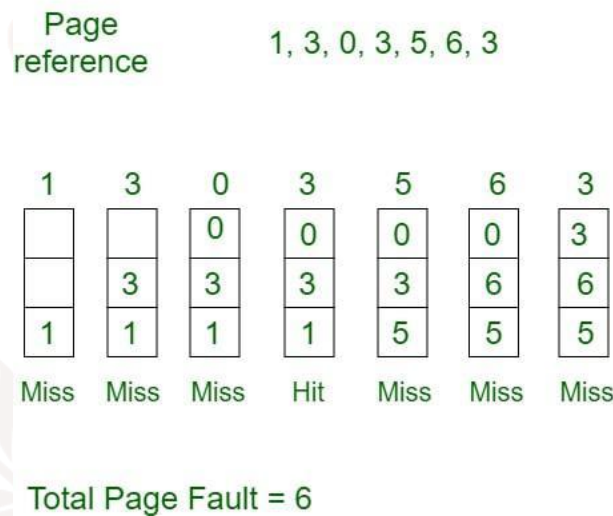


Figure 3: FIFO Page Replacement Algorithm

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20241023184328270600/page1.jpg>

The figure demonstrates the First-In, First-Out (FIFO) page replacement algorithm using a sequence of page references: 1, 3, 0, 3, 5, 6, 3. The diagram shows a memory with three page frames. As each page reference occurs, the algorithm brings the page into memory if it's not already present. For instance, pages 1, 3, and 0 initially result in misses as they fill the frames. When page 3 is referenced again, it's a hit. However, when new pages like 5 and 6 are referenced, they replace the oldest page in memory (1, then 3, then 0), leading to further misses. The illustration clearly indicates each "Miss" and "Hit", culminating in a "Total Page Fault = 6", which reflects the number of times a page had to be brought into memory from secondary storage.

3.2. Optimal (Belady's) Algorithm

Belady's Optimal algorithm defines the lower bound on page-fault rate by making the replacement decision that will incur the fewest future faults. It evicts the page whose next reference lies furthest in the future.

Working Principle

- For each page resident in memory at the time of replacement, determine the time of its next use.
- Evict the page with the greatest next-use distance, since retaining pages used sooner avoids immediate faults.
- In cases where a page will never be used again, it is the obvious candidate for eviction.

Belady's optimal page-replacement algorithm provides an idealised benchmark against which all practical schemes are measured. By evicting the page whose next use is furthest in the future, it achieves the lowest possible fault rate for any given reference string. This clairvoyant approach offers valuable insight into the theoretical limits of paging performance and helps gauge how close real world algorithms come to optimal behaviour.

In practice, however, the optimal algorithm is untenable. It demands perfect knowledge of all future memory references, information that is fundamentally unavailable at runtime. Furthermore, attempting to simulate or predict every forthcoming access for all pages would impose an overwhelming computational burden. As a result, although Belady's optimal method remains an indispensable yardstick, it is relegated to theoretical study rather than direct implementation in operating systems.

Performance Considerations

- While impossible to implement exactly, approximations of optimal behaviour guide design of heuristics.
- Research often compares real-world algorithms against the Optimal curve to quantify their efficiency gap.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3													No. of Page frame - 4	
7	0	1	2	0	3	0	4	2	3	0	3	2	3		
			2	2	2	2	2	2	2	2	2	2	2		
		1	1	1	1	1	4	4	4	4	4	4	4		
	0	0	0	0	0	0	0	0	0	0	0	0	0		
7	7	7	7	7	3	3	3	3	3	3	3	3	3		
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit		
Total Page Fault = 6															

Figure 4: Optimal Page Replacement Algorithm

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20241023184349191529/page2.jpg>

The figure demonstrates the optimal page replacement algorithm using a given page reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3, with a memory capacity of four page frames. This algorithm, which is theoretically optimal but practically unfeasible, works by replacing the page that will not be used for the longest period in the future. The diagram shows the state of the four frames for each page reference, indicating whether it results in a "Miss" or a "Hit". For example, when page '4' is referenced and results in a miss, the algorithm replaces page '7' because '7' is the one that will not be accessed again for the longest duration among the pages currently in memory. The final tally shows a "Total Page Fault = 6".

3.3. Least Recently Used (LRU)

Least Recently Used seeks to approximate the Optimal algorithm by using past behaviour as a proxy for future needs, on the assumption that pages used recently are likely to be used again soon.

Working Principle

- Each page table entry is augmented with a timestamp or counter indicating last access time.
- On a page fault, the OS selects the page with the oldest timestamp (i.e. the one least recently used).
- The new page replaces the victim, and timestamps are updated on every memory reference.

Implementation Techniques

1. Counter Implementation

- Each time a page is referenced, load a global counter value into a slot in its page table entry.
- On replacement, the page with the smallest counter is evicted.

2. Stack Implementation

- Maintain a stack of page numbers. Upon each access, move the referenced page's entry to the top.
- The bottom entry of the stack is the least recently used.

3. Approximation Schemes

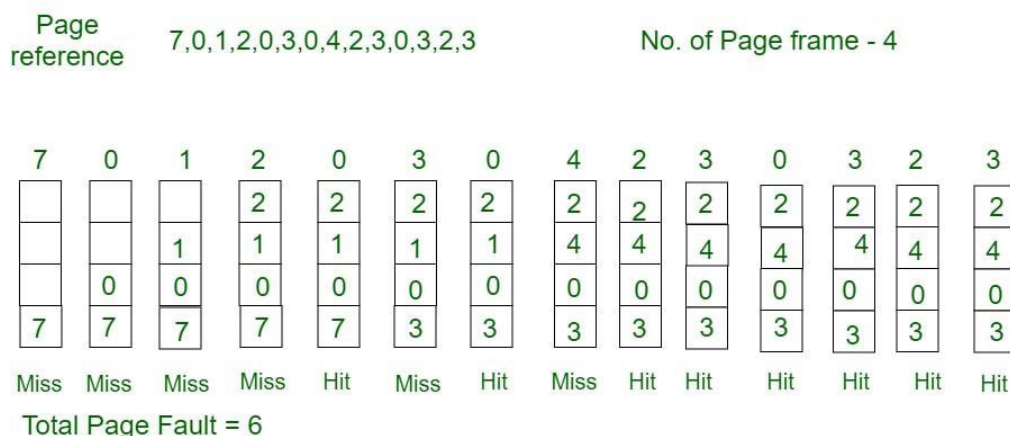
- Clock with Reference Bits: Uses a circular list and a single "use" bit per page.
- Enhanced Second-Chance: Incorporates both reference and dirty bits to refine choices.

Least recently used (LRU) capitalises on the principle of temporal locality by evicting the page that has not been accessed for the longest period. This approach often achieves near optimal performance across a variety of workloads, since it retains the most recently used pages that are more likely to be needed again. Moreover, LRU is free from Belady's anomaly, ensuring that increasing the number of frames never paradoxically raises the page fault count.

Despite its merits, LRU can impose considerable overhead on the system. Every memory reference must update access timestamps or manipulate a linked structure to track recency, which consumes CPU cycles and memory bandwidth. Efficient real time implementations typically depend on hardware support, such as reference bits and specialised registers, to approximate true LRU behaviour without prohibitive software overhead.

Performance Considerations

- LRU is typically the standard against which simpler heuristics are measured.
- Approximate LRU algorithms, such as the Clock, provide a favourable compromise between overhead and fault rate.



Here LRU has same number of page fault as optimal but it may differ according to question.

Figure 5: Least Recently Used Algorithm

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20241023184405860059/page3.jpg>

The figure illustrates the application of the LRU page replacement algorithm with a page reference string of 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3, and a memory capacity of four page frames. This algorithm replaces the page that has not been used for the longest period of time. For each page reference, the diagram shows the current state of the four memory frames and whether the reference results in a "Miss" or a "Hit". For instance, when page '4' is referenced, and a replacement is needed, the algorithm removes page '7' because it was the least recently used among the pages currently in memory. The example concludes with a "Total Page Fault = 6".

3.4. Clock (Second-Chance)

The Clock algorithm offers an efficient approximation to LRU without the full overhead of timestamps or stacks. It organises frames in a circular list, each with a use bit.

Working Principle

- When a page is loaded, its use bit is set to 1.
- The algorithm maintains a pointer (the "hand of the clock") to examine frames in cyclic order.
- On a page fault:
 1. Examine the frame under the pointer.
 2. If its use bit is 0, select it for eviction.
 3. If its use bit is 1, clear the bit and advance the pointer to the next frame.

4. Repeat until a victim is found.

The clock page replacement algorithm maintains a single reference bit for each frame and a rotating pointer that sweeps through frames in search of a victim. Because it requires only one bit per frame and a simple pointer, its overhead is minimal. In practice, clock delivers performance close to that of true LRU: pages that have been referenced recently receive a “second chance” and are not immediately evicted. Furthermore, it does not suffer from Belady’s anomaly, so granting additional frames will not unexpectedly increase the page fault rate.

Nonetheless, clock is only an approximation of LRU and can occasionally give a second chance to pages that were hardly used in a meaningful sense, leading to suboptimal eviction choices. In the worst case—when many pages have their reference bits set— the pointer may circle through numerous frames before finding one to replace, which can introduce latency. Although this behaviour seldom negates clock’s overall efficiency, it does illustrate the trade off between low implementation cost and the precision of recency tracking.

Performance Considerations

- Clock is widely adopted in commercial operating systems due to its simplicity and effectiveness.
- Variants such as Enhanced Clock incorporate the dirty bit to further refine eviction choices and minimise write-backs.

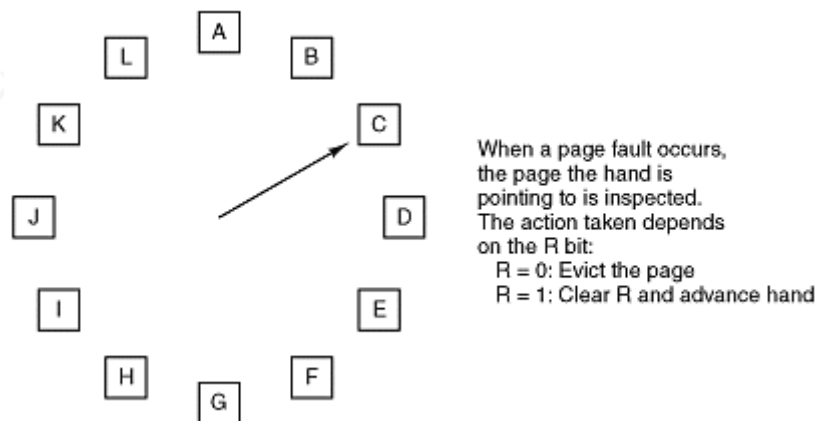


Figure 6: Clock Page Replacement Algorithm

Source: <https://productdevelop.blogspot.com/2010/01/clock-page-replacement-algorithm.html>

The figure illustrates a page replacement algorithm that aims to approximate the Least Recently Used (LRU) algorithm with less overhead. Pages in memory are arranged in a circular list, conceptually

forming a clock face, with a "page hand" pointing to the next page to be inspected. Each page has an associated 'R bit' (reference bit). When a page fault occurs, the algorithm inspects the page pointed to by the hand. If the R bit is 0, the page is 'Evicted' as it has not been recently accessed. If the R bit is 1, it means the page has been recently used; the R bit is then 'Clear[ed] R' to 0, and the hand advances to the next page, giving that page another chance to be referenced. This process continues until a page with an R bit of 0 is found and evicted.

3.5. LFU and Other Variants

While LRU and its approximations focus on recency, other algorithms consider frequency of access. The Least Frequently Used (LFU) strategy evicts pages accessed the fewest times.

LFU (Least Frequently Used)

- Each page maintains a count of how many times it has been referenced.
- On replacement, the page with the lowest count is evicted.
- Counters may be periodically decremented to give ageing, preventing long-lived pages from retaining inflated counts.

Least frequently used (LFU) capitalises on access frequency by favouring pages that have been referenced most often. Over time, this ensures that pages exhibiting long term popularity remain resident in memory, making LFU particularly effective in environments where a small working set is accessed repeatedly. By maintaining counts of page references, the algorithm adapts to stable usage patterns, ensuring that consistently needed pages are not evicted prematurely.

However, LFU can suffer from staleness: a page that was heavily accessed in the distant past may continue to hold a high count even after it is no longer in active use. This can prevent newer but more relevant pages from entering memory. Moreover, the bookkeeping required to update reference counts on every access—and to implement mechanisms for decaying or ageing these counts—introduces additional overhead, both in terms of processing and storage, which may offset the benefits in some workloads.

Other Notable Variants

1. Most Recently Used (MRU)

- Evicts the page most recently used, under the assumption that once accessed, a page may not be needed again soon.
- Useful in certain database access patterns.

2. Hybrid Schemes

- LRU-2: Tracks the time of the second most recent reference, giving preference to pages accessed twice in quick succession.
- LFUDA (LFU with Dynamic Aging): Adjusts counts based on elapsed time to prevent staleness.

3. ARC (Adaptive Replacement Cache)

- Balances recency and frequency by maintaining two queues (recently used and frequently used) and dynamically adjusting their sizes based on observed hits.

4. CAR (Clock with Adaptive Replacement)

- Integrates Clock mechanism with adaptive sizing of recency/frequency lists.

Frequency based methods perform exceptionally well when a stable 'hot set' of pages exists, as they ensure that frequently accessed pages remain resident in memory. However, without ageing techniques to diminish the priority of pages that were popular only in the past, cache pollution can occur, whereby once hot pages continue to occupy frames despite no longer being in active use. Hybrid approaches address this dilemma by blending competing heuristics—such as recency and frequency—so as to combine their respective strengths while compensating for their individual weaknesses.

Page-replacement algorithms form the fulcrum of virtual memory performance. Simplicity, overhead and fault-rate minimisation must be balanced:

- **FIFO** offers ease of implementation but risks Belady's anomaly and poor locality exploitation.
- **Optimal** serves as the unattainable gold standard against which real algorithms are gauged.
- **LRU** approximates optimal behaviour accurately yet incurs substantial metadata management cost.
- **Clock** strikes a practical compromise with minimal overhead and robust performance.
- **LFU** and hybrid variants address long-term frequency patterns and adapt dynamically to changing workloads.

There is no universally best algorithm; instead, operating systems choose the strategy that best aligns with their workload characteristics, hardware capabilities and performance goals. Understanding the trade-offs in each method enables informed design and tuning for reliable, efficient virtual memory management.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which page replacement algorithm can paradoxically exhibit an increase in page faults when the number of frames is increased? a) First In First Out b) Least Recently Used c) Optimal (Belady's) Algorithm d) Clock (Second Chance)
7	What essential information does Belady's Optimal algorithm require to make its replacement decision? a) The time of each page's last use b) The number of times each page has been accessed c) The sequence and timing of all future page references d) The process priority levels
8	In a pure stack implementation of Least Recently Used, how is the page to be evicted identified? a) It is at the top of the stack b) It has the smallest access counter c) It resides at the bottom of the stack d) It has its reference bit cleared
9	What mechanism does the Clock (Second Chance) algorithm use to give pages an opportunity to stay resident? a) It tracks each page's timestamp b) It maintains a count of page accesses c) It uses a single use (reference) bit per frame d) It predicts future references via history
10	Which page replacement strategy evicts the page with the lowest number of references, potentially requiring periodic ageing of counts? a) First In First Out b) Least Frequently Used c) Most Recently Used d) Adaptive Replacement Cache

4. IMPLEMENTATION AND PERFORMANCE CONSIDERATIONS

Designing an effective page-replacement subsystem involves more than merely choosing an algorithm. While the logical correctness of a strategy is essential, its practical viability depends on implementation overhead, susceptibility to pathological behaviours, interaction with hardware support, and the capacity to maintain throughput under varying workloads. This chapter examines four key considerations:

1. **Algorithm Overhead and Data Structures:** The resources and complexity required to track page usage.
2. **Belady's Anomaly:** The counterintuitive situation in which adding memory framing can worsen performance.
3. **Thrashing and Its Mitigation:** How excessive page faults can stall a system, and what policies prevent this.
4. **Hardware versus Software Support:** The division of labour between processor features and operating-system routines.

Together, these topics guide system designers in balancing trade-offs between speed, complexity, and memory efficiency.

4.1. Algorithm Overhead and Data Structures

Page-replacement algorithms differ not only in fault-rate performance but also in the bookkeeping they require. Practical implementations must strike a balance between algorithmic sophistication and the cost of maintaining the information that the algorithm depends upon.

Core Data Structures

- **Counters and Timestamps** - Many algorithms, such as Least Recently Used, rely on tracking either the count of accesses or the time since last use. Counters demand increment operations on each memory reference; timestamps require reading a global clock value.
- **Stacks and Lists** - Stack-based approaches maintain an ordered list of pages, promoting an accessed page to the top. This entails pointer updates or node-relinking on each reference, which can become costly at high reference rates.

- **Circular Buffers** - Clock algorithms use a circular list or array of frames, each augmented with a single reference bit. The “hand” pointer advances until it locates a candidate for eviction, resetting bits en route.
- **Frequency Tables** - Less Frequently Used algorithms depend on tables recording how often each page has been accessed. Without ageing, counts can grow without bound, requiring periodic decay operations to prevent stale pages from dominating.

Time and Space Overhead

A. Per-Reference Cost

- Updates to counters, timestamps or lists on every memory access add CPU overhead.
- For high-performance systems, even a few extra cycles per reference can be significant.

B. Memory Footprint

- Recording metadata for each page consumes RAM.
- In systems with very large virtual address spaces, multi-level tables and large per-page records can strain memory resources.

C. Complexity of Updates

- Simple algorithms like FIFO require constant-time pointer movements, whereas LRU stacks involve moving elements within a linked structure.
- Algorithms requiring global scans (for example, to find the least-referenced page) incur $O(n)$ operations for n frames.

D. Hardware Versus Software Maintenance

- When hardware provides reference bits, the OS can clear them periodically and approximate LRU without full timestamping.
- Absence of such support forces the OS to maintain software tables and perform extra work on each reference.

In designing page replacement algorithms, there is an inherent trade off between accuracy and overhead. Techniques that track page usage with fine granularity tend to reduce page faults, yet each additional check or timestamp update adds to the per reference cost. Conversely, simpler methods such as FIFO or Clock keep bookkeeping minimal but may incur a higher fault rate under complex workloads. As systems scale to thousands of frames, algorithms that perform $O(n)$ scans for each decision become prohibitively expensive; hence, any practical scheme must avoid frequent full table traversals to remain efficient.

Underlying these algorithms are core data structures—counters, stacks and circular buffers—that hold the metadata needed for replacement decisions. Every extra bit of state information consumes memory and translates into CPU cycles during page references. In many cases, hardware support such as reference bits in page table entries or flags in the TLB can offload much of this work from software, shifting the burden into silicon and thereby reducing software overhead without compromising the precision of usage tracking.

4.2. Belady's Anomaly

Belady's Anomaly is the phenomenon where, under certain page-replacement strategies, increasing the number of allocated frames can actually increase the number of page faults. Discovered by László Belády in the late 1960s, the anomaly runs counter to intuitive expectations and highlights the need for careful algorithm selection.

Illustration of the Anomaly

Consider a reference string and two configurations:

- **Reference String:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- **FIFO with 3 Frames:**
 - Frames evolve: [1], [1,2], [1,2,3], [4,2,3], [4,1,3], [4,1,2], [5,1,2], [5,1,2], [5,1,2], [5,1,2], [5,1,2], [5,1,2]
 - Total faults: 9
- **FIFO with 4 Frames:**
 - Frames evolve: [1], [1,2], [1,2,3], [1,2,3,4], [1,2,3,4], [1,2,3,4], [5,2,3,4], [5,1,3,4], [5,1,2,4], [5,1,2,3], [5,1,2,3], [5,1,2,3]
 - Total faults: 10

Thus, allocating an extra frame increased faults from 9 to 10.

Causes of Anomaly

- **Queue Discipline:** FIFO evicts the oldest page irrespective of future use, so larger queues may retain pages that delay eviction of those that should go.
- **Lack of Locality Awareness:** FIFO does not consider recency; it may keep cold pages longer as the queue grows.

Algorithms Immune to the Anomaly

- **Least Recently Used:** By always evicting the page with the oldest access time, LRU avoids paradoxical behaviour.
- **Optimal (Belady's) Algorithm:** Since it minimises future faults exactly, adding frames cannot worsen performance.
- **Clock:** As an approximation to LRU, it typically does not exhibit Belady's Anomaly.

Significance in System Design - Belady's Anomaly emphasises that:

- **More Memory Is Not Always Better:** Simply increasing frame counts without changing the replacement scheme may not yield performance improvements.
- **Algorithm Selection Matters:** Systems requiring predictable scaling should favour anomaly-immune strategies.
- **Analytical Validation:** Designers must simulate algorithms under realistic traces to detect such counterintuitive outcomes.

Belady's anomaly describes the counter intuitive situation in which increasing the number of available frames actually leads to a higher page fault rate. This phenomenon can afflict simple first in, first out schemes and their variants, since they may evict pages that would soon be reused despite having more memory at their disposal. By contrast, anomaly immune algorithms such as least recently used, optimal and clock consistently avoid this issue, ensuring that additional frames cannot worsen performance. In practice, any replacement strategy—whether simple or sophisticated—must be evaluated against a range of representative workloads to confirm that it behaves as expected and does not succumb to pathological cases.

0
1
2

Number of frames = 3

0
1
4
5

Number of frames = 4

As the set of pages in memory with 4 frames is not a subset of memory with 3 frames, the property of stack algorithm fails in FIFO.

Figure 7: Belady's Anomaly

Source: <https://media.geeksforgeeks.org/wp-content/uploads/stackbased.png>

The figure illustrates Belady's Anomaly, a phenomenon where increasing the number of available memory frames can, counter-intuitively, lead to an increase in the number of page faults when using the First-In, First-Out (FIFO) page replacement algorithm. The diagram shows two scenarios side-by-side. The left panel, with "Number of frames = 3", depicts a state where pages 0, 1, and 2 are in memory. The right panel, with "Number of frames = 4", shows a different set of pages (0, 1, 4, 5) present in memory for the same page reference string. As the accompanying text explains, "As the set of pages in memory with 4 frames is not a subset of memory with 3 frames, the property of stack algorithm fails in FIFO," meaning that simply adding more frames does not guarantee fewer page faults, a characteristic unique to certain non-stack algorithms like FIFO.

4.3. Thrashing and Its Mitigation

When the system spends a majority of its time servicing page faults rather than executing instructions, it is said to be thrashing. This condition often arises when the combined working sets of active processes exceed the available physical memory, resulting in continuous page replacement with little forward progress.

Detecting Thrashing

- **High Page Fault Rate:** A sudden spike in faults per second indicates that processes lack sufficient frames to hold their working sets.
- **Low CPU Utilisation:** The processor remains idle while awaiting I/O completion for page loads.
- **Increased I/O Traffic:** Swap-in and swap-out operations dominate disk activity.

Causes of Thrashing

1. **Excessive Multiprogramming:** Too many processes compete for limited memory.
2. **Poor Replacement Policy:** An algorithm that does not respect locality may evict pages prematurely.
3. **Large or Dynamic Working Sets:** Applications with unpredictable or extensive memory footprints may overwhelm static allocations.

Mitigation Strategies

- **Working Set Model**
 - Track each process's working set size.
 - Ensure sum of working sets remains below physical memory.

- If exceeded, reduce multiprogramming by suspending or swapping out processes.
- **Page Fault Frequency (PFF) Control**
 - Define acceptable fault interval thresholds for each process.
 - If a process's fault rate crosses an upper threshold, allocate more frames; if below a lower threshold, reclaim frames.
- **Load Control**
 - Limit the number of processes admitted to execution based on current memory load.
 - Delay admission of new processes when memory pressure is high.
- **Local Replacement Policies**
 - Confine each process to its own frame allocation, preventing runaway demands from displacing other processes' pages.
- **Dynamic Adjustment of Degree of Multiprogramming**
 - Temporarily reduce the number of active processes to restore a working set balance.

Effects of Mitigation

- **Reduced Fault Rate:** Adequate frame allocation lowers the likelihood of evicting pages still in active use.
- **Improved Throughput:** More CPU cycles are spent on computation rather than I/O.
- **Predictable Performance:** Systems become resilient to workload surges by controlling memory admission.

To mitigate thrashing, the operating system must ensure that each process's working set is matched by an adequate number of frames. One approach is to monitor overall fault rates and, when they exceed a threshold, temporarily reduce the degree of multiprogramming by suspending or swapping out whole processes. Alternatively, dynamic frame allocation schemes can redistribute frames from less active processes to those whose working sets have grown, thereby preventing any single process from monopolising resources. By adjusting both process load and frame assignments in real time, the system maintains a balance that avoids the extremes of resource starvation and excessive paging.

4.4. Hardware versus Software Support

A robust virtual memory system combines hardware features with operating-system routines. The balance between silicon assistance and algorithmic complexity shapes both performance and cost.

Hardware Support Features

- **Reference and Dirty Bits**
 - Automatically set by the MMU on page access and write operations.
 - Enable efficient detection of page usage and streamlined replacement decisions.
- **Translation Lookaside Buffer (TLB)**
 - Caches recent address translations to reduce page-table walk times.
 - Requires hardware-driven invalidation or shutdown on context switches.
- **Page-Table Walk Logic**
 - Some processors provide a hardware mechanism to traverse multi-level page tables without OS intervention.
 - Reduces the number of memory accesses needed for translation.
- **Large or Huge Pages**
 - Support for larger page sizes (for example, 2 MB or 1 GB) diminishes TLB pressure and reduces table size.
 - Trade-off: increased internal fragmentation.
- **Prefetching Mechanisms**
 - Hardware can speculatively fetch nearby pages based on detected access patterns.
 - Complements software-driven prepaging.

Software Support Responsibilities

- **Page Table Management**
 - Allocate, free and potentially reorganise multi-level structures.
 - Handle page faults and coordinate with hardware traps.
- **Replacement Algorithm Implementation**
 - Use hardware-provided bits or software counters depending on available features.
 - Perform maintenance tasks such as clearing reference bits or decaying frequency counters.
- **Swap-Space Management**
 - Maintain free swap slots.
 - Schedule disk operations to hide latency (for instance, by grouping writes).
- **Consistency and Coherency**
 - Manage TLB invalidations or shutdowns in multiprocessor environments.
 - Ensure that shared pages reflect the latest writes (for example, via cache coherence protocols).

Trade offs between hardware and software implementations of page replacement centre on performance, cost and adaptability. Enhanced hardware support—such as dedicated reference bits, trap mechanisms and nested page table structures—can dramatically reduce operating system overhead and speed up memory management, but it comes at the expense of greater chip complexity, higher power consumption and irreversible design choices. By contrast, purely software based schemes run on commodity hardware with no specialised features, offering the flexibility to revise or tune algorithms through updates, yet they often incur CPU stalls and increased per reference costs. As workloads grow in scale and concurrency, hardware acceleration tends to maintain performance with minimal impact on the operating system, whereas software solutions may struggle to keep pace under heavy multiprogramming or vast address spaces.

Ultimately, the most effective systems blend both approaches. Hardware mechanisms handle low level tasks—such as tracking page access and performing rapid translations—while the operating system oversees fault handling, replacement decisions and consistency checks. Striking the right balance involves evaluating platform requirements, cost constraints and target performance: a design that integrates modest hardware support with well optimised software routines can deliver the responsiveness of dedicated silicon alongside the adaptability of code.

Effective virtual memory management requires more than choosing a replacement algorithm. Designers must weigh the overhead of data structures and per-access bookkeeping against fault-rate benefits; recognise and avoid pathological behaviours such as Belady's Anomaly; detect and mitigate thrashing through dynamic process and memory control; and strike an appropriate balance between hardware acceleration and operating-system flexibility.

By carefully analysing each dimension—algorithmic complexity, anomalous outcomes, system workload dynamics and platform capabilities—engineers can craft a page-replacement subsystem that sustains high throughput, predictable performance and efficient memory use, even as applications grow in scale and diversity.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Which category of overhead concerns the extra CPU cycles consumed on each memory access to update replacement metadata? a) Memory Footprint b) Per Reference Cost c) Complexity of Updates d) Hardware Maintenance
12	Belady's Anomaly refers to which counterintuitive situation? a) Reducing the number of frames increases the fault rate b) Increasing the number of frames increases the fault rate c) The fault rate remains constant regardless of frame count d) The system thrashes when too many frames are allocated
13	Which of the following is a primary indicator of thrashing in a system? a) High CPU utilisation b) Low page fault rate c) Low CPU utilisation d) Reduced I/O traffic
14	Which mitigation strategy involves delaying the admission of new processes when memory pressure is high? a) Working Set Model b) Page Fault Frequency Control c) Load Control d) Local Replacement Policy
15	Which hardware feature assists the operating system by automatically marking pages as accessed or modified? a) Translation Lookaside Buffer b) Reference and Dirty Bits c) Huge Page Support d) Prefetching Mechanism

5. SUMMARY

- Page faults occur when a process references a page not resident in physical memory, prompting the CPU to generate an interrupt through which the operating system validates the access, loads the needed page into a free frame (or evicts a victim if none are free), updates tables and resumes execution.
- Frame allocation strategies—equal, proportional and priority based—determine how many frames each process receives, with global policies allowing any process to evict from the entire frame pool and local policies confining replacement to frames already assigned to the faulting process.
- The principles of temporal and spatial locality underpin the working set model, whereby the set of pages used within a recent window of time guides dynamic frame allocation to hold active pages in memory and reduce fault rates.
- Key performance metrics include page fault rate (faults per memory reference), throughput (processes completed per unit time), CPU utilisation (time spent executing versus waiting for I/O) and response time, while thrashing is detected by high fault rates coupled with low CPU utilisation and heavy swap I/O.
- The First In First Out algorithm evicts the oldest loaded page via a simple circular queue, offering minimal overhead but risking Belady's anomaly, where adding frames counter intuitively raises the fault count.
- Belady's Optimal (clairvoyant) algorithm provides a theoretical lower bound on faults by evicting the page whose next reference lies furthest in the future, serving as a benchmark though impractical without advance knowledge of all page references.
- Least Recently Used approximates optimal behaviour by tracking last access times—through counters, stack structures or hardware reference bits—and evicts the page unused for the longest period, at the cost of per reference overhead.
- The Clock (Second Chance) algorithm mimics LRU with low overhead, organising frames in a circular list each with a single reference bit; the “hand” clears bits on inspection and only evicts when it finds a frame whose bit remains zero.
- Frequency based schemes such as LFU evict the page with the fewest accesses, often enhanced by ageing to avoid staleness, while hybrid methods (for example ARC or CAR) blend recency and frequency heuristics to adapt to changing workloads.

- Practical implementation demands a balance between metadata overhead and fault rate reduction, avoidance of pathological cases like Belady's anomaly, mitigation of thrashing via working set or load control policies, and judicious use of hardware support (reference/dirty bits, TLBs and prefetching) alongside operating system routines.



6. GLOSSARY

Page fault	- An event triggered when a process attempts to access a page that is not currently loaded into physical memory, causing the operating system to fetch it from secondary storage.
Frame	- A fixed size block of physical memory into which pages are loaded for execution.
Thrashing	- A state in which the system spends the majority of its time swapping pages in and out of memory rather than executing useful work.
Temporal locality	- The tendency for a process to access the same memory locations repeatedly within a short time interval.
Spatial locality	- The tendency for a process to access memory locations that are close to recently accessed addresses.
Working set	- The collection of pages that a process has referenced during a defined recent time window, used to guide dynamic frame allocation.
Belady's Anomaly	- The counterintuitive phenomenon where increasing the number of available frames under certain algorithms (notably FIFO) leads to a higher page fault rate.
Translation Lookaside Buffer (TLB)	- A specialised cache that stores recent virtual to physical address translations to accelerate memory access.
Clock algorithm	- A page replacement scheme that organises frames in a circular list, using a single reference bit per page to approximate least recently used behaviour.
Least Frequently Used (LFU)	- A replacement strategy that evicts the page with the lowest access count, often incorporating ageing to prevent stale pages from remaining resident too long.

7. TERMINAL QUESTIONS

1. Explain the sequence of steps the operating system follows when handling a page fault, from interrupt to resuming the process.
2. Compare equal and proportional frame allocation strategies, highlighting one advantage and one drawback of each.
3. Describe how the working set model uses temporal and spatial locality to guide dynamic frame allocation.
4. Define page fault rate and throughput, and discuss how they can be used together to evaluate a replacement algorithm's effectiveness.
5. Outline the causes of Belady's Anomaly and explain why FIFO is susceptible while LRU is not.
6. Contrast the Clock algorithm with true LRU in terms of implementation complexity and page fault performance.
7. Discuss how ageing techniques address the staleness problem in LFU, and give one limitation of LFU even with ageing.
8. Describe two methods by which an operating system can detect thrashing and one policy it can employ to mitigate it.
9. Identify three hardware features that support virtual memory management and explain how each reduces operating system overhead.
10. Analyse the trade offs between software only and hardware accelerated page replacement implementations, considering both performance and adaptability.

8. ANSWERS

Self-Assessment Questions

1. b) Examine the faulting address to confirm the reference is valid
2. c) Proportional Allocation
3. b) Spatial Locality
4. b) CPU Utilisation
5. b) Clock (Second Chance)
6. a) First In First Out
7. c) The sequence and timing of all future page references
8. c) It resides at the bottom of the stack
9. c) It uses a single use (reference) bit per frame
10. b) Least Frequently Used
11. b) Per Reference Cost
12. b) Increasing the number of frames increases the fault rate
13. c) Low CPU utilisation
14. c) Load Control
15. b) Reference and Dirty Bits

Terminal Questions Answers

Answers 1: When a page fault occurs, the CPU triggers an interrupt and hands control to the operating system's fault handler. The handler checks the faulting address for validity, then looks for a free frame; if none is available, it selects a victim page and writes it back if modified. Finally, it loads the required page into the frame, updates the page tables and restarts the interrupted instruction. Refer section 2.1

Answers 2: Equal allocation gives each process the same fixed number of frames, which is fair and simple to manage but can penalise large processes. Proportional allocation assigns frames based on each process's share of virtual pages, matching demand better yet requiring accurate size information and still misjudging working set needs. Refer section 2.2

Answers 3: The working set model tracks pages referenced in a recent time window, using temporal locality to retain recently used pages and spatial locality to keep pages near those accesses. By

adjusting frame allocations to cover each process's active page set, it aims to minimise faults without wasting memory. Refer section 2.3

Answers 4: Page fault rate is the proportion of memory references that cause faults, while throughput measures the number of processes completed per unit time. Together, a low fault rate and high throughput indicate an effective replacement algorithm, balancing reduced disk I/O with manageable overhead. Refer section 2.4

Answers 5: Belady's Anomaly arises when a non stack algorithm like FIFO evicts pages without regard to future use, so adding frames may keep cold pages longer and increase faults. FIFO's simple queue discipline makes it vulnerable, whereas LRU's recency based evictions ensure that more frames cannot worsen the fault count. Refer section 4.2

Answers 6: True LRU tracks precise last use times, requiring timestamps or stacks and updating on every access, which yields low fault rates but high overhead. The Clock algorithm approximates LRU with a single reference bit per frame and a rotating pointer, cutting metadata costs at the expense of occasional suboptimal evictions. Refer sections 3.3 and 3.4

Answers 7: Ageing in LFU periodically decays reference counts so that formerly popular pages lose priority if they fall out of use, reducing cache pollution. Even with ageing, LFU can struggle when access patterns shift suddenly, as the history of past references may delay adaptation. Refer section 3.5

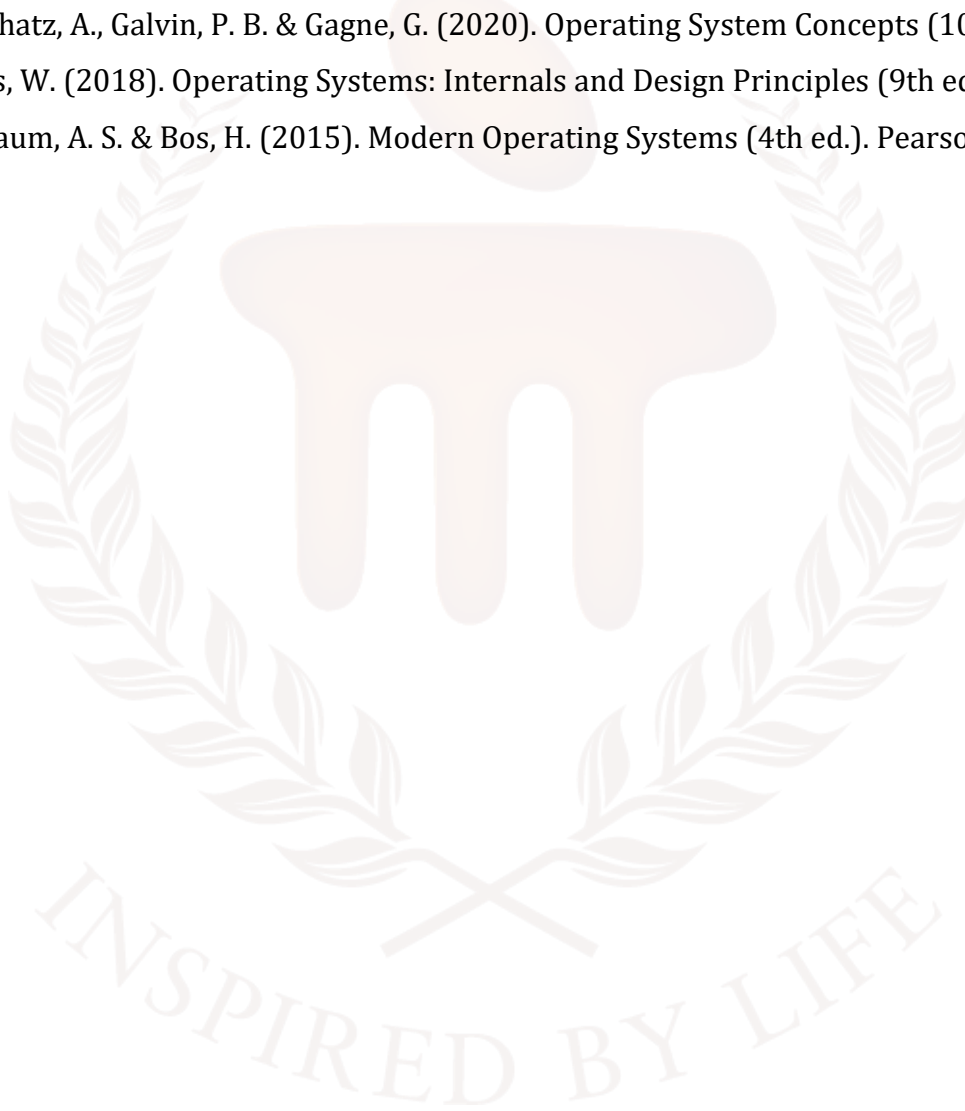
Answers 8: An operating system can detect thrashing by observing a sharp rise in page fault rate alongside low CPU utilisation, or by monitoring heavy swap in/out I/O. To mitigate it, it might apply a working set model to suspend processes whose frame demands exceed available memory or use page fault frequency control to rebalance frame allocations. Refer section 4.3

Answers 9: Reference and dirty bits let the OS identify accessed or modified pages without per access software updates, reducing overhead. The TLB caches recent virtual to physical mappings to avoid full page table walks, while hardware page table walk logic can traverse multi level tables automatically, all of which cut down on OS intervention. Refer section 4.4

Answers 10: Software only replacement runs on generic hardware, offering flexibility to tune algorithms but suffering higher CPU and memory overhead per reference. Hardware accelerated implementations offload tasks such as setting reference bits and managing TLBs to silicon, improving performance but adding design complexity and reducing adaptability. Refer section 4.4

9. REFERENCES

1. Arpaci Dusseau, R. H. & Arpaci Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces. Arpaci Dusseau Books.
2. Bovet, D. P. & Cesati, M. (2005). Understanding the Linux Kernel (3rd ed.). O'Reilly Media.
3. Silberschatz, A., Galvin, P. B. & Gagne, G. (2020). Operating System Concepts (10th ed.). Wiley.
4. Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson.
5. Tanenbaum, A. S. & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.

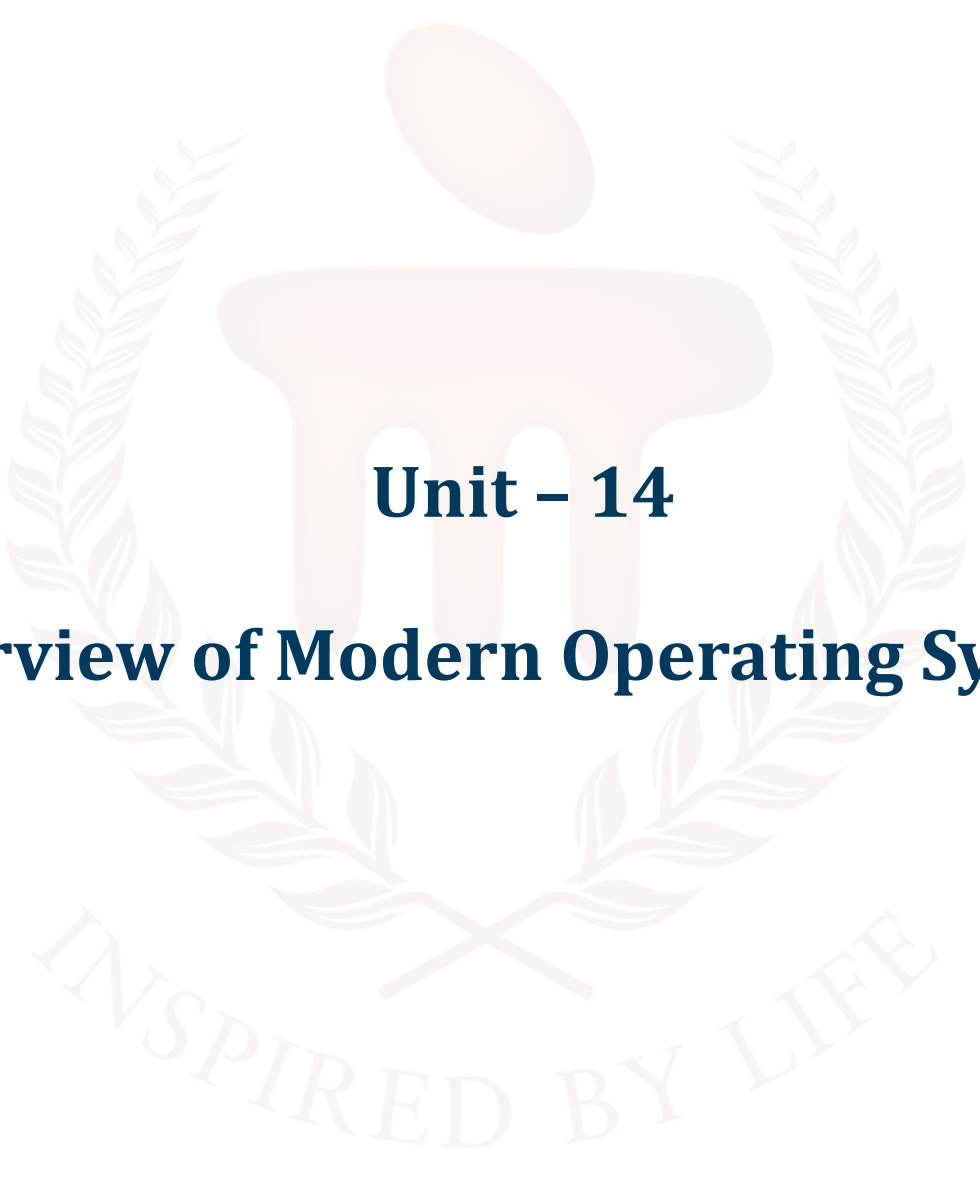


BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 3



DCA2108
OPERATING SYSTEMS



Unit – 14

Overview of Modern Operating Systems

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Objectives	-	-	
2	Overview of Modern Operating Systems	-	-	7 - 12
2.1	Architectural Models (Monolithic, Microkernel, Hybrid)	1	-	
2.2	Kernel Types and Services	2	-	
2.3	User Interfaces (CLI, GUI, Web-Based)	-	-	
2.4	Security and Access Control Mechanisms	-	-	
2.5	Modularisation and Extensibility	-	1	
3	Case Study on UNIX, Linux and Windows	-	-	13 - 21
3.1	Historical Evolution and Design Goals	3	-	
3.2	Architectural Comparison	-	-	
3.3	Process and Memory Management Features	-	-	
3.4	File System Organisation	4	-	
3.5	Security Implementations	5	2	
4	Introduction to Cloud Operating Systems	-	-	22 - 28
4.1	Defining Cloud Operating Systems	-	-	
4.2	Virtualisation and Container Support	6, 7, 8	-	
4.3	Resource Orchestration and Scheduling	-	-	
4.4	Scalability and Elasticity Mechanisms	-	-	
4.5	Fault Tolerance and High Availability	-	3	
5	Future Directions and Considerations	-	-	29 - 33
5.1	Edge and Fog Computing Integration	-	-	
5.2	Serverless and Function-as-a-Service Models	-	-	
5.3	Security Challenges in Distributed Clouds	9	-	
5.4	Management of Heterogeneous Resources	-	4	

6	Summary	-	-	34
7	Glossary	-	-	35 - 36
8	Terminal Questions	-	-	37
9	Answers	-	-	38 - 40
10	References	-	-	41



1. INTRODUCTION

In the previous chapter, we explored the concept of page replacement, which is central to efficient memory management. The discussion included the need for replacing pages when memory is full and various strategies used by operating systems to select which pages to remove. We studied important page-replacement algorithms such as FIFO, LRU, and Optimal, examining how each of these influences system performance under different workloads. These concepts provided a strong foundation in understanding how operating systems manage limited memory resources in multi-programmed environments.

This chapter moves beyond the internal mechanisms and introduces the broader landscape of modern operating systems in use today. It begins with an examination of major operating systems such as UNIX, Linux, and Windows. These systems serve as the backbone of computing in various domains—from servers and desktops to embedded systems. Each operating system is built on distinct philosophies and architectural decisions, yet they share several fundamental responsibilities like process management, memory handling, and file system organisation. Through detailed study, we can better understand how these systems have evolved to meet the growing demands of hardware diversity, user expectations, and software complexity.

The chapter also presents an emerging area of interest: cloud operating systems. These platforms manage not a single machine but entire data centres and distributed environments. They are designed to support virtualisation, scalable resource allocation, and the orchestration of services across multiple physical hosts. The role of cloud OSs such as Google's Container-Optimised OS and Microsoft's Azure-based platforms is increasing, especially as businesses shift towards remote infrastructure and platform services. By contrasting traditional and cloud operating systems, we gain insight into how operating systems are adapting to modern computing trends.

To study this chapter effectively, begin by revisiting the basic responsibilities of an operating system. From there, examine each case study to identify the core similarities and differences between the systems. Pay special attention to system architecture, kernel design, and how each system handles process and memory management. As cloud systems are a relatively new area, focus on their features, use cases, and architectural implications. Diagrams comparing traditional and cloud-based system layers can offer clarity. Treat this chapter not just as a technical overview but as a bridge to understanding the real-world applications of the concepts studied earlier.

- Compare the architecture and design principles of UNIX, Linux, Windows, and cloud operating systems.
- Distinguish key process and memory management approaches in modern operating systems.
- Evaluate the strengths and limitations of traditional versus cloud-based operating systems.
- Classify operating systems based on their usage context and system architecture.
- Analyse the evolution of operating systems in response to technological and user demands.



2. OVERVIEW OF MODERN OPERATING SYSTEMS

Operating systems of today present a rich tapestry of design choices, each shaped by decades of research and practical experience. They must juggle hardware control, resource scheduling, security enforcement and user interaction, all while maintaining reliability. In this section, we examine the principal architectures, core services, interface options, security frameworks and the ways in which contemporary systems remain flexible and extensible.

2.1. Architectural Models

Modern kernels tend to follow one of three paradigms—monolithic, microkernel or hybrid—each offering a distinct balance between performance, isolation and complexity.

Monolithic Kernels - These systems embed all core functions—device drivers, file systems, memory management and process scheduling—into a single large executable running in kernel space. Because components call one another directly, performance is typically excellent. However, the tight coupling means that a fault in any one part can propagate across the entire system.

- **Performance:** Minimal inter-component overhead.
- **Stability risk:** Single bug may crash the kernel.
- **Maintenance:** Complex to update or isolate individual services.

Microkernels - In contrast, microkernels include only the most essential services (such as low-level memory and IPC) within the privileged core. Additional services execute in user space as separate processes, communicating via message passing. The result is improved fault isolation—if one service fails, the kernel remains unaffected—but at the cost of increased communication overhead.

- **Isolation:** Better fault containment.
- **Overhead:** IPC latency can reduce throughput.
- **Upgradability:** Individual services replaced without kernel rebuild.

Hybrid Kernels - Hybrid designs attempt to combine the speed of monolithic kernels with the modularity of microkernels. They keep certain subsystems (for example, file system abstraction or device management) in kernel space to reduce IPC, while others run as user-mode services.

- **Trade-off:** Balanced performance and modularity.
- **Examples:** Windows NT lineage, macOS XNU, modern Linux with loadable modules.

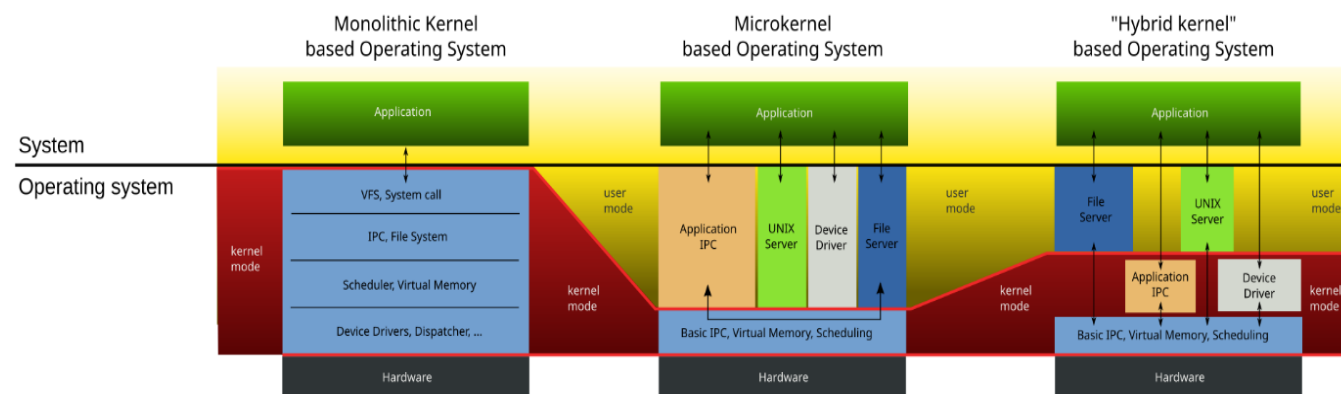


Figure 1: Operating System Kernel Architecture

Source: <https://upload.wikimedia.org/wikipedia/commons/thumb/d/d0/OS-structure2.svg/1920px-OS-structure2.svg.png>

The figure provides a comparative schematic of three distinct operating system kernel architectures: monolithic, microkernel, and hybrid kernels, highlighting the placement of various system services in either kernel or user space. The monolithic kernel centralises most operating system services, such as file systems, IPC, scheduling, virtual memory, and device drivers, within the kernel mode for direct hardware access. In contrast, the microkernel architecture moves many of these services, like device drivers and file servers, into user mode, leaving only essential functions such as basic IPC, virtual memory, and scheduling in the kernel. The hybrid kernel model, as depicted, combines aspects of both, placing some services in kernel mode for performance while others, like file and UNIX servers, operate in user mode.

2.2. Kernel Types and Services

Irrespective of architectural choice, every kernel must provide a set of fundamental services that enable the operating system to function.

Modern kernels can be classified further by their approach to extensibility:

Monolithic with Modules: Support for loadable kernel modules allows drivers and features to be added or removed at runtime.

Exokernels: Expose hardware primitives directly to applications, offering minimal abstraction and maximal performance, albeit at the expense of portability and safety.

Regardless of type, a kernel typically offers:

- A. Process Management:** Kernels create, schedule and terminate processes or threads. Scheduling algorithms (such as round-robin or multi-level feedback queues) govern CPU allocation, aiming for fairness and responsiveness.
- B. Memory Management:** Through paging or segmentation, kernels control which parts of physical memory processes may access. Virtual memory techniques allow processes to operate within their own address spaces, with the kernel handling translation via page tables.
- C. File System Support:** Abstractions for organising, storing and retrieving data on secondary storage. Kernels manage metadata, enforce permissions and provide caching to improve performance.
- D. Device Management:** Device drivers within the kernel mediate between hardware peripherals and user-level requests. Standardised driver interfaces permit new hardware to be supported without altering core kernel code.
- E. Inter-Process Communication (IPC):** Mechanisms such as pipes, message queues or shared memory enable processes to exchange data or synchronise actions.
- F. Interrupt and Exception Handling:** The kernel responds to hardware interrupts (for example, signalling I/O completion) and software exceptions (such as division by zero), ensuring the system can react to events in real time.

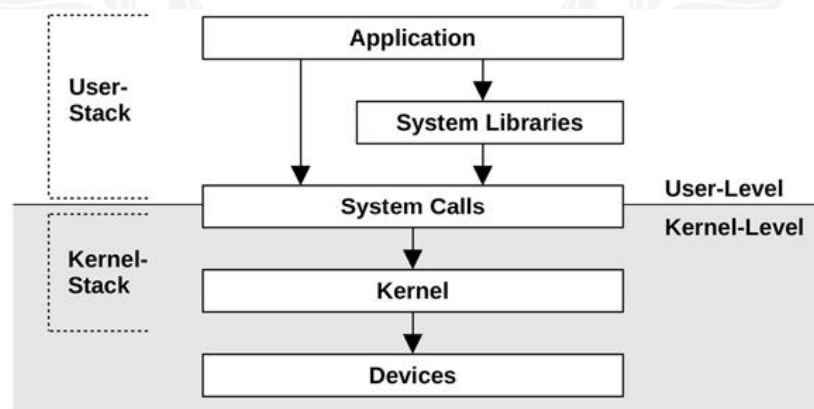


Figure 2: Kernel Internals in Operating System

Source: <https://www.informit.com/articles/article.aspx?p=3104531&seqNum=2>

The figure presents a layered view of an operating system's architecture, illustrating the interactions from user applications down to the hardware. At the topmost layer, applications operate, which can interact with 'System Libraries' to access common functionalities. These libraries, or direct application requests, then initiate 'System Calls', which serve as the interface between user-level processes and the operating system's 'Kernel'. The kernel, operating in a privileged mode, then

interacts with the underlying 'Devices' (hardware) to perform requested operations. The diagram also indicates the presence of a 'User-Stack' and 'Kernel-Stack', signifying distinct memory areas for managing function calls and local variables at each level.

2.3. User Interfaces

While the kernel works behind the scenes, user interfaces define how people or automated scripts interact with the system's capabilities.

Command-Line Interface (CLI) - Text-based interaction remains indispensable for servers, embedded devices and power users. Shell environments (bash, PowerShell, zsh) provide scripting, piping and automation features that GUI alone cannot match.

- **Advantages:** Low resource use; fine-grained control; easily automated.
- **Disadvantages:** Steeper learning curve; less discoverable for beginners.

Graphical User Interface (GUI) - GUIs employ windows, icons and menus to present a more intuitive experience. Components such as window managers or desktop environments sit atop the kernel, using graphical APIs (for example, X11 or Wayland on Linux).

- **Advantages:** User-friendly; visual feedback; supports multimedia.
- **Disadvantages:** Higher resource demands; can obscure system complexity.

Web-Based Interfaces - Increasingly, even desktop operating systems incorporate browser-based dashboards for tasks such as updates, system monitoring or remote administration. This approach ensures cross-platform compatibility and can deliver a consistent look and feel to distributed environments.

- **Advantages:** Platform-agnostic; accessible remotely.
- **Disadvantages:** Dependent on web server components; potential security surface.

2.4. Security and Access Control Mechanisms

Security is woven through every layer of a modern operating system. Trust boundaries are enforced via authentication, authorisation and auditing.

Authentication: Methods include passwords, public-key cryptography and biometric factors. Secure authentication prevents unauthorised users from gaining entry to the system.

Authorisation and Access Control: Once authenticated, users require permissions to read, write or execute resources. Common models include:

- **Discretionary Access Control (DAC):** Owners assign permissions to their files and processes.
- **Mandatory Access Control (MAC):** Central policies (often based on security labels) determine access, typical in high-security environments.
- **Role-Based Access Control (RBAC):** Permissions are granted to roles rather than individual users, simplifying administration in large organisations.

Key Security Services

- **Encryption:** Both full-disk and file-level encryption guard data at rest.
- **Sandboxing/Containers:** Applications run within confined environments to limit potential damage.
- **Auditing and Logging:** System events are recorded for forensic analysis and compliance.
- **Firewall and Network Filtering:** Controls inbound and outbound traffic at the kernel's network stack.

2.5. Modularisation and Extensibility

To cater for ever-changing requirements, modern kernels support dynamic extension:

- **Loadable Kernel Modules (LKMs)** allow new drivers or file systems to integrate without recompiling the entire kernel.
- **Plug-in Frameworks** in user-space enable additional services (for example, network protocol stacks) to be added on demand.

Modularisation in operating system design allows different components to function independently, improving maintainability and flexibility. This means features can be tailored to suit specific system requirements, and individual modules can be updated or replaced without disturbing the core system. Fault isolation is another benefit, as faulty modules can be safely removed without impacting other parts of the system.

However, extensibility brings its own set of challenges. Managing compatibility between modules and different kernel versions requires careful version control and testing. Poorly developed or malicious modules can pose security threats, and dependency issues may arise when modules rely on specific versions or other components, potentially leading to instability.

SELF-ASSESSMENT QUESTIONS - 1

Multiple Choice Questions	
1	Which kernel architecture places most services, including device drivers and file systems, within a single large executable in kernel space? a) Microkernel b) Monolithic c) Hybrid d) Exokernel
2	In which kernel design are core services kept minimal in kernel space, with other services running in user space and communicating by message passing? a) Monolithic b) Hybrid c) Microkernel d) Exokernel
3	Which of the following is not a fundamental service provided by all modern kernels? a) Process management b) Virtual memory management c) Web-based interface provisioning d) Interrupt and exception handling
4	Which access control model relies on centrally defined policies and security labels to determine permissions? a) Discretionary Access Control (DAC) b) Role-Based Access Control (RBAC) c) Mandatory Access Control (MAC) d) Capability-based Access Control
5	What mechanism allows additional device drivers or file systems to be added to a running kernel without a full rebuild? a) Static linking b) Loadable Kernel Modules c) Hybrid layering d) User-space plug-in frameworks

3. CASE STUDY ON UNIX, LINUX AND WINDOWS

The study of UNIX, Linux and Windows as operating systems offers invaluable insight into how different design philosophies and development communities have shaped modern computing. Each system addresses fundamental tasks—process scheduling, memory management, file storage, security enforcement and user interaction—in distinct ways, motivated by particular historical contexts and intended use cases. By examining their evolution, architectural underpinnings, core management features, file system structures and security frameworks, we gain a comprehensive perspective on the trade offs and innovations that continue to influence system design.

3.1. Historical Evolution and Design Goals

UNIX emerged in 1969 at Bell Labs, created by Ken Thompson and Dennis Ritchie. Frustrated with the complexity of existing systems, they sought a simpler, portable environment. Implemented in the C programming language by 1973, UNIX became the first operating system to support cross platform compilation. Its guiding principles emphasised small, composable utilities; a hierarchical file system; and text streams as a universal interface between programs. These ideas fostered a thriving community of academic and commercial offshoots, such as BSD at the University of California, Berkeley.

Linux, initiated by Linus Torvalds in 1991, was conceived as a free and open counterpart to UNIX. By licensing under the GNU General Public Licence, Linux permitted collaborative development, leading to rapid expansion of supported architectures and device drivers. The kernel's modular design—supporting loadable modules—enabled contributors worldwide to add functionality without centralised control. Goals included maintaining UNIX compatibility, encouraging community involvement and evolving with emerging hardware trends.

Windows traces its origin to MS DOS in 1981, but the significant pivot came with Windows NT in 1993. Designed by a team led by Dave Cutler, NT (New Technology) adopted a clean room architecture distinct from MS DOS. Its objectives were enterprise reliability, hardware independence via a Hardware Abstraction Layer (HAL), and built in security suitable for corporate environments. NT blended microkernel ideas—such as isolated subsystems—with a monolithic driver model to achieve both performance and modularity. Subsequent versions have maintained backward compatibility through subsystems and widened use across desktops and servers.

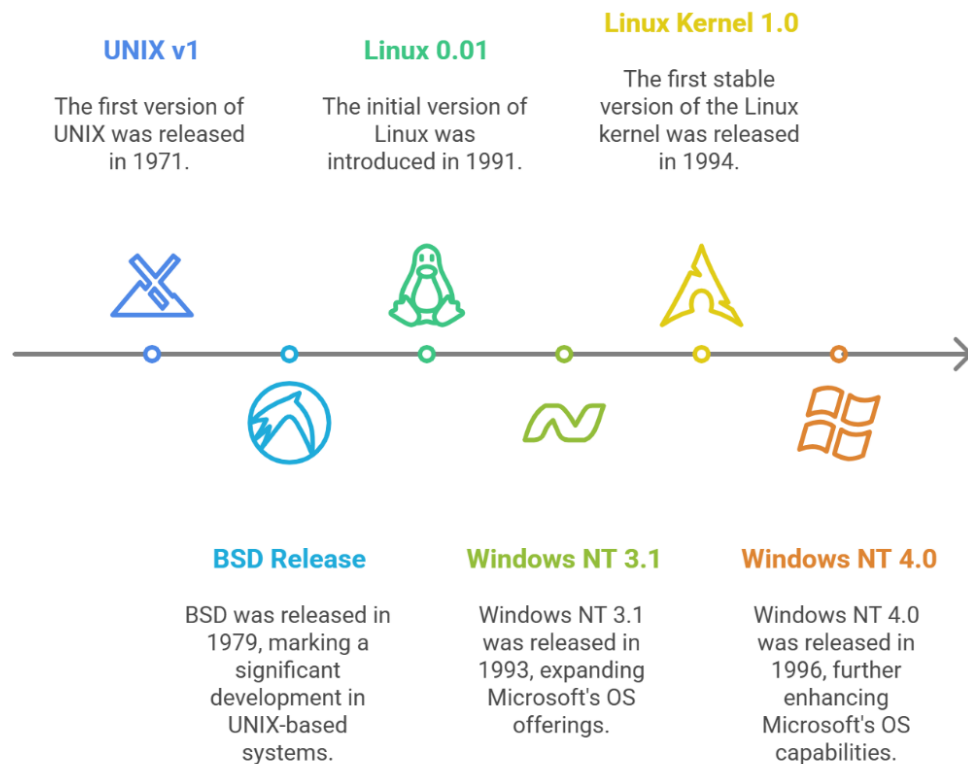


Figure 3: Operating System Timeline

The figure presents a chronological overview of significant milestones in the history of operating systems. It begins with the release of UNIX v1 in 1971, marking a foundational moment. Eight years later, in 1979, the BSD Release emerged, representing a key development in UNIX-based systems. The timeline then progresses to 1991 with the introduction of Linux 0.01, followed by Windows NT 3.1 in 1993, which expanded Microsoft's operating system offerings. Subsequently, 1994 saw the release of Linux Kernel 1.0, its first stable version, and finally, Windows NT 4.0 was released in 1996, further enhancing Microsoft's capabilities.

3.2. Architectural Comparison

At their cores, UNIX, Linux and Windows share a kernel plus user space utilities model, yet their internal arrangements reveal different emphases on performance, modularity and extensibility.

UNIX (BSD lineage) operates with a monolithic kernel where all primary services (process scheduler, virtual file system, device drivers) reside in supervisor mode. This tight integration offers efficient component interaction but poses maintenance challenges: any change requires rebuilding the entire kernel. Over time, BSD variants introduced dynamically loadable kernel modules, improving flexibility.

Linux maintains a monolithic style kernel but supports loadable kernel modules (LKMs). This enables the insertion or removal of device drivers and filesystems at runtime without rebooting. Internally, Linux is structured into subsystems—such as the Virtual File System (VFS), network stack and memory manager—each communicating through clearly defined interfaces. The kernel's build system and Git based workflow facilitate distributed development.

Windows NT embodies a hybrid kernel architecture. A minimal microkernel layer handles low level tasks like inter processor communication and address translation. Above this lie executive subsystems—Object Manager, Memory Manager, I/O Manager—executing in kernel mode but functionally separated. Device drivers also run in kernel mode, interacting through standard driver interfaces. User mode subsystems (Win32, POSIX, OS/2) interface with the kernel via system call stubs, allowing multiple application environments concurrently.

Comparative summary:

- **Kernel type:** UNIX monolithic; Linux monolithic with modules; Windows hybrid.
- **Module support:** Optional in BSD, central in Linux, structured via Windows driver model.
- **Development model:** Academic and open in UNIX/Linux; commercial with documented APIs in Windows.

3.3. Process and Memory Management Features

The strategies for process scheduling and memory allocation directly affect system responsiveness and resource utilisation.

Process Management

- **UNIX/BSD:** Utilises a simple round robin scheduler with priority ageing to prevent starvation. Processes are represented by process control blocks (PCBs) containing state, scheduling parameters and parent/child relationships. UNIX signals provide asynchronous notifications (for example, SIGINT or SIGTERM), allowing processes to handle interrupts gracefully.
- **Linux:** Offers multiple scheduling classes. The default Completely Fair Scheduler (CFS) aims to allocate CPU time proportionally to process weights, ensuring fairness while maintaining low latency. Real time classes (SCHED_FIFO, SCHED_RR) cater to time critical applications. Linux threads are implemented as discrete tasks sharing address spaces, enabling lightweight concurrency.

- **Windows NT:** Implements a priority based, pre-emptive scheduler with 32 priority levels—ranging from idle to real time. Each thread is assigned a quantum length that may be dynamically adjusted based on thread importance and I/O activity, balancing foreground interactivity against background throughput.

Memory Management

- **UNIX/BSD:** Early BSD kernels adopted the Mach virtual memory system, introducing demand paging and copy-on-write semantics. A clock-like page replacement algorithm approximates Least Recently Used (LRU) behaviour.
- **Linux:** Employs a two-tier cache for file pages (page cache and swap cache). The kernel uses an LRU list with adaptive thresholding to decide when to reclaim pages. Transparent Huge Pages (THP) support larger page sizes without application changes, enhancing performance for memory-intensive workloads.
- **Windows NT:** The Memory Manager maintains a working set for each process, tracking pages recently accessed. When memory is constrained, the system trims working sets or swaps pages to the page file. Windows also provides memory-mapped files and section objects to facilitate inter-process sharing and efficient file access.

3.4. File System Organisation

Efficient and reliable file systems are essential for data integrity and performance.

UNIX/BSD File Systems

- **UFS/FFS (Fast File System):** Divides disk into cylinder groups, reducing fragmentation and improving locality. Uses inodes to represent files, storing metadata separately from data blocks.
- **ZFS (in OpenZFS on BSD):** Introduces pooled storage, end-to-end checksums, copy-on-write transactions and snapshots. Provides self-healing capabilities by detecting and repairing silent corruption.

Linux File Systems

- **ext2/3/4 Series:** Ext4 adds extents (contiguous block mappings), delayed allocation and journaling for rapid recovery after crashes.
- **Btrfs:** Designed for modern workloads, with copy-on-write, subvolumes, online defragmentation and built-in RAID support.

- **XFS and ReiserFS:** Optimised for large files and directories, offering dynamic allocation and advanced journalling.

Windows NTFS

- **Master File Table (MFT):** Central metadata table listing every file and directory. Attributes within MFT records include file size, timestamps, security descriptors and data runs.
- **Features:** Journalling of metadata updates, per file encryption (EFS), compression and sparse files. Supports alternate data streams and extended attributes.

File system organisation varies notably across UNIX, Linux, and Windows, though all three platforms implement journalling to guard against data corruption during crashes or power failures. In UNIX and Linux, file systems are generally handled as independent kernel subsystems that can be loaded or removed as needed, offering modularity and support for a wide range of file system types. Windows, on the other hand, integrates NTFS more tightly into the kernel, making it a core component of the system's design and performance. Advanced file systems like ZFS (used in some UNIX variants) and Btrfs (commonly used in Linux) go a step further by incorporating volume management and snapshot functionality directly within the file system itself, reducing the need for separate tools to manage partitions, backups, and consistency.

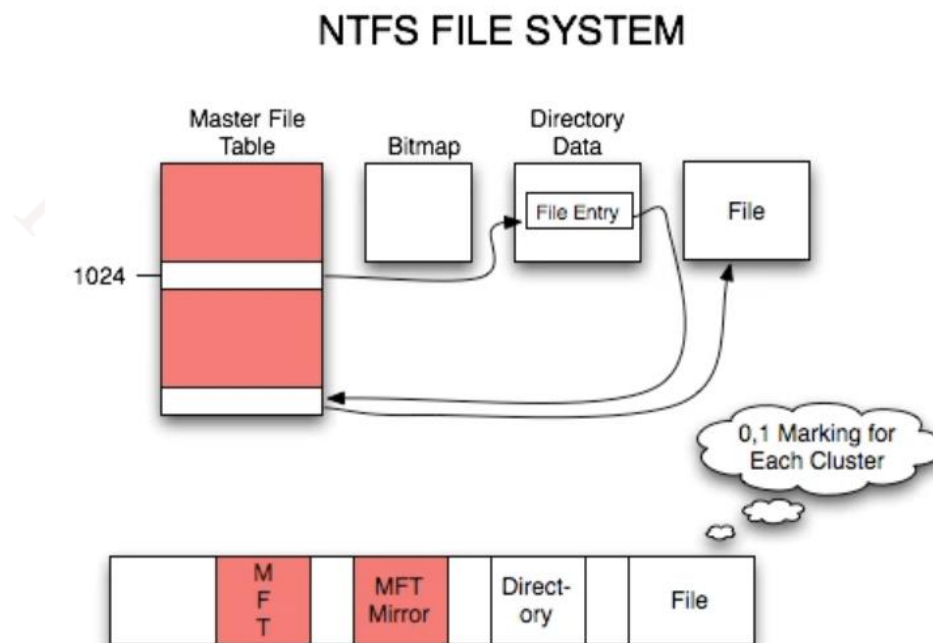


Figure 4: NTFS File System

Source: <https://cdn.geekboots.com/geek/ntfs-file-system-1657714415393.webp>

The figure, "NTFS File System", illustrates the layered architecture of the New Technology File System (NTFS), detailing how data is organised and accessed. At its core is the Master File Table (MFT), which contains entries for all files and directories on the volume, including a dedicated entry for the MFT itself at entry 0. The diagram shows how a file entry within the directory data points back to the MFT for file metadata. The MFT also interacts with a bitmap, which tracks the allocation status of clusters on the disk, represented by 0 or 1 markings. Below this, a conceptual layout of the disk shows the MFT, MFT Mirror, Directory, and File areas, indicating the physical arrangement of these components.

3.5. Security Implementations

Operating system security encompasses authentication, authorisation and auditing, ensuring only permitted actions occur and that events are traceable.

UNIX/BSD Security

- **Traditional DAC:** Each file has owner/group/other bits indicating read, write and execute permissions. Users may change permissions on owned files via `chmod` and `chown`.
- **Capsicum (FreeBSD):** Introduces capability mode, allowing processes to drop all global rights and operate only on pre opened file descriptors.
- **TrustedBSD:** Implements Mandatory Access Control (MAC) frameworks, enabling policy enforcement beyond standard UNIX permissions.

Linux Security

- **DAC by default:** Standard UNIX style permissions and user/group model.
- **SELinux and AppArmor:** Linux Security Modules (LSM) provide MAC policies. Administrators define fine grained rules specifying allowed operations per process, greatly limiting damage from exploits.
- **Namespaces and cgroups:** Form the basis of container isolation, separating process views of resources such as process IDs, network interfaces and file systems.

Windows NT Security

- **Access Control Lists (ACLs):** Each securable object (files, registry keys, services) carries a discretionary ACL listing user and group permissions.
- **Security Reference Monitor:** Enforces every access request against object ACLs in kernel mode, ensuring centralised policy application.

- **User Account Control (UAC):** Encourages least privilege by running applications with standard user rights by default and prompting for elevation when needed.

Cryptographic and Isolation Features

- **Encryption:** BitLocker on Windows; LUKS on Linux; geli on BSD. These subsystems integrate with boot loaders and system components to secure data at rest.
- **Sandboxing and Virtualisation:** Tools such as FreeBSD jails, Linux containers, and Windows Hyper V isolate workloads at different granularities, preventing cross application interference.
- **Auditing:** All three platforms provide logging frameworks—syslog on UNIX/Linux; Event Tracing for Windows—capturing security relevant events.

Table 1: Access Control Matrix

System	Subject	Object	Access Control Mechanism	Access Rights
UNIX (DAC)	User alice	File /report.txt	Owner-based DAC (Discretionary Access Control)	Read, Write
	Group staff	File /report.txt	Group permissions	Read
	Others	File /report.txt	World permissions	No access
Linux (MAC)	Role admin	Process sshd	SELinux or AppArmor enforced Mandatory Access Control	Full access (execute, control)
	Role guest	File /etc/passwd	SELinux/AppArmor policy-defined restrictions	Read-only or denied based on label enforcement
Windows (ACL)	User bob	File C:\data.docx	ACL (Access Control List) with fine-grained permissions	Read, Modify
	Group users	File C:\data.docx	ACL entries per user or group	Read-only
	User admin	Process explorer	Privilege-based control via ACL and token permissions	Full access

- **UNIX (DAC):** Users and groups have ownership-based access control. Rights are assigned directly to the file owner, the group, and others.

- **Linux (MAC):** MAC systems like SELinux and AppArmor apply policy rules that override user discretion, enforcing access based on security labels.
- **Windows (ACL):** Each object includes an access control list specifying which users or groups have what kind of permissions. These allow more precise control than traditional UNIX-style permissions.

UNIX, Linux and Windows each embody a unique set of design choices shaped by history, community and intended deployment. UNIX introduced the philosophy of simplicity, portability and composability. Linux extended this ethos under an open source model, adding modern capabilities such as sophisticated scheduling, containerisation and diverse file systems. Windows NT targeted enterprise robustness and backward compatibility, forging a hybrid kernel that balanced modularity with performance. By analysing their architectures, management strategies, file organisation and security frameworks, one appreciates the enduring trade offs that drive operating system innovation and the foundations upon which future systems will continue to evolve.

SELF-ASSESSMENT QUESTIONS - 2

Multiple Choice Questions	
6	Which operating system was first implemented in the C programming language to support cross-platform compilation? a) Linux b) UNIX c) Windows NT d) BSD
7	Which kernel type does Windows NT employ, combining microkernel principles with monolithic driver performance? a) Microkernel b) Exokernel c) Hybrid d) Monolithic with modules
8	Which scheduling class in Linux is designed to allocate CPU time proportionally based on process weights? a) SCHED_FIFO b) Completely Fair Scheduler

	c) Round robin with priority ageing d) Multi-level feedback queue
9	Which file system feature is common to both ZFS on BSD and Btrfs on Linux? a) Master File Table b) Delayed allocation c) Copy-on-write transactions d) Cylinder groups
10	In SELinux and AppArmor, what type of access control model is enforced to restrict process actions? a) Discretionary Access Control b) Role-Based Access Control c) Mandatory Access Control d) Capability-Based Access Control



4. INTRODUCTION TO CLOUD OPERATING SYSTEMS

Cloud operating systems represent a paradigm shift from traditional, single-host OS designs to platforms engineered for distributed, on-demand resource pooling. They orchestrate virtualised and containerised workloads across clusters of machines, ensuring efficient utilisation, resilience and seamless scalability. In this chapter, we explore what defines a cloud OS, how it integrates virtualisation and containers, the approaches to scheduling and orchestration, the mechanisms that enable elastic scaling, and the features that underpin fault tolerance and high availability.

4.1. Defining Cloud Operating Systems

A cloud operating system is not simply an OS running in a data centre; rather, it is a cohesive software layer that abstracts underlying hardware, network and storage into unified, on-demand services. Key characteristics include:

- **Abstraction of Resources:** Compute, storage and network are presented as services that can be allocated dynamically.
- **Multi-tenancy:** Multiple users or applications share the same infrastructure without interfering with each other.
- **Self-service Provisioning:** Users request and release resources through APIs or portals, without direct administrator intervention.
- **Elastic Capacity:** Resources expand or contract automatically in response to load.

While public cloud providers offer their own proprietary platforms (for example, AWS, Azure and Google Cloud), open-source cloud OS projects—such as OpenStack and Apache CloudStack—allow organisations to build private or hybrid clouds using similar concepts.

4.2. Virtualisation and Container Support

Virtualisation and containers form the foundation of cloud OS environments by enabling efficient resource sharing and isolation.

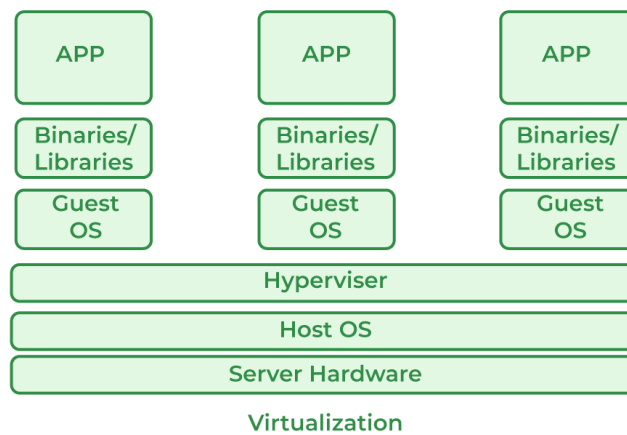


Figure 5: Virtualisation Architecture

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20230324174647/Virtualization.png>

The figure illustrates a typical architecture for virtualisation, depicting how multiple applications can run concurrently on a single set of server hardware. At the base is the 'Server Hardware'. Above this sits the 'Host OS', which manages the underlying hardware resources. On top of the Host OS, a 'Hypervisor' layer is introduced; this software is responsible for creating and managing virtual machines. Each virtual machine, or 'Guest OS', runs its own operating system, along with its 'Binaries/Libraries' and 'Applications' (APP). This setup allows for efficient resource utilisation by enabling multiple isolated environments to operate independently on shared physical infrastructure.

Virtualisation - Hypervisors, or virtual machine monitors, allow multiple guest operating systems to run concurrently atop a host. Two types prevail:

- **Type 1 (Bare-metal) Hypervisors:** Run directly on hardware (examples: Xen, VMware ESXi, Microsoft Hyper-V).
- **Type 2 (Hosted) Hypervisors:** Operate within a host OS (examples: KVM on Linux, VirtualBox).

Hypervisors manage CPU scheduling, memory allocation and I/O emulation. They enable:

- **Isolation:** Each VM is segregated, preventing cross-interference.
- **Portability:** VMs can be migrated live between hosts for maintenance.
- **Backward Compatibility:** Legacy OS and applications may run unmodified.

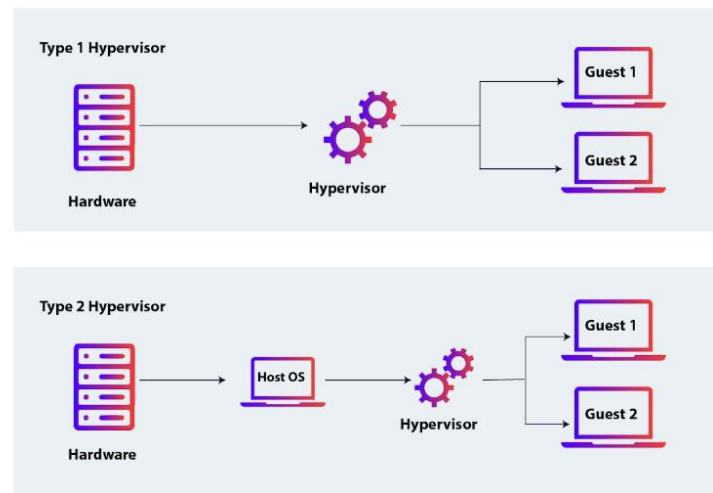


Figure 6: Types of Hypervisors

Source: <https://www.appviewx.com/wp-content/uploads/2022/10/hypervisor-types.png>

The figure differentiates between Type 1 and Type 2 hypervisors, illustrating their respective architectural placements in a virtualised environment. The upper diagram shows a Type 1 hypervisor, also known as a "bare-metal" hypervisor, which runs directly on the underlying 'Hardware'. This hypervisor directly manages the hardware resources and then provisions these resources to various 'Guest' operating systems, such as Guest 1 and Guest 2, offering high performance and security. In contrast, the lower diagram depicts a Type 2 hypervisor, or "hosted" hypervisor, which operates as an application on top of an existing 'Host OS'. This Host OS then runs on the hardware, and the hypervisor, in turn, manages the 'Guest' operating systems, typically seen in desktop virtualisation scenarios.

Containers - Containers share the host kernel but isolate processes, file systems and network namespaces. Popular container engines include Docker, containerd and CRI-O. Features include:

- **Lightweight Footprint:** Faster startup than VMs and lower resource overhead.
- **Immutable Images:** Reproducible environments from development through production.
- **Orchestration Integration:** Platforms such as Kubernetes manage container lifecycles across clusters.

Many cloud OS platforms support both VM and container workloads, offering flexibility for varied application requirements.

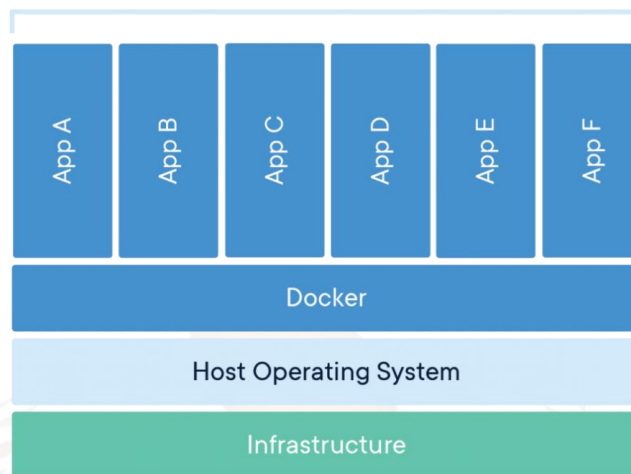


Figure 7: Containerization

Source: <https://www.docker.com/app/uploads/2021/11/container-what-is-container.png>

The figure illustrates the layered architecture of containerisation, demonstrating how multiple applications can run on a single host. At the lowest level is the 'Infrastructure', representing the underlying hardware. Above this resides the 'Host Operating System', which provides the core functionalities. On top of the operating system, 'Docker' (or a similar containerisation platform) acts as a runtime environment, enabling the packaging and execution of applications in isolated units called containers. Each 'Containerized Application', such as App A through App F, encapsulates its own code, runtime, system tools, libraries, and settings, ensuring consistency across different environments without the overhead of full virtual machines.

4.3. Resource Orchestration and Scheduling

Efficient scheduling and orchestrating workloads across a distributed infrastructure are central to cloud operating systems. Orchestration layers coordinate the deployment, scaling and health of services.

Schedulers - Schedulers determine where and when workloads run based on resource availability, policy constraints and quality-of-service requirements. Common scheduling strategies include:

- **Bin packing:** Consolidate workloads to minimise active hosts and conserve energy.
- **Spread scheduling:** Distribute workloads evenly to reduce contention.
- **Priority-based:** Assign high-priority workloads to preferred resources.

Examples of schedulers:

- **OpenStack Nova Scheduler:** Pluggable filters and weighting modules to decide VM placement.
- **Kubernetes Scheduler:** Binds pods to nodes using predicates (e.g. resource checks) and priorities.

Orchestration - Beyond initial placement, orchestration systems manage full lifecycle: deployment, scaling, healing and updates.

- **Declarative APIs:** Users specify the desired state; the orchestrator converges the actual state accordingly.
- **Self-healing:** Failed instances are detected and replaced automatically.
- **Rolling updates and canary deployments:** Allow software changes with minimal disruption.

4.4. Scalability and Elasticity Mechanisms

A defining benefit of cloud OS platforms is the ability to adapt capacity to workload demands, ensuring performance under variable conditions while optimising cost.

Horizontal Scaling: Adding or removing instances (VMs or containers) to match load. Advantages:

- **Linear capacity growth:** More nodes share the workload.
- **Failure isolation:** Faults affect only a subset of instances.

Vertical Scaling: Adjusting resources (CPU, memory) within an existing instance. While simpler to implement, it often requires downtime or specialised mechanisms for live resizing.

Auto-scaling: Automated policies trigger scaling actions based on metrics such as CPU usage, request latency or custom application signals. Components include:

- **Metrics collectors:** Gather performance data at regular intervals.
- **Policy engine:** Evaluates metrics against thresholds.
- **Actuators:** Initiate scale-up or scale-down operations.

Service Discovery: As instances appear and disappear, service discovery mechanisms update routing information, ensuring that clients locate healthy endpoints. Techniques involve:

- **DNS-based discovery:** Dynamic DNS records.
- **Key-value stores:** Central registries such as etcd or Consul.
- **Load balancers:** Route traffic to active backends.

4.5. Fault Tolerance and High Availability

Cloud operating systems must ensure minimal service interruption despite hardware failures, network partitions or software errors.

Redundancy - Deploying multiple replicas of critical components across failure domains (racks, data centres). Redundancy strategies include:

- **Active-active:** All replicas serve traffic concurrently, balancing load.
- **Active-passive:** Standby instances remain idle until primary failure.

Health Monitoring and Probes - Regular liveness and readiness checks verify application health. Failures trigger automated recovery:

- **Liveness probes:** Restart failed containers or VMs.
- **Readiness probes:** Control service ingress based on application readiness.

State Management - Maintaining consistency of application state across failures is challenging. Approaches include:

- **Stateless services:** Avoid persistent state within instances, pushing state to external datastores.
- **Stateful sets with persistent volumes:** Orchestrators ensure stable identity and storage binding for stateful workloads.

Data Replication and Backup - Distributed file systems and databases replicate data across nodes or regions. Backup services capture periodic snapshots and archive them externally.

Disaster Recovery - Cloud OS platforms often support multi-region deployment for geo-redundancy. Strategies encompass:

- **Failover:** Automatic switch to a secondary region on primary outage.
- **Active-active geo-clusters:** Continuous operation across multiple sites with global load balancing.

Cloud operating systems combine complex subsystems to present a unified, resilient and scalable platform. By abstracting hardware through virtualisation and containers, orchestrating workloads intelligently, enabling automatic scaling and embedding fault-tolerance features, they support the demands of modern applications and services. Understanding these components and their interactions is essential for designing, deploying and managing cloud-native systems.

SELF-ASSESSMENT QUESTIONS - 3

Multiple Choice Questions	
11	Which characteristic of a cloud operating system refers to presenting compute, storage and network as services that can be allocated on demand? a) Multi-tenancy b) Abstraction of resources c) Elastic capacity d) Self-service provisioning
12	Which hypervisor type runs directly on the server hardware without relying on a host OS? a) Type 2 (Hosted) b) Type 1 (Bare-metal) c) Container runtime d) Hosted kernel
13	Which scheduling strategy aims to consolidate workloads so as to reduce the number of active hosts and conserve energy? a) Spread scheduling b) Priority-based scheduling c) Bin packing d) Round robin
14	Which scaling approach involves adding or removing virtual machines or containers to match fluctuating load? a) Vertical scaling b) Auto-scaling c) Horizontal scaling d) Service discovery
15	What term describes the deployment of multiple replicas of critical components across different failure domains to ensure uninterrupted service? a) Redundancy b) Sharding c) Checkpointing d) Versioning

5. FUTURE DIRECTIONS AND CONSIDERATIONS

As computing continues to evolve, operating systems—particularly those underpinning cloud and distributed environments—must adapt to new paradigms. Emerging trends such as edge and fog computing, serverless architectures, complex security landscapes and the proliferation of heterogeneous hardware pose fresh challenges. In this section, we explore four key areas likely to shape the next generation of operating-system design and management: the integration of edge and fog computing, the rise of serverless and Function-as-a-Service models, security concerns inherent in distributed clouds, and strategies for managing increasingly varied resources.

5.1. Edge and Fog Computing Integration

The traditional cloud paradigm, centred on large, centralised data centres, faces limitations in latency-sensitive and bandwidth-constrained scenarios. Edge and fog computing address these by extending computation and storage closer to data sources.

Edge computing places processing on or near devices such as IoT sensors, mobile phones or local gateways. This reduces round-trip delays and alleviates network congestion. Fog computing, a related concept, introduces an intermediate layer of micro data centres or network nodes to further distribute workloads along the path between the core cloud and edge devices.

Key considerations for OS support in edge and fog environments include:

- **Lightweight Footprint:** Operating systems for edge devices must impose minimal overhead on constrained hardware. Real-time capabilities are often required for time-critical tasks.
- **Distributed Resource Management:** A unified control plane must view edge, fog and cloud nodes as part of a continuum, enabling seamless workload migration and load balancing based on proximity and resource availability.
- **Security at the Periphery:** Edge nodes operate in less secure environments than centralised data centres. OS-level hardening, secure boot, trusted execution environments and hardware attestation help ensure trustworthiness.
- **Local Data Processing and Caching:** To limit data transfer costs and preserve privacy, operating systems should provide mechanisms for on-device analytics and policy-driven data filtering before forwarding to the cloud.

5.2. Serverless and Function-as-a-Service Models

Serverless computing abstracts the notion of servers entirely, enabling developers to deploy discrete functions that execute in response to events. Function-as-a-Service platforms automatically manage scaling, provisioning and life cycles of these functions.

Fundamental OS and runtime challenges in serverless contexts include:

- **Cold versus Warm Starts:** Invoking a function on demand may incur latency if runtime environments must be created afresh (cold start). Operating systems and lightweight runtimes must minimise boot time through techniques such as snapshotting or pre-warmed containers.
- **Multi-Tenancy and Resource Isolation:** Numerous untrusted functions may execute on the same host. The OS must enforce strict isolation using containers, sandboxing or lightweight virtualisation, balancing security against performance.
- **Fine-Grained Scheduling:** Function invocations are often short lived and bursty. Scheduling policies must rapidly assign compute resources to transient tasks while avoiding thrashing and ensuring fairness.
- **Stateless Execution and Ephemeral Storage:** Serverless functions typically lack persistent local storage. Operating systems must integrate seamlessly with distributed object stores or ephemeral file systems, presenting a coherent interface to developers.

5.3. Security Challenges in Distributed Clouds

Distributed cloud models—spanning central, fog and edge layers—amplify the attack surface, necessitating robust security strategies across all tiers.



Figure 8: Security Challenges in Distributed Clouds

Major OS-level security concerns include:

- **Dynamic Trust Management:** As workloads migrate between nodes with varying trust levels, operating systems must track and enforce provenance, applying stricter controls when code moves from a secure cloud core to less-trusted edge nodes.
- **Secure Communication Channels:** Ensuring confidentiality and integrity of data in transit requires mechanisms such as kernel-based IPsec, TLS acceleration and hardware offload support.
- **Runtime Threat Detection:** Traditional perimeter-based defences falter in dynamic environments. Operating systems must incorporate anomaly detection and trusted monitoring agents to identify malicious behaviour in real time.
- **Patch Management and Attestation:** Thousands of distributed nodes must be kept up to date without disrupting critical operations. OS-level support for secure over the air updates, rollback facilities and remote attestation helps maintain a consistent, trustworthy state.

5.4. Management of Heterogeneous Resources

The rise of specialised hardware—such as GPUs, FPGAs, AI accelerators and domain specific chips—has led to highly heterogeneous data centre and edge environments. Operating systems must evolve to orchestrate these diverse resources effectively.

Important facets of heterogeneous resource management include:

- **Unified Resource Abstraction:** Presenting varied compute units behind common interfaces (e.g., PCIe, CXL) allows schedulers to treat hardware uniformly while still exploiting specialised capabilities.
- **Dynamic Workload Placement:** OS algorithms should assess the computational profile of tasks—such as throughput, latency and parallelism requirements—and match them to the most suitable accelerator or general purpose core.
- **Driver and Firmware Ecosystem:** A modular driver architecture enables the on the fly addition of support for new devices. Operating systems must safeguard against incompatible driver versions through strict module versioning and sandboxing.
- **Power and Thermal Management:** Heterogeneous hardware exhibits different power and heat characteristics. The OS must implement fine grained power capping, DVFS control and thermal throttling policies to maintain reliability and efficiency.
- **Quality of Service Enforcement:** In multi tenant deployments, operating systems should isolate performance interference and guarantee minimum resource shares, preventing noisy neighbours from degrading accelerator performance.

As the computing landscape diverges from monolithic data centres towards a spectrum of edge, fog and cloud resources, operating systems must transcend traditional boundaries. They require lightweight footprints for peripheral devices, intelligent orchestration for serverless bursts, robust security across trust domains and sophisticated mechanisms for managing diverse hardware. Addressing these areas will be critical in crafting resilient, efficient and secure platforms for the next generation of distributed applications and services.

SELF-ASSESSMENT QUESTIONS - 4

Multiple Choice Questions

- | | |
|----|---|
| 16 | Which term refers to an intermediate layer of micro data centres situated between edge devices and central cloud data centres?

a) Edge computing
b) Fog computing
c) Serverless computing
d) Hybrid cloud |
|----|---|

17	In a Function-as-a-Service environment, which challenge describes the latency incurred when a runtime environment is initialised afresh for an invocation? a) Warm start b) Cold start c) Throttling delay d) Snapshot lag
18	Which operating-system-level mechanism is commonly used to ensure confidentiality and integrity of data in transit across distributed cloud environments? a) Load balancing b) Kernel-based IPsec c) Container orchestration d) Filesystem encryption
19	In the context of managing heterogeneous hardware resources, what allows the operating system's scheduler to present varied compute units behind common interfaces? a) Dynamic workload placement b) Unified resource abstraction c) Power capping d) Firmware sandboxing
20	Which security strategy involves tracking and enforcing the origin and trustworthiness of workloads as they migrate between nodes with different security levels? a) Secure boot b) Dynamic trust management c) Remote attestation d) Key-value discovery

6. SUMMARY

- Modern operating systems adopt monolithic, microkernel or hybrid architectures to balance performance, fault isolation and modularity.
- All kernels, whether monolithic with modules, exokernel or hybrid, provide core services such as process scheduling, memory management, file systems, device drivers and inter-process communication.
- Users interact with operating systems through command-line interfaces for fine-grained control, graphical interfaces for ease of use, or web-based dashboards for remote administration.
- Security is enforced via authentication, authorisation and auditing, employing models such as discretionary access control, mandatory access control and role-based control alongside encryption and sandboxing.
- Modular design through loadable kernel modules and plug-in frameworks ensures that services and drivers can be added or updated without rebuilding the entire kernel.
- UNIX, Linux and Windows illustrate differing philosophies: UNIX emphasises simplicity and composability, Linux combines open-source collaboration with modular monolithic design, and Windows NT blends microkernel concepts with a monolithic driver model.
- Process and memory management vary across systems: UNIX uses round-robin scheduling and simple paging, Linux offers the Completely Fair Scheduler and adaptive LRU page reclamation, and Windows NT provides priority-based scheduling with working-set trimming.
- File systems range from UFS/FFS and ZFS in BSD variants to ext4 and Btrfs in Linux, and NTFS in Windows, each offering features such as journalling, copy-on-write, snapshots and pooled storage.
- Cloud operating systems abstract compute, storage and network as on-demand services, support both virtual machines via Type 1 and Type 2 hypervisors and lightweight containers, and orchestrate workloads with schedulers and declarative APIs.
- Future directions demand support for edge and fog computing with minimal footprint and secure boot, serverless environments with rapid cold-start mitigation, robust security across distributed trust domains, and management of GPUs, FPGAs and other specialised hardware.

7. GLOSSARY

Monolithic Kernel	-	A kernel architecture in which all core services—such as device drivers, file systems and memory management—reside in a single large executable running in privileged mode, offering minimal inter-component overhead but risking system-wide instability if one component fails.
Microkernel	-	A minimalist kernel design that implements only essential services (for example, basic memory management and inter-process communication) in kernel space, with all other services running in user space to improve fault isolation at the expense of increased messaging overhead.
Hybrid Kernel	-	An architecture that combines elements of monolithic and microkernel designs by retaining certain subsystems in kernel mode for performance while moving others into user mode, aiming to strike a balance between speed and modularity.
Loadable Kernel Module (LKM)	-	A binary component that can be inserted into or removed from a running kernel without recompilation, enabling dynamic extension of functionality such as new device drivers or file system support.
Hypervisor (Virtual Machine Monitor)	-	Software that creates and manages virtual machines by abstracting hardware resources into isolated guest environments; Type 1 hypervisors run directly on bare metal, while Type 2 hypervisors operate atop a host operating system.
Container	-	A lightweight runtime environment that shares the host operating system's kernel but isolates processes, file systems and network namespaces to provide reproducible and efficient application deployment.
Scheduler	-	A kernel component responsible for determining which process or thread runs at any given time, using strategies such as bin-packing, priority-based or fair-sharing algorithms to balance throughput and responsiveness.
Elasticity	-	The capability of a cloud platform to expand or contract its allocated resources—such as virtual machines or containers—automatically in response to workload changes, ensuring performance while optimising cost.

Edge Computing	-	A distributed computing paradigm that locates processing and storage close to data sources (for example, IoT devices or local gateways) to reduce latency and conserve network bandwidth.
Mandatory Access Control (MAC)	-	A security model in which a central authority defines labels and policies that govern access to resources, overriding individual user decisions to enforce uniform protection levels across the system.



8. TERMINAL QUESTIONS

1. Compare monolithic, microkernel and hybrid architectures, highlighting how each balances performance, fault isolation and complexity.
2. Describe the fundamental services offered by an operating system kernel and explain why each is essential for system operation.
3. Discuss the benefits and limitations of command-line, graphical and web-based user interfaces in modern operating systems.
4. Explain the principles of discretionary, mandatory and role-based access control, and how these models are applied to protect system resources.
5. Analyse how modularisation through loadable kernel modules and plug-in frameworks enhances extensibility and maintainability in operating systems.
6. Outline the historical development and guiding design goals of UNIX, Linux and Windows, emphasising how context and community shaped each system.
7. Compare the process scheduling and memory management techniques used by UNIX, Linux and Windows, and assess their impact on performance and responsiveness.
8. Describe the organisation and key features of file systems in UNIX/BSD, Linux and Windows, noting how they ensure data integrity and support advanced functionality.
9. Define the concept of a cloud operating system, and explain how virtualisation, containerisation and orchestration contribute to resource pooling and elasticity.
10. Evaluate the challenges and strategies associated with edge and fog integration, serverless computing, distributed-cloud security and heterogeneous resource management in future operating systems.

9. ANSWERS

Self-Assessment Questions

1. b) Monolithic
2. c) Microkernel
3. c) Web-based interface provisioning
4. c) Mandatory Access Control
5. b) Loadable Kernel Modules
6. b) UNIX
7. c) Hybrid
8. b) Completely Fair Scheduler
9. c) Copy-on-write transactions
10. c) Mandatory Access Control
11. b) Abstraction of resources
12. b) Type 1 (Bare-metal)
13. c) Bin packing
14. c) Horizontal scaling
15. a) Redundancy
16. b) Fog computing
17. b) Cold start
18. b) Kernel-based IPsec
19. b) Unified resource abstraction
20. b) Dynamic trust management

Terminal Questions Answers

Answers 1 : Monolithic kernels integrate all core services—such as scheduling, memory management and device drivers—into a single privileged executable, offering high performance at the expense of fault isolation; microkernels reduce the kernel to essential functions and run other services in user space for stronger isolation but incur messaging overhead; hybrid kernels seek a compromise by keeping performance-critical subsystems in kernel space while moving others to user mode to balance speed and modularity. Refer to Section 2.1 Architectural Models

Answers 2: Every operating-system kernel must manage processes (creation, scheduling and termination), control memory (via paging or segmentation and address translation), provide file-system abstractions, handle device I/O through drivers, enable inter-process communication, and respond to interrupts and exceptions to ensure reliability and responsiveness. Each service is essential for isolating workloads, organising data, mediating hardware access and coordinating concurrent activities. Refer to Section 2.2 Kernel Types and Services

Answers 3: Command-line interfaces offer precise control, low resource use and powerful scripting but present a steep learning curve; graphical interfaces improve user accessibility and visual feedback at higher resource cost and potential complexity; web-based dashboards enable remote, platform-agnostic administration but introduce dependencies on web services and expand the system's attack surface. Each interface suits different use cases and user skill levels. Refer to Section 2.3 User Interfaces

Answers 4: Discretionary access control lets resource owners grant permissions to users and groups, offering flexibility but relying on user judgement; mandatory access control enforces central policies based on security labels to provide uniform, high-assurance protection; role-based access control assigns permissions to roles rather than individuals, simplifying management in large environments while ensuring consistent privilege allocation. These models form the foundation of modern OS security. Refer to Section 2.4 Security and Access Control Mechanisms

Answers 5: Loadable kernel modules and plug-in frameworks allow new drivers, file systems or services to integrate at runtime without rebuilding the kernel, improving maintainability and reducing downtime. Modularisation confines faults to individual components, simplifies updates and supports customisation for varied deployments, though it demands careful version control and dependency management. Refer to Section 2.5 Modularisation and Extensibility

Answers 6: UNIX began in 1969 at Bell Labs as a portable, C-based system emphasising simplicity and composable tools; Linux emerged in 1991 under an open-source licence to extend UNIX principles with community-driven modularity; Windows NT launched in 1993 with a hybrid design prioritising enterprise reliability, hardware abstraction and backward compatibility. Each was shaped by its developers' goals and target environments. Refer to Section 3.1 Historical Evolution and Design Goals

Answers 7: UNIX/BSD employs a round-robin scheduler with priority ageing and a simple clock-based paging algorithm; Linux uses the Completely Fair Scheduler for proportional CPU sharing, real-

time classes and an adaptive LRU page-reclamation scheme; Windows NT applies a 32-level, priority-based preemptive scheduler and maintains per-process working sets with trimming and swap-file support. These choices affect latency, throughput and resource utilisation. Refer to Section 3.3 Process and Memory Management Features

Answers 8: UNIX/BSD file systems like UFS/FFS use cylinder groups and inodes for efficient layout; ZFS adds pooled storage, copy-on-write, checksums and snapshots; Linux ext4 provides extents, delayed allocation and journalling while Btrfs offers subvolumes and RAID; Windows NTFS centres on the Master File Table with metadata journalling, encryption and alternate data streams. Each design ensures integrity, performance and feature richness. Refer to Section 3.4 File System Organisation

Answers 9: A cloud operating system abstracts compute, storage and network into on-demand services, supports virtual machines via Type 1 and Type 2 hypervisors, and runs containers that share the host kernel for lightweight isolation. Orchestration layers schedule workloads based on policy and health checks, enabling dynamic resource pooling, resilience and automated scaling. Refer to Sections 4.1, 4.2 and 4.3

Answers 10: Future systems must support latency-sensitive edge and fog nodes with minimal OS footprints, optimise cold-start latency and isolation in serverless platforms, enforce security across distributed trust domains, and manage diverse accelerators through unified abstractions, driver modularity, power management and quality-of-service controls. These strategies will shape resilient, efficient distributed environments. Refer to Sections 5.1–5.4

10. REFERENCES

1. Silberschatz, A., Galvin, P. B. & Gagne, G. (2018) Operating System Concepts. 10th edn. Hoboken, NJ: John Wiley & Sons.
2. Tanenbaum, A. S. (2015) Modern Operating Systems. 4th edn. Harlow: Pearson Education.
3. Stallings, W. (2018) Operating Systems: Internals and Design Principles. 9th edn. Harlow: Pearson Education.
4. Buyya, R., Vecchiola, C. & Selvi, S. T. (2013) Mastering Cloud Computing. New York: McGraw-Hill Education.
5. Hwang, K., Fox, G. C. & Dongarra, J. (2012) Distributed and Cloud Computing: From Parallel Processing to the Internet of Things. Amsterdam: Elsevier.